

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2019

Huấn luyện viên: Zhang Ruizhe

Tháng 5 năm 2019

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2019

IOI China candidate team papers / National training team 2019 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2019. Tập được tạo bằng XeTeX và có lớp văn bản đọc được; tuy nhiên các trang có watermark chéo, nên kết quả trích xuất xen thêm chữ từ watermark vào nội dung chính. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, mục lục và mười bốn bài viết đầu tiên: “Tính chất và ứng dụng của hai loại dãy truy hồi” của Zhong Ziqian, “Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị” của Wang Xiuhuan, “Báo cáo ra đề *Bài toán nhỏ dễ*” của Yang Junzhao, “Bàn về bài toán tô màu đỉnh của đồ thị” của Gao Jiaxuan, “Bàn về các bài toán liên quan tới đếm đường đi trên lưới” của Dai Yan, “Mở rộng một số bài toán số học trong thi thuật toán và bước đầu về số nguyên Gauss” của Li Jiaheng, “Báo cáo ra đề *Bài luyện cơ bản về cây tròn-vuông*” của Fan Zhiyuan, “Báo cáo ra đề *Đếm điểm nguyên* và một số nghiên cứu về số nguyên Gauss” của Xu Yixuan, “Bàn về thuật toán chia để trị trên cây” của Zhang Zheyu, “Báo cáo ra đề *Tổng tổ hợp*” của Wu Siyang, và “Bàn về một lớp cấu trúc dữ liệu ngắn gọn” của Wang Siqi, “Thuật toán liên quan tới truy vấn chu kỳ xâu con và ứng dụng” của Chen Sunli, và “Báo cáo ra đề *Công viên*” của Wu Zuotong, và “Bàn về cấu trúc dữ liệu có thể truy vết lịch sử” của Kong Chaozhe.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc năm 2019*. Trang bìa ghi huấn luyện viên là Zhang Ruizhe và thời điểm phát hành là tháng 5 năm 2019. Tập PDF gồm 231 trang.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Tính chất và ứng dụng của hai loại dãy truy hồi	Zhong Ziqian
17	Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị	Wang Xiuhua
27	Báo cáo ra đề <i>Bài toán nhỏ dễ</i>	Yang Junzhao
38	Bàn về bài toán tô màu đỉnh của đồ thị	Gao Jiaxuan
55	Bàn về các bài toán liên quan tới đếm đường đi trên lưới	Dai Yan
69	Mở rộng một số bài toán số học trong thi thuật toán và bước đầu về số nguyên Gauss	Li Jiaheng
89	Báo cáo ra đề <i>Bài luyện cơ bản về cây tròn-vuông</i>	Fan Zhiyuan
104	Báo cáo ra đề <i>Đếm điểm nguyên</i> và một số nghiên cứu về số nguyên Gauss	Xu Yixuan
122	Bàn về thuật toán chia để trị trên cây	Zhang Zheyu
130	Báo cáo ra đề <i>Tổng tổ hợp</i>	Wu Siyang
141	Bàn về một lớp cấu trúc dữ liệu ngắn gọn	Wang Siqi
152	Thuật toán liên quan tới truy vấn chu kỳ xâu con và ứng dụng	Chen Sunli
165	Báo cáo ra đề <i>Công viên</i>	Wu Zuotong
192	Bàn về cấu trúc dữ liệu có thể truy vết lịch sử	Kong Chaozhe
202	Bàn về ứng dụng của ma trận Young trong thi tin học	Yuan Fangzhou

Tính chất và ứng dụng của hai loại dãy truy hồi

Zhong Ziqian, Trường Trung học số 3 Phúc Châu

Tóm tắt

Dãy truy hồi tuyến tính và dãy truy hồi đa thức là hai loại dãy truy hồi thường gặp trong toán học. Bài viết này giới thiệu định nghĩa, tính chất và thuật toán liên quan của hai loại dãy ấy, đồng thời trình bày một số ứng dụng của chúng trong thi tin học.

Lời nói đầu

Dãy truy hồi tuyến tính đã được đưa vào giới thi thuật toán ít nhất năm năm, nhưng vẫn chưa được phổ biến thật rộng rãi. Dãy truy hồi đa thức là một mở rộng tự nhiên của dãy truy hồi tuyến tính và chỉ mới được đưa vào thi tin học trong khoảng hai năm gần đây. Bài viết này mong muốn giới thiệu có hệ thống các tính chất và công dụng của hai loại dãy này trong thi tin học, để người đọc có đường hướng khi suy nghĩ về các bài toán liên quan.

Bài viết trước hết giới thiệu dãy truy hồi tuyến tính ở mục 1, rồi giới thiệu dãy truy hồi đa thức ở mục 2. Với cả hai loại dãy, bài viết trình bày định nghĩa, tính chất, thuật toán liên quan và bài ví dụ thực tế; riêng với dãy truy hồi tuyến tính còn có thêm một số ứng dụng liên quan tới đại số tuyến tính.

1. Dãy truy hồi tuyến tính

1.1. Định nghĩa

Định nghĩa 1.1. Ta gọi dãy có độ dài hữu hạn là dãy hữu hạn, và dãy có độ dài vô hạn là dãy vô hạn.

Định nghĩa 1.2. Ta gọi bậc của hạng tử cao nhất trong chuỗi lũy thừa hình thức F là bậc của F , ký hiệu $\deg(F)$, có thể bằng ∞ . Riêng đa thức không được quy ước có bậc là âm vô cùng $(-\infty)$.

Định nghĩa 1.3. Với dãy hữu hạn $\{a_0, a_1, a_2, \dots, a_{n-1}\}$, ta định nghĩa hàm sinh của nó là đa thức

$$A(x) = \sum_{i=0}^{n-1} a_i x^i.$$

Với dãy vô hạn $\{a_0, a_1, a_2, \dots\}$, ta định nghĩa tương tự hàm sinh của nó là chuỗi lũy thừa hình thức

$$A(x) = \sum_{i=0}^{\infty} a_i x^i.$$

Định nghĩa 1.4. Với dãy vô hạn $\{a_0, a_1, a_2, \dots\}$ và dãy hữu hạn không rỗng $\{r_0, r_1, \dots, r_{m-1}\}$, nếu với mọi $p \geq m-1$ ta có

$$\sum_{k=0}^{m-1} a_{p-k} r_k = 0,$$

thì gọi r là một công thức truy hồi tuyến tính của a . Nếu $r_0 = 1$, ta gọi r là công thức truy hồi tuyến tính của a . Dãy vô hạn có tồn tại công thức truy hồi tuyến tính được gọi là dãy truy hồi tuyến tính.

Với dãy hữu hạn $\{a_0, a_1, \dots, a_{n-1}\}$ định nghĩa tương tự, chỉ yêu cầu đẳng thức trên với $m-1 \leq p \leq n-1$. Bậc của công thức truy hồi tuyến tính là độ dài của nó trừ một. Công thức truy hồi tuyến tính của a có bậc nhỏ nhất được gọi là công thức truy hồi tuyến tính ngắn nhất của a .

1.2. Tính chất cơ bản và phương pháp nhận biết

Định lý 1.1. Với dãy vô hạn a và dãy hữu hạn không rỗng r như trên, gọi A và R là hàm sinh tương ứng của chúng. Khi đó r là công thức truy hồi tuyến tính của a khi và chỉ khi tồn tại đa thức S có bậc không vượt quá $m - 2$ sao cho

$$AR + S = 0.$$

Với dãy hữu hạn $\{a_0, \dots, a_{n-1}\}$, điều kiện tương ứng là

$$AR + S \equiv 0 \pmod{x^n}.$$

Chứng minh. Ta chứng minh trường hợp dãy vô hạn; trường hợp dãy hữu hạn tương tự. Với $k \geq m - 1$, hệ số của x^k trong $A(x)R(x)$ là

$$[x^k](A(x)R(x)) = \sum_{j=0}^{m-1} r_j a_{k-j} = 0.$$

Chọn S thích hợp để triệt tiêu các hệ số bậc thấp là đủ. $\square$

Hệ quả 1.1. Với dãy vô hạn a , gọi A là hàm sinh của nó. Khi đó a là dãy truy hồi tuyến tính khi và chỉ khi tồn tại một đa thức R có hệ số tự do bằng 1 và một đa thức S sao cho

$$A = \frac{S}{R}.$$

Bậc của công thức truy hồi tuyến tính ngắn nhất của a là giá trị nhỏ nhất có thể của $\max(\deg(R), \deg(S) + 1)$. Chứng minh là chuyển về trực tiếp từ định lý 1.1.

Định lý 1.2. Với ma trận $n \times n$ là M , dãy $\{I, M, M^2, M^3, \dots\}$ là dãy truy hồi tuyến tính, và bậc của công thức truy hồi tuyến tính ngắn nhất không vượt quá n .

Chứng minh. Gọi p là đa thức đặc trưng của M . Ta có $\deg(p) = n$, $[x^n]p(x) = 1$, và theo định lý Cayley–Hamilton, $p(M) = 0$. Viết $p(x) = \sum_{i=0}^n c_{n-i}x^i$. Từ

$$\sum_{i=0}^n c_{n-i}M^i = 0$$

và nhân thêm M^j , suy ra

$$\sum_{i=0}^n c_i M^{j+n-i} = 0.$$

Vậy $\{c_0, c_1, \dots, c_n\}$ là một công thức truy hồi tuyến tính bậc n của dãy ma trận này. $\square$

Định lý 1.3. Với các dãy truy hồi tuyến tính $a = \{a_i\}$, $b = \{b_i\}$, dãy hữu hạn $\{t_0, \dots, t_{m-1}\}$ và hằng số p , các dãy sau đều là dãy truy hồi tuyến tính:

$$\{t_0, \dots, t_{m-1}, a_0, a_1, \dots\}, \quad \{a_i p^i\}, \quad \{a_{i+1}\}, \quad \{a_i + b_i\},$$

$$\left\{ \sum_{j=0}^i a_j b_{i-j} \right\}, \quad \{a_i b_i\}.$$

Các chứng minh khá đơn giản, dùng hệ quả 1.1 và định lý 1.2 là đủ; do giới hạn độ dài nên lược bỏ.

1.3. Thuật toán liên quan

1.3.1. Tìm công thức truy đầy tuyến tính ngắn nhất

Trước hết xét dãy hữu hạn và giả sử mọi phép toán nằm trên một trường. Cách đơn giản là khử Gauss, với dãy dài n tốn $O(n^3)$. Thuật toán Berlekamp–Massey giảm thời gian xuống $O(n^2)$.

Với dãy $\{a_0, \dots, a_{n-1}\}$, Berlekamp–Massey tìm công thức truy đầy tuyến tính ngắn nhất $r^{(i)}$ cho mỗi tiền tố $\{a_0, \dots, a_i\}$. Đặt $|r^{(i)}| - 1 = l_i$. Rõ ràng $r^{(-1)} = \{1\}$ và $l_{i-1} \leq l_i$.

Bổ đề 1.1. Nếu $r^{(i-1)}$ không phải công thức truy đầy tuyến tính ngắn nhất của $\{a_0, \dots, a_i\}$, thì

$$l_i \geq \max(l_{i-1}, i + 1 - l_{i-1}).$$

Chứng minh. Phản chứng, giả sử $l_i \leq i - l_{i-1}$. Đặt

$$r^{(i-1)} = \{p_0, p_1, \dots, p_{l_{i-1}}\}, \quad r^{(i)} = \{q_0, q_1, \dots, q_{l_i}\}.$$

Theo định nghĩa của $r^{(i)}$, với $l_i \leq j \leq i$,

$$a_j = - \sum_{t=1}^{l_i} q_t a_{j-t}.$$

Do đó

$$\begin{aligned} - \sum_{j=1}^{l_{i-1}} p_j a_{i-j} &= \sum_{j=1}^{l_{i-1}} p_j \sum_{k=1}^{l_i} q_k a_{i-j-k} \\ &= \sum_{k=1}^{l_i} q_k \sum_{j=1}^{l_{i-1}} p_j a_{i-j-k} = - \sum_{k=1}^{l_i} q_k a_{i-k} = a_i. \end{aligned}$$

Vậy $r^{(i-1)}$ vẫn là công thức truy đầy cho tiền tố dài hơn, mâu thuẫn. $\square$

Dấu bằng trong bổ đề có thể đạt được. Gọi A là hàm sinh của a , còn $R^{(i)}$ là hàm sinh của $r^{(i)}$. Theo định lý 1.1 tồn tại $S^{(i)}$ sao cho

$$AR^{(i)} \equiv S^{(i)} \pmod{x^{i+1}}, \quad \deg(S^{(i)}) \leq l_i - 1.$$

Nếu $AR^{(i-1)} \equiv S^{(i-1)} \pmod{x^{i+1}}$, đặt $R^{(i)} = R^{(i-1)}$. Ngược lại

$$AR^{(i-1)} \equiv S^{(i-1)} + dx^i \pmod{x^{i+1}}.$$

Xét lần gần nhất công thức tăng bậc: tồn tại $p < i$ và c sao cho

$$AR^{(p-1)} \equiv S^{(p-1)} + cx^p \pmod{x^{p+1}}.$$

Nhân với $x^{i-p}dc^{-1}$ rồi trừ hai đồng dư, ta đặt được

$$R^{(i)} = R^{(i-1)} - x^{i-p}dc^{-1}R^{(p-1)}.$$

Khi đó $l_i = \max(l_{i-1}, i + 1 - l_{i-1})$. Chỉ cần duyệt i tăng dần, tính $R^{(i)}$ và cập nhật p, c khi $l_i > l_{i-1}$. Độ phức tạp là $O(n^2)$.

Định lý 1.4. Nếu dãy truy hồi tuyến tính a có công thức truy đầy tuyến tính ngắn nhất bậc không vượt quá s , thì công thức truy đầy tuyến tính ngắn nhất của tiền tố $\{a_0, a_1, \dots, a_{2s-1}\}$ chính là công thức truy đầy tuyến tính ngắn nhất của a . Chứng minh dùng bổ đề 1.1: nếu lần đầu thất bại ở vị trí $t \geq 2s$, bậc mới ít nhất $t + 1 - s > s$, mâu thuẫn. Vì vậy chỉ cần biết chặn trên s , tính $2s$ hạng đầu và chạy Berlekamp–Massey.

1.3.2. Tính một hạng của dãy truy hồi tuyến tính

Giả sử a thỏa công thức $\{r_0, \dots, r_{m-1}\}$, cần tính a_k . Đặt

$$G(F) = \sum_{i=0}^{\infty} a_i [x^i] F(x), \quad S(x) = \sum_{i=0}^{m-1} x^{m-1-i} r_i.$$

Từ công thức truy hồi, $G(S(x)T(x)) = 0$ với mọi đa thức T . Chia

$$x^k = S(x)U(x) + R(x), \quad \deg R < \deg S,$$

ta được

$$G(x^k) = G(x^k \bmod S(x)).$$

Dùng lũy thừa nhanh để tính $x^k \bmod S(x)$, rồi lấy tổ hợp tuyến tính các hạng đầu của a . Nếu phép lấy dư hai đa thức bậc $O(m)$ làm trong $O(m \log m)$, tổng thời gian là $O(m \log m \log k)$ [4].

1.4. Ứng dụng thường gặp

Nhờ định lý 1.2, dãy truy hồi tuyến tính thường gặp trong đại số tuyến tính. Dưới đây giả sử tính toán modulo một số nguyên tố lớn p .

1.4.1. Dãy vectơ và dãy ma trận

Với dãy vectơ hàng n chiều $\{t_0, t_1, \dots\}$, lấy ngẫu nhiên vectơ cột v rồi xét dãy vô hướng $\{t_0 v, t_1 v, \dots\}$. Theo bổ đề Schwartz–Zippel [5], với xác suất ít nhất $1 - n/p$, công thức truy đầy ngắn nhất của dãy vô hướng trùng với công thức của dãy vectơ.

Với dãy ma trận $n \times m$, lấy ngẫu nhiên vectơ hàng u và vectơ cột v , rồi xét $\{ut_0 v, ut_1 v, ut_2 v, \dots\}$. Xác suất thành công ít nhất $1 - (n + m)/p$.

1.4.2. Đa thức tối thiểu của ma trận

Đa thức tối thiểu của M là đa thức khác không bậc nhỏ nhất f sao cho $f(M) = 0$. Nếu $\{r_0, \dots, r_m\}$ là công thức truy đầy của $\{I, M, M^2, \dots\}$, thì

$$\sum_{i=0}^m r_{m-i} M^i = 0, \quad f(x) = \sum_{i=0}^m r_{m-i} x^i$$

là một đa thức triệt tiêu. Vậy đa thức tối thiểu tương ứng với công thức truy đầy tuyến tính ngắn nhất của dãy ma trận này, có bậc không vượt quá n .

Cụ thể, với u, v ngẫu nhiên, tính $\{uv, uMv, uM^2v, \dots, uM^{2n}v\}$. Vì $uM^{i+1} = (uM^i)M$, ma trận đặc tồn $O(n^3)$. Nếu M thưa có e vị trí khác không, mỗi bước tốn $O(n + e)$, tổng thời gian $O(n(n + e))$.

1.4.3. Tối ưu quy hoạch động

Xét quy hoạch động

$$f(i, j) = \sum_{t=0}^{m-1} f(i-1, t) c(t, j) \quad (i \geq 1, j \in [0, m)).$$

Gọi $F(i)$ là vectơ hàng các giá trị $f(i, j)$, và C là ma trận của c . Khi đó $F(i) = F(i-1)C$, nên $F(n) = F(0)C^n$. Thay vì lũy thừa ma trận $O(m^3 \log n)$, dùng định lý 1.2 để suy ra $\{F(n)\}$ là dãy truy hồi tuyến tính, tìm công thức truy đầy từ các hạng đầu rồi dùng mục 1.3.2. Độ phức tạp được nêu là

$$O(m^3 + m \log m \log n).$$

1.4.4. Hệ tuyến tính thưa

Cho ma trận đầy hạng A và vectơ hàng b , cần tìm x sao cho $Ax = b$. Vì $x = A^{-1}b$, xét công thức truy đầy ngắn nhất $\{r_0, \dots, r_{m-1}\}$ của $\{b, Ab, A^2b, \dots\}$. Ta có

$$\sum_{i=0}^{m-1} A^i b r_{m-1-i} = 0, \quad r_{m-1} \neq 0.$$

Nhân với A^{-1} , suy ra

$$A^{-1}b = -\frac{1}{r_{m-1}} \sum_{i=0}^{m-2} A^i b r_{m-2-i}.$$

Nếu A có e vị trí khác không, tính dãy $\{b, Ab, \dots, A^{2n}b\}$ tốn $O(ne)$, tổng thời gian $O(n(n+e))$.

1.4.5. Định thức ma trận thưa

Để tính $\det(A)$, tìm đa thức tối thiểu của ma trận thưa. Nếu mỗi giá trị riêng có bội hình học bằng một, đa thức tối thiểu là đa thức đặc trưng; hệ số tự do nhân $(-1)^n$ là định thức. Để tăng xác suất thỏa điều kiện này, nhân A với ma trận đường chéo ngẫu nhiên B . Khi đó AB thỏa với xác suất ít nhất $1 - (2n^2 - n)/p$ [7]. Tính $\det(AB)$, rồi chia cho $\det(B)$, là tích các phần tử đường chéo. Độ phức tạp $O(n(n+e))$.

1.4.6. Hạng ma trận thưa

Với ma trận $n \times m$ A , hạng của ma trận vuông bằng cấp trừ bội đại số của giá trị riêng 0. Nếu đa thức đặc trưng bằng đa thức tối thiểu nhân một lũy thừa của x , chỉ cần tìm đa thức tối thiểu rồi bỏ các thừa số x .

Sinh ngẫu nhiên ma trận đường chéo D_1 cỡ $m \times m$. Ma trận AD_1A^T đối xứng và có cùng hạng với A với xác suất ít nhất $1 - n^2/p$. Sinh tiếp ma trận đường chéo D_2 ; ma trận $D_2AD_1A^TD_2$ thỏa điều kiện đặc trưng/tối thiểu với xác suất ít nhất $1 - 4n^2/p$ [7]. Vì các ma trận đường chéo và A, A^T đều thưa, thời gian vẫn là $O(n(n+e))$.

1.5. Bài ví dụ

Ví dụ 1. Gapless Filling with Tetrominoes. Nguồn: Bytedance Camp 2019 Day3 Division A, Problem G, có chỉnh sửa.

Cho lưới $n \times 4$, cần dùng n tetromino phủ kín, mỗi ô đúng một lần. In số phương án modulo số nguyên tố p , với

$$1 \leq n \leq 10^9, \quad 2 \leq p \leq 10^9 + 9.$$

Dùng quy hoạch động nén trạng thái: $f[t][S]$ là số cách sau khi xét t hàng, trong đó S là tập ô chưa phủ ở ba hàng cuối. Đáp án là $f[n][\emptyset]$. Lời giải chính thức tìm các trạng thái đạt được rồi chuyển bằng ma trận. Theo định lý 1.2, $\{f[u][\emptyset]\}_{u=0}^\infty$ là dãy truy hồi tuyến tính; tìm công thức truy đầy bằng mục 1.3.1 rồi nhảy hạng bằng mục 1.3.2. Thử nghiệm thực tế tìm được công thức bậc 34.

Ví dụ 2. Expected Value. Nguồn: Ildar Gainullin Contest 1, Problem E, có giản lược.

Cho đồ thị phẳng đơn, vô hướng, liên thông. Quân cờ bắt đầu ở đỉnh 1, mỗi bước chọn đều một cạnh kề và đi theo cạnh đó. Tính kỳ vọng số bước để tới đỉnh n , modulo số nguyên tố p , với

$$1 \leq n \leq 2000, \quad 10^9 < p < 1.01 \times 10^9.$$

Đồ thị phẳng có $m \leq 3n - 6$ cạnh [8]. Gọi f_x là kỳ vọng từ đỉnh x , d_x là bậc của x . Khi đó

$$f_x = \begin{cases} 1 + \sum_{(x,y) \in E} \frac{f_y}{d_x}, & x \neq n, \\ 0, & x = n. \end{cases}$$

Mà trận hệ số chỉ có $O(n + m)$ vị trí khác không, nên dùng mục 1.4.4, độ phức tạp $O(n^2)$. Một cách khác xem ở [9].

2. Dãy truy hồi đa thức

Thuật ngữ tiếng Anh thường dùng cho loại dãy này là *P-recursive sequences*.

2.1. Định nghĩa

Ta dùng lại các định nghĩa dãy hữu hạn, dãy vô hạn, bậc chuỗi lũy thừa hình thức và hàm sinh. Giả sử tính toán trên trường $\mathbb{K}$.

Định nghĩa 2.1. Với dãy vô hạn $a = \{a_i\}$ và dãy đa thức hữu hạn không rỗng $\{P_0, \dots, P_{m-1}\}$, nếu $P_0 \neq 0$ và với mọi $p \geq m - 1$

$$\sum_{k=0}^{m-1} a_{p-k} P_k(p) = 0,$$

thì gọi dãy P là công thức truy hồi đa thức của a . Dãy có tồn tại công thức như vậy gọi là dãy truy hồi đa thức. Với dãy hữu hạn, yêu cầu đẳng thức cho $m - 1 \leq p \leq n - 1$. Bậc truy hồi của $\{r_0, \dots, r_{m-1}\}$ là $m - 1$, bậc đa thức là $\max_i \deg(r_i)$.

Định nghĩa 2.2. Chuỗi lũy thừa hình thức $A(x)$ gọi là vi phân hữu hạn nếu tồn tại đa thức

$$Q_0(x), \dots, Q_{m-1}(x),$$

với $Q_{m-1} \neq 0$, sao cho

$$\sum_{i=0}^{m-1} Q_i(x) A^{(i)}(x) = 0.$$

Chuỗi $A(x)$ gọi là đại số nếu nó đại số trên $\mathbb{K}(x)$, tức là nghiệm của một phương trình đa thức có hệ số trong trường phân thức hữu tỷ $\mathbb{K}(x)$. Thuật ngữ tiếng Anh của “vi phân hữu hạn” là *D-finite*.

2.2. Tính chất cơ bản và phương pháp nhận biết

Định lý 2.1. Nếu hàm sinh thường của $a = \{a_i\}$ là $A(x)$, thì a là dãy truy hồi đa thức khi và chỉ khi A vi phân hữu hạn.

Chứng minh. Chiều đủ: vì

$$x^j A^{(i)}(x) = \sum_{n=0}^{\infty} a_{n+i-j} x^n \prod_{t=1}^i (n+t-j)$$

(các hạng có chỉ số âm xem là 0), thay vào quan hệ vi phân rồi lấy hệ số x^n với n đủ lớn sẽ cho một công thức truy hồi đa thức.

Chiều cần: viết công thức truy hồi dưới dạng

$$\sum_{k=0}^{m-1} a_{p+k} P'_k(p) = 0 \quad (p \geq 0).$$

Mỗi $P'_i(n)$ có thể biểu diễn thành tổ hợp tuyến tính của $\prod_{t=0}^{j-1} (n+i-t)$. Mặt khác

$$x^i \sum_{n=0}^{\infty} a_{n+i} x^n \prod_{t=0}^{j-1} (n+i-t) = x^i A^{(j)}(x) - \sum_{n=0}^{i-1} a_n x^n \prod_{t=0}^{j-1} (n-t).$$

Gom hạng sẽ được

$$\sum_i Q'_i(x) A^{(i)}(x) = S(x)$$

với S là đa thức. Nếu $S \neq 0$, lấy đạo hàm $\deg(S) + 1$ lần ở hai vế. $\square$

Bổ đề 2.1. Chuỗi $u(x)$ vi phân hữu hạn khi và chỉ khi

$$\dim_{\mathbb{K}(x)} \text{span}_{\mathbb{K}(x)} \{u(x), u'(x), \dots\}$$

hữu hạn. Một chiều lấy quan hệ phụ thuộc tuyến tính giữa $u, u', \dots$; chiều còn lại dùng quan hệ vi phân để biểu diễn mọi đạo hàm bậc cao bằng hữu hạn đạo hàm đầu. $\square$

Bổ đề 2.2. Nếu u đại số theo x , thì u' cũng đại số. Thật vậy, với đa thức tối tiểu $P(x, u) = 0$, lấy đạo hàm:

$$0 = \frac{\partial P(x, y)}{\partial x} \Big|_{y=u} + u' \frac{\partial P(x, y)}{\partial y} \Big|_{y=u},$$

nên

$$u' = - \frac{\frac{\partial P(x, y)}{\partial x} \Big|_{y=u}}{\frac{\partial P(x, y)}{\partial y} \Big|_{y=u}}.$$

Định lý 2.2. Nếu u vi phân hữu hạn, v đại số, và $u(v(x))$ là một chuỗi lũy thừa hình thức, thì $u(v(x))$ vi phân hữu hạn.

Chứng minh. Đặt $y = u(v(x))$. Từ quy tắc dây chuyền, mọi $y^{(i)}$ là tổ hợp tuyến tính của

$$u(v(x)), u'(v(x)), \dots$$

với hệ số trong $\mathbb{K}[v', v'', \dots]$. Theo bổ đề 2.2, các $v^{(i)}$ nằm trong $\mathbb{K}(x, v)$. Vì u vi phân hữu hạn, $\text{span}_{\mathbb{K}(x, v)} \{u(v), u'(v), \dots\}$ hữu hạn chiều; do v đại số, $[\mathbb{K}(x, v) : \mathbb{K}(x)]$ hữu hạn. Bởi bổ đề 2.1, y vi phân hữu hạn. $\square$

Định lý 2.3. Với các dãy truy hồi đa thức $a = \{a_i\}$, $b = \{b_i\}$, dãy hữu hạn $\{t_0, \dots, t_{m-1}\}$ và hằng số p , các dãy sau đều là dãy truy hồi đa thức:

$$\{t_0, \dots, t_{m-1}, a_0, a_1, \dots\}, \quad \{a_i p^i\}, \quad \{a_{i+1}\}, \quad \{a_i + b_i\}, \\ \left\{ \sum_{j=0}^i a_j b_{i-j} \right\}, \quad \{a_i b_i\}.$$

Ba tính chất đầu theo định nghĩa. Với tổng, dùng $V_t = \text{span}_{\mathbb{K}(x)} \{t, t', t'', \dots\}$ và $V_{u+v} \subseteq V_u \oplus V_v$. Với tích chập, hàm sinh là uv , dùng quy tắc tích và ánh xạ $\varphi : V_u \otimes V_v \rightarrow \mathbb{K}((x))$, $\varphi(y \otimes z) = yz$. Tính chất cuối phức tạp hơn, xem định lý 6.4.12 trong [10].

2.3. Thuật toán liên quan

2.3.1. Tìm công thức truy hồi đa thức

Vì bậc đa thức chưa biết, thường liệt kê hoặc suy ra bậc s , rồi tìm công thức bậc truy hồi nhỏ nhất dưới bậc đó. Với công thức bậc $m \{P_0, \dots, P_m\}$,

$$\sum_{k=0}^m a_{p-k} P_k(p) = 0 \quad (m \leq p \leq n-1).$$

Đặt $Q_i(x) = P_i(x+m)$, ta có

$$\sum_{k=0}^m a_{p+k} Q_k(p) = 0 \quad (0 \leq p \leq n-m-1).$$

Giả sử bậc truy hồi không vượt quá m_0 , đặt $t_{k,u} = [x^u]Q_k(x)$; số biến là $(m_0 + 1)(s + 1)$. Lấy $n \geq (s + 2)(m_0 + 1)$ hạng đầu và lập $n - m_0$ phương trình. Ma trận không đầy hạng vì nhân mọi Q_i với một hằng số vẫn là nghiệm; ta khử theo thứ tự các biến, gán giá trị khác không cho biến tự do đầu tiên sao cho nhận được $Q_m \neq 0$. Sau đó dùng nhị thức để đổi ngược về P_i . Độ phức tạp $O(n^2ms)$. Với dữ liệu ngẫu nhiên và n đủ lớn, xác suất tìm đúng công thức là cao.

2.3.2. Tính một hạng của dãy truy hồi đa thức

Giả sử a thỏa công thức bậc $m \{P_0, \dots, P_m\}$, biết $a_0, \dots, a_{m-1}$, và bậc đa thức là d . Nếu $P_0(x) \neq 0$ với mọi số nguyên $x \in [m, n]$, có thể truy đẩy trực tiếp:

$$a_p = -\frac{1}{P_0(p)} \sum_{j=1}^m a_{p-j} P_j(p),$$

tốn $O(nmd)$.

Khi $m, d \ll n$, tính trước $\prod_{i=m}^n P_0(i)a_n$, rồi chia cho $\prod_{i=m}^n P_0(i)$. Với $n' = n - m + 1$, đặt

$$u_i = \left\{ a_j \prod_{t=0}^{i-1} P_0(t+m) \right\}_{j=i}^{i+m-1}.$$

Khi đó $u_{n'}$ chứa $\prod_{i=m}^n P_0(i)a_n$. Quan hệ $u_i M(i) = u_{i+1}$ được mô tả bởi ma trận

$$M(i) = \begin{bmatrix} 0 & 0 & \dots & 0 & 0 & -P_m(i+m) \\ P_0(i+m) & 0 & \dots & 0 & 0 & -P_{m-1}(i+m) \\ 0 & P_0(i+m) & \dots & 0 & 0 & -P_{m-2}(i+m) \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & P_0(i+m) & 0 & -P_2(i+m) \\ 0 & 0 & \dots & 0 & P_0(i+m) & -P_1(i+m) \end{bmatrix},$$

nên

$$u_{n'} = u_0 \prod_{i=0}^{n'-1} M(i).$$

Xem $M(x)$ là ma trận đa thức bậc không vượt quá d . Chọn ngưỡng S và tính

$$T_S(x) = \prod_{i=0}^{S-1} M(Sx + i).$$

Mỗi phần tử có bậc không vượt quá dS . Ta chỉ cần các giá trị điểm tại $x = 0, 1, \dots, \lfloor (n' - S)/S \rfloor$, rồi nhân các ma trận giá trị ấy và các hạng lẻ còn lại.

Để tính nhanh các giá trị điểm, đặt $T_w(x) = \prod_{i=0}^{w-1} M(x+i)$ và nhân đôi w , dùng

$$T_{2w}(x) = T_w(x)T_w(x+w).$$

Phép dịch giá trị điểm dùng nội suy Lagrange: nếu biết $h(0), \dots, h(d)$, thì

$$\begin{aligned} h(m+k) &= \sum_{i=0}^d h(i) \prod_{\substack{j=0 \\ j \neq i}}^d \frac{m+k-j}{i-j} \\ &= \left(\prod_{j=0}^d (m+k-j) \right) \left(\sum_{i=0}^d \frac{h(i)}{i!(d-i)!(-1)^{d-i}} \frac{1}{m+k-i} \right). \end{aligned}$$

Phần tích có thể lấy bằng tích tiền tố trên đoạn $[k-d, k+d]$, phần còn lại là tích chập, xử lý bằng FFT trong $O(d \log d)$.

Chọn S nhỏ nhất thỏa $dS \geq (n' - S)/S$, khi đó

$$S = O(\sqrt{n/d}), \quad T = O\left(\sqrt{nd}(m^3 + m^2 \log(nd))\right).$$

Nếu $P_0 \neq 1$, tích $\prod_{i=m}^n P_0(i)$ cũng tính bằng cùng phương pháp với dãy bậc 1, không đổi nút nghẽn.

2.4. Bài ví dụ

Ví dụ 3. Chef and Same Old Recurrence 2. Nguồn: CodeChef JADUGAR2, có chỉnh sửa.

Cho K, A, B , định nghĩa

$$\begin{cases} a_1 = K, \\ a_n = Aa_{n-1} + B \sum_{i=1}^{n-1} a_i a_{n-i}, \quad n > 1. \end{cases}$$

Cần tính $a_1, \dots, a_N$ modulo $10^9 + 7$, nhưng chỉ in xor của các giá trị, với $1 \leq N \leq 10^7, 1 \leq B, K < 10^9 + 7, 0 \leq A < 10^9 + 7$. Đặt $a_0 = 0$ và gọi g là hàm sinh. Từ truy hồi suy ra

$$g = Kx + Axg + Bg^2.$$

Vậy g đại số trên $\mathbb{K}(x)$, nên vi phân hữu hạn; a là dãy truy hồi đa thức. Dùng mục 2.3.1 để tìm công thức rồi truy đẩy, thời gian $O(N)$.

Ví dụ 4. Jump Jump Jump. Nguồn: Grand Prix of Zhejiang, Problem J, có chỉnh sửa.

Một con thỏ bắt đầu ở $(0, 0)$, mỗi bước chọn đều một trong k vectơ (dx_i, dy_i) khác nhau rồi nhảy theo vectơ ấy. Tại mỗi $(x, x), x \geq 1$, có một bẫy. Cho n , với mỗi $x \in [1, n]$ hãy tính xác suất thỏ rơi vào bẫy (x, x) modulo 998244353. Các $dx_i, dy_i \in [0, 3]$ và không đồng thời bằng 0; $1 \leq n \leq 10^5, 1 \leq k \leq 15$.

Định lý 2.4. Nếu $F(s, t)$ là chuỗi lũy thừa hình thức hữu tỷ theo s, t , thì

$$\text{diag } F = \sum_n x^n [s^n t^n] F(s, t)$$

là chuỗi lũy thừa hình thức đại số theo x . Chứng minh xem [10, định lý 6.3.3].

Gọi p_t là xác suất trong mô hình không có bẫy rằng thỏ từng đi qua (t, t) . Đặt

$$G(s, t) = \frac{1}{k} \sum_{i=1}^k s^{dx_i} t^{dy_i}, \quad F(s, t) = \sum_{i=0}^{\infty} G(s, t)^i = \frac{1}{1 - G(s, t)}.$$

Khi đó $p_t = [x^t]$ diag F , nên theo định lý 2.4 và 2.2, p là dãy truy hồi đa thức. Tính một số hạng đầu bằng quy hoạch động, tìm công thức bằng mục 2.3.1, rồi truy đẩy được $p_1, \dots, p_n$.

Gọi h_t là xác suất rơi vào bẫy (t, t) . Khi đó

$$h_t = p_t - \sum_{s=1}^{t-1} h_s p_{t-s}.$$

Với

$$P = \sum_{i=1}^{\infty} x^i p_i, \quad H = \sum_{i=1}^{\infty} x^i h_i,$$

ta có $H = P - HP$, tức $H = P/(1 + P)$. Biết $P \bmod x^{n+1}$, dùng nghịch đảo đa thức để tính $H \bmod x^{n+1}$ trong $O(n \log n)$ [4].

Ví dụ 5. Đếm đơn giản. Nguồn: bài kiểm tra lẫn nhau của đội tuyển tập huấn năm 2019, có chỉnh sửa.

Tính số đồ thị có hướng không chu trình có nhãn trên n đỉnh, mỗi cạnh có một trong k màu, mỗi đỉnh có không quá một cạnh đi ra, và số cạnh đi vào mỗi đỉnh thuộc S . Hai đồ thị khác nhau nếu khác cạnh hoặc khác màu cạnh.

$$1 \leq n \leq 9 \times 10^8, \quad 1 \leq k \leq 10^7, \quad 0 \in S, \quad S \subseteq [0, 3].$$

Đối tượng ứng sang rừng có gốc có nhãn: mỗi đỉnh có số con thuộc S , mỗi cạnh có k màu. Gọi f_n là số cây hợp lệ, $g_{a,n}$ là số rừng gồm đúng a cây, h_n là số rừng. Dùng hàm sinh mũ $A(x) = \sum a_i x^i / i!$. Gọi các hàm sinh là $F(x), G_a(x), H(x)$. Từ tính chất hàm sinh mũ [12],

$$G_a(x) = \frac{F(x)^a}{a!}, \quad F(x) = \sum_{t \in S} x k^t G_t(x) = \sum_{t \in S} x k^t \frac{F(x)^t}{t!},$$

$$H(x) = \sum_{t=0}^{\infty} \frac{F(x)^t}{t!} = e^{F(x)}.$$

Đáp án là $n![x^n]H(x)$. Vì F đại số và e^x vi phân hữu hạn, định lý 2.2 cho $H = e^F$ vi phân hữu hạn. Theo định lý 2.3, $\{t![x^t]H(x)\}$ là dãy truy hồi đa thức. Dùng mục 2.3.1 để khử ra công thức, rồi mục 2.3.2 để tính đáp án.

3. Tổng kết

Dãy truy hồi tuyến tính và dãy truy hồi đa thức là hai loại dãy rất thường gặp, có ứng dụng rộng rãi trong toán học, nhưng hiện vẫn chưa phổ biến trong thi tin học. Bài viết đã giới thiệu định nghĩa, tính chất, thuật toán liên quan và ứng dụng của chúng, với hy vọng giúp người đọc có hiểu biết ban đầu và mong rằng sẽ có thêm nhiều bài toán thú vị thuộc lớp này xuất hiện trong thi tin học.

Trong lĩnh vực này vẫn còn nhiều vấn đề cần giải quyết. Do trình độ tác giả còn hạn chế, mong bài viết có thể đóng vai trò gợi mở và thu hút thêm nhiều độc giả nghiên cứu các bài toán liên quan.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn. Xin cảm ơn các thầy cô Huang Zhigang, Song Lilin, Huang Xiaoyan và Wei Lizhen đã bồi dưỡng và dạy dỗ tôi. Xin cảm ơn Du Yuhao, Mao Xiao, Wang Xiuhuan, Zhu Zhenting đã giúp đỡ bài viết này. Xin cảm ơn Kong Chaozhe, Liu Cheng'ao, Zeng Yaohui, Zhang Qingchuan, Wu Hang đã kiểm tra bản thảo. Xin cảm ơn cha mẹ đã thấu hiểu và ủng hộ tôi.

Tài liệu tham khảo

1. Mao Xiao. Một số nghiên cứu về công thức truy hồi của dãy số. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2017, 2017.
2. Wikipedia contributors. Cayley–Hamilton theorem. Wikipedia, https://en.wikipedia.org/wiki/Cayley-Hamilton_theorem.
3. Massey J. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory, 1969.
4. Peng Yuxiang. Introduction to Polynomials. WC2016, 2016.
5. Wikipedia contributors. Schwartz–Zippel lemma. Wikipedia, https://en.wikipedia.org/wiki/Schwartz-Zippel_lemma.
6. Wiedemann, Douglas. Solving sparse linear equations over finite fields. IEEE Transactions on Information Theory 32.1, 1986.
7. Chen, Li, et al. Efficient matrix preconditioners for black box linear algebra. Linear Algebra and its Applications 343, 2002.
8. Wikipedia contributors. Planar graph. Wikipedia, https://en.wikipedia.org/w/index.php?title=Planar_graph.
9. Wang Xiuhan. Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2019, 2019.
10. Stanley RP. Enumerative Combinatorics. Volume 62 of Cambridge Studies in Advanced Mathematics, 1999.
11. Min-25. Find Linear Recurrence with Polynomial Coefficients. <https://min-25.hatenablog.com/entry/2018/05/10/21280>.
12. Jin Ce. Phép toán và ứng dụng của hàm sinh. Trao đổi học viên WC2015, 2015.

Bàn về bài toán bước đi ngẫu nhiên trên mô hình đồ thị

Wang Xiuhuan, Trường Trung học số 7 Thành Đô

Tóm tắt

Bài toán bước đi ngẫu nhiên là một lớp bài toán thường gặp trong thi tin học, và cũng có ứng dụng rộng rãi trong đời sống thực tế, chẳng hạn thuật toán xếp hạng trang web của Google. Bài viết này xoay quanh bài toán ấy, lần lượt khảo sát từ ba góc độ: đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Bài viết giới thiệu vài phương pháp thường dùng để giải lớp bài toán này, đồng thời so sánh ưu nhược điểm của chúng khi xử lý các kiểu bài khác nhau, với hy vọng giúp người đọc có đường hướng khi gặp bài toán dạng này.

1. Dẫn nhập

Trong các cuộc thi thuật toán những năm gần đây, số bài về bước đi ngẫu nhiên dần tăng lên. Lớp bài toán này biến hóa rất nhiều, cách giải tùy từng đề, nên thường khiến người giải khó bắt đầu.

Bài viết chủ yếu khảo sát từ ba góc độ: đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Ta sẽ giới thiệu nhiều phương pháp giải các bài toán này và so sánh ưu nhược điểm của chúng trên các kiểu bài khác nhau, với hy vọng giúp người đọc có đường hướng khi gặp bài toán dạng này.

Mục 2 giới thiệu định nghĩa của bài toán bước đi ngẫu nhiên. Mục 3 giới thiệu hai phương pháp cho bài toán bước đi ngẫu nhiên trên đồ thị lưới, có độ phức tạp lần lượt là $O(n\sqrt{n})$ và $O(n^2)$. Mục 4 giới thiệu một phương pháp giải bài toán bước đi ngẫu nhiên trên đồ thị thưa với một cặp đỉnh xuất phát và kết thúc cố định, độ phức tạp $O(n^2)$. Mục 5 giới thiệu phương pháp giải bài toán trên đồ thị tổng quát cho mọi cặp đỉnh xuất phát và kết thúc, độ phức tạp $O(n^3)$. Mục 6 so sánh ưu nhược điểm của các phương pháp này, và mục 7 tổng kết toàn văn.

2. Định nghĩa

Cho một đồ thị đơn có hướng $G = (V, E)$, trong đó $V = \{v_1, v_2, \dots, v_{|V|}\}$, một đỉnh xuất phát $v_s \in V$ và một đỉnh kết thúc $v_t \in V$. Mỗi cạnh $e = (v_x, v_y)$ có trọng số dương w_e , thỏa

$$\forall v_x \in V \setminus \{v_t\}, \quad \sum_{(v_x, v_y) \in E} w_{(v_x, v_y)} = 1,$$

và từ mọi đỉnh v_x đều tồn tại một đường đi tới v_t . Một quân cờ bắt đầu từ đỉnh xuất phát; mỗi giây, nếu hiện đang ở v_x , nó chọn cạnh ra (v_x, v_y) với xác suất $w_{(v_x, v_y)}$ rồi đi tới v_y ; khi tới đỉnh kết thúc thì dừng. Cần tính kỳ vọng thời gian cần dùng¹².

Thật ra bài toán này cũng có thể viết dưới dạng ma trận. Định nghĩa ma trận P bởi

$$P_{x,y} = \begin{cases} w_{(v_x, v_y)}, & (v_x, v_y) \in E \text{ và } x \neq t, \\ 0, & (v_x, v_y) \notin E \text{ hoặc } x = t. \end{cases}$$

Đáp án cần tìm là

$$\sum_{k \geq 0} k \times (P^k)_{s,t},$$

¹Nếu trọng số bằng 0, có thể coi cạnh ấy không tồn tại.

²Để tiện mô tả, định nghĩa trong bài viết này hơi khác định nghĩa của bài toán bước đi ngẫu nhiên thực tế.

trong đó $(P^k)_{s,t}$ biểu thị xác suất lần đầu tới đỉnh kết thúc sau khi đi k bước. Khi đồ thị hữu hạn và mọi đỉnh đều tới được đỉnh kết thúc, từ định nghĩa của P có thể chứng minh mọi trị riêng của nó đều nhỏ hơn 1, nên đáp án chắc chắn hội tụ.

Để tiện mô tả, nếu không nói khác, trong bài này n luôn chỉ $|V|$, còn m chỉ $|E|$.

Ngoài ra, trong bài này, đồ thị thưa là đồ thị có số cạnh cùng bậc với số đỉnh.

3. Đồ thị lưới

Ví dụ 1. Circles of Waiting. Nguồn: Tinkoff Internship Warmup Round 2018 and Codeforces Round #475 (Div. 1), Problem E.

Một quân cờ ban đầu được đặt tại điểm $(0, 0)$ trên mặt phẳng tọa độ vuông góc. Mỗi giây quân cờ di chuyển ngẫu nhiên. Giả sử nó hiện ở (x, y) ; giây tiếp theo, nó có xác suất p_1 đi tới $(x - 1, y)$, xác suất p_2 đi tới $(x, y - 1)$, xác suất p_3 đi tới $(x + 1, y)$, và xác suất p_4 đi tới $(x, y + 1)$. Bảo đảm $p_1 + p_2 + p_3 + p_4 = 1$. Cần tính kỳ vọng thời gian cho tới khi nó đi tới một vị trí có khoảng cách Euclid tới gốc tọa độ lớn hơn R .

$$0 \leq R \leq 50, \quad p_1, p_2, p_3, p_4 > 0,$$

đáp án lấy modulo $10^9 + 7$.

3.1. Cách làm đơn giản

Gọi $f(i, j)$ là kỳ vọng thời gian để quân cờ, khi đang ở (i, j) , đi tới một vị trí có khoảng cách Euclid tới gốc tọa độ lớn hơn R . Phương trình chuyển là

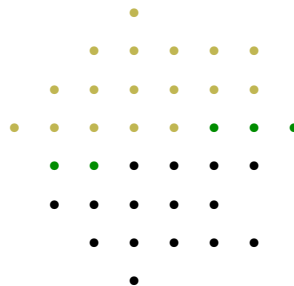
$$f(i, j) = \begin{cases} p_1 f(i - 1, j) + p_2 f(i, j - 1) + p_3 f(i + 1, j) + p_4 f(i, j + 1) + 1, & i^2 + j^2 \leq R, \\ 0, & i^2 + j^2 > R. \end{cases}$$

Do phép chuyển không có thứ tự tô-pô, cần dùng khử Gauss để giải. Độ phức tạp thời gian là $O(R^6)$, không thể qua bài này.

3.2. Khử trực tiếp

Chú ý rằng đa số hệ số trong các phương trình cần khử đều bằng 0. Khi khử chỉ tính trên những vị trí có giá trị khác 0 thì có thể giảm độ phức tạp [1].

Xét quá trình khử. Ta khử các phương trình theo thứ tự từ trên xuống dưới trong hệ tọa độ, và trong cùng một lớp thì từ trái sang phải. Những phương trình đã khử được tô vàng, các điểm kề với điểm vàng được tô xanh, các điểm còn lại được tô đen, như sơ đồ sau:



Tiếp theo ta cần khử phương trình tương ứng với ô xanh kế tiếp. Trong phương trình này, chỉ các hệ số của biến tương ứng với ô xanh và ô đen đầu tiên bên dưới nó là có thể khác 0; đồng thời, chỉ trong các

phương trình tương ứng với ô xanh và ô đen đầu tiên bên dưới nó, hệ số của biến tương ứng với ô hiện tại mới có thể khác 0.

Chú ý rằng chỉ có $O(R)$ ô xanh, nên thời gian khử một phương trình là $O(R^2)$. Tổng cộng chỉ có $O(R^2)$ phương trình, vì vậy độ phức tạp thời gian giảm còn $O(R^4)$, đủ qua bài này.

3.3. Phương pháp ẩn chính

Số phương trình và số biến đều là $O(R^2)$. Nếu có thể thu nhỏ quy mô xuống $O(R)$, thì khử Gauss đơn giản là đủ.

Đặt biến tương ứng với ô đầu tiên từ trái sang phải trên mỗi hàng làm ẩn chính; tổng cộng có $2R + 1$ ẩn chính. Ta tìm cách biểu diễn các biến tương ứng với những ô còn lại thành hàm tuyến tính theo các ẩn chính này. Xét từng cột từ trái sang phải. Với mỗi ô (i, j) trong cột hiện tại, chú ý rằng $f(i, j), f(i - 1, j), f(i, j - 1), f(i, j + 1)$ đều đã là hàm tuyến tính theo các ẩn chính. Chuyển về phương trình truy hồi, ta được

$$f(i + 1, j) = \frac{f(i, j) - p_1 f(i - 1, j) - p_2 f(i, j - 1) - p_4 f(i, j + 1) - 1}{p_3}.$$

Như vậy ta có biểu diễn tuyến tính của $f(i + 1, j)$ theo các ẩn chính. Nếu $(i + 1, j)$ đã có khoảng cách Euclid tới gốc tọa độ lớn hơn R , thì ta thu được một phương trình $f(i + 1, j) = 0$. Cuối cùng, ta sẽ có $2R + 1$ phương trình, rồi chỉ cần khử Gauss trên các phương trình ấy.

Trong giai đoạn truy đẩy các hàm tuyến tính theo ẩn chính, có $O(R^2)$ biến; truy đẩy một biến tốn $O(R)$ thời gian. Sau đó, quy mô bài toán đã được rút xuống $O(R)$. Hai phần đều có độ phức tạp $O(R^3)$, nên tổng độ phức tạp thời gian cũng là $O(R^3)$, đủ qua bài này.

3.4. So sánh hai cách làm

Ta so sánh hai cách làm ở nhiều khía cạnh.

Về độ phức tạp thời gian, phương pháp ẩn chính trên đồ thị lưới có độ phức tạp xấu nhất $O(n\sqrt{n})$ (đạt lớn nhất khi chiều dài và chiều rộng của lưới đều cùng bậc $O(\sqrt{n})$); phương pháp khử trực tiếp trên đồ thị lưới có độ phức tạp xấu nhất $O(n^2)$, nên phương pháp ẩn chính tốt hơn.

Về độ chính xác, với một số bài cần tính trên số thực thay vì lấy modulo, khử trực tiếp chính xác hơn phương pháp ẩn chính.

Về phạm vi áp dụng, hai cách làm phù hợp với những tình huống khác nhau.

Khi trong đồ thị lưới có chướng ngại, hoặc xác suất đi qua một số cạnh bằng 0, phương pháp ẩn chính cần thêm một ẩn chính cho mỗi chướng ngại hoặc mỗi cạnh xác suất 0. Khi số lượng chướng ngại hoặc cạnh xác suất 0 nhiều hơn $O(R)$, độ phức tạp của phương pháp ẩn chính sẽ tăng, trong khi độ phức tạp của khử trực tiếp vẫn không đổi.

Nhưng phương pháp ẩn chính còn có thể khử những phương trình chuyển tương tự trên đồ thị lưới, chẳng hạn

$$f(i, j) = p_1 f(i + 1, j) + p_2 f(i, j + 1) + p_3 f(\text{pre}(i, j)) + 1,$$

trong đó $\text{pre}(i, j) = (x, y)$, $x \leq i$, $y \leq j$, là giá trị do đề bài cho. Phân tích độ phức tạp của khử trực tiếp không áp dụng được cho mô hình này.

Ngoài ra, việc tính định thức của ma trận kề của đồ thị lưới cũng không thể dùng phương pháp ẩn chính, mà chỉ có thể dùng khử trực tiếp để tối ưu độ phức tạp thời gian.

Tóm lại, hai cách làm đều có sở trường riêng; cần phân tích bài cụ thể để chọn cách thích hợp.

4. Đồ thị thưa

Ví dụ 2. Expected Value. Nguồn: Ildar Gainullin Contest 1, Problem E, có chỉnh sửa.

Cho một đồ thị đơn vô hướng liên thông phẳng $G = (V, E)$. Một quân cờ ban đầu được đặt tại v_1 ; mỗi giây quân cờ chọn đều một cạnh nối với đỉnh hiện tại rồi đi tới đỉnh ở đầu kia. Cần tính kỳ vọng thời gian để tới v_n .

$$n \leq 2000,$$

đáp án lấy modulo p , trong đó p là một số nguyên tố được sinh ngẫu nhiên trong khoảng $[10^9, 1.01 \times 10^9]$.

Một kết luận là số cạnh của đồ thị phẳng cùng bậc với số đỉnh. Cụ thể, ta có:

Định lý 4.1. Một đồ thị phẳng có n đỉnh ($n \geq 3$) thì số cạnh m không vượt quá $3n - 6$ [2].

Vì cách làm của bài này có thể mở rộng sang mọi đồ thị thưa, nên phần thảo luận sau chỉ dựa trên điều kiện số cạnh và số đỉnh cùng bậc, không cần các tính chất khác của đồ thị phẳng.

4.1. Kiến thức cơ sở

Định nghĩa 4.1. Mọi đa thức $p(\lambda)$ thỏa $p(A) = 0$ được gọi là đa thức triệt tiêu của ma trận A .

Định nghĩa 4.2. Ký hiệu I_n là ma trận đơn vị cấp n . Định nghĩa đa thức đặc trưng của một ma trận $n \times n$ là

$$p(\lambda) = \det(\lambda I_n - A),$$

trong đó $\det$ là định thức của ma trận.

Dễ thấy đa thức đặc trưng của một ma trận cấp n có bậc không vượt quá n .

Định lý 4.2 (Cayley–Hamilton [3]). Đa thức đặc trưng của mọi ma trận đều là đa thức triệt tiêu của nó.

Vì vậy, đa thức triệt tiêu có bậc nhỏ nhất của một ma trận cấp n cũng có bậc không vượt quá n .

4.2. Giải bài toán ban đầu

Chú ý rằng kỳ vọng thời gian đi được là

$$E(t) = \sum_{i \geq 0} \Pr[t > i].$$

Nếu ta tính được xác suất sau khi đi i bước mà vẫn chưa kết thúc, thì cộng trên mọi $i \geq 0$ sẽ được đáp án.

Gọi $f(i, j)$ là xác suất sau khi đi i bước, quân cờ đang dừng ở v_j và chưa từng đi tới v_n . Khi đó

$$f(i, j) = \sum_{(v_k, v_j) \in E} \frac{f(i-1, k)}{\deg_k} \quad (j \neq n),$$

trong đó $\deg_k$ là bậc của v_k .

Chú ý rằng phép chuyển của f không phụ thuộc vào i , nên có thể xem mỗi lần chuyển là nhân thêm một ma trận, tức $f_{i+1} = f_i M$. Vì đa thức triệt tiêu nhỏ nhất của M có bậc không vượt quá n , nên công thức truy hồi ngắn nhất của f cũng có độ dài không vượt quá n . Do đó

$$\Pr[t > i] = \sum_{j=1}^{n-1} f(i, j)$$

cũng có công thức truy hồi ngắn nhất dài không vượt quá n . Ta có thể tính $\Pr[t > 0], \Pr[t > 1], \dots, \Pr[t > 3n]$ trong $O(nm)$ thời gian, rồi dùng thuật toán Berlekamp–Massey [4] để tìm công thức truy hồi ngắn nhất của $\Pr[t > i]$ trong $O(n^2)$ thời gian.

Xét cách tính hàm sinh của một dãy truy hồi tuyến tính bậc k . Không mất tính tổng quát, giả sử với $i \geq i_0$ ta có

$$a_i = \sum_{j=1}^k c_j a_{i-j}.$$

Gọi hàm sinh của a và c lần lượt là $A(x)$ và $C(x)$. Khi đó

$$A(x) = A(x)C(x) + A_0(x),$$

trong đó $A_0(x)$ do các hạng $i < i_0$ quyết định.

Quay lại bài toán ban đầu. Vì ta tìm được công thức truy hồi ngắn nhất của $\Pr[t > i]$, nên có thể tìm $C(x)$ và $A_0(x)$ (định nghĩa như đoạn trước). Chuyển về được

$$A(x) = \frac{A_0(x)}{1 - C(x)}.$$

Giá trị cần tìm là $\sum_{i \geq 0} [x^i]A(x)$; để thấy giá trị này chính là $A(1)$. Thay $x = 1$ vào rồi giải là xong. Vì modulo là số nguyên tố ngẫu nhiên, có thể coi mẫu số sẽ không bằng 0.

Như vậy, ta giải được bài này trong $O(nm + n^2)$ thời gian. Do số đỉnh và số cạnh cùng bậc, trong bài này độ phức tạp có thể coi là $O(n^2)$.

Ngoài ra, bài này còn có một cách làm khác; xem tài liệu tham khảo [5].

5. Đồ thị tổng quát

Ví dụ 3. Frank. Nguồn: 2018 ACM-ICPC Asia Nanjing Regional Contest, Problem F, có chỉnh sửa.

Cho một đồ thị đơn có hướng liên thông mạnh $G = (V, E)$. Với mọi $1 \leq s \leq n, 1 \leq t \leq n, s \neq t$, hãy trả lời bài toán sau.

Một quân cờ ban đầu được đặt tại v_s ; mỗi giây quân cờ chọn đều một cạnh ra của đỉnh hiện tại rồi đi tới đỉnh mà cạnh ấy trở tới. Cần tính kỳ vọng thời gian để tới v_t .

$$3 \leq n \leq 400.$$

5.1. Phân tích và chuyển hóa

Gọi $p_{i,j}$ là xác suất quân cờ, khi ở v_i , chọn cạnh ra (v_i, v_j) để đi tới v_j ; đặc biệt, nếu cạnh ra không tồn tại thì xác suất bằng 0. Gọi $f_{i,j}$ là kỳ vọng thời gian để đi ngẫu nhiên từ v_i tới v_j ; đặc biệt, $f_{i,i} = 0$. Khi $i \neq j$, phương trình chuyển là

$$f_{i,j} = 1 + \sum_{1 \leq k \leq n} p_{i,k} f_{k,j}.$$

Khi $i = j$, gọi g_i là kỳ vọng thời gian để bắt đầu từ v_i , đi ngẫu nhiên và lần đầu quay lại v_i . Khi đó

$$f_{i,i} = 1 - g_i + \sum_{1 \leq k \leq n} p_{i,k} f_{k,i}.$$

Để dễ quan sát, ta viết phương trình chuyển dưới dạng ma trận. Gọi P là ma trận chuyển của đồ thị, F là ma trận đáp án, I là ma trận đơn vị cấp n , J là ma trận toàn 1 cấp n , và G là ma trận cấp n thỏa $G_{i,i} = g_i$, các vị trí khác bằng 0. Khi đó

$$F = J - G + PF.$$

Nếu tính được G , ta chỉ cần giải phương trình [6]

$$(I - P)F = J - G.$$

5.2. Cách tính G

Định nghĩa 5.1. Định nghĩa phân bố dừng [7] của một ma trận chuyển cấp n là một vectơ n chiều π , thỏa

$$\sum_{i=1}^n \pi_i = 1, \quad \pi P = \pi.$$

Giá trị ở mỗi chiều của π đều nằm trong đoạn $[0, 1]$.

Ta rất dễ tìm được ý nghĩa thực tế của phân bố dừng. Nếu tại một thời điểm nào đó quân cờ dừng ở v_i với xác suất π_i , thì ở mọi thời điểm về sau quân cờ vẫn thỏa phân bố xác suất này. Ta có thể dùng khử Gauss để giải hệ và tìm π trong $O(n^3)$ thời gian. Vậy π có quan hệ gì với G ?

Định lý 5.1. Với mọi $1 \leq i \leq n$, ta có

$$\pi_i g_i = 1.$$

Chứng minh. Từ $F = J - G + PF$, chuyển vế được

$$G = PF + J - F.$$

Nhân đồng thời hai vế với π ở bên trái:

$$\pi G = \pi PF + \pi J - \pi F.$$

Theo định nghĩa của π , ta có $\pi P = \pi$, do đó

$$\pi G = \pi J.$$

Vì vậy

$$\pi_i g_i = \sum_{j=1}^n \pi_j = 1.$$

Mệnh đề được chứng minh. $\square$

Vậy bằng cách đưa vào phân bố dừng, ta có thể tính G trong $O(n^3)$ thời gian.

5.3. Giải bài toán ban đầu

Trong quá trình giải phương trình, ta gặp một vấn đề: $I - P$ không đầy hạng, nên không thể giải bằng cách nhân ma trận nghịch đảo.

Định nghĩa 5.2. Định nghĩa một cây khung có hướng gốc $v_r \in V$ của đồ thị có hướng $G = (V, E)$ là một đồ thị con $T = (V, A)$ của G thỏa:

1. Với mọi $v_i \neq v_r$, bậc ra của v_i bằng 1.
2. Bậc ra của v_r bằng 0.
3. Trong T không tồn tại chu trình.

Bổ đề 5.1 (Định lý cây ma trận trên đồ thị có hướng [8]). Với một đồ thị có hướng G , gọi D là ma trận bậc ra của nó, tức

$$D_{i,i} = d_i, \quad D_{i,j} = 0 \quad (i \neq j),$$

trong đó d_i là bậc ra của v_i ; gọi A là ma trận kề của nó. Khi đó số cây khung có hướng gốc v_r bằng định thức của $D - A$ sau khi bỏ hàng r và cột r .

Định lý 5.2. Với ma trận chuyển P của một đồ thị liên thông mạnh $G = (V, E)$, hạng của $I - P$ bằng $n - 1$.

Chứng minh. Vì nhân một hàng của ma trận với một hằng số khác 0 không làm đổi hạng, ta nhân hàng thứ i của $I - P$ với bậc ra của v_i , thu được một ma trận mới L . Chỉ cần chứng minh hạng của L bằng $n - 1$.

Do tổng trên mỗi hàng của L đều bằng 0, cộng tất cả các vectơ cột của L sẽ thu được vectơ không; tức các vectơ này phụ thuộc tuyến tính. Vì thế hạng của L không phải n .

Để thấy L bằng ma trận bậc ra của G trừ đi ma trận kề của nó. Theo bổ đề 5.1, định thức của L sau khi bỏ hàng i và cột i biểu thị số cây khung có hướng gốc v_i .

Vì G liên thông mạnh, với mọi đỉnh được chọn làm gốc, số cây khung có hướng đều khác 0. Tức là sau khi bỏ hàng i và cột i , L vẫn đầy hạng.

Vì thêm một cột không làm hạng giảm, mọi vectơ hàng của L sau khi bỏ hàng i là độc lập tuyến tính. Do đó hạng của L là $n - 1$. $\square$

Quay lại bài toán ban đầu, xét cách giải phương trình trong bài. Để tiện, ta viết phương trình dưới dạng $AX = B$, trong đó A, B đã biết, cần tìm X . Do A không đầy hạng, nghiệm có vô số; trước hết ta tìm một nghiệm riêng.

Khử Gauss đồng thời trên A và B . Khử $n - 1$ hàng đầu của A về dạng chỉ còn đường chéo chính và cột thứ n có giá trị, còn hàng cuối cùng toàn 0, tức dạng sau:

$$\begin{bmatrix} 1 & 0 & 0 & \cdots & 0 & a_1 \\ 0 & 1 & 0 & \cdots & 0 & a_2 \\ 0 & 0 & 1 & \cdots & 0 & a_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 & a_{n-1} \\ 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix} X = \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & \cdots & b_{1,n-1} & b_{1,n} \\ b_{2,1} & b_{2,2} & b_{2,3} & \cdots & b_{2,n-1} & b_{2,n} \\ b_{3,1} & b_{3,2} & b_{3,3} & \cdots & b_{3,n-1} & b_{3,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ b_{n-1,1} & b_{n-1,2} & b_{n-1,3} & \cdots & b_{n-1,n-1} & b_{n-1,n} \\ 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix}.$$

Đặt $X_{n,i} = 0$, ta giải được một nghiệm riêng, ký hiệu là Y . Tiếp theo ta cần điều chỉnh nghiệm riêng thành nghiệm thật sự.

Chú ý rằng $X_{i,i} = 0$. Xét theo ý nghĩa tổ hợp, ta có

$$Y_{i,j} = 1 + Y_{j,j} + \sum_{k=1}^n P_{i,k} X_{k,j},$$

và dễ suy ra

$$X_{i,j} = Y_{i,j} - Y_{j,j}.$$

Cuối cùng, ta giải được bài toán này trong $O(n^3)$ thời gian.

6. So sánh và thảo luận

Về độ phức tạp thời gian, trên đồ thị lưới có phương pháp ẩn chính $O(n\sqrt{n})$; thuật toán trên đồ thị thưa là $O(n^2)$; còn trên đồ thị tổng quát, độ phức tạp cao hơn, là $O(n^3)$.

Về góc độ bài toán được giải, thuật toán trên đồ thị lưới có thể giải cho trường hợp mọi đỉnh trong đồ thị đều làm đỉnh xuất phát; thuật toán trên đồ thị thưa chỉ có thể giải nhanh cho một đỉnh xuất phát và một đỉnh kết thúc; thuật toán trên đồ thị tổng quát thì giải cho mọi cặp đỉnh làm xuất phát và kết thúc. Nếu các thuật toán trên đồ thị lưới, đồ thị thưa phải giải cho nhiều đỉnh kết thúc, hoặc thuật toán trên đồ thị thưa phải giải cho nhiều đỉnh xuất phát, thì cần thêm thời gian.

Về phạm vi áp dụng, vì đồ thị lưới là tập con của đồ thị thưa, còn đồ thị thưa là tập con của đồ thị tổng quát, nên thuật toán trên đồ thị lưới có phạm vi áp dụng nhỏ nhất, còn thuật toán trên đồ thị tổng quát có phạm vi áp dụng lớn nhất.

Về độ chính xác, do thuật toán Berlekamp–Massey có độ chính xác rất kém, nên trong các bài cần tính trên số thực, thông thường không thể dùng thuật toán trên đồ thị thưa; còn thuật toán trên đồ thị lưới và đồ thị tổng quát đều có thể chọn dùng.

Tóm lại, vài thuật toán này đều có ưu nhược điểm riêng. Việc dùng thuật toán nào thường tùy bài; khi giải lớp bài này, thí sinh nên phân tích tính chất của đề rồi chọn phương pháp phù hợp nhất.

7. Tổng kết

Bài viết đã nghiên cứu bài toán bước đi ngẫu nhiên trên mô hình đồ thị, lần lượt giới thiệu vài thuật toán khác nhau trên đồ thị lưới, đồ thị thưa và đồ thị tổng quát. Chú ý rằng phần lớn các thuật toán này được thảo luận trên nền tảng ma trận; điều đó cho thấy ma trận là một công cụ mạnh để giải bài toán bước đi ngẫu nhiên. Với bài toán bước đi ngẫu nhiên, sau khi chuyển thành dạng ma trận, ta thường sẽ dễ bắt tay giải hơn.

Đồng thời, các thuật toán được nhắc tới trong bài không chỉ dùng được cho bước đi ngẫu nhiên, mà còn có thể gợi mở cho những bài toán khác: khử trực tiếp trong đồ thị lưới gợi ý rằng với một số ma trận thưa có tính chất đặc biệt, ta có thể dùng một số cắt tía để giảm độ phức tạp; phương pháp ẩn chính có thể dùng cho một lớp bài toán giải hệ phương trình; thuật toán trong phần đồ thị thưa gợi ý cách dùng truy hồi tuyến tính để giải một số bài toán liên quan tới ma trận; phân bố dừng trong phần đồ thị tổng quát là công cụ mạnh để xử lý các bài toán liên quan tới xích Markov, và nó còn nhiều tính chất đáng khai thác.

Thật ra bài toán bước đi ngẫu nhiên còn rất nhiều điểm đáng nghiên cứu. Mong rằng bài viết có thể đóng vai trò gợi mở và thu hút thêm nhiều độc giả nghiên cứu lớp bài toán này.

8. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy Lin Yang và thầy Zhang Junliang của Trường Trung học số 7 Thành Đô đã quan tâm và hướng dẫn tôi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn.

Xin cảm ơn đàn anh Yang Jingqin đã giúp đỡ tôi.

Xin cảm ơn các tiền bối Chen Songyang, Tang Jingzhe và Du Yuhao đã truyền cảm hứng và giúp đỡ tôi trong quá trình viết bài.

Xin cảm ơn tiền bối Chen Songyang, tiền bối Tang Jingzhe và bạn Fang Lixing đã kiểm tra bản thảo.

Tài liệu tham khảo

1. Romanov, V. Editorial Tinkoff Internship Warmup Round 2018 and Codeforces Round #475 (Div. 1 + Div. 2). Codeforces, <https://codeforces.com/blog/entry/58991>.
2. Wikipedia contributors. Planar graph. Wikipedia, https://en.wikipedia.org/wiki/Planar_graph.
3. Hamilton, W. R. Lectures on Quaternions. Dublin, 1853.
4. Massey, J. L. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory, IT-15(1): 122–127, 1969. doi:10.1109/TIT.1969.1054260.
5. Zhong Ziqian. Tính chất và ứng dụng của hai loại dãy truy hồi. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2019, 2019.
6. Tang, J. 2018-2019 ACM-ICPC, Asia Nanjing Regional Contest (Online Mirror on Gym). Codeforces, <https://codeforces.com/blog/entry/63057>.
7. Wikipedia contributors. Stationary Distribution. Wikipedia, https://en.wikipedia.org/wiki/Stationary_Distribution.
8. Chaiken, S.; Kleitman, D. Matrix Tree Theorems. Journal of Combinatorial Theory, Series A, 24(3): 377–381, 1978. doi:10.1016/0097-3165(78)90067-5, ISSN 0097-3165.

Báo cáo ra đề *Bài toán nhỏ dễ*

Yang Junzhao, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này giới thiệu bài thứ ba trong vòng tự kiểm tra thứ năm của đội ứng viên, do tác giả ra đề. Đây là một bài cấu trúc dữ liệu lấy bối cảnh duy trì sự thay đổi mực nước trong các bể nước. Bài toán cần trước hết chuyển mô hình vật lý của sự biến đổi mực nước thành một mô hình toán học để duy trì, rồi dùng kỹ thuật cấu trúc dữ liệu để tối ưu và giải quyết.

1. Ý chính đề bài

1.1. Mô tả đề bài

Có n bể nước có cùng diện tích đáy, xếp thành một hàng. Hai bể kề nhau dùng chung một vách ngăn; chiều cao vách ngăn giữa bể thứ i và bể thứ $i + 1$ là b_i . Vách ở ngoài cùng bên trái và bên phải có thể xem là cao vô hạn. Ban đầu tất cả các bể đứng yên, mực nước ban đầu của bể thứ i là a_i . Bạn cần duy trì hai loại thao tác sau:

- Cho x và h , khoan một lỗ nhỏ ở độ cao h trên vách ngăn giữa bể thứ x và bể thứ $x + 1$. Sau đó mực nước của một số bể sẽ thay đổi; bạn phải chờ cho đến khi các bể ấy đứng yên trở lại.
- Cho x , hỏi mực nước hiện tại của bể thứ x .

1.2. Dạng nhập

Dữ liệu được đọc từ chuẩn nhập.

Dòng đầu gồm hai số nguyên n, q , lần lượt là số bể và số thao tác.

Dòng thứ hai gồm n số thực a_i , cách nhau bởi dấu cách, biểu thị mực nước ban đầu.

Dòng thứ ba gồm $n - 1$ số thực b_i , cách nhau bởi dấu cách, biểu thị chiều cao vách ngăn ban đầu.

Tiếp theo là q dòng, mỗi dòng mô tả một thao tác.

1 x h biểu thị thao tác loại một, bảo đảm $1 \leq x < n$. Chú ý h là số thực.

2 x biểu thị thao tác loại hai, bảo đảm $1 \leq x \leq n$.

Mọi số thực xuất hiện trong đầu vào có không quá bảy chữ số sau dấu thập phân.

1.3. Dạng xuất

In ra chuẩn xuất.

Với mỗi thao tác loại hai, in một dòng là đáp án.

Dòng cuối cùng in n số thực cách nhau bởi dấu cách, biểu thị mực nước của từng bể sau khi mọi thao tác kết thúc.

Sai số tuyệt đối trong phạm vi 10^{-7} được chấp nhận.

1.4. Gợi ý và thuyết minh bổ sung

Trong bài này, ta dùng một mô hình lý tưởng. Tức là ta giả sử thể tích nước không đổi, đồng thời bỏ qua sức căng bề mặt, ma sát, v.v. Có thể hiểu rằng sự thay đổi mực nước sau khi khoan lỗ phù hợp với trực giác đời sống. Cụ thể, với hai mực nước kề nhau a_i, a_{i+1} , nếu vách ngăn giữa chúng có một lỗ ở độ cao h (hoặc chiều cao vách ngăn ban đầu là h), thì nếu $a_i > h$ và $a_i > a_{i+1}$, sẽ có nước chảy từ i sang $i + 1$. Đối xứng, nếu $a_{i+1} > h$ và $a_{i+1} > a_i$, sẽ có nước chảy từ $i + 1$ sang i . Nước chảy từ x sang y nghĩa là a_x liên tục giảm, a_y liên tục tăng, và tổng của chúng được giữ nguyên.

1.5. Giới hạn và quy ước

Với mọi dữ liệu, $1 \leq n, q \leq 100000$, $0 \leq a_i, h \leq 100$, và với $1 \leq i < n$ có $b_i \geq \max(a_i, a_{i+1})$. Mọi số thực xuất hiện trong đầu vào có không quá bảy chữ số sau dấu thập phân.

- Subtask 1 (1 điểm): $1 \leq n, q \leq 10$;
- Subtask 2 (10 điểm): $1 \leq n, q \leq 300$;
- Subtask 3 (10 điểm): $1 \leq n, q \leq 2000$, với $i > 1$ có $a_i = 0$;
- Subtask 4 (10 điểm): $1 \leq n, q \leq 2000$;
- Subtask 5 (30 điểm): với $i > 1$ có $a_i = 0$;
- Subtask 6 (39 điểm): không có ràng buộc đặc biệt.

Giới hạn thời gian: 4s.

Giới hạn bộ nhớ: 1024MB.

2. Phân tích lời giải

2.1. Thuật toán một

Đây là một bài cấu trúc dữ liệu. Trước hết xét mô phỏng trực tiếp theo đề. Bài toán yêu cầu duy trì quá trình nước chảy từ nơi cao xuống nơi thấp. Nhưng khác với một bài cấu trúc dữ liệu thông thường, vấn đề không đơn giản như vậy, vì mô tả đề không phải là ngôn ngữ đã được chương trình hóa; do đó trước hết ta cần xây dựng mô hình.

Vì vách ngăn chỉ liên quan tới giá trị nhỏ hơn giữa chiều cao ban đầu của nó và độ cao lỗ khoan, nên trong phần sau ta trực tiếp xem chiều cao vách ngăn (tức b_i) là giá trị nhỏ hơn ấy.

Gợi ý trong đề đã đưa ra một mô hình khả dĩ. Mỗi lần chỉ xét hai bể kề nhau; nếu chúng ở trạng thái mất cân bằng thì điều chỉnh chúng về cân bằng, cho đến khi không còn hai bể kề nhau nào mất cân bằng.

Mô hình này không dễ duy trì. Xét trường hợp có ba bể, nước từ bể thứ nhất chảy sang hai bể sau; cuối cùng mực nước của cả ba bể đáng ra phải bằng nhau, nhưng vì mỗi lần chỉ điều chỉnh mực nước của hai bể kề nhau, không thể làm cho chúng bằng nhau trong hữu hạn bước, quá trình này có thể tiếp diễn vô hạn. Tuy vậy, vì ta không cần đáp án chính xác mà chỉ cần bảo đảm chín chữ số chính xác (bao gồm hàng đơn vị, hàng chục và bảy chữ số sau dấu thập phân), nên khi hai độ cao rất gần nhau ta có thể dừng lại. Từ đó có thuật toán một:

Thuật toán một. Đặt $\varepsilon = 10^{-8}$. Với $1 \leq i < n$, gọi vị trí này là mất cân bằng khi và chỉ khi

$$a_i \geq \max(b_i, a_{i+1}) + \varepsilon \quad \text{hoặc} \quad a_{i+1} \geq \max(b_i, a_i) + \varepsilon.$$

Với mỗi lần sửa đổi, ta liên tục tìm trạng thái mất cân bằng và điều chỉnh, cho đến khi không còn trạng thái mất cân bằng. Nếu $a_i \geq \max(b_i, a_{i+1}) + \varepsilon$, lượng nước thay đổi là

$$\min\left(a_i - b_i, \frac{a_i - a_{i+1}}{2}\right),$$

ngược lại lượng nước thay đổi là

$$\min\left(a_{i+1} - b_i, \frac{a_{i+1} - a_i}{2}\right).$$

Độ phức tạp thuật toán: $O(q \times \text{một hằng số lớn phụ thuộc vào độ chính xác})$.

Điểm kỳ vọng: có thể qua subtask một, kỳ vọng 1 điểm.

2.2. Thuật toán hai

Ta cần một thuật toán thời gian đa thức, có độ phức tạp không phụ thuộc vào độ chính xác. Có thể quan sát một số tính chất:

- Sau mỗi lần sửa chiều cao vách ngăn, xét mực nước của hai bể kề với vách ấy, nước chỉ chảy từ cao xuống thấp. Vì vậy hoặc không có gì xảy ra, hoặc nước chảy từ trái sang phải, hoặc nước chảy từ phải sang trái.
- Nước chảy từ trái sang phải và từ phải sang trái là đối xứng. Khi phân tích tính chất bên dưới, không mất tính tổng quát giả sử nước chảy từ trái sang phải; còn khi bàn cách duy trì thông tin thì xét ảnh hưởng của cả hai hướng.

Vấn đề chính của thuật toán một là không giải quyết được cân bằng của nhiều bể. Ta xét cải tiến thuật toán một. Gọi một đoạn $[l, r]$ là liên thông khi và chỉ khi đồng thời thỏa ba điều kiện sau:

- $1 \leq l \leq r \leq n$;
- với $l \leq i \leq r$, có $a_i = a_l$;
- với $l \leq i < r$, có $b_i < \max(a_i, a_{i+1})$.

Chú ý rằng khi mực nước thay đổi, mọi bể trong cùng một bình thông nhau thay đổi cùng lúc, tức cùng tăng hoặc cùng giảm một thể tích nước như nhau. Dựa trên đoạn liên thông, ta thiết kế thuật toán hai:

Thuật toán hai. Thay vì xét hai bể kề nhau như cách trước, ta xét hai đoạn liên thông kề nhau, rồi để nước chảy từ cao xuống thấp cho đến khi điều kiện liên thông bị phá vỡ hoặc giữa chúng không còn chênh lệch độ cao. Trong cài đặt cụ thể, cần hỗ trợ:

- gộp hai đoạn liên thông;
- tách một đoạn liên thông thành hai đoạn liên thông;
- cộng/trừ đồng loạt trên một đoạn liên thông;
- hỏi chiều cao vách ngăn lớn nhất trong một đoạn liên thông.

Độ phức tạp thuật toán: dễ thấy sau mỗi lần điều chỉnh, đầu trái hoặc đầu phải của đoạn liên thông đang ở phía cao hơn sẽ dịch sang phải ít nhất một vị trí, nên sau mỗi thao tác chỉ cần điều chỉnh $O(n)$ lần. Vì vậy độ phức tạp của mỗi thao tác sửa là $O(n \times \text{độ phức tạp điều chỉnh})$. Tùy cách cài đặt, mỗi lần điều chỉnh có thể đạt $O(n)$, $O(\log n)$ hoặc $O(1)$. Trường hợp $O(1)$ có thể dùng hàng đợi đơn điệu, và mỗi thao tác sửa cần thêm $O(n)$. Tổng độ phức tạp có thể đạt $O(qn^2)$, $O(qn \log n)$ hoặc $O(qn)$.

Điểm kỳ vọng: tùy cách cài đặt, có thể qua hai subtask đầu hoặc bốn subtask đầu, kỳ vọng 11 đến 31 điểm.

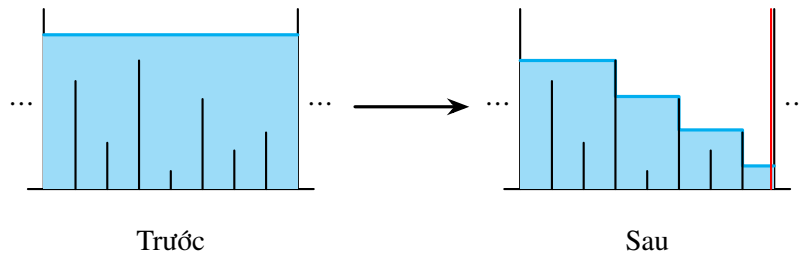
2.3. Thuật toán ba

Tổng số lần gộp hoặc tách đoạn liên thông trong thuật toán hai có thể đạt bậc hai, nên không thể qua hai subtask cuối. Chú ý rằng tuy trong thuật toán hai vẫn còn thao tác dư thừa (xét một dãy bể dạng bậc thang, nước ở bên trái có thể chảy xuôi theo bậc thang đến phía dưới bên phải, còn các đoạn liên thông ở giữa không thay đổi), ta vẫn có thể dựng phương án sao cho mỗi thao tác hoặc gộp $O(n)$ đoạn liên thông, hoặc tách một đoạn liên thông thành $O(n)$ đoạn liên thông. Nói cách khác, tồn tại một phương án khiến tổng các trị tuyệt đối của hiệu số lượng đoạn liên thông giữa hai thao tác kế nhau đạt bậc hai.

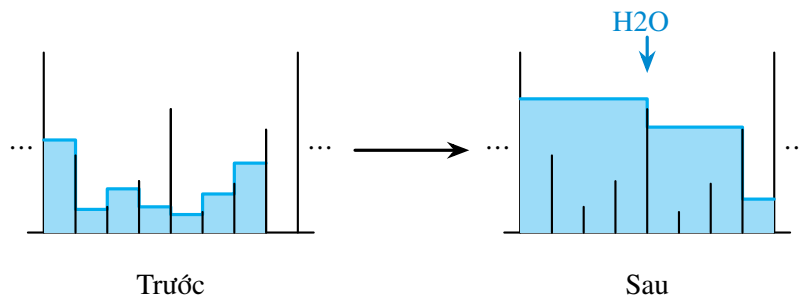
Ta suy nghĩ bài toán từ một góc nhìn khác. Có thể thấy nếu trong một thao tác mà duy trì biến đổi của các bể theo thời gian thì sẽ rất rắc rối; ta xét cách tính trực tiếp kết quả sau khi thay đổi.

Sau khi phân tích bình tĩnh, có ba tình huống cần xử lý:

1. Sau khi nước chảy đi, để lại một đoạn “bậc thang”.
2. Khi nước chảy qua, nó lấp đầy một số “hố”.
3. Nước chảy xuống bị vách ngăn chặn lại, tạo thành một “hồ”.



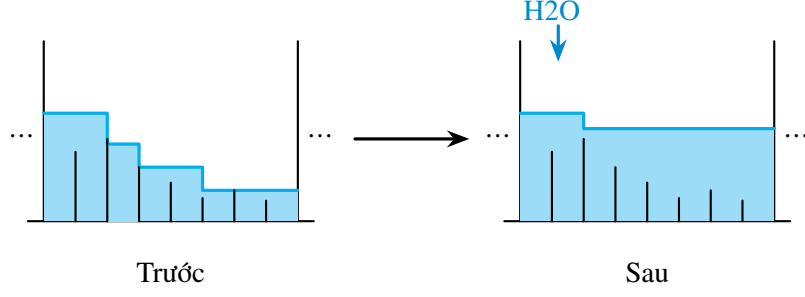
Hình 1: Sau khi nước chảy đi, để lại một đoạn bậc thang.



Hình 2: Khi nước chảy qua, nó lấp đầy một số hố.

Ta có thể chia toàn bộ quá trình biến đổi thành ba giai đoạn theo ba hình trên. Giả sử ta sửa chiều cao vách ngăn giữa bể thứ x và bể thứ $x + 1$ thành b'_x . Đặt $h = \max(b'_x, a_{x+1})$. Khi đó để nước chảy từ trái sang phải cần có $a_x > h$.

- Giai đoạn một: giả sử mực nước của bể thứ $x + 1$ là âm vô cùng, ta cần tính có bao nhiêu nước sẽ chảy đi. Chú ý chỉ mực nước trong đoạn liên thông mới thay đổi. Gọi đầu trái của đoạn liên thông là l . Gọi thể tích nước chảy đi là V . (Xem Hình 1.)



Hình 3: Nước bị vách ngăn chặn lại và tạo thành hồ.

- Giai đoạn hai: giả sử V đơn vị thể tích nước chảy từ bể thứ l sang phải, tìm r lớn nhất sao cho thể tích cần để lấp các hố từ l đến r không vượt quá V . (Xem Hình 2.)
- Giai đoạn ba: gọi lượng nước còn lại sau khi lấp hố là V' . Ta cần đổ lượng nước còn lại vào một số bể phía bên phải. Điều này tương đương với tìm h' sao cho trong đoạn $[l, r]$, thể tích cần để nâng mọi bể có mực nước nhỏ hơn h' lên đúng h' bằng V' . (Xem Hình 3.)

Chú ý rằng ba giai đoạn này không chỉ cần truy vấn mà còn cần sửa mực nước. Tuy các thao tác trong các giai đoạn ấy nhìn có vẻ khó tối ưu, suy nghĩ kỹ sẽ thấy chúng có điểm tương tự. Xét thao tác sau:

Với một đoạn $[l, r]$, đặt

$$FL(l, r, h) = \sum_{i=l}^r \max \left(\max_{j=i}^{r-1} b_j, h \right).$$

Đối xứng, đặt

$$FR(l, r, h) = \sum_{i=l}^r \max \left(\max_{j=l}^{i-1} b_j, h \right).$$

Ký hiệu $ML(l, r, h)$ là thao tác sửa sau: với $l \leq x \leq r$, đặt

$$a_x = \max \left(\max_{i=x}^{r-1} b_i, h \right).$$

Đối xứng, $MR(l, r, h)$ là thao tác sửa sau: với $l \leq x \leq r$, đặt

$$a_x = \max \left(\max_{i=l}^{x-1} b_i, h \right).$$

Nếu biểu diễn hậu tố max (hoặc tiền tố max) của chiều cao vách ngăn trên hình, ta gọi đường giống như đường đồng mức này là đường viền; tổng mực nước trên đường viền, tức tổng thể tích nước dưới đường viền, được gọi là mực nước đường viền của đoạn này. Ý nghĩa thực tế của hàm này (lấy FL làm ví dụ) là: giả sử ban đầu trong đoạn không có nước, vách ngăn ở hai bên đoạn cao vô hạn, ta chậm rãi đổ nước vào bể ngoài cùng bên trái cho đến khi mực nước ở bể ngoài cùng bên phải đạt h ; khi đó hàm cho biết thể tích nước cần dùng.

Lúc này ba giai đoạn có thể được mô tả hình thức bằng mực nước đường viền. Dùng $SUM(l, r)$ biểu thị tổng mực nước từ bể thứ l đến bể thứ r trong đoạn $[l, r]$.

- Giai đoạn một: tìm đầu trái l của đoạn liên thông. Tính $V = SUM(l, x) - FL(l, x, h)$. Thực hiện thao tác $ML(l, x, h)$.
- Giai đoạn hai: tìm r lớn nhất sao cho $FL(l, r, a_r) - SUM(l, r) \leq V$, đặt $V' = V - FL(l, r, a_r) + SUM(l, r)$. Thực hiện thao tác $ML(l, r, a_r)$.
- Giai đoạn ba: tìm h lớn nhất sao cho $FL(l, r, h) - SUM(l, r) \leq V'$. Thực hiện thao tác $ML(l, r, h)$.

Ở đây giai đoạn ba dùng tính đơn điệu của hàm FL theo h , tức nếu $h \leq h'$ thì $FL(l, r, h) \leq FL(l, r, h')$.

Ta phát hiện có thể hợp nhất đẹp ba quá trình này thành một quá trình:

- Tìm đầu trái l của đoạn liên thông. Tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(l, r, h) \leq \text{SUM}(l, r)$. Sau đó thực hiện $ML(l, r, h)$.

Chú ý ở đây phải bảo đảm r là lớn nhất trước, rồi mới bảo đảm h là lớn nhất; nếu không sẽ xuất hiện trạng thái mất cân bằng. Hàm FL ở đây còn có một tính đơn điệu nữa, tức đơn điệu theo r : nếu $r \leq r'$ thì

$$FL(l, r, a_r) \leq FL(l, r', a_{r'}).$$

Ta thấy nếu có một cấu trúc dữ liệu hỗ trợ truy vấn nhanh FL, FR và hỗ trợ nhanh các thao tác ML, MR , chỉ cần hai lần chặt nhị phân là giải được bài toán.

Thuật toán ba. Dùng phương pháp trên. Tính FL, FR bằng vét cạn $O(n)$, và sửa ML, MR bằng vét cạn.

Độ phức tạp thuật toán: vì mỗi thao tác sửa cần chặt nhị phân, độ phức tạp là $O(qn \log n)$. Chú ý phép chặt nhị phân trong giai đoạn ba có thể trực tiếp chặt trên miền giá trị, hoặc dùng tính chất nó là hàm từng đoạn để chặt trên các mực nước trong đoạn. Nếu dùng cách trước, $O(\log n)$ trong phân tích độ phức tạp có thể phải đổi thành $O(\text{số lần chặt miền giá trị})$. Để tránh sai số độ chính xác, số lần chặt này cần đủ lớn (khoảng 60). Các phân tích độ phức tạp của những thuật toán sau cũng có vấn đề tương tự.

Điểm kỳ vọng: có thể qua bốn subtask đầu, kỳ vọng 31 điểm.

2.4. Thuật toán bốn

Phân tính chất đặc biệt thỏa rằng ban đầu chỉ bể thứ nhất có nước. Ta thấy phần subtask này có một tính chất đẹp: mảng a không tăng, tức với mọi $1 \leq i < n$ có $a_i \geq a_{i+1}$. Kết luận này có thể suy ra từ mô hình ban đầu: khi điều chỉnh hai bể kề nhau, quan hệ thứ tự tương đối của chúng không thay đổi, nên nếu ban đầu thỏa tính chất này thì về sau chắc chắn vẫn thỏa.

Ta còn thấy đoạn có nước nhất định chỉ gồm K bể đầu, và với mọi $1 \leq i < K$, chắc chắn có $a_i \geq b_i$. Sau khi phân tích bình tĩnh, ba giai đoạn trước có thể rút gọn thành:

- Giai đoạn một: tìm đầu trái l của đoạn liên thông. Tính $V = \text{SUM}(l, x) - FL(l, x, h)$. Thực hiện thao tác $ML(l, x, h)$.
- Giai đoạn hai: tìm h lớn nhất sao cho $FL(l, r, h) - \text{SUM}(l, r) \leq V'$. Thực hiện thao tác $ML(l, r, h)$. Nếu h lớn hơn b_K , tăng K lên một, rồi sửa b_K , lặp lại quá trình này.

Vì giai đoạn hai ở đây thỏa mảng a giảm, và với mọi $l \leq i < r$ chắc chắn có $a_i \geq b_i$ (thật ra tính chất này cũng tồn tại trong giai đoạn ba ở trên), nên FL ở đây có thể tính trực tiếp bằng chặt nhị phân, tức tính tổng các giá trị lớn hơn h và số lượng giá trị nhỏ hơn h .

Ở đây còn có một cách đơn giản hơn. Vì tổng thể tích nước V cố định, ta có thể không duy trì mực nước thật. Ta chỉ cần biết bể có nước ngoài cùng bên phải là bể nào và mực nước của bể ấy, từ đó suy ra mực nước của mọi bể. Ta xét chỉ duy trì chiều cao vách ngăn, khi truy vấn thì tìm bể có nước ngoài cùng bên phải r và mực nước h của bể ấy. Do tính đơn điệu đã nhắc ở trên, ta chỉ cần tìm:

- tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(1, r, h) \leq V$.

Như vậy ta chỉ cần hỗ trợ thao tác FL . Tuy nhiên bên dưới vẫn giới thiệu một cách gần với lời giải chuẩn, tức cách duy trì FL, FR, ML, MR .

Ta thấy cần duy trì thông tin đoạn và thao tác đoạn, nên không khó nghĩ tới cây đoạn.

Trước hết cây đoạn chắc chắn cần duy trì tổng mực nước trong đoạn. Ta còn cần hỗ trợ truy vấn nhanh hai hàm FL và FR của đoạn với h bất kỳ, và hỗ trợ các phép sửa ML và MR với h bất kỳ; cây đoạn truyền thống không giải quyết trực tiếp được.

Xét một nút cây đoạn $[l, r]$ duy trì thông tin các bể trong $[l, r]$, cùng thông tin $r - l$ vách ngăn ở giữa. Giả sử các nút con đã có thể duy trì mực nước SUM, vách ngăn lớn nhất, và các thao tác FL, FR . Đặt

$$m = \left\lfloor \frac{l + r}{2} \right\rfloor,$$

khi đó hai nút con của $[l, r]$ là $[l, m]$ và $[m + 1, r]$. Ký hiệu

$$\text{HMAX}(l, r) = \max_{i=l}^{r-1} b_i.$$

Ta thấy

$$\begin{aligned} FL(l, r, h) &= FL(m + 1, r, h) \\ &\quad + FL(l, m, \max(h, \max(\text{HMAX}(m + 1, r), b_m))). \end{aligned}$$

Nếu cứ đệ quy trực tiếp thì là $O(r - l)$, nhưng nếu có thể chỉ chọn một phía để đệ quy thì sẽ đạt $O(\log(r - l))$.

Nếu $h \geq \text{HMAX}(m + 1, r)$, thì

$$FL(m + 1, r, h) = (r - m)h,$$

lúc này chỉ cần đệ quy tính phía còn lại. Nếu $h < \text{HMAX}(m + 1, r)$, chú ý rằng cần tính nhanh

$$FL(l, m, \max(h, \max(\text{HMAX}(m + 1, r), b_m))),$$

mà

$$\max(h, \max(\text{HMAX}(m + 1, r), b_m)) = \max(\text{HMAX}(m + 1, r), b_m),$$

đây là một hằng số không phụ thuộc vào h . Vì vậy ta cần ghi trong nút cây đoạn giá trị

$$FL(l, m, \max(\text{HMAX}(m + 1, r), b_m)).$$

Việc duy trì giá trị này cần gọi hàm FL của nút con; nhưng chỉ cần mọi nút trong cây con đều duy trì giá trị này, thì mọi hàm FL đều có thể tính trong $O(\log n)$.

Xét cách sửa. Chú ý rằng thao tác ML và MR về bản chất là thao tác sửa đoạn, nên các lazy tag không tương thích với nhau; có thể trực tiếp ghi đè, không cần xét gộp tag. Khi đánh tag ML hoặc MR trên cây đoạn, ta đồng thời ghi lại tham số h . Khi đánh tag, xét cách sửa nhanh thông tin đoạn; chú ý rằng cập nhật mực nước SUM chỉ cần truy vấn hàm FL hoặc FR . Loại tag này cũng rất dễ đẩy xuống.

Ta thu được một cây đoạn có thể hỗ trợ tổng mực nước, hỗ trợ các thao tác FL, FR, ML, MR . Vì hàm update (gộp thông tin hai con), hàm pushdown (đẩy tag xuống), cũng như các thao tác FL, FR, ML, MR trên một đoạn đơn đều là $O(\log n)$, nên độ phức tạp của thao tác trên đoạn bất kỳ là $O(\log^2 n)$.

Thuật toán bốn. Dùng phương pháp trên. Tính FL và sửa ML trong $O(\log^2 n)$. Cũng có thể trực tiếp dùng truy vấn FL để hỏi bể ngoài cùng bên phải và mực nước của bể ấy.

Độ phức tạp thuật toán: vì mỗi thao tác sửa cần chặt nhị phân, độ phức tạp là $O(q \log^3 n)$. Nếu trong giai đoạn hai đổi chặt nhị phân thành chặt ngay trên cây đoạn, có thể giảm độ phức tạp xuống $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua năm subtask đầu (cần chú ý hằng số), kỳ vọng 31 đến 61 điểm.

2.5. Thuật toán năm, thuật toán sáu

Thật ra cấu trúc dữ liệu trong thuật toán bốn đã có thể dùng để giải toàn bộ bài toán. Nhưng ta còn lại một vấn đề:

- Tìm đầu trái l của đoạn liên thông. Tìm r lớn nhất, rồi tìm h lớn nhất, sao cho $FL(l, r, h) \leq \text{SUM}(l, r)$.

Thuật toán năm. Dùng phương pháp trên. Tính FL, FR , sửa ML, MR trong $O(\log^2 n)$; thực hiện chặt nhị phân cộng truy vấn FL, FR trên cây đoạn trong $O(\log^3 n)$.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^3 n)$.

Điểm kỳ vọng: có thể qua mọi subtask (cần chú ý hằng số), kỳ vọng 31 đến 100 điểm.

Việc tìm h lớn nhất ở đây có thể làm trong $O(\log^2 n)$ bằng cách phỏng theo thuật toán trên. Mấu chốt là tìm r lớn nhất như thế nào. Chặt nhị phân trực tiếp là $O(\log^3 n)$. Ta thấy về bản chất nó yêu cầu nối một nút cây đoạn vào bên phải một đoạn bất kỳ, rồi truy vấn hàm FL hoặc FR của đoạn mới. Có thể xét trực tiếp tạo một nút cây đoạn mới để lưu thông tin đoạn mới này, chú ý mỗi nút cần lưu hai con trái phải của nó. Ta thấy sau khi gộp hai đoạn, truy vấn FL và FR vẫn là $O(\log n)$, do đó khi chặt trên cây đoạn, chỉ cần duy trì nút mới được gộp ra này là có thể giảm độ phức tạp xuống $O(\log^2 n)$.

Thuật toán sáu. Dùng phương pháp trên. Tính FL, FR , sửa ML, MR trong $O(\log^2 n)$; thực hiện chặt trên cây đoạn trong $O(\log^2 n)$.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua mọi subtask, kỳ vọng 100 điểm.

2.6. Các thuật toán khác

Do tính chất đặc biệt của bài này, còn một số thuật toán khác chưa được thảo luận; ở đây chỉ nhắc ngắn gọn.

Thuật toán bảy. Khi mô phỏng vết cạn, đẩy nước từng ô một và duy trì một hàng đợi đơn điệu.

Độ phức tạp thuật toán: tổng độ phức tạp $O(nq)$.

Điểm kỳ vọng: có thể qua bốn subtask đầu, kỳ vọng 31 điểm.

Thuật toán tám. Trong giai đoạn hai của ba giai đoạn ở trên, có thể trực tiếp sửa vết cạn; khi các giá trị trong đoạn của một nút cây đoạn là đơn điệu thì trực tiếp gán cả đoạn. Có thể chứng minh số nút được thăm là $O(\log n)$ trung bình, nhưng vì thao tác pushdown của cây đoạn cần $O(\log n)$, tổng độ phức tạp không đổi.

Độ phức tạp thuật toán: tổng độ phức tạp $O(q \log^2 n)$.

Điểm kỳ vọng: có thể qua mọi subtask, kỳ vọng 100 điểm.

3. Thiết lập dữ liệu kiểm thử

Bài này có tổng cộng 82 test. Có 11 loại dữ liệu ngẫu nhiên khác nhau, 3 loại dữ liệu thỏa tính chất đặc biệt, và 4 loại dữ liệu dựng tay. Trong đó dữ liệu dựng tay đồng thời kiểm tra độ phức tạp thời gian và độ chính xác của thuật toán. Đáp án cuối cùng được sinh bởi chương trình chuẩn dùng long double.

Ở đây chỉ nhắc tới một cách dựng dữ liệu có yêu cầu rất cao về độ chính xác.

Gọi m là một tham số (xấp xỉ một phần ba của n), đặt

$$S = \sum_{i=1}^m i.$$

Vì mọi số trong đầu vào của bài này sau khi nhân 10^7 đều là số nguyên, phần dưới trực tiếp dùng số nguyên để biểu thị giá trị đã nhân lên.

Chia toàn bộ dãy bể thành ba phần: phần thứ nhất cung cấp nước, phần thứ hai là một bậc thang, phần thứ ba nhận nước; mực nước trong phần thứ hai giảm dần. Mỗi phần có đúng m bể, và đặt đường chân trời là S .

Ban đầu mực nước và chiều cao vách ngăn của phần thứ nhất và phần thứ hai bằng nhau, phần thứ ba không có nước. Mỗi lần hạ chiều cao vách ngăn ngoài cùng bên phải của phần thứ nhất đúng S , rồi hạ vách ngăn ngoài cùng bên trái của phần thứ ba xuống S . Bảo đảm sau thao tác đầu tiên, S đơn vị thể tích nước này sẽ lấp bằng các bể của phần thứ hai; sau thao tác thứ hai, toàn bộ nước sẽ chảy vào dưới đường chân trời trong bể ngoài cùng bên trái của phần thứ ba, và không để lại hồ. Sau mỗi lần sửa, hỏi mực nước của bể ngoài cùng bên trái trong phần thứ ba.

Cuối cùng nói thêm vì sao bài này yêu cầu miền giá trị nhỏ hơn 100 và yêu cầu sai số tuyệt đối bảy chữ số. Trước hết, ta có thể thay đổi chiều cao vách ngăn của bể để thực hiện thao tác tương tự lấy phần thập phân trên mực nước, nên lúc này dùng sai số tuyệt đối hay sai số tương đối về bản chất là giống nhau. Mặt khác, vì lượng nước ban đầu không đổi, nếu có thuật toán duy trì các khối liên thông cùng mực nước thì khi miền giá trị nhỏ sẽ không thuận tiện để dựng dữ liệu có số lượng khối liên thông biến đổi lớn; vì vậy tác giả chọn dùng chín chữ số có nghĩa. Điều này có yêu cầu tương đối cao đối với độ chính xác của double; khi cài đặt nên cố gắng dùng phương pháp tính toán dấu phẩy động có sai số nhỏ.

4. Ước lượng độ khó

Subtask đầu tiên chỉ cần phân tích và mô phỏng đơn giản; dự kiến mọi thí sinh làm bài này đều có thể qua.

Subtask thứ hai cần thí sinh suy nghĩ một chút là có thể qua; dự kiến 80% thí sinh đều có thể qua.

Subtask thứ ba và thứ tư cần thí sinh tối ưu đơn giản thuật toán vét cạn, hoặc trực tiếp dùng một thuật toán vét cạn hiệu quả; dự kiến 60% thí sinh đều có thể qua.

Subtask thứ năm cần thí sinh phân tích và nghiên cứu bài toán sâu hơn, suy nghĩ kỹ rồi đưa ra cách làm; có độ khó tư duy nhất định. Độ khó code trung bình. Dự kiến 40% thí sinh đều có thể qua.

Subtask thứ sáu cần thí sinh tiếp tục phân tích, nghiên cứu và tối ưu bài toán; độ khó code cao. Dự kiến 20% thí sinh có thể qua.

Bài này ngoài độ khó tư duy và độ khó code, còn yêu cầu thí sinh phân tích và xử lý cẩn thận sai số độ chính xác của phép tính dấu phẩy động. Nếu thí sinh không phân tích kỹ thao tác, có thể còn đưa ra cách làm khá phức tạp. Nhìn chung độ khó của bài này khá cao.

5. Tổng kết

Cây đoạn được dùng cuối cùng trong bài này khác với cây đoạn thông thường. Về bản chất, mỗi nút của cây đoạn thông thường duy trì một tập hợp thông tin, và thông tin của mỗi nút có thể được lấy riêng ra. Cây đoạn trong bài này không chỉ duy trì một tập hợp thông tin, nó còn dựa vào cây nhị phân để hình thành một cấu trúc đệ quy; dù sửa hay hỏi một nút, đều phải dựa vào các nút con của nó. Khi truy vấn có

nhu cầu đặc biệt, ta còn có thể lấy ra một số nút để hình thành một cấu trúc đệ quy nằm ngoài cây nhị phân, điều này hơi giống tư tưởng của cây đoạn bền vững.

Kiến thức cần cho lời giải đúng của bài này chỉ liên quan tới cây đoạn, nhưng nó kiểm tra năng lực chuyển hóa mô hình, năng lực tư duy, năng lực phân tích vấn đề và năng lực code của thí sinh. Tuy đây là một bài cấu trúc dữ liệu, nó khác với bài cấu trúc dữ liệu truyền thống: không chỉ kiểm tra năng lực xử lý dữ liệu của thí sinh mà còn có yêu cầu cao về tư duy. Bài này không kiểm tra một cấu trúc dữ liệu mà thí sinh chưa nghiên cứu nghiêm túc hoặc chưa nắm chắc, mà kiểm tra cây đoạn, thứ thí sinh quen thuộc nhất, cơ bản nhất, và người thi NOIP đều biết dùng. Điều này cho thấy một bài hay không nhất thiết phải dựa vào cấu trúc dữ liệu hay kiến thức hỏ lánh, cũng không cần chỉ dựa vào lượng code khổng lồ để nâng ngưỡng. Người ra đề hy vọng bài này có thể đóng vai trò gợi mở, và mong được thấy mọi người nghĩ ra nhiều bài cấu trúc dữ liệu đa dạng hơn, thú vị hơn.

6. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn.

Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã quan tâm và hướng dẫn.

Xin cảm ơn bạn Wang Zeyuan đã thảo luận thuật toán với tôi và giúp kiểm tra đề.

Xin cảm ơn bạn Chen Sunli đã thẩm định bản thảo bài viết này.

Xin cảm ơn những bạn học đã đồng hành cùng tôi trong những năm qua.

Xin cảm ơn tất cả các thầy cô đã bồi dưỡng và hướng dẫn tôi.

Xin cảm ơn cha mẹ đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. CodeChef. Count on a Treap, problem code: COT5.
2. 2017 Multi-University Training Contest 5, problem 1005.

Bàn về bài toán tô màu đỉnh của đồ thị

Gao Jiaxuan, Trường Trung học Tưởng niệm Tôn Trung Sơn, thành phố Trung Sơn, tỉnh Quảng Đông

Tóm tắt

Bài viết này giới thiệu một nội dung quan trọng trong bài toán tô màu đồ thị: bài toán tô màu đỉnh. Tô màu đỉnh có hai phần quan trọng: sắc số và đa thức sắc. Bài viết giới thiệu các định lý liên quan tới sắc số và một số thuật toán thực dụng để phân định k -tô màu được cũng như để tìm sắc số của đồ thị; đồng thời, bài viết giới thiệu thuật toán xóa-co để tính đa thức sắc của đồ thị, rồi nêu các tối ưu của thuật toán này trên đồ thị thưa và đồ thị dày.

Lời nói đầu

Bài toán tô màu đồ thị là một nội dung quan trọng trong lý thuyết đồ thị, từ xưa tới nay luôn là một chủ đề được nghiên cứu rộng rãi; ví dụ nổi tiếng là định lý bốn màu. Đồng thời, tô màu đồ thị có ứng dụng quan trọng trong các bài toán lập lịch, phân phối thanh ghi và quy hoạch.

Bài toán tô màu đồ thị có vài dạng: tô màu đỉnh, tô màu cạnh và một số bài toán tô màu khác; trong đó tô màu đỉnh là một dạng rất quan trọng. Trong thi tin học cũng có những bài liên quan tới tô màu đỉnh, chẳng hạn tính sắc số của một đồ thị, tính số cách k -tô màu một đồ thị; nhưng dữ liệu của lớp bài này thường rất nhỏ, và lời giải chuẩn thường dùng chuỗi lũy thừa theo tập hợp³. Vì vậy tác giả nghiên cứu sâu hơn về bài toán tô màu đỉnh, với hy vọng khiến nhiều người chú ý hơn tới bài toán này.

Bài viết chia thành ba phần. Phần thứ nhất đưa ra một số ký hiệu và định nghĩa sẽ dùng. Phần thứ hai xoay quanh sắc số trong bài toán tô màu đỉnh, trình bày các định lý và thuật toán thực dụng liên quan. Phần thứ ba giới thiệu thuật toán xóa-co⁴ để tính đa thức sắc của đồ thị và đưa ra các tối ưu của tác giả cho thuật toán ấy.

1. Định nghĩa và quy ước

Để tiện mô tả, trước hết ta đưa ra một số định nghĩa.

Nếu không nói rõ thêm, đồ thị trong bài đều là đồ thị vô hướng không có cạnh bội và không có khuyên.

Với $G = (V, E)$, bài viết dùng $n = |V|$ để chỉ kích thước tập đỉnh V , và dùng $m = |E|$ để chỉ kích thước tập cạnh E .

Định nghĩa 1.1. (Tô màu đỉnh) Tô màu đỉnh là việc trong một đồ thị vô hướng $G = (V, E)$, gán cho mỗi đỉnh x một màu $c(x)$, sao cho phương án tô màu thỏa

$$\forall (u, v) \in E, \quad c(u) \neq c(v).$$

Định nghĩa 1.2. (Phương án k -tô màu hợp lệ) Trong đồ thị vô hướng G , một phương án tô màu dùng không quá k màu được gọi là phương án k -tô màu hợp lệ; nếu đồ thị vô hướng G tồn tại một phương án k -tô màu, ta nói G là k -tô màu được.

³Xem chi tiết trong bài “Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa theo tập hợp” của Lu Kaifeng, tập luận văn đội tuyển quốc gia năm 2015.

⁴Nguyên văn: Deletion-Contraction Algorithm.

Định nghĩa 1.3. (Tập độc lập) Một tập độc lập của đồ thị vô hướng $G = (V, E)$ được định nghĩa là một tập con S của V , sao cho các đỉnh trong S đôi một không kề nhau. Nói hình thức, I là một tập độc lập của G khi và chỉ khi

$$I \subseteq V \quad \text{và} \quad \forall u, v \in I, (u, v) \notin E.$$

Hệ quả 1.1. Trong một phương án k -tô màu hợp lệ của đồ thị vô hướng G , tập các đỉnh mang một màu bất kỳ đều là một tập độc lập của G .

Định nghĩa 1.4. (Đồ thị song liên thông) Ta định nghĩa đồ thị vô hướng G là song liên thông khi và chỉ khi sau khi bỏ đi một đỉnh bất kỳ của G , đồ thị G vẫn liên thông.

Định nghĩa 1.5. (Thành phần song liên thông) Một thành phần song liên thông của đồ thị vô hướng G được định nghĩa là một đồ thị con song liên thông cực đại của G . Hai thành phần song liên thông khác nhau bất kỳ trong G chứa nhiều nhất một đỉnh chung; những đỉnh như vậy được gọi là đỉnh khớp. Một đồ thị song liên thông không có đỉnh khớp.

Định nghĩa 1.6. (Đồ thị con cảm sinh) Trong đồ thị vô hướng $G = (V, E)$, với tập đỉnh $S \subseteq V$, đồ thị con cảm sinh bởi S trên G được định nghĩa là đồ thị con gồm các đỉnh trong S và các cạnh trong E có cả hai đầu mút đều thuộc S , ký hiệu là $G[S]$.

Định nghĩa 1.7. (Clique) Một đồ thị vô hướng $G = (V, E)$ là một clique khi và chỉ khi giữa hai đỉnh khác nhau bất kỳ trong V đều có cạnh nối; nói hình thức,

$$\forall u, v \in V, u \neq v \Rightarrow (u, v) \in E.$$

Ký hiệu K_n chỉ clique có kích thước n .

Định nghĩa 1.8. (Clique lớn nhất và số clique) Clique lớn nhất của đồ thị vô hướng $G = (V, E)$ được định nghĩa là một tập con lớn nhất S của V sao cho đồ thị con cảm sinh bởi S trên G là một clique. Kích thước của clique lớn nhất của G được gọi là số clique của G , ký hiệu $\omega(G)$.

Định nghĩa 1.9. Trong đồ thị vô hướng $G = (V, E)$, dùng $\Delta(G)$ để chỉ bậc lớn nhất trong G , và dùng $\Delta(x)$ để chỉ bậc của đỉnh x .

Để tiện mô tả, bài viết dùng các số nguyên dương để đánh số các màu khác nhau; trong một phương án k -tô màu hợp lệ, các màu được đánh số bởi $1..k$.

2. Sắc số

2.1. Định nghĩa

Định nghĩa 2.1. (Sắc số) Sắc số của đồ thị vô hướng G được định nghĩa là số nguyên dương nhỏ nhất k sao cho G là k -tô màu được; sắc số của G cũng được ký hiệu $\chi(G)$.

2.2. Cận của sắc số

Có nhiều định lý và kết luận thú vị về cận của sắc số $\chi(G)$; ở đây nêu một vài kết luận cơ bản.

Kết luận 2.1. Trên đồ thị vô hướng $G = (V, E)$, sắc số không nhỏ hơn số clique, tức

$$\chi(G) \geq \omega(G).$$

Kết luận 2.2. Trên đồ thị vô hướng $G = (V, E)$,

$$\chi(G)(\chi(G) - 1) \leq 2m.$$

Chứng minh. Xét một phương án tô màu của G . Nếu tồn tại hai màu p, q sao cho không có $(u, v) \in E$ nào thỏa u được tô màu p và v được tô màu q , thì đổi màu tất cả các đỉnh màu p thành màu q vẫn thu được một phương án tô màu hợp lệ. Từ đó suy ra trong một phương án $\chi(G)$ -tô màu, số cạnh ít nhất là $\chi(G)(\chi(G) - 1)/2$, tức $\chi(G)(\chi(G) - 1) \leq 2m$. $\square$

2.3. Tô màu tham lam

Thông qua thuật toán tô màu tham lam sau, ta có thể thấy

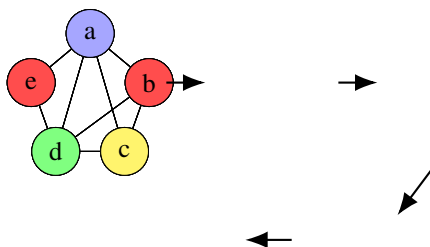
$$\chi(G) \leq \Delta(G) + 1.$$

Quy trình của thuật toán tô màu tham lam như sau. Trước hết chọn một dãy a , trong đó a là một hoán vị của tất cả các đỉnh trong G , và ban đầu mọi đỉnh đều được đặt là chưa tô màu.

Sau đó, duyệt từng đỉnh trong a theo thứ tự từ trước ra sau. Với đỉnh x , gọi c là màu có số hiệu nhỏ nhất sao cho trong các đỉnh hiện kề với x , không có đỉnh nào đã được tô màu c . Dùng màu c tô cho đỉnh x .

Dễ thấy số hiệu màu của mỗi đỉnh chắc chắn không vượt quá số đỉnh kề với nó và đứng trước nó trong dãy, cộng thêm một. Theo quá trình trên, ta thu được một phương án tô màu dùng không quá $\Delta(G) + 1$ màu.

Ví dụ, trong hình dưới, thứ tự tô màu là $[b, e, c, a, d]$, và số hiệu của các màu đỏ/vàng/xanh dương/xanh lục lần lượt là 1, 2, 3, 4:



Trong sơ đồ trên, các màu được thêm dần theo thứ tự b, e, c, a, d ; đây cũng là thuật toán tô màu theo dãy. Dãy a được chọn có ảnh hưởng then chốt tới kết quả của thuật toán.

2.4. Đồ thị phẳng

Định nghĩa 2.2. (Đồ thị phẳng) Đồ thị vô hướng G là đồ thị phẳng khi và chỉ khi có thể vẽ G trên mặt phẳng sao cho mọi cạnh chỉ cắt nhau tại các đỉnh.

Trong bài toán tô màu đỉnh trên đồ thị phẳng, định lý bốn màu chiếm vị trí rất quan trọng; nội dung của nó là:

Định lý 2.1. (Định lý bốn màu) Mọi đồ thị phẳng đều tồn tại phương án 4-tô màu hợp lệ.

Vì định lý bốn màu không phải trọng tâm của bài viết, ở đây không dành nhiều dung lượng giới thiệu.

Dưới đây đưa ra cách tìm một phương án 6-tô màu cho đồ thị phẳng. Trước hết có kết luận:

Kết luận 2.3. Trong đồ thị phẳng tồn tại ít nhất một đỉnh có bậc không vượt quá 5.

Mời người đọc tự chứng minh.

Sau khi có kết luận này, không khó để có một thuật toán tìm phương án 6-tô màu cho đồ thị phẳng $G = (V, E)$: trước hết tìm trong đồ thị một đỉnh x có bậc không vượt quá 5; sau khi tìm được phương án 6-tô màu hợp lệ trong $G - x$, tìm màu c có số hiệu nhỏ nhất chưa được dùng trong các đỉnh kề với x , rồi dùng màu c tô cho x . Như vậy có thể thu được một phương án 6-tô màu hợp lệ.

2.5. Đồ thị dây cung

Định nghĩa 2.3. (Đồ thị dây cung) Đồ thị vô hướng G là đồ thị dây cung khi và chỉ khi với mọi chu trình C trong G có kích thước lớn hơn 3, trên chu trình C tồn tại hai đỉnh không kề nhau trên chu trình nhưng có cạnh nối với nhau; cạnh ấy cũng được gọi là một dây cung.

Định nghĩa 2.4. (Thứ tự loại bỏ hoàn hảo) Một thứ tự loại bỏ hoàn hảo của đồ thị dây cung $G = (V, E)$ được định nghĩa là một dãy P gồm tất cả các đỉnh trong V , sao cho với mọi đỉnh v , các đỉnh kề với v và đứng sau v trong dãy P , cùng với v , cảm sinh trên G một clique.

Trong đồ thị dây cung $G = (V, E)$, đảo ngược một thứ tự loại bỏ hoàn hảo của G để được dãy P , rồi chạy thuật toán tô màu tham lam ở mục 2.3 theo dãy P , ta thu được một phương án $\omega(G)$ -tô màu hợp lệ. Vì với mọi đồ thị đều có $\chi(G) \geq \omega(G)$, suy ra với mọi đồ thị dây cung $G = (V, E)$ đều có

$$\chi(G) = \omega(G).$$

2.6. Định lý Brooks

Định lý 2.2. (Định lý Brooks) Khi $G = (V, E)$ không phải đồ thị đầy đủ và G không phải chu trình lẻ, ta có

$$\chi(G) \leq \Delta(G).$$

Chứng minh. Chứng minh dưới đây dùng quy nạp. Gọi d là bậc lớn nhất $\Delta(G)$ của đồ thị vô hướng $G = (V, E)$, và gọi $n = |V|$ là số đỉnh. Với d , giả sử định lý đúng cho mọi trường hợp có d nhỏ hơn; khi $d \leq 2$, định lý hiển nhiên đúng. Với d, n , giả sử định lý đúng cho mọi trường hợp có n nhỏ hơn. Quy nạp bắt đầu từ $n = d + 1$; trường hợp này hiển nhiên đúng, vì $G \neq K_{d+1}$, nên chắc chắn tìm được hai đỉnh không kề nhau, có thể tô chúng bằng cùng một màu để thu được một phương án d -tô màu hợp lệ. Do đó trong phần chứng minh dưới đây giả sử $n \geq d + 2$ và $d > 2$.

Trường hợp 1. G không phải một thành phần song liên thông, tức tồn tại đỉnh khớp v . Gọi $C_1, \dots, C_t$ là các tập đỉnh của các thành phần liên thông của $G - v$. Theo giả thiết quy nạp, có thể d -tô màu các đồ thị con cảm sinh trên G bởi các tập $C_1 \cup \{v\}, \dots, C_t \cup \{v\}$. Bằng cách đổi tên màu, có thể gộp các phương án d -tô màu ấy lại để được một phương án d -tô màu hợp lệ của G .

Trường hợp 2. Trong G tồn tại hai đỉnh không kề nhau v, w sao cho $G - v - w$ không liên thông. Gọi A là tập đỉnh của một thành phần liên thông của $G - v - w$, và đặt $B = V \setminus (A \cup \{v, w\})$. Khi đó v, w đều có ít nhất một cạnh nối tới các đỉnh trong A và trong B . Gọi G_1 là đồ thị con cảm sinh trên G bởi $A \cup \{v, w\}$, và G_2 là đồ thị con cảm sinh trên G bởi $B \cup \{v, w\}$.

Nếu cả $G_1 + vw$ và $G_2 + vw$ đều không phải K_{d+1} , thì theo quy nạp có thể tìm phương án d -tô màu hợp lệ cho hai đồ thị ấy; đổi tên màu rồi gộp hai phương án lại là đủ.

Ngược lại, giả sử $G_1 + vw$ là K_{d+1} (đồ thị con cảm sinh bởi A trên G là K_{d-1}). Khi đó có thể tô cả v và w bằng màu 1, rồi tô các đỉnh trong A lần lượt bằng các màu $2..d$. Mặt khác, gộp hai đỉnh v, w trong $G_2 + vw$ để được G_3 . Theo quy nạp, G_3 tồn tại phương án d -tô màu hợp lệ; đổi tên màu thì có thể gộp với phương án phía trước. Trường hợp $G_2 + vw$ là K_{d+1} tương tự.

Trường hợp 3. Trong G không có đỉnh khớp, và cũng không tồn tại hai đỉnh v, w sao cho $G - v - w$ không liên thông. Gọi x là một đỉnh có bậc lớn nhất. Trong các đỉnh kề với x , tìm hai đỉnh không kề nhau a, b . Khi đó $G - a - b$ chắc chắn là đồ thị liên thông. Trong $G - a - b$, chạy duyệt rộng (*bfs*) từ x để được dãy P ; đảo ngược dãy P , rồi thêm a, b vào đầu dãy để được dãy Q . Chạy thuật toán tô màu tham lam ở mục 2.3 theo dãy này sẽ thu được một phương án d -tô màu hợp lệ.

Vì đầu dãy là a, b , nên a, b đều có màu 1. Với mọi đỉnh trong $V \setminus \{a, b, x\}$, đều có ít nhất một đỉnh kề đứng sau nó trong Q . Với x , vì a, b đều có màu 1, nên chắc chắn có thể dùng tại đỉnh x một màu có số hiệu không vượt quá d . $\square$

Ghi chú trích xuất hình: bản gốc có các sơ đồ màu minh họa ba trường hợp trong chứng minh định lý Brooks. Các sơ đồ ấy chỉ biểu diễn trực quan việc tách theo đỉnh khớp, tách theo hai đỉnh v, w , rồi đổi tên màu để gộp các phương án; phần lập luận phía trên đã ghi lại đầy đủ nội dung toán học.

2.7. Phán định k -tô màu được

Mục này đưa ra một số thuật toán phán định đồ thị vô hướng $G = (V, E)$ có k -tô màu được hay không.

Với $k = 2$. Phán định một đồ thị có 2-tô màu được hay không chính là phán định đồ thị ấy có phải đồ thị hai phía hay không; chỉ cần tô đen-trắng trực tiếp trên đồ thị. Độ phức tạp thời gian là $O(n + m)$.

Với $k = 3$. Để phán định một đồ thị có 3-tô màu được hay không, có thể dùng thuật toán Bron–Kerbosch để tìm tập độc lập cực đại, như Zhong Zhixian đã nhắc tới trong bài “Bàn về bài toán tập độc lập trong thi tin học” thuộc tập luận văn đội tuyển quốc gia năm 2017. Độ phức tạp thời gian là $O(3^{n/3}n^2)$.

Với k bất kỳ. Khi k là bất kỳ, tồn tại cách làm dùng quy hoạch động và thuật toán tìm tập độc lập cực đại, với độ phức tạp thời gian $O(2.445^n)$. Ở đây tác giả đưa ra thuật toán dùng chuỗi lũy thừa theo tập hợp để phán định đồ thị có k -tô màu được hay không.

Tích chập hợp. Định nghĩa tích chập hợp như sau. Với hai chuỗi lũy thừa theo tập hợp f, g trên biến tập hợp x , đặt

$$f = \sum_{S \subseteq U} f_S x^S, \quad g = \sum_{S \subseteq U} g_S x^S.$$

Định nghĩa $f \cdot g = h$, trong đó h cũng là một chuỗi lũy thừa theo tập hợp và hệ số h_S thỏa

$$h_S = \sum_{L \subseteq U} \sum_{R \subseteq U} [L \cup R = S] f_L g_R. \quad (1)$$

Định nghĩa biến đổi Möbius của chuỗi lũy thừa theo tập hợp f là

$$\hat{f}_S = \sum_{T \subseteq S} f_T.$$

Có thể dùng thuật toán chia để trị để tính $\hat{f}$. Ngược lại, có thể định nghĩa phép đảo Möbius của $\hat{f}$ là f . Theo nguyên lý bù trừ, ta có

$$f_S = \sum_{T \subseteq S} (-1)^{|S|-|T|} \hat{f}_T.$$

Lấy biến đổi Möbius hai vế của (1), ta được

$$\begin{aligned}\hat{h}_S &= \sum_{L \subseteq U} \sum_{R \subseteq U} [L \cup R \subseteq S] f_L g_R \\ &= \left(\sum_{L \subseteq U} [L \subseteq S] f_L \right) \left(\sum_{R \subseteq U} [R \subseteq S] g_R \right) \\ &= \hat{f}_S \hat{g}_S.\end{aligned}$$

Vì vậy, lần lượt lấy biến đổi Möbius của f, g để được $\hat{f}, \hat{g}$, ta có thể tính $\hat{h}$, rồi lấy đảo Möbius của $\hat{h}$ để thu được h . Độ phức tạp thời gian là $O(n2^n)$.

Trong các bài thi tin học, đề bài thường cho một môđun P ; các hệ số của chuỗi lũy thừa theo tập hợp được tính trong vành số nguyên modulo P .

Xét cách dùng tích chập hợp để phán định đồ thị vô hướng G có thể k -tô màu được hay không. Theo định nghĩa tô màu đỉnh, tập đỉnh của mỗi màu đều là một tập độc lập. Do đó phán định đồ thị vô hướng $G = (V, E)$ có thể k -tô màu được hay không tương đương với phán định có thể tìm trong $G = (V, E)$ k tập độc lập $P_1, \dots, P_k$ sao cho

$$P_1 \cup P_2 \cup \dots \cup P_k = V$$

hay không.

Ta có thuật toán sau. Với đồ thị vô hướng $G = (V, E)$, định nghĩa chuỗi lũy thừa theo tập hợp f bởi

$$f = \sum_{S \subseteq V} [S \text{ là tập độc lập trong } G] x^S.$$

Tính biến đổi Möbius $\hat{f}$ của f . Tính $\hat{g}$, trong đó

$$\hat{g}_S = \hat{f}_S^k.$$

Lấy đảo Möbius của $\hat{g}$ để được g . Nếu $g_V = 0$, điều này cho thấy G không tồn tại phương án k -tô màu hợp lệ; ngược lại G có thể k -tô màu được.

Độ phức tạp thời gian của thuật toán trên là $O(2^n n)$.

2.8. Tìm sắc số của đồ thị

Khi tìm sắc số của đồ thị vô hướng $G = (V, E)$, có thể tiếp tục dùng ý tưởng tích chập hợp trong việc phán định đồ thị có k -tô màu được hay không.

Phần chính của thuật toán là giống nhau; điểm khác là ta liệt kê k từ nhỏ tới lớn rồi phán định G có k -tô màu được hay không.

Quy trình thuật toán như sau. Với đồ thị vô hướng $G = (V, E)$, định nghĩa chuỗi lũy thừa theo tập hợp f bởi

$$f = \sum_{S \subseteq V} [S \text{ là tập độc lập trong } G] x^S.$$

Tính biến đổi Möbius $\hat{f}$ của f .

Liệt kê k từ nhỏ tới lớn. Gọi $\hat{g}$ là chuỗi lũy thừa theo tập hợp trong đó $\hat{g}_S = \hat{f}_S^k$. Dùng nguyên lý bù trừ để tính trong $O(2^n)$ giá trị

$$g_V = \sum_{S \subseteq V} (-1)^{|V|-|S|} \hat{g}_S.$$

Nếu $g_V \neq 0$, ta có thể cho rằng sắc số của G là $\chi(G) = k$.

Độ phức tạp thời gian của thuật toán trên là $O(2^n n)$.

3. Đa thức sắc

3.1. Định nghĩa và quy ước

Định nghĩa 3.1. (Số k -tô màu) Số k -tô màu của đồ thị vô hướng G được định nghĩa là số phương án tô màu hợp lệ cho G bằng k màu; hai phương án khác nhau khi và chỉ khi tồn tại một đỉnh có màu khác nhau trong hai phương án.

Định nghĩa 3.2. (Đa thức sắc) Đa thức sắc của đồ thị vô hướng G , ký hiệu $P(G, x)$, được định nghĩa là một đa thức theo x , sao cho khi dùng k màu để tô màu, thay $x = k$ vào $P(G, x)$ thì nhận được số k -tô màu của G .

Định nghĩa 3.3. (k -phân hoạch độc lập) Một phương án k -phân hoạch độc lập của đồ thị vô hướng $G = (V, E)$ được định nghĩa là việc chia tập đỉnh V thành k tập không rỗng $S_1, S_2, \dots, S_k$, mỗi đỉnh thuộc đúng một tập, và thỏa với mọi $i \in [1, k]$, S_i đều là tập độc lập của G . Hai phương án k -phân hoạch độc lập khác nhau khi và chỉ khi tồn tại hai đỉnh u, v sao cho trong một phương án chúng nằm trong cùng một tập, còn trong phương án kia chúng nằm trong hai tập khác nhau.

Định nghĩa 3.4. (Dãy phân hoạch độc lập) Dãy phân hoạch độc lập của đồ thị vô hướng $G = (V, E)$ được định nghĩa là dãy h , trong đó h_k biểu thị số phương án k -phân hoạch độc lập của G . Dễ thấy $h_n = 1$ và với mọi $i > n$, $h_i = 0$.

Định nghĩa 3.5. (Đa thức phân hoạch độc lập) Với đồ thị vô hướng G , gọi h là dãy phân hoạch độc lập của nó. Định nghĩa đa thức phân hoạch độc lập của G là

$$H(G, x) = \sum_i h_i x^i.$$

Thông thường trong một đồ thị có quy mô nhất định, các hệ số của đa thức sắc sẽ rất lớn. Trong thi tin học, người ta thường xét các hệ số ấy sau khi lấy modulo một số nào đó, chẳng hạn 998244353 hoặc $10^9 + 7$. Vì vậy trong phần sau, ta giả định mọi phép toán đều được thực hiện theo modulo một số nguyên tố đủ lớn M .

3.2. Tính chất

Tính chất 3.1. Với đồ thị vô hướng G , giữa đa thức sắc

$$P(G, x) = \sum_i a_i x^i$$

và dãy phân hoạch độc lập h tồn tại quan hệ sau:

$$P(G, x) = \sum_i h_i x^i.$$

Ở đây x^i biểu thị lũy thừa giảm bậc i của x , tức

$$x^i = \prod_{j=1}^i (x - i + j).$$

Vì $h_n = 1$, và với mọi $i > n$ đều có $h_i = 0$, bậc của đa thức $P(G, x)$ bằng n .

Tính chất 3.2. Sắc số của đồ thị vô hướng G là số nguyên k nhỏ nhất làm cho $P(G, k)$ khác không; nói hình thức,

$$\chi(G) = \min\{k \in \mathbb{N} : P(G, k) > 0\}.$$

Tính đúng đắn của tính chất 3.2 suy ra trực tiếp từ định nghĩa sắc số.

Tính chất 3.3. Với đồ thị vô hướng $G = (V, E)$, giá trị $P(G, -1) \times (-1)^{|V|}$ bằng số phương án định hướng phi chu trình⁵ của G . Ở đây định hướng phi chu trình là số cách chọn hướng cho mỗi cạnh của G để nhận được đồ thị có hướng G' là đồ thị có hướng không chu trình; hai phương án khác nhau khi và chỉ khi có một cạnh (u, v) mang hướng khác nhau trong hai phương án.

Chứng minh lược bỏ.

Tính chất 3.4. Với đồ thị vô hướng $G = (V, E)$, giả sử G có c thành phần liên thông $G_1, \dots, G_c$. Khi đó trong đa thức sắc $P(G, x)$:

- các hệ số của các hạng $x^0, \dots, x^{c-1}$ đều bằng 0;
- các hệ số của các hạng $x^c, \dots, x^n$ đều khác 0, và dấu của chúng luân phiên nhau;
- hệ số của hạng x^n bằng 1;
- hệ số của hạng x^{n-1} bằng $-|E|$;
- $P(G, x) = P(G_1, x)P(G_2, x) \cdots P(G_c, x)$.

Tính chất 3.5. G là cây khi và chỉ khi

$$P(G, x) = x(x-1)^{n-1}.$$

3.3. Đa thức sắc của đồ thị đặc biệt

Trong một số đồ thị đặc biệt, có thể trực tiếp tìm đa thức sắc:

Clique K_n	$x(x-1)(x-2) \cdots (x-(n-1))$
Đồ thị không cạnh $\overline{K_n}$	x^n
Cây có n đỉnh	$x(x-1)^{n-1}$
Chu trình đơn C_n	$(x-1)^n + (-1)^n(x-1)$

Trong đồ thị dây cung, ngoài việc sắc số $\chi(G)$ bằng số clique $\omega(G)$, đa thức sắc cũng có thể tính trực tiếp. Với đồ thị dây cung $G = (V, E)$, tìm một thứ tự loại bỏ hoàn hảo P của G . Gọi x_i là số đỉnh kề với đỉnh P_i và đứng sau P_i trong P . Khi đó

$$P(G, x) = (x - x_1)(x - x_2) \cdots (x - x_n).$$

3.4. Chuỗi lũy thừa theo tập hợp

Có thể dùng chuỗi lũy thừa theo tập hợp để tìm sắc số của đồ thị; cũng có thể dùng nó để tìm đa thức sắc của đồ thị.

⁵Nguyên văn: Acyclic orientation.

Trước hết xét cách tính số phương án tô màu đồ thị $G = (V, E)$ bằng k màu. Định nghĩa chuỗi lũy thừa theo tập hợp f theo biến tập hợp x :

$$f = \sum_{S \subseteq V} [S \text{ là tập độc lập trong } G] \cdot u^{|S|} \cdot x^S.$$

Chú ý rằng chuỗi lũy thừa theo tập hợp ở đây khác với mục 2.8: f_S , tức hệ số của x^S , không còn là một số đơn thuần, mà là một đa thức theo u ; các phép toán liên quan được thực hiện theo modulo u^{n+1} .

Tìm biến đổi Möbius $\hat{f}$ của f . Tính chuỗi lũy thừa theo tập hợp $\hat{g}$, trong đó

$$\hat{g}_S = (f_S)^k.$$

Tính đảo Möbius g của $\hat{g}$. Hệ số của u^n trong g_V chính là số phương án tô màu G bằng k màu.

Lấy $k = 1..n$, thuật toán trên có thể tính được số phương án tô màu G bằng $1..n$ màu trong thời gian $O(2^n n^3)$. Gọi $w(k)$ là số phương án tô màu bằng k màu.

Vì đa thức sắc $P(G, x)$ của G là một đa thức bậc n và hằng số tự do bằng 0, các giá trị $w(1..n)$ nhận được ở trên chính là các giá trị điểm của $P(G, x)$ tại $x = 1..n$. Do đó có thể dùng nội suy Lagrange để tìm $P(G, x)$.

Nội suy Lagrange. Với một đa thức $F(x)$ theo x , bậc của nó là c . Nếu biết $c + 1$ điểm đôi một khác nhau trên $F(x)$,

$$(x_0, y_0), (x_1, y_1), \dots, (x_c, y_c),$$

thì

$$F(x) = \sum_{i=0}^c y_i \prod_{i \neq j} \frac{x - x_j}{x_i - x_j}.$$

Dùng nội suy Lagrange, có thể sử dụng các giá trị điểm

$$(0, 0), (1, w(1)), (2, w(2)), \dots, (n, w(n))$$

để tính $P(G, x)$.

3.5. Thuật toán xóa-co

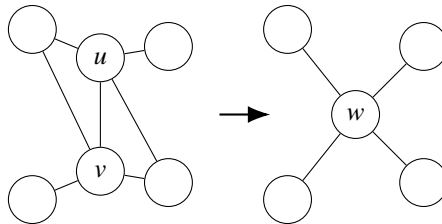
Định nghĩa 3.6. (Co cạnh) Trong đồ thị $G = (V, E)$, co cạnh $e = (u, v)$ là thao tác xóa e khỏi G , rồi gộp hai đỉnh u và v thành một đỉnh mới w ; các cạnh nối với w tương ứng với những cạnh vốn nối với u hoặc v .

Nói hình thức hơn, trong $G = (V, E)$, co cạnh (u, v) thu được một đồ thị mới $G' = (V', E')$. Trước hết gộp u, v thành một đỉnh mới w , khi đó

$$V' = \{w\} \cup (V \setminus \{u, v\}).$$

Sau đó, trong tập cạnh E' , với mọi $x, y \notin \{u, v\}$, $(x, y) \in E'$ khi và chỉ khi $(x, y) \in E$; với mọi $x \notin \{u, v\}$, $(w, x) \in E'$ khi và chỉ khi $(u, x) \in E$ hoặc $(v, x) \in E$.

Hình dưới là một minh họa cho thao tác co cạnh.⁶



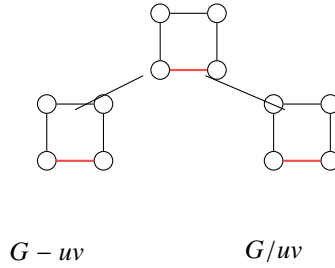
⁶Hình trong bản gốc lấy từ mục từ “Contraction” trên Wikipedia; ở đây vẽ lại giản lược bằng TikZ.

Đúng như tên gọi, nội dung của thuật toán xóa-co dựa trên việc xóa và co cạnh. Với một cạnh (u, v) trong G , ta có

$$P(G, x) = P(G - uv, x) - P(G/uv, x). \quad (2)$$

Ở đây $G - uv$ chỉ đồ thị thu được bằng cách xóa cạnh (u, v) khỏi G , và G/uv chỉ đồ thị thu được bằng cách co cạnh (u, v) trên G .

Tính trực tiếp theo đẳng thức trên sẽ cho thuật toán xóa-co cơ bản nhất. Ví dụ, hình dưới biểu thị cây đệ quy khi thực hiện thuật toán xóa-co trên một đồ thị nhỏ:



Vì dạng đệ quy của thuật toán trên giống với dãy Fibonacci⁷, trong trường hợp xấu nhất độ phức tạp là

$$\left(\frac{1 + \sqrt{5}}{2}\right)^{|V|+|E|} \in O(1.62^{|V|+|E|}).$$

Đáng tiếc là trong phần lớn trường hợp, hiệu suất của thuật toán này kém xa độ phức tạp tính toán bằng chuỗi lũy thừa theo tập hợp.

Dưới đây tối ưu thuật toán xóa-co.

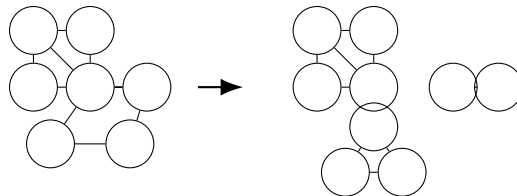
3.6. Tối ưu thuật toán xóa-co trên đồ thị thưa

3.6.1. Tách thành phần song liên thông

Với $G = (V, E)$, giả sử trong G có k thành phần song liên thông $C_1, C_2, \dots, C_k$, và G có s thành phần liên thông. Khi đó

$$P(G, x) = \frac{1}{x^{k-s}} \prod_{i=1}^k P(C_i, x).$$

Với G , trước hết dùng thuật toán Tarjan tách G thành các thành phần song liên thông. Sau khi tìm được đa thức sắc của từng thành phần song liên thông, dùng công thức trên để tính đa thức sắc của G .



3.6.2. Đường đi bậc hai

Qua quan sát, dễ thấy trong đồ thị thưa có rất nhiều đỉnh có bậc không vượt quá 2.

⁷Dãy Fibonacci thỏa $f_0 = 1, f_1 = 1, f_i = f_{i-1} + f_{i-2} \ (i \geq 2)$.

Khi đỉnh u có bậc 0 trong G ,

$$P(G, x) = P(G - u, x) \cdot x.$$

Khi đỉnh u có bậc 1 trong G ,

$$P(G, x) = P(G - u, x) \cdot (x - 1).$$

Với đỉnh bậc 2, để giảm số lần tính toán, xét đồng thời xóa—co nhiều cạnh và dùng các đa thức sắc đã tổng kết phía trước cho đồ thị đặc biệt để tính nhanh.

Để trình bày tốt hơn, đưa vào khái niệm “đường đi bậc hai”.

Định nghĩa 3.7. (Đường đi bậc hai) Một đường đi bậc hai trong G được định nghĩa là một đường đi đơn mà các đỉnh ngoài điểm đầu và điểm cuối đều có bậc 2 trong G . Nói hình thức, một đường đi độ dài k

$$P = (p_1, p_2, \dots, p_{k+1})$$

là một đường đi bậc hai khi và chỉ khi:

- P là một đường đi đơn;
- với mọi $i \in [2, k]$, $\Delta(p_i) = 2$.

Trước hết nhắc lại trường hợp đường thẳng và chu trình. Đa thức sắc của một đường thẳng độ dài l là $x(x - 1)^{l-1}$. Đa thức sắc của một chu trình độ dài l là

$$(x - 1)^l + (-1)^l(x - 1).$$

Với một đường đi bậc hai độ dài k

$$P = (p_1, p_2, \dots, p_{k+1}),$$

nếu đã xác định màu của p_1 và p_{k+1} , thì số cách tô màu $p_2, \dots, p_k$ là bao nhiêu?

Chia thành hai trường hợp:

Trường hợp 1. Màu tô cho p_1 và p_{k+1} giống nhau. Trường hợp này có thể xem là tìm đa thức sắc của chu trình độ dài k , tức

$$(x - 1)^k + (-1)^k(x - 1).$$

Vì màu của p_1 và p_{k+1} đã được xác định, số phương án tương ứng là

$$\frac{(x - 1)^k + (-1)^k(x - 1)}{x}.$$

Trường hợp 2. Màu tô cho p_1 và p_{k+1} khác nhau. Trường hợp này có thể xem là lấy số cách tô màu đường thẳng độ dài k trừ đi số cách tô màu chu trình độ dài k , tức

$$x(x - 1)^k - (x - 1)^k - (-1)^k(x - 1) = (x - 1)^{k+1} - (-1)^k(x - 1).$$

Vì màu của p_1 và p_{k+1} đã được xác định, số cách tô màu $p_2, \dots, p_k$ là

$$\frac{(x - 1)^{k+1} - (-1)^k(x - 1)}{x}.$$

Vì vậy, với một đường đi bậc hai P , gọi G_1 là đồ thị thu được bằng cách xóa khỏi G tất cả các đỉnh trên P trừ điểm đầu và điểm cuối; gọi G_2 là đồ thị thu được bằng cách co tất cả các cạnh trên P trong G . Theo hai trường hợp trên, ta có

$$\begin{aligned} P(G, x) &= \frac{(x-1)^k - (-1)^k}{x} (P(G_1, x) - P(G_2, x)) + \frac{(x-1)^k + (-1)^k(x-1)}{x} P(G_2, x) \\ &= \frac{(x-1)^k - (-1)^k}{x} P(G_1, x) + (-1)^k P(G_2, x). \end{aligned}$$

Thật ra công thức

$$P(G, x) = P(G - uv, x) - P(G/uv, x)$$

là trường hợp $k = 1$ của công thức này.

Trong G , trước hết tìm một đường đi bậc hai cực dài P , rồi dùng công thức trên để tính P .

3.7. Tối ưu thuật toán xóa-co trên đồ thị dày

Trong đồ thị dày $G = (V, E)$, với một cạnh $(u, v) \notin E$, có đẳng thức sau:

$$P(G, x) = P(G + uv, x) + P(G/uv, x). \quad (3)$$

Công thức (3) thật ra là biến dạng của công thức (2).

Để tận dụng tốt hơn tính chất đặc biệt của đồ thị dày, xét trước hết việc tìm đa thức phân hoạch độc lập $H(G, x)$ của G , rồi dựa vào tính chất 3.1 để dùng $H(G, x)$ tìm $P(G, x)$.

Với $H(G, x)$, ta có

$$H(G, x) = H(G + uv, x) + H(G/uv, x).$$

Để tiện mô tả, đặt

$$\overline{H}(G, x) = H(\overline{G}, x).$$

3.7.1. Đỉnh khớp

Khác với tình huống trong đồ thị thưa, ở đây ta sẽ tách theo đỉnh khớp của đồ thị bù của đồ thị dày.

Với một đỉnh khớp u trong $\overline{G}$, gọi G_1 là một thành phần song liên thông của $\overline{G}$ chứa u , và gọi G_2 là đồ thị con cảm sinh trên $\overline{G}$ bởi các đỉnh của $\overline{G}$ sau khi bỏ đi $G_1 - u$. Khi đó:

$$\begin{aligned} H(G, x) &= \overline{H}(G_1, x) \overline{H}(G_2 - u, x) + \overline{H}(G_2, x) \overline{H}(G_1 - u, x) \\ &\quad - x \cdot \overline{H}(G_2 - u, x) \overline{H}(G_1 - u, x). \end{aligned}$$

Điều cần chú ý là công thức trên rất khác kết quả thu được ở mục 3.6.1; ưu điểm của nó là tối ưu dạng đệ quy liên quan tới số đỉnh và số cạnh trong công thức (3).

3.7.2. Đường đi bậc hai

Trong đồ thị bù của đồ thị dày có nhiều đỉnh bậc 2. Được gợi ý từ mục 3.6.2, có thể xét tối ưu một đường đi bậc hai cực dài trong đồ thị bù.

Gọi đường đi đơn

$$P = (p_1, p_2, \dots, p_{k+1})$$

là một đường đi bậc hai cực dài trong $\overline{G}$, và đặt $u = p_1$, $v = p_{k+1}$.

Khi $k = 1$, trực tiếp tính trên G :

$$H(G, x) = H(G + uv, x) + H(G/uv, x).$$

Khi $k = 2$, trực tiếp tính trên (u, p_2) :

$$H(G, x) = H(G + up_2, x) + H(G/up_2, x).$$

Khi $k > 2$, trước hết tính trên G đối với cạnh (u, p_2) :

$$H(G, x) = H(G + up_2, x) + H(G/up_2, x),$$

rồi trong hai đồ thị thu được tiếp tục tính công thức trước đó đối với (p_k, v) .

Trong đồ thị vô hướng G' thu được ở trên, sẽ xuất hiện trường hợp một thành phần liên thông của $\overline{G'}$ là một đường thẳng.

Với một đồ thị L_r có r đỉnh mà đồ thị bù của nó là một đường thẳng, đa thức phân hoạch độc lập của nó là

$$H(L_r, x) = \sum_{i=\lceil r/2 \rceil}^r \binom{i}{r-i} x^i.$$

Có thể thấy tối ưu đường đi bậc hai trong đồ thị thưa và trong đồ thị bù của đồ thị dày rất khác nhau. Trường hợp trước tương đối đơn giản, còn trường hợp sau phức tạp hơn; nó chỉ đưa ra một thứ tự tính theo cạnh, hình thức cũng không đẹp bằng trường hợp trước. Về bản chất, mục đích là cố gắng tách ra các thành phần liên thông mà đồ thị bù của chúng là đường thẳng, rồi dùng công thức để giảm số lần tính toán.

3.8. Các tối ưu khác của thuật toán xóa-co

Trong thuật toán xóa-co, với một số đồ thị đặc biệt như chu trình, cây, có thể trực tiếp tính đa thức sắc của chúng.

Trong ứng dụng thực tế, người ta cũng thường dùng các cách làm liên quan tới phán định đẳng cấu đồ thị để giảm số lần tính toán của thuật toán xóa-co.

Tổng kết

Đối với sắc số, bài viết đưa ra một số cận cơ bản của sắc số trong đồ thị, đồng thời đưa ra các thuật toán phán định k -tô màu được và tính sắc số của đồ thị.

Đối với đa thức sắc, bài viết đưa ra các tính chất của đa thức sắc và thuật toán xóa-co để tính đa thức sắc, đồng thời đưa ra các tối ưu trên đồ thị thưa và đồ thị dày. Để trình bày rõ hơn, bài viết đưa vào khái niệm “đường đi bậc hai”; trong tối ưu dành cho đồ thị dày, bài viết đưa vào các khái niệm “dãy phân hoạch độc lập” và “đa thức phân hoạch độc lập” để tối ưu đồ thị dày.

Thuật toán kinh điển dùng chuỗi lũy thừa theo tập hợp để tính đa thức sắc, do hiệu suất thời gian và không gian, chỉ có thể áp dụng cho đồ thị có số đỉnh nhỏ. Tối ưu thuật toán xóa-co trên đồ thị thưa khiến việc tính đa thức sắc trên đồ thị có số đỉnh lớn hơn trở nên khả thi.

Trong bài toán tô màu đỉnh vẫn còn nhiều nội dung thú vị hơn nữa. Hy vọng bài viết này có thể gợi suy nghĩ cho người đọc và khiến nhiều người quan tâm hơn tới bài toán tô màu đỉnh.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp cơ hội học tập và trao đổi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe đã hướng dẫn và giúp đỡ.

Xin cảm ơn cha mẹ đã bồi dưỡng và quan tâm tôi trong nhiều năm qua.

Xin cảm ơn thầy Song Xinbo và thầy Xiong Chao của Trường Trung học Tưởng niệm Tôn Trung Sơn đã quan tâm và giúp đỡ trong nhiều năm qua.

Xin cảm ơn bạn Cao Tianyou của Trường Trung học Tưởng niệm Tôn Trung Sơn, và các bạn Fan Zhiyuan, Zhang Zheyu của Trường Trung học Xuejun Hàng Châu đã thẩm định bản thảo bài viết này.

Xin cảm ơn các thầy cô và bạn học đã khích lệ, giúp đỡ tôi.

Tài liệu tham khảo

1. Wikipedia tiếng Anh, Graph coloring, https://en.wikipedia.org/wiki/Graph_coloring.
2. Wikipedia tiếng Anh, Chromatic polynomial, https://en.wikipedia.org/wiki/Chromatic_polynomial.
3. Zhong Zhixian. Bàn về bài toán tập độc lập trong thi tin học. Tập luận văn đội tuyển quốc gia IOI 2017.
4. Lu Kaifeng. Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa theo tập hợp. Tập luận văn đội tuyển quốc gia IOI 2015.

Bàn về các bài toán liên quan tới đếm đường đi trên lưới

Dai Yan, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Mô hình đường đi trên lưới là một mô hình bài toán thường gặp trong thi tin học; trong đó một dạng khá phổ biến là các bài toán đếm đường đi Dyck. Tác giả nhận thấy trong thi thuật toán, người giải thường dùng hàm sinh hoặc quy hoạch động để xử lý lớp bài này. Sau khi nghiên cứu kỹ lớp bài toán ấy, tác giả thấy rằng nhiều bài có thể được giải nhanh bằng cách dựng song ánh. Bài viết đưa ra phương pháp đếm hai loại đường đi Dyck đặc biệt, cũng như phương pháp đếm khi giới hạn số đỉnh nhọn. Đồng thời, tác giả khảo sát một số bài toán đếm đường đi trên lưới không giao nhau và nêu mối liên hệ giữa hai lớp bài toán này.

1. Dẫn nhập

Bài toán đường đi trên lưới là mô hình kinh điển trong tổ hợp, và rất thường gặp trong thi tin học. Rất nhiều bài trong số đó lấy việc đếm đường đi Dyck làm mô hình, chẳng hạn bài thứ hai ngày thứ nhất NOI 2018, *Bubble Sort*.⁸

Dựng song ánh là một cách làm thường gặp trong đếm tổ hợp. So với hàm sinh hoặc quy hoạch động mà người giải tin học thường dùng, cách này nhanh hơn, đẹp hơn, độ khó cài đặt thấp hơn, và trong phần lớn trường hợp cũng có thể tối ưu được độ phức tạp thời gian. Tuy vậy, trong cộng đồng OI hiện nay, tư tưởng giải bài này còn khá ít gặp.

Mục 2 chủ yếu thảo luận hai loại đường đi Dyck đặc biệt: đường đi (n, m) -Dyck khi n, m nguyên tố cùng nhau, và đường đi t -Dyck bậc n ; đồng thời đưa ra cách đếm khi giới hạn số đỉnh nhọn.

Mục 3 chủ yếu thảo luận các bài toán đếm đường đi Dyck không giao nhau, đường đi tự do không giao nhau, và đường đi tự do không chạm nhau, đồng thời nêu một mối liên hệ giữa các bài toán ở mục 2 và mục 3.

Mục 4 đưa ra một ứng dụng thực tế của đếm đường đi Dyck.

2. Đường đi Dyck

2.1. Đường đi trên lưới

Định nghĩa 2.1. Trong mặt phẳng tọa độ Descartes, điểm có cả hoành độ và tung độ đều là số nguyên được gọi là điểm lưới. Đường đi trên lưới phẳng là một đường đi từ một điểm lưới tới một điểm lưới khác, chỉ đi qua các điểm lưới. Độ dài của đường đi trên lưới là số bước mà đường đi ấy sử dụng.

2.2. Đường đi tự do

Định nghĩa 2.2. Với một đường đi trên lưới từ $(0, 0)$ tới (n, m) , nếu nó chỉ dùng bước lên $U = (0, 1)$ và bước ngang $L = (1, 0)$, ta gọi nó là một đường đi tự do (n, m) (*free path*).

⁸<http://uoj.ac/problem/394>

Định lý 2.1. Gọi $\mathcal{F}(n, m)$ là tập các đường đi tự do (n, m) , và $F(n, m) = \#\mathcal{F}(n, m)$ là số lượng phần tử của tập ấy. Khi đó số đường đi tự do (n, m) là

$$F(n, m) = \binom{n+m}{n}. \quad (3)$$

Chứng minh. Một đường đi tự do từ $(0, 0)$ tới (n, m) tương ứng duy nhất với một dãy gồm n bước ngang L và m bước lên U . Mỗi dãy như vậy cũng xác định duy nhất một đường đi tự do. Vì vậy số đường đi tự do (n, m) bằng số cách chọn n vị trí đặt L trong $n + m$ vị trí, tức $\binom{n+m}{n}$. $\square$

2.3. Đếm đường đi (n, m) -Dyck

Định nghĩa 2.3. Với một đường đi tự do từ $(0, 0)$ tới (n, m) , nếu nó luôn không đi xuống dưới đường chéo $y = \frac{m}{n}x$, ta gọi nó là một đường đi (n, m) -Dyck.

Gọi $\mathcal{D}(n, m)$ là tập các đường đi (n, m) -Dyck, và $D(n, m) = \#\mathcal{D}(n, m)$ là số lượng của chúng.

Ta xét dãy L, U tương ứng với một đường đi (n, m) -Dyck.

Định nghĩa 2.4. Với hai đường đi trên lưới P, Q từ $(0, 0)$ tới (n, m) , trong đó

$$P = u_1 u_2 \cdots u_{n+m}, \quad Q = v_1 v_2 \cdots v_{n+m},$$

và $u_i, v_i \in \{L, U\}$, nếu tồn tại i sao cho

$$u_{i+1} \cdots u_{n+m} u_1 \cdots u_i = v_1 v_2 \cdots v_{n+m},$$

ta nói hai đường đi P, Q là tương đương. Ký hiệu toàn bộ các đường đi tương đương với P là $[P]$.

Định nghĩa 2.5. Với một đường đi bất kỳ P , đặt

$$P_k = u_{k+1} u_{k+2} \cdots u_{n+m} u_1 \cdots u_k.$$

Khi đó

$$[P] = \{P_k \mid k = 1, 2, 3, \dots, n+m\}.$$

Định nghĩa chu kỳ của P là số k nhỏ nhất sao cho $P = P_k$, ký hiệu $\text{period}(P)$. Rõ ràng $\#[P] = \text{period}(P)$.

Tiếp theo, ta đưa ra hai bổ đề.

Bổ đề 2.1. Nếu $(n, m) = 1$, tức n, m nguyên tố cùng nhau, thì với mọi $P \in \mathcal{F}(n, m)$,

$$\text{period}(P) = n + m. \quad (4)$$

Chứng minh. Hiển nhiên $\text{period}(P) \leq n + m$. Ta chứng minh bằng phản chứng rằng $\text{period}(P) = n + m$.

Giả sử $\text{period}(P) < n + m$, gọi nó là r . Đặt $P = u_1 u_2 \cdots u_{n+m}$, và $n + m = a \cdot r + b$, với $b < r$. Theo định nghĩa chu kỳ, ta có

$$P = P_r = P_{2r} = \cdots = P_{ar}.$$

Nếu $b \neq 0$, thì $P = P_b$ với $b < r$, mâu thuẫn với định nghĩa của $\text{period}(P)$. Do đó $b = 0$, tức $n + m = a \cdot r$. Vì $r < n + m$, nên $a > 1$.

Giả sử trong $u_1 u_2 \cdots u_r$ có đúng x bước L và y bước U . Khi đó $n = a \cdot x, m = a \cdot y$, suy ra $(n, m) = a \neq 1$, mâu thuẫn. Vậy $\text{period}(P)$ không thể nhỏ hơn $n + m$, nên $\text{period}(P) = n + m$. $\square$

Bổ đề 2.2. Khi n và m nguyên tố cùng nhau, trong $[P]$ có đúng một đường đi (n, m) -Dyck.

Chứng minh. Trước hết chứng minh sự tồn tại.

Nếu P không phải một đường đi (n, m) -Dyck, thì nó có ít nhất một điểm nằm dưới đường chéo. Trong các điểm ấy, lấy điểm v xa đường chéo nhất, cắt P tại điểm này thành hai đường con L_1, L_2 . Khi đó $P = L_1 L_2$. Dựng đường đi mới $\tilde{P} = L_2 L_1$; rõ ràng $\tilde{P}$ là một đường đi (n, m) -Dyck. Vậy sự tồn tại được chứng minh.

Tiếp theo chứng minh tính duy nhất.

Từ quá trình trên, ta chỉ cần chứng minh rằng trong các điểm nằm dưới đường chéo, điểm xa đường chéo nhất là duy nhất. Giả sử ngược lại có hai điểm như vậy, gọi là $v_1(x_1, y_1), v_2(x_2, y_2)$, với

$$0 < x_1 < x_2 < n, \quad 0 < y_1 < y_2 < m.$$

Đường thẳng qua hai điểm này song song với đường chéo $y = \frac{m}{n}x$, nên chúng có cùng hệ số góc:

$$k = \frac{y_2 - y_1}{x_2 - x_1} = \frac{m}{n}.$$

Điều này mâu thuẫn với việc n và m nguyên tố cùng nhau. Vậy tính duy nhất được chứng minh. $\square$

Từ hai bổ đề trên, ta nhận được ngay kết quả sau.

Định lý 2.2. Khi (n, m) nguyên tố cùng nhau, số đường đi (n, m) -Dyck là

$$D(n, m) = \frac{1}{n+m} \binom{n+m}{n}. \quad (5)$$

2.4. Đếm đường đi (n, m) -Dyck có k đỉnh nhọn

Định nghĩa 2.6. Trong một đường đi tự do từ $(0, 0)$ tới (n, m) , nếu hai bước liên tiếp là UL , ta gọi đó là một đỉnh nhọn (*peak*); nếu hai bước liên tiếp là LU , ta gọi đó là một đáy (*valley*).

Định lý 2.3. Gọi $\mathcal{F}(n, m; k)$ là tập mọi đường đi tự do (n, m) có đúng k đỉnh nhọn, và $F(n, m; k) = \#\mathcal{F}(n, m; k)$. Số đường đi tự do từ $(0, 0)$ tới (n, m) có đúng k đỉnh nhọn là

$$F(n, m; k) = \binom{n}{k} \binom{m}{k}. \quad (6)$$

Chứng minh. Ta gọi điểm lưới nằm giữa hai bước UL của một đỉnh nhọn là điểm đỉnh. Lấy một đường đi bất kỳ $P \in \mathcal{F}(n, m; k)$, giả sử k điểm đỉnh của nó là

$$(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k).$$

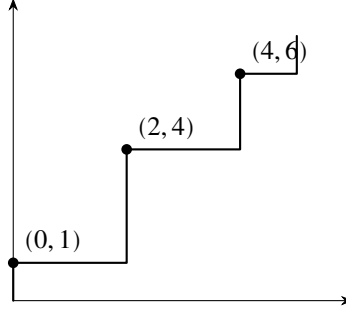
Tọa độ các điểm đỉnh thỏa

$$0 \leq x_1 < x_2 < \dots < x_k \leq n-1, \quad 0 \leq y_1 < y_2 < \dots < y_k \leq m.$$

Trước hết chọn $x_1, x_2, \dots, x_k$, rồi chọn $y_1, y_2, \dots, y_k$. Số bộ k điểm nguyên thỏa điều kiện là $\binom{n}{k} \binom{m}{k}$.

Sau khi chọn các điểm này, ta nối chúng bằng các đoạn gồm toàn bước U hoặc toàn bước L , sẽ nhận được một đường đi tự do thỏa điều kiện.

Dễ thấy mỗi đường đi tự do tương ứng duy nhất với các điểm đỉnh này, và mỗi cách chọn điểm đỉnh cũng xác định duy nhất một đường đi tự do. Nói cách khác, đường đi tự do và cách chọn điểm đỉnh song ánh với nhau, nên số đường đi tự do là $\binom{n}{k} \binom{m}{k}$. $\square$



Hình 4: $n = 5, m = 7$, các điểm đỉnh được chọn là $(0, 1), (2, 4), (4, 6)$.

Định lý 2.4. Gọi $\mathcal{F}^{UL}(n, m; k)$ là tập mọi đường đi tự do (n, m) có đúng k đỉnh nhọn, bắt đầu bằng U và kết thúc bằng L , và $F^{UL}(n, m; k) = \#\mathcal{F}^{UL}(n, m; k)$. Khi đó

$$F^{UL}(n, m; k) = \binom{n-1}{k-1} \binom{m-1}{k-1}. \quad (7)$$

Chứng minh. Mọi phần tử của $\mathcal{F}^{UL}(n, m; k)$ đều bắt đầu bằng U và kết thúc bằng L , nên hoành độ của điểm đỉnh đầu tiên và tung độ của điểm đỉnh cuối cùng đã được xác định. Do đó số cách chọn k điểm nguyên là $\binom{n-1}{k-1} \binom{m-1}{k-1}$.

Dựng đường đi tương tự chứng minh định lý 2.3, ta nhận được một đường đi tự do bắt đầu bằng U và kết thúc bằng L . Vậy số đường đi tự do bằng số cách chọn điểm đỉnh: $\binom{n-1}{k-1} \binom{m-1}{k-1}$. $\square$

Định lý 2.5. Gọi $\mathcal{D}(n, m; k)$ là tập mọi đường đi (n, m) -Dyck có đúng k đỉnh nhọn, và $D(n, m; k) = \#\mathcal{D}(n, m; k)$. Khi n, m nguyên tố cùng nhau, số đường đi (n, m) -Dyck có đúng k đỉnh nhọn là

$$D(n, m; k) = \frac{1}{k} \binom{n-1}{k-1} \binom{m-1}{k-1}. \quad (8)$$

Chứng minh. Với một đường đi bất kỳ P , nếu nó có k đỉnh nhọn thì chắc chắn có $k-1$ đáy, gọi là $v_1, v_2, \dots, v_{k-1}$. Cắt P tại các đáy này, ta chia được nó thành k đường con $L_1, L_2, \dots, L_k$, mỗi đường con chứa đúng một đỉnh nhọn, và

$$P = L_1 L_2 \cdots L_k.$$

Với một đường đi Dyck bất kỳ, tại mỗi trong k điểm đỉnh của nó đều có thể ánh xạ thành một đường đi tự do: với đỉnh thứ i , nếu đường con chứa nó là L_i , dựng

$$\tilde{P} = L_{i+1} L_{i+2} \cdots L_k L_1 \cdots L_i.$$

Vì vậy, mỗi đỉnh nhọn của mỗi đường đi Dyck tương ứng duy nhất với đúng một phần tử trong $\mathcal{F}^{UL}(n, m; k)$.

Ngược lại, với một đường đi tự do bất kỳ $\tilde{P} \in \mathcal{F}^{UL}(n, m; k)$, ta cũng cắt nó theo các đáy. Xét điểm xa đường chéo nhất ở phía dưới đường chéo; điểm này chắc chắn là một đáy, gọi là v_j . Cắt $\tilde{P}$ tại v_j thành L_1, L_2 , rồi đặt $P = L_2 L_1$. Theo bổ đề 2.2,

$$P = L_2 L_1 \in \mathcal{D}(n, m; k).$$

Do đó mỗi phần tử trong $\mathcal{F}^{UL}(n, m; k)$ tương ứng duy nhất với một đỉnh nhọn của một đường đi Dyck. Suy ra

$$D(n, m; k) = \frac{1}{k} F^{UL}(n, m; k) = \frac{1}{k} \binom{n-1}{k-1} \binom{m-1}{k-1}.$$

$\square$

2.5. Đếm đường đi t -Dyck

Định nghĩa 2.7. Với một đường đi tự do từ $(0, 0)$ tới (n, m) , nếu nó luôn không đi xuống dưới đường chéo $y = t \cdot x$, ta gọi nó là một đường đi t -Dyck. Đặc biệt, nếu $m = t \cdot n$, ta gọi nó là một đường đi t -Dyck bậc n .

Chú ý rằng vì $(n, t \cdot n) = n \neq 1$, nên đường đi t -Dyck bậc n không thỏa điều kiện áp dụng của các định lý trên.

Định lý 2.6. Số đường đi t -Dyck bậc n có k đỉnh nhọn là

$$D(n, tn; k) = \frac{1}{k} \binom{n-1}{k-1} \binom{tn}{k-1}. \quad (9)$$

Chứng minh. Đặt $m = t \cdot n + 1$, khi đó rõ ràng $(n, m) = 1$. Thay vào công thức (8), ta có

$$D(n, tn+1; k) = \frac{1}{k} \binom{n-1}{k-1} \binom{tn+1-1}{k-1} = \frac{1}{k} \binom{n-1}{k-1} \binom{tn}{k-1}.$$

Xét một phần tử bất kỳ $P \in \mathcal{D}(n, tn+1; k)$. Vì $\frac{tn+1}{n} > t \geq 2$, hai bước đầu của P chắc chắn đều là bước lên U . Gọi P' là đường đi nhận được sau khi bỏ bước lên đầu tiên.

Vì giữa hai đường thẳng $y = \frac{tn+1}{n}x$ và $y = tx$ không có điểm nguyên nào, việc bỏ bước U đầu tiên không khiến P' đi xuống dưới $y = tx$. Do đó $P' \in \mathcal{D}(n, tn; k)$.

Tương tự, với mọi $P' \in \mathcal{D}(n, tn; k)$, đặt $P = UP'$ thì $P \in \mathcal{D}(n, tn+1; k)$. Vì vậy các phần tử của $\mathcal{D}(n, tn; k)$ và $\mathcal{D}(n, tn+1; k)$ song ánh với nhau. Suy ra

$$D(n, tn; k) = D(n, tn+1; k) = \frac{1}{k} \binom{n-1}{k-1} \binom{tn}{k-1}.$$

□

Định lý 2.7. Số đường đi t -Dyck bậc n là

$$D(n, tn) = \frac{1}{tn+1} \binom{tn+n}{n}. \quad (10)$$

Chứng minh.

$$\begin{aligned} D(n, tn) &= \sum_{k=1}^n D(n, tn; k) \\ &= \sum_{k=1}^n \frac{1}{k} \binom{n-1}{k-1} \binom{tn}{k-1} \\ &= \frac{1}{tn+1} \binom{tn+n}{n}. \end{aligned}$$

□

Thật ra, số đường đi t -Dyck từ $(0, 0)$ tới (n, m) có đúng k đỉnh nhọn đã được I. P. Goulden và L. G. Serrano đưa ra trong tài liệu tham khảo 3.

Định lý 2.8. Số đường đi t -Dyck từ $(0, 0)$ tới (n, m) có k đỉnh nhọn là

$$D_t(n, m; k) = \frac{m+1-tn}{n} \binom{n}{k} \binom{m}{k-1}. \quad (11)$$

Định lý 2.9. Số đường đi t -Dyck từ $(0, 0)$ tới (n, m) là

$$D_t(n, m) = \frac{m - tn + 1}{n + 1} \binom{n + m}{n}. \quad (12)$$

Do chứng minh của hai định lý này khá phức tạp, ở đây xin lược bỏ. Bạn đọc quan tâm có thể tự tra cứu thêm.

2.6. Một định nghĩa tương đương khác của đường đi Dyck

Đường đi Dyck bậc n là đường đi trên lưới nằm phía trên trục x , dùng bước lên $U_1 = (1, 1)$ và bước xuống $D = (1, -1)$, đi từ $(0, 0)$ tới $(2n, 0)$.

Đường đi t -Dyck bậc n là đường đi trên lưới nằm phía trên trục x , dùng bước lên $U_t = (1, t)$ và bước xuống $D = (1, -1)$, đi từ $(0, 0)$ tới $((t + 1)n, 0)$.

Ký hiệu toàn bộ các đường đi t -Dyck bậc n là $\widetilde{\mathcal{D}}_t(n)$, với $\widetilde{D}_t(n) = \#\widetilde{\mathcal{D}}_t(n)$. Ký hiệu toàn bộ các đường đi t -Dyck có k đỉnh nhọn là $\widetilde{\mathcal{D}}_t(n; k)$, với $\widetilde{D}_t(n; k) = \#\widetilde{\mathcal{D}}_t(n; k)$.

Giữa $\widetilde{D}_t(n)$ và $D(n, tn)$ tồn tại một song ánh. Một cách dựng song ánh như sau: với P , đảo ngược thứ tự để được P' , rồi thay L bằng U_t và thay U bằng D . Ví dụ, nếu

$$P = UUULUL \in \mathcal{D}(2, 4),$$

thì

$$P' = LULUUU, \quad \widetilde{P} = U_t D U_t D D D.$$

3. Đường đi trên lưới không giao nhau

3.1. Đếm cặp đường đi Dyck không giao nhau bậc n

Định nghĩa 3.1. Hai đường đi Dyck P, Q từ $(0, 0)$ tới (n, n) được gọi là một cặp đường đi Dyck bậc n . Nếu Q luôn không xuyên qua P , thì (P, Q) là một cặp đường đi Dyck không giao nhau; ngược lại gọi là cặp đường đi Dyck giao nhau.

Định lý 3.1. Số cặp đường đi Dyck không giao nhau bậc n là

$$\begin{vmatrix} C_n & C_{n+1} \\ C_{n+1} & C_{n+2} \end{vmatrix}, \quad (13)$$

trong đó C_n là số Catalan thứ n .⁹

Chứng minh. Với một cặp đường đi Dyck bậc n bất kỳ (P, Q) , dựng ánh xạ $(P, Q) \mapsto (\widetilde{P}, \widetilde{Q})$, trong đó

$$\widetilde{P} = UUPLL, \quad \widetilde{Q} = Q.$$

Rõ ràng (P, Q) giao nhau khi và chỉ khi $(\widetilde{P}, \widetilde{Q})$ giao nhau. Vì vậy, số cặp đường đi không giao nhau (P, Q) bằng số cặp đường đi Dyck $(\widetilde{P}, \widetilde{Q})$ trừ đi số cặp giao nhau $(\widetilde{P}, \widetilde{Q})$. Số trước hiển nhiên là $C_{n+2}C_n$; ta xét cách tính số sau.

Với một cặp đường đi giao nhau bất kỳ $(\widetilde{P}, \widetilde{Q})$, gọi x là giao điểm đầu tiên. Điểm x chia $\widetilde{P}$ và $\widetilde{Q}$ thành hai đường con, ký hiệu $\widetilde{P}_1, \widetilde{P}_2$ và $\widetilde{Q}_1, \widetilde{Q}_2$.

⁹Công thức tổng quát: $C_n = \frac{1}{n+1} \binom{2n}{n}$.

Đặt

$$\theta(\tilde{P}, \tilde{Q}) = (\tilde{P}_1 \tilde{Q}_2, \tilde{Q}_1 \tilde{P}_2),$$

trong đó $\tilde{P}_1 \tilde{Q}_2$ là một đường đi Dyck bậc $n+1$ từ $(0,0)$ tới $(n+1, n+1)$, còn $\tilde{Q}_1 \tilde{P}_2$ là một đường đi Dyck bậc $n+1$ từ $(1,1)$ tới $(n+2, n+2)$.

Trong cùng một ô vuông $(n+2) \times (n+2)$, một đường đi Dyck bậc $n+1$ từ $(0,0)$ tới $(n+1, n+1)$ và một đường đi Dyck bậc $n+1$ từ $(1,1)$ tới $(n+2, n+2)$ chắc chắn có giao điểm. Chỉ cần cắt tại giao điểm đầu tiên rồi thực hiện cùng thao tác là có thể khôi phục $\tilde{P}$ và $\tilde{Q}$, nên θ là một song ánh.

Do đó số cặp giao nhau $(\tilde{P}, \tilde{Q})$ bằng số cặp $(\tilde{P}_1 \tilde{Q}_2, \tilde{Q}_1 \tilde{P}_2)$, tức C_{n+1}^2 . Vậy số cặp đường đi Dyck không giao nhau bậc n là

$$C_{n+2}C_n - C_{n+1}^2 = \begin{vmatrix} C_n & C_{n+1} \\ C_{n+1} & C_{n+2} \end{vmatrix}.$$

□

Thật ra, định lý này đã được M. DeSainte-Catherine và G. Viennot mở rộng trong tài liệu tham khảo 4 cho trường hợp k đường đi Dyck không giao nhau bậc n .

Định lý 3.2. Số cách chọn k đường đi Dyck không giao nhau bậc n là

$$\det(C_{n-i-j})_{i,j=1}^k = \begin{vmatrix} C_{n-2} & C_{n-3} & \cdots & C_{n-k} & C_{n-k-1} \\ C_{n-3} & C_{n-4} & \cdots & C_{n-k-1} & C_{n-k-2} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ C_{n-k-1} & C_{n-k-2} & \cdots & C_{n-2k+1} & C_{n-2k} \end{vmatrix}. \quad (14)$$

3.2. Đếm cặp đường đi tự do không giao nhau

Định nghĩa 3.2. Giả sử P, Q là hai đường đi tự do. Nếu Q luôn không xuyên qua P , ta gọi (P, Q) là một cặp đường đi tự do không giao nhau từ $(0,0)$ tới (n, m) , ký hiệu F_{nc} (*non-crossing*). Nếu P, Q không có điểm chung nào ngoài điểm đầu và điểm cuối, ta gọi (P, Q) là một cặp đường đi tự do không chạm nhau, ký hiệu F_{nt} (*non-touching*).

Định lý 3.3. Số cặp đường đi tự do không chạm nhau từ $(0,0)$ tới (n, m) là

$$F_{nt}(n, m) = \frac{1}{n} \binom{n+m-1}{n-1} \binom{n+m-2}{n-1}. \quad (15)$$

Chứng minh. Với một cặp đường đi tự do không chạm nhau bất kỳ (P, Q) , không mất tính tổng quát giả sử P nằm phía trên Q . Khi đó bước đầu tiên của P chắc chắn là U , bước cuối cùng chắc chắn là L . Với Q thì ngược lại: bước đầu tiên chắc chắn là L , bước cuối cùng chắc chắn là U .

Bỏ hai bước đầu và cuối của P, Q , nhận được một cặp đường đi tự do $(\tilde{P}, \tilde{Q})$, tức

$$P = U\tilde{P}L, \quad Q = L\tilde{Q}U.$$

Theo định nghĩa, P, Q không có điểm chung, nên $\tilde{P}, \tilde{Q}$ là cặp đường đi tự do không giao nhau. Rõ ràng cặp không giao nhau $(\tilde{P}, \tilde{Q})$ song ánh với cặp không chạm nhau (P, Q) .

Số cặp đường đi tự do không giao nhau $(\tilde{P}, \tilde{Q})$ bằng số cặp đường đi tự do $(\tilde{P}, \tilde{Q})$ trừ đi số cặp giao nhau $(\tilde{P}, \tilde{Q})$. Số trước là

$$F(n-1, m-1)^2 = \left(\binom{n+m-2}{n-1} \right)^2.$$

Ta xét cách tính số cặp giao nhau.

Gọi x là giao điểm đầu tiên của $\tilde{P}$ và $\tilde{Q}$. Điểm x chia $\tilde{P}$ và $\tilde{Q}$ thành hai đường con $\tilde{P}_1, \tilde{P}_2$ và $\tilde{Q}_1, \tilde{Q}_2$. Khi đó $\tilde{P}_1\tilde{Q}_2$ là đường đi tự do từ $(0, 1)$ tới $(n, m-1)$, còn $\tilde{Q}_1\tilde{P}_2$ là đường đi tự do từ $(1, 0)$ tới $(n-1, m)$.

Tương tự chứng minh định lý 3.1, ánh xạ

$$\theta(\tilde{P}, \tilde{Q}) = (\tilde{P}_1\tilde{Q}_2, \tilde{Q}_1\tilde{P}_2)$$

là một song ánh. Vì thế số cặp đường đi tự do giao nhau $(\tilde{P}, \tilde{Q})$ bằng số cặp đường đi tự do $(\tilde{P}_1\tilde{Q}_2, \tilde{Q}_1\tilde{P}_2)$, tức

$$F(n, m-2)F(n-2, m) = \binom{n+m-2}{n} \binom{n+m-2}{n-2}.$$

Do đó

$$\begin{aligned} F_{nt}(n, m) &= \binom{n+m-2}{n-1}^2 - \binom{n+m-2}{n} \binom{n+m-2}{n-2} \\ &= \frac{1}{n} \binom{n+m-1}{n-1} \binom{n+m-2}{n-1}. \end{aligned}$$

□

Định lý 3.4. Số cặp đường đi tự do không giao nhau từ $(0, 0)$ tới (n, m) là

$$F_{nc}(n, m) = \frac{1}{n+1} \binom{n+m+1}{n} \binom{n+m}{n}. \quad (16)$$

Chứng minh. Tương tự chứng minh định lý 3.3, ta xét dựng một cặp đường đi tự do không chạm nhau $(\tilde{P}, \tilde{Q})$ từ $(0, 0)$ tới $(n+1, m+1)$: đặt

$$\tilde{P} = UPL, \quad \tilde{Q} = LQU.$$

Ta nhận được song ánh $\theta^{-1}(\tilde{P}, \tilde{Q}) = (P, Q)$. Do đó

$$\begin{aligned} F_{nc}(n, m) &= F_{nt}(n+1, m+1) \\ &= \frac{1}{n+1} \binom{n+1+m+1-1}{n+1-1} \binom{n+1+m+1-2}{n} \\ &= \frac{1}{n+1} \binom{n+m+1}{n} \binom{n+m}{n}. \end{aligned}$$

□

3.3. Quan hệ giữa cặp đường đi tự do không chạm nhau và đường đi Dyck có k đỉnh nhọn

Ta có thể thấy số cặp đường đi tự do không chạm nhau từ $(0, 0)$ tới $(k, n-k+1)$ bằng số đường đi Dyck bậc n có k đỉnh nhọn,¹⁰ đều bằng

$$\frac{1}{k} \binom{n}{k-1} \binom{n-1}{k-1}.$$

Thật ra, ta có thể chứng minh giữa hai đối tượng này tồn tại một song ánh.

¹⁰Đó là số Narayana $N(n, k)$, xem dãy A001263 trong tài liệu tham khảo 10.

Chứng minh. Với một cặp đường đi tự do không chạm nhau bất kỳ (P, Q) từ $(0, 0)$ tới $(k, n - k + 1)$, khai triển chúng thành dãy U, L :

$$P = \underbrace{UU \cdots UL}_{i_1} \underbrace{U \cdots UL}_{i_2} \cdots L \underbrace{U \cdots UL}_{i_k},$$

$$Q = L \underbrace{U \cdots UL}_{j_1} \underbrace{U \cdots UL}_{j_2} \cdots L \underbrace{U \cdots UL}_{j_k}.$$

Khi đó

$$\sum_{t=1}^k i_t = n - k, \quad \sum_{t=1}^k j_t = n - k.$$

Đặt

$$R = \rho(P, Q) = \underbrace{U \cdots UL}_{i_1+1} \underbrace{L \cdots UL}_{j_1+1} \underbrace{U \cdots UL}_{i_2+1} \underbrace{L \cdots L}_{j_2+1} \cdots \underbrace{U \cdots UL}_{i_k+1} \underbrace{L \cdots L}_{j_k+1}.$$

Khi đó R là một đường đi lưới bậc n có k đỉnh nhọn. Vì (P, Q) là cặp đường đi tự do không chạm nhau, ta có

$$\begin{aligned} i_1 + 1 &> j_1, \\ i_1 + i_2 + 1 &> j_1 + j_2, \\ &\vdots \\ \sum_{t=1}^k i_t + 1 &> \sum_{t=1}^k j_t, \quad \sum_{t=1}^k i_t \geq \sum_{t=1}^k j_t. \end{aligned}$$

Vậy R là một đường đi Dyck. Một ví dụ dựng R từ P, Q là

$$\begin{aligned} P &= ULUUULLUUL, & Q &= LLULUULUUU, \\ \{i\} &= \{0, 3, 0, 2\}, & \{j\} &= \{0, 1, 2, 2\}, \\ R &= ULUUUULLULLLUUULLL. \end{aligned}$$

Cách dựng ánh xạ ngược của ρ và chứng minh cũng tương tự, nên ρ là một song ánh. $\square$

4. Ứng dụng thực tế

Histogram là một công cụ thường gặp trong thống kê. Nó có thể thể hiện trạng thái dao động của đối tượng thống kê, qua đó truyền đạt trực quan thông tin về trạng thái của quá trình. Sau khi nghiên cứu trạng thái dao động của đối tượng thống kê, ta có thể nắm được trạng thái của quá trình và cải tiến tốt hơn. Ngoài phân tích thủ công, hình dạng của histogram cũng có thể được chuyển hóa thành đường đi trên lưới, chẳng hạn đường đi Dyck, để nâng cao hiệu suất phân tích.

4.1. Dựng song ánh

Định nghĩa 4.1. Với một đường đi Dyck P , định nghĩa độ cao của mỗi bước trong P là giá trị tung độ lớn hơn trong hai đầu mút của bước ấy. Dãy gồm độ cao của mọi bước trong P được gọi là dãy độ cao của P .

Ta xét cách dựng ánh xạ φ từ đường đi Dyck tới histogram. Trong dãy độ cao của P , xét mỗi đoạn liên tiếp cực đại có cùng độ cao. Giả sử một đoạn cực đại có độ cao h , độ dài c ; ta ánh xạ nó thành $\lfloor c/2 \rfloor$ cột có độ cao h . Dãy nhận được chính là dãy độ cao của histogram $\varphi(P)$.

Ví dụ, với đường đi Dyck

$$P = UDUDUUUDUDDUDUUDUDDUDDUDUDD,$$

dãy độ cao của nó là

$$1111123333222233332221111221.$$

Ta nhận được dãy độ cao của histogram tương ứng $\varphi(P)$:

$$113322332112.$$



Hình 5: Một đường đi Dyck và histogram tương ứng của nó.

Tiếp theo, ta xét ánh xạ ngược φ^{-1} . Giả sử dãy độ cao của histogram B là

$$h_0^{a_0} h_1^{a_1} \dots h_{k+1}^{a_{k+1}}.$$

Bằng cách chèn các $a_i = 0$ ở vị trí thích hợp, có thể giả sử

$$h_0 = h_{k+1} = 0, \quad |h_i - h_{i+1}| = 1 \quad (0 \leq i \leq k).$$

Với $i = 1, 2, \dots, k$, thay lần lượt $h_i^{a_i}$ như sau, ta thu được đường đi Dyck $\varphi^{-1}(B)$:

$$\begin{cases} (UD)^{a_i} U, & h_{i-1} < h_i \wedge h_i < h_{i+1}, \\ (UD)^{a_i}, & h_{i-1} < h_i \wedge h_i > h_{i+1}, \\ (DU)^{a_i}, & h_{i-1} > h_i \wedge h_i < h_{i+1}, \\ (DU)^{a_i} D, & h_{i-1} > h_i \wedge h_i > h_{i+1}. \end{cases}$$

Ví dụ, với histogram $B = \varphi(P)$ ở trên, dãy độ cao 113322332112 có thể được viết lại theo dạng trên, và nhận được đường đi Dyck tương ứng

$$\varphi^{-1}(B) = (UD)^2 U U (UD)^2 (DU)^2 (UD)^2 (DU) D (DU)^2 (UD) D = P.$$

Vì vậy φ là một song ánh.

4.2. Khai thác tính chất

Trước hết định nghĩa một số đại lượng thống kê trên histogram và đường đi Dyck.

Với histogram $B \in \mathcal{B}$, có thể xem nó là một đường đi trên lưới gồm bước ngang $H = (1, 0)$, bước lên $U = (0, 1)$, và bước xuống $D = (0, -1)$. Ký hiệu $\#H(B)$, $\#U(B)$, $\#D(B)$ lần lượt là số bước H , U , D trong B . Định nghĩa nửa chu vi (*semiperimeter*) của B là tổng số bước lên và bước ngang, ký hiệu

$$\text{sp}(B) = \#H(B) + \#U(B).$$

Định nghĩa diện tích (*area*) của B là diện tích vùng kín do B và trục x bao lại, ký hiệu $\text{area}(B)$.

Định nghĩa một đỉnh nhọn (*peak*) là một lần xuất hiện của $UH^\ell D$, với $\ell \geq 1$. Ký hiệu $\text{pk}(B)$ là số đỉnh nhọn trong B . Tương tự, định nghĩa một đáy (*valley*) là một lần xuất hiện của $DH^\ell U$, với $\ell \geq 1$, và ký hiệu $\text{val}(B)$ là số đáy trong B . Vì đỉnh nhọn và đáy luôn xuất hiện xen kẽ, đồng thời phần tử đầu và cuối chắc chắn đều là đỉnh nhọn, nên

$$\text{pk}(B) = \text{val}(B) + 1.$$

Gọi $\text{shv}(B)$ là tổng độ cao của các đáy trong B , và $\#H_1(B)$ là số đáy có độ cao 1 trong B .

Với đường đi Dyck $P \in \mathcal{D}$, định nghĩa đỉnh nhọn là một lần xuất hiện của UD , và hồi quy (*return*) là một bước xuống khiến đường đi Dyck chạm lại trục x . Ký hiệu $\Lambda(P)$ và $\text{ret}(P)$ lần lượt là số đỉnh nhọn và số hồi quy trong P . Gọi $\text{shp}(P)$ là tổng độ cao của các đỉnh nhọn trong P .

Với histogram B hoặc đường đi Dyck P , định nghĩa $\text{iniU}(B)$ (hoặc $\text{iniU}(P)$) và $\text{finD}(B)$ (hoặc $\text{finD}(P)$) lần lượt là số bước lên U ban đầu và số bước xuống D kết thúc.

Ví dụ 1. Với histogram B trong hình trên, ta có

$$\begin{aligned}\#H(B) &= 12, & \#H_1(B) &= 4, & \#U(B) &= 5, & \#D(B) &= 5, \\ \text{sp}(B) &= \#H(B) + \#U(B) = 17, & \text{iniU}(B) &= 1, & \text{finD}(B) &= 2, \\ \text{area}(B) &= 24, & \text{pk}(B) &= 3, & \text{shv}(B) &= 3.\end{aligned}$$

Ta có định lý sau.

Định lý 4.1. Với $P \in \mathcal{D}$ và $B = \varphi(P) \in \mathcal{B}$, các kết luận sau đúng:

- $\text{sl}(P) = \text{sp}(B) - \text{pk}(B)$;
- $\Lambda(P) = \#H(B) - \text{val}(B)$;
- $\text{shp}(P) = \text{area}(B) - \text{shv}(B)$;
- $\text{iniU}(P) = \text{iniU}(B)$;
- $\text{finD}(P) = \text{finD}(B)$;
- nếu độ cao lớn nhất của P là 1, thì $\text{ret}(P) = \#H_1(B)$, ngược lại $\text{ret}(P) = \#H_1(B) + 1$.

Các kết luận này đều không khó chứng minh; chỉ cần suy ra trực tiếp từ định nghĩa của ánh xạ. Ở đây ta lược bỏ, bạn đọc quan tâm có thể tự suy diễn.

Bảng sau tổng hợp các ký hiệu được định nghĩa trong mục này và quan hệ giữa chúng:

Histogram	Ý nghĩa	Dyck	Quan hệ
$\text{sp}(B)$	nửa chu vi của B	$\text{sl}(P)$	$\text{sl}(P) = \text{sp}(B) - \text{pk}(B)$
$\text{pk}(B)$	số đỉnh nhọn trong B	$\Lambda(P)$	$\Lambda(P) = \#H(B) - \text{val}(B)$
$\text{val}(B)$	số đáy trong B	$\text{ret}(P)$	$\text{ret}(P) = \#H_1(B) + 1$, khi độ cao của P không phải 1
$\text{shv}(B)$	tổng độ cao đáy trong B	$\text{shp}(P)$	$\text{shp}(P) = \text{area}(B) - \text{shv}(B)$
$\text{iniU}(B)$	số bước U ban đầu	$\text{iniU}(P)$	$\text{iniU}(P) = \text{iniU}(B)$
$\text{finD}(B)$	số bước D kết thúc	$\text{finD}(P)$	$\text{finD}(P) = \text{finD}(B)$

5. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã hướng dẫn.

Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã ủng hộ và hướng dẫn.

Xin cảm ơn các thầy cô, bạn học và bạn bè đã giúp đỡ tôi.

Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi chu đáo.

Tài liệu tham khảo

1. Blanco S. A.; Petersen T. K. Counting Dyck paths by area and rank. *Annals of Combinatorics*, 18(2): 171–197, 2014.
2. Chen W. Y. C.; Pang S. X. M.; Qu E. X. Y.; et al. Pairs of noncrossing free Dyck paths and noncrossing partitions. *Discrete Mathematics*, 309(9): 2834–2838, 2009.
3. Goulden I. P.; Serrano L. G. Maintaining the spirit of the reflection principle when the boundary has arbitrary integer slope. *Journal of Combinatorial Theory, Series A*, 104(2): 317–326, 2003.
4. DeSainte-Catherine M.; Viennot G. Enumeration of certain Young tableaux with bounded height. In *Combinatoire énumérative*, pp. 58–67. Springer, Berlin, Heidelberg, 1986.
5. Deutsch E.; Elizalde S. A bijection between bargraphs and Dyck paths. *Discrete Applied Mathematics*, 251: 340–344, 2018.
6. Lu Qinglin; Chang Haiting. Đếm tham số cộng tính của đường đi Dyck tổng quát. *Journal of Xuzhou Normal University (Natural Science)*, 2009(1): 3.
7. Sun Xuezhi. Một số bài toán đếm trên hai loại đường đi Dyck tổng quát. Luận văn, Đại học Sư phạm Hoa Đông, 2014.
8. Sun Yidong; Jia Cangzhi. Đếm đường đi Dyck có đỉnh nhọn tăng nghiêm ngặt. *Journal of Mathematical Research and Exposition*, 2007(2): 6.
9. Zhang Xiaoguang. Nghiên cứu bài toán đường đi K trong tổ hợp. Luận văn, Đại học Công nghệ Đại Liên, 2008.
10. Sloane, N. J. A. On-Line Encyclopedia of Integer Sequences, <http://oeis.org>, 2002.

Mở rộng một số bài toán số học trong thi thuật toán và bước đầu về số nguyên Gauss

Li Jiaheng, Trường Trung học trực thuộc Đại học Công nghiệp Tây Bắc

Tóm tắt

Bài viết bắt đầu từ một bài mô phỏng NOIP nội bộ, khảo sát ngắn gọn một số đặc điểm của lý thuyết chia hết trên các cấu trúc số học, rồi thử chuyển lý thuyết chia hết sang số nguyên Gauss. Từ đó, bài viết mở rộng một số bài toán số học thường gặp trong thi thuật toán sang phạm vi số nguyên Gauss.

1. Kiến thức chuẩn bị

Để tiện cho việc thảo luận và giúp người đọc theo dõi, mục này nêu một số khái niệm trong đại số trừu tượng được dùng trong bài, cùng các tính chất đơn giản của chúng. Bạn đọc đã có kiến thức liên quan có thể bỏ qua phần tương ứng.

1.1. Phép toán hai ngôi và hệ đại số

Định nghĩa 1.1.1 (Phép toán hai ngôi). Cho một tập X . Ta gọi ánh xạ $f : X^2 \rightarrow X$ là một phép toán hai ngôi trên X . Tức với mỗi cặp có thứ tự $(a, b) \in X^2$, tồn tại một $h \in X$ xác định tương ứng với nó, ký hiệu

$$h = f(a, b)$$

hoặc

$$h = a \circ b.$$

Định nghĩa 1.1.2 (Tính giao hoán và tính kết hợp). Với phép toán hai ngôi $\circ$ định nghĩa trên tập X , nếu với mọi $a, b \in X$ đều có

$$a \circ b = b \circ a,$$

thì gọi phép toán hai ngôi này là giao hoán. Nếu với mọi $a, b, c \in X$ đều có

$$(a \circ b) \circ c = a \circ (b \circ c), \tag{17}$$

thì gọi phép toán hai ngôi này là kết hợp.

Định nghĩa 1.1.3. Cho $S \subset X$, và $\circ$ là một phép toán hai ngôi trên X . Nếu với mọi $a, b \in S$ đều có

$$a \circ b \in S,$$

thì nói tập S đóng với phép toán $\circ$.

Định nghĩa 1.1.4 (Hệ đại số). Một tập X trên đó đã định nghĩa một số phép toán hai ngôi $\circ, *, \dots$ được gọi là một hệ đại số. Chẳng hạn hệ đại số trên có thể ký hiệu là $(X, \circ, *, \dots)$.

1.2. Nửa nhóm và nhóm

Định nghĩa 1.2.1 (Nửa nhóm và nửa nhóm giao hoán). Ta gọi hệ đại số $(X, \circ)$ là một nửa nhóm khi và chỉ khi phép toán hai ngôi $\circ$ có tính kết hợp. Đặc biệt, nếu phép toán hai ngôi $\circ$ trong nửa nhóm là giao hoán, thì gọi $(X, \circ)$ là nửa nhóm giao hoán.

Định nghĩa 1.2.2 (Phần tử đơn vị của nửa nhóm, nửa nhóm có đơn vị). Phần tử e trong nửa nhóm $(X, \circ)$ được gọi là phần tử đơn vị nếu với mọi $a \in X$ đều có

$$a \circ e = e \circ a = a. \quad (18)$$

Nửa nhóm có phần tử đơn vị được gọi là nửa nhóm có đơn vị.

Định lý 1.2.1. Phần tử đơn vị trong một nửa nhóm hoặc không tồn tại, hoặc tồn tại duy nhất.

Chứng minh. Giả sử e, e' đều là phần tử đơn vị của nửa nhóm $(X, \circ)$. Khi đó theo (18),

$$e = e \circ e' = e',$$

mâu thuẫn với $e \neq e'$. Vì vậy nếu phần tử đơn vị tồn tại thì nó phải là duy nhất. $\square$

Định nghĩa 1.2.3 (Phần tử khả nghịch). Giả sử nửa nhóm có đơn vị $(X, \circ)$ có phần tử đơn vị là e . Với phần tử $a \in X$, nếu tồn tại $b \in X$ sao cho

$$a \circ b = b \circ a = e, \quad (19)$$

thì gọi a là phần tử khả nghịch, hay phần tử đơn vị theo nghĩa nhân, và gọi b là phần tử nghịch đảo của a , ký hiệu $b = a^{-1}$.

Định lý 1.2.2. Mỗi phần tử khả nghịch trong nửa nhóm có nghịch đảo duy nhất.

Chứng minh. Giả sử b, b' đồng thời là nghịch đảo của phần tử a trong nửa nhóm $(X, \circ)$. Theo (17), (18), (19), ta có

$$b' = (b \circ a) \circ b' = b \circ (a \circ b') = b,$$

mâu thuẫn với $b \neq b'$. Vì vậy nếu nghịch đảo tồn tại thì nó phải là duy nhất. $\square$

Định nghĩa 1.2.4 (Nhóm và nhóm giao hoán). Cho $(X, \circ)$ là một nửa nhóm. Nếu mọi phần tử của nó đều khả nghịch, thì gọi $(X, \circ)$ là một nhóm. Đặc biệt, nếu phép toán hai ngôi $\circ$ là giao hoán, thì gọi nó là nhóm giao hoán.

1.3. Vành

Định nghĩa 1.3.1 (Vành và vành giao hoán). Cho một tập X , trên đó định nghĩa hai phép toán hai ngôi $\oplus$ và $\odot$. Hệ đại số $(X, \oplus, \odot)$ được gọi là một vành nếu nó thỏa các điều kiện sau:

1. $(X, \oplus)$ là một nhóm giao hoán, thường gọi là nhóm cộng của vành;
2. $(X, \odot)$ là một nửa nhóm, thường gọi là nửa nhóm nhân của vành;
3. với mọi $a, b, c \in X$, ta có

$$a \odot (b \oplus c) = (a \odot b) \oplus (a \odot c),$$

$$(b \oplus c) \odot a = (b \odot a) \oplus (c \odot a).$$

Đặc biệt, nếu phép toán $\odot$ cũng giao hoán, thì gọi hệ đại số này là một vành giao hoán.

Định nghĩa 1.3.2 (Phần tử không). Nếu nhóm cộng $(X, \oplus)$ của vành $(X, \oplus, \odot)$ có phần tử đơn vị o , thì gọi o là phần tử không của vành X .

Định nghĩa 1.3.3 (Phần tử đơn vị). Nếu nửa nhóm nhân $(X, \odot)$ của vành $(X, \oplus, \odot)$ có phần tử đơn vị e , thì gọi e là phần tử đơn vị của vành X .

Định nghĩa 1.3.4 (Đơn vị). Phần tử khả nghịch ϵ của nửa nhóm nhân $(X, \odot)$ của vành $(X, \oplus, \odot)$ được gọi là đơn vị của vành X .

Định nghĩa 1.3.5 (Phần tử liên kết). Cho ϵ là một đơn vị của vành $(X, \oplus, \odot)$. Với $a, b \in X$, nếu $a \odot \epsilon = b$, thì gọi a và b là hai phần tử liên kết với nhau, ký hiệu $a \sim b$.

Hiển nhiên quan hệ liên kết là một quan hệ tương đương, tức có tính đối xứng, bắc cầu và phản xạ.

Định nghĩa 1.3.6 (Ước không). Ký hiệu phần tử không của vành $(X, \oplus, \odot)$ là o . Nếu $o \neq a, b \in X$ và $a \odot b = o$, thì gọi a là ước không trái của X , và b là ước không phải của X . Rõ ràng trong vành giao hoán, ước không trái và phải trùng nhau.

Định nghĩa 1.3.7 (Miền nguyên). Cho X là một vành giao hoán có ít nhất hai phần tử. Nếu X có phần tử đơn vị và không có ước không, thì gọi X là một miền nguyên.

2. Một số ký hiệu và quy ước

Trong bài viết này, nếu không nói khác đi, ta không phân biệt tập M và hệ đại số xây dựng trên M . Ý nghĩa cụ thể có thể suy ra từ ngữ cảnh.

Để diễn đạt ngắn gọn, khi không gây nhập nhằng, ta dùng o, e, ϵ lần lượt để chỉ phần tử không, phần tử đơn vị và đơn vị trong hệ đại số tương ứng.

$\mathbb{Z}[i]$ ký hiệu tập $\{a + bi \mid a, b \in \mathbb{Z}\}$, trong đó $i^2 = -1$ là đơn vị ảo, và các số trong $\mathbb{Z}[i]$ được gọi là số nguyên Gauss. Các ký hiệu như $\mathbb{Q}[i]$ được hiểu tương tự.

3. Dẫn nhập bài toán

3.1. Mô tả đề bài

Cho hai số khác không $x, y \in \mathbb{Z}[i]$. Cần tìm và xuất ra số phức có mô-đun nhỏ nhất trong tất cả các số phức khác không có dạng $ax + by$, trong đó $a, b \in \mathbb{Z}[i]$.

Bảo đảm giá trị tuyệt đối của phần thực và phần ảo của x, y không vượt quá 10^9 .

3.2. Thảo luận

Bài này đến từ kỳ thi mô phỏng NOIP năm 2018 của trường tôi.

Đề gốc có một phần điểm cho trường hợp phần ảo của x, y đều bằng 0, và dạng $ax + by$ rất dễ gợi nhớ tới định lý Bezout. Vì vậy với phần điểm này chỉ cần xuất ước chung lớn nhất của hai số nguyên hữu tỉ.

Nay bài toán được mở rộng từ số nguyên hữu tỉ một chiều sang $\mathbb{Z}[i]$ hai chiều. Một ý tưởng tự nhiên là xét riêng từng chiều. Tiếc rằng cách này sai. Ví dụ, với $x = 1 + 2i, y = 3 + i$, nếu lấy ước chung lớn

nhất theo từng chiều thì đáp án nhận được là $1 + i$; nhưng chú ý rằng $y = (1 - i)x$, nên

$$|ax + by| = |ax + (1 - i)bx| \geq |x| = \sqrt{5}.$$

Rõ ràng $|1 + i| = \sqrt{2} < \sqrt{5}$, mâu thuẫn.

Một ý tưởng khác là liệu có thể xây dựng trên $\mathbb{Z}[i]$ một lý thuyết chia hết giống như trên số nguyên hữu tỉ, từ đó dùng kết luận tương tự định lý Bezout để giải quyết bài toán hay không.

Vì vậy, trước hết ta cần hiểu lý thuyết chia hết trong số nguyên hữu tỉ, rồi sau khi nắm đủ cấu trúc số học của nó, thử mở rộng lý thuyết ấy.

4. Lý thuyết chia hết trong số nguyên hữu tỉ $\mathbb{Z}$

Dù phần lớn bạn đọc bài này đã biết lý thuyết chia hết của số nguyên, để lập luận được đầy đủ, mục này liệt kê không chứng minh một số khái niệm cơ bản và tính chất đơn giản của lý thuyết chia hết trong số nguyên.

Từ định nghĩa dễ thấy $(\mathbb{Z}, +, \times)$ là một miền nguyên. Để thống nhất với phần sau, mục này sẽ diễn đạt một số khái niệm bằng ngôn ngữ miền nguyên.

Định nghĩa 4.1 (Chia hết). Cho $a, b \in \mathbb{Z}$ và $a \neq 0$. Nếu tồn tại $x \in \mathbb{Z}$ sao cho $b = ax$, thì nói a chia hết b , ký hiệu $a \mid b$. Đồng thời gọi a là một ước của b , và b là một bội của a .

Với $0 \neq b \in \mathbb{Z}$, rõ ràng ϵ và $\pm b$ đều là các ước hiển nhiên của b ; các ước khác của b được gọi là ước không hiển nhiên.

Định nghĩa 4.2 (Số bất khả quy). Cho $p \in \mathbb{Z}$, $p \neq 0, \pm 1$. Nếu p không có ước không hiển nhiên, thì gọi p là một số bất khả quy trong $\mathbb{Z}$.

Định nghĩa 4.3 (Số nguyên tố). Nếu $p \in \mathbb{Z}$ thỏa: với mọi $a, b \in \mathbb{Z}$, từ $p \mid ab$ luôn suy ra $p \mid a$ hoặc $p \mid b$, thì gọi p là một số nguyên tố trong $\mathbb{Z}$.

Như ta đã biết, trong $\mathbb{Z}$, số bất khả quy và số nguyên tố trùng nhau, nên ta không phân biệt chúng.¹¹ Trong phần sau, để tránh nhầm lẫn, số nguyên tố trong $\mathbb{Z}$ sẽ được gọi là số nguyên tố hữu tỉ.

Định lý 4.1 (Phép chia có dư). Cho $a, b \in \mathbb{Z}$ và $a \neq 0$. Khi đó tồn tại duy nhất một cặp $q, r \in \mathbb{Z}$ sao cho

$$b = qa + r, \quad 0 \leq r < |a|.$$

Định nghĩa 4.4 (Ước chung). Cho $a, b \in \mathbb{Z}$. Nếu $d \in \mathbb{Z}$, $d \mid a$, $d \mid b$, thì gọi d là một ước chung của a, b .

Định nghĩa 4.5 (Ước chung lớn nhất). Cho $a, b \in \mathbb{Z}$ không đồng thời bằng 0. Ước lớn nhất trong các ước chung của a, b được gọi là ước chung lớn nhất của a, b , ký hiệu $\gcd(a, b)$.

Định lý 4.2 (Định lý Bezout). Cho $a, b \in \mathbb{Z}$. Khi đó tồn tại $x, y \in \mathbb{Z}$ sao cho

$$ax + by = \gcd(a, b).$$

¹¹Thật ra, trong lý thuyết số, “số nguyên tố” và “số bất khả quy” là hai khái niệm khác nhau.

Định lý 4.3 (Định lý cơ bản của số học). Cho $a \in \mathbb{Z}$, $a \neq 0, \pm 1$. Khi đó a có thể được biểu diễn duy nhất dưới dạng

$$a = \epsilon p_1^{m_1} \cdots p_s^{m_s},$$

trong đó $p_1 < p_2 < \cdots < p_s$ là các số nguyên tố hữu tỉ dương khác nhau.

Vì quan hệ liên kết là một quan hệ tương đương, ta có thể chia toàn bộ phần tử của $\mathbb{Z}$ thành các lớp tương đương đôi một rời nhau theo quan hệ liên kết, rồi chỉ định một phần tử đại diện trong mỗi lớp. Như vậy ta nhận được một tập đại diện của $\mathbb{Z}$, chẳng hạn $\mathbb{N}$. Do các phần tử liên kết có cùng tính chất chia hết, khi thảo luận các tính chất chỉ liên quan tới chia hết, ta có thể chỉ xét trong một tập đại diện nào đó.

5. Cấu trúc số học của hệ lý thuyết chia hết

Không khó nhận thấy phần lớn khái niệm và tính chất ở mục 4 được xây dựng trên cơ sở $(\mathbb{Z}, +, \times)$ là một miền nguyên, chứ không dùng những tính chất riêng của tập số nguyên $\mathbb{Z}$, chẳng hạn tính rời rạc hay quan hệ lớn nhỏ. Vì vậy, giả sử $(M, \oplus, \odot)$ là một miền nguyên, ta có thể mở rộng các khái niệm trong mục 4.

Để tiện, với $\alpha, \beta \in M$, ta dùng $\alpha\beta$ để chỉ $\alpha \odot \beta$.

Định nghĩa 5.1 (Chia hết). Cho $\alpha, \beta \in M$ và $\alpha \neq o$. Nếu tồn tại $\tau \in M$ sao cho $\beta = \alpha\tau$, thì nói α chia hết β , ký hiệu $\alpha \mid \beta$. Đồng thời gọi α là một ước của β , và β là một bội của α .

Với $o \neq \beta \in M$, rõ ràng ϵ và $\epsilon\beta$ đều là các ước hiển nhiên của β ; các ước khác của β được gọi là ước không hiển nhiên.

Định nghĩa 5.2 (Số bất khả quy). Cho $\xi \in M$, $\xi \neq o, \epsilon$. Nếu ξ không có ước không hiển nhiên, thì gọi ξ là một số bất khả quy trong M .

Định nghĩa 5.3 (Số nguyên tố). Cho $\xi \in M$. Nếu với mọi $\alpha, \beta \in M$, từ $\xi \mid \alpha\beta$ đều suy ra $\xi \mid \alpha$ hoặc $\xi \mid \beta$, thì gọi ξ là một số nguyên tố.

Định lý 5.1. Mọi số nguyên tố đều là số bất khả quy.

Chứng minh. Giả sử số nguyên tố $\xi = \alpha\beta$ không bất khả quy, trong đó $\alpha, \beta \neq \epsilon$. Theo định nghĩa số nguyên tố, phải có $\xi \mid \alpha$ hoặc $\xi \mid \beta$.

Không mất tính tổng quát, giả sử $\xi \mid \alpha$. Kết hợp với $\alpha \mid \xi$ ta có $\xi \sim \alpha$, từ đó $\beta = \epsilon$, mâu thuẫn. Vì vậy mọi số nguyên tố đều bất khả quy. $\square$

Định nghĩa 5.4 (Ước chung). Cho $\alpha, \beta \in M$. Nếu $\delta \in M$, $\delta \mid \alpha$ và $\delta \mid \beta$, thì gọi δ là một ước chung của α, β .

Khi thử mở rộng khái niệm ước chung lớn nhất, ta gặp một vấn đề: giữa các phần tử của miền nguyên M chưa chắc đã định nghĩa quan hệ lớn nhỏ. Do đó ta cần một cách khác để mô tả ước chung “lớn nhất”.

Định nghĩa 5.5 (Ước chung lớn nhất). Cho $\alpha, \beta \in M$ không đồng thời bằng o . Nếu tồn tại $\Delta \in M$ thỏa $\Delta \mid \alpha$, $\Delta \mid \beta$, và với mọi ước chung δ của α, β đều có $\delta \mid \Delta$, thì gọi Δ là ước chung lớn nhất của α, β , ký hiệu $\gcd(\alpha, \beta)$.

Cần chỉ ra rằng định nghĩa này không bảo đảm sự tồn tại của Δ .¹² Tuy nhiên, có thể chứng minh rằng nếu Δ tồn tại thì nó tất yếu là duy nhất theo nghĩa liên kết.

Định nghĩa 5.6 (Miền nguyên phân tích được). Nếu với mọi $\alpha \in M$, $\alpha \neq o, \epsilon$, α chắc chắn có thể và chỉ có thể biểu diễn thành tích của hữu hạn số bất khả quy trong M :

$$\alpha = \xi_1 \cdots \xi_r, \quad (20)$$

thì gọi M là một miền nguyên phân tích được. Ở đây “chỉ có thể biểu diễn” nghĩa là không tồn tại một biểu diễn như sau: với một số nguyên dương n bất kỳ cho trước, α có thể biểu diễn thành tích của n phần tử không phải đơn vị, và trong đó có phần tử không bất khả quy.

Định lý 5.2. Nếu với mọi $\alpha \neq o \in M$, α chỉ có hữu hạn ước đôi một không liên kết, thì M là một miền nguyên phân tích được.

Chứng minh. Ký hiệu $n(\alpha)$ là số ước không liên kết của α . Rõ ràng khi $\alpha \neq o, \epsilon$ luôn có $n(\alpha) \geq 2$.

Khi $n(\alpha) = 2$, α là bất khả quy, kết luận đúng.

Giả sử kết luận đúng khi $2 \leq n(\alpha) < k$. Khi $n(\alpha) = k$, α chắc chắn không bất khả quy, tức tồn tại $\alpha = \beta\gamma$ và $\beta, \gamma \neq \epsilon$. Hiển nhiên $n(\alpha) > n(\beta), n(\gamma)$. Theo giả thiết quy nạp, β, γ đều biểu diễn được thành tích các số bất khả quy, nên α cũng vậy. Kết luận được chứng minh. $\square$

Với miền nguyên phân tích được, ta có các mệnh đề sau.

Mệnh đề 1. Cho $\alpha, \beta \in M$ không đồng thời bằng o . Khi đó $\Delta = \gcd(\alpha, \beta)$ tồn tại, và tồn tại $x, y \in M$ sao cho

$$\alpha x \oplus \beta y = \Delta.$$

Mệnh đề 2. Mọi số bất khả quy trong M đều là số nguyên tố.

Mệnh đề 3. Phân tích (20) là duy nhất nếu không kể thứ tự và liên kết. Cụ thể, nếu M_0 là một tập đại diện của M , thì α có thể được biểu diễn duy nhất, không kể thứ tự, dưới dạng

$$\alpha = \epsilon \eta_1^{n_1} \cdots \eta_s^{n_s},$$

trong đó $\eta_j \in M_0$ là các số bất khả quy đôi một khác nhau.

Định nghĩa 5.7 (Miền nguyên phân tích duy nhất). Ta gọi miền nguyên phân tích được thỏa mệnh đề 3 là miền nguyên phân tích duy nhất.

Định lý 5.3. Nếu mệnh đề 2 đúng trong M , $\xi_1, \xi_2 \in M$ là hai số nguyên tố khác nhau, $\alpha \in M$, và $\xi_1 \mid \alpha, \xi_2 \mid \alpha$, thì $\xi_1 \xi_2 \mid \alpha$.

Chứng minh. Từ $\xi_1 \mid \alpha$, đặt $\alpha = \xi_1 \beta$. Rõ ràng $\xi_2 \nmid \xi_1$. Theo mệnh đề 2, $\xi_2 \mid \beta$, do đó $\xi_1 \xi_2 \mid \alpha$. $\square$

Định lý 5.4. Các mệnh đề 1 đến 3 đôi một tương đương.

Chứng minh định lý này có thể xem trong phần liên quan của tài liệu tham khảo [1].

¹²Thật vậy, trong một số miền nguyên có trường hợp Δ không tồn tại.

6. Lý thuyết chia hết trong số nguyên Gauss $\mathbb{Z}[i]$

Hiển nhiên $(\mathbb{Z}[i], +, \times)$ là một miền nguyên, nên ta có thể đưa các định nghĩa 5.1 đến 5.5 của mục trước vào đây, và không nhắc lại nữa.

Định nghĩa 6.1 (Chuẩn). Cho $\alpha = a + bi \in \mathbb{Q}[i]$. Ta gọi $N(\alpha) = a^2 + b^2$ là chuẩn của α .

Định lý 6.1. Cho $\alpha, \beta \in \mathbb{Q}[i]$. Khi đó

$$N(\alpha\beta) = N(\alpha)N(\beta).$$

Nếu $\alpha \mid \beta$ thì $N(\alpha) \mid N(\beta)$.

Định lý 6.2. Cho $n \in \mathbb{N}$. Trong $\mathbb{Z}[i]$, số phần tử có chuẩn bằng n là hữu hạn.

Hai định lý trên là hiển nhiên, nên ở đây không chứng minh. Từ định lý 6.2 và định lý 5.2 suy ra:

Định lý 6.3. $\mathbb{Z}[i]$ là miền nguyên phân tích được.

Định lý 6.4 (Phép chia có dư). Cho $\alpha, \beta \in \mathbb{Z}[i]$ và $\alpha \neq 0$. Khi đó chắc chắn tồn tại $\eta, \gamma \in \mathbb{Z}[i]$ sao cho

$$\beta = \eta\alpha + \gamma, \quad 0 \leq N(\gamma) < N(\alpha). \quad (21)$$

Chứng minh. Đặt $\tau = r + si \in \mathbb{Q}[i]$ và $\beta = \tau\alpha$. Lấy $u, v \in \mathbb{Z}$ lần lượt là các số nguyên gần r, s nhất. Khi đó $|r - u| \leq \frac{1}{2}$ và $|s - v| \leq \frac{1}{2}$.

Đặt $\eta = u + vi$. Khi đó

$$N(\tau - \eta) = |r - u|^2 + |s - v|^2 \leq \frac{1}{2}.$$

Do $\gamma = (\tau - \eta)\alpha$, ta có

$$N(\gamma) = N(\tau - \eta)N(\alpha) \leq \frac{1}{2}N(\alpha) < N(\alpha).$$

□

Định lý 6.5. Mệnh đề 1 đúng trong $\mathbb{Z}[i]$. Tức với $\alpha, \beta \in \mathbb{Z}[i]$ không đồng thời bằng 0, $\Delta = \gcd(\alpha, \beta)$ chắc chắn tồn tại, và tồn tại $x, y \in \mathbb{Z}[i]$ sao cho

$$\alpha x + \beta y = \Delta.$$

Chứng minh. Đặt $S = \{\alpha x + \beta y \mid x, y \in \mathbb{Z}[i]\}$, và gọi δ là phần tử khác không có chuẩn nhỏ nhất trong S . Từ $\delta \in S$ suy ra với mọi $\eta \mid \alpha, \eta \mid \beta$, ta đều có $\eta \mid \delta$. Với một phần tử v bất kỳ trong S , theo định lý 6.4, viết

$$v = \eta\delta + \gamma, \quad 0 \leq N(\gamma) < N(\delta).$$

Từ tính nhỏ nhất của $N(\delta)$, suy ra $N(\gamma) = 0$, tức $\gamma = 0$, và $\delta \mid \gamma$. Theo định nghĩa, $\delta = \gcd(\alpha, \beta)$, $\gcd(\alpha, \beta)$ tồn tại. Lại do $\delta \sim \Delta$, ta được $\Delta \in S$. □

Từ đây và định lý 5.4 suy ra:

Định lý 6.6. $\mathbb{Z}[i]$ là miền nguyên phân tích duy nhất.

Định lý 6.7. Cho $n \in \mathbb{N}$. Số phần tử trong $\mathbb{Z}[i]$ có chuẩn bằng n là

$$E(n) = 4 \sum_{d|n} \chi(d),$$

trong đó¹³

$$\chi(d) = \begin{cases} (-1)^{(d-1)/2}, & 2 \nmid d, \\ 0, & 2 \mid d. \end{cases}$$

Định lý 6.8. $\xi \in \mathbb{Z}[i]$ là số nguyên tố khi và chỉ khi ξ thỏa một trong các điều kiện sau:

1. $N(\xi) = 2$;
2. $N(\xi) = p$, trong đó $p \equiv 1 \pmod{4}$ là số nguyên tố hữu tỉ dương;
3. $\xi \sim p$, trong đó $p \equiv 3 \pmod{4}$ là số nguyên tố hữu tỉ dương.

Chứng minh. Tính đủ: trường hợp (1) có thể kiểm tra trực tiếp; trường hợp (2) suy ra từ định lý 6.1 và định nghĩa 5.2. Với trường hợp (3), giả sử $\xi = \alpha\beta$ và $\alpha, \beta \neq \epsilon$. Khi đó

$$p^2 = N(\xi) = N(\alpha)N(\beta).$$

Từ $N(\alpha), N(\beta) > 1$, suy ra $N(\alpha) = N(\beta) = p$. Theo định lý 6.7 không tồn tại số nguyên Gauss có chuẩn bằng p , mâu thuẫn. Vì vậy ξ là số nguyên tố.

Tính cần: nếu ξ là số nguyên tố, theo định lý 4.3,

$$\xi \bar{\xi} = N(\xi) = q_1 \cdots q_r,$$

trong đó q_j là số nguyên tố hữu tỉ dương. Do ξ là số nguyên tố, tồn tại một q_j , giả sử là q_1 , sao cho $\xi \mid q_1$. Từ đó

$$N(\xi) \mid N(q_1) = q_1^2,$$

nên $N(\xi) = q_1$ hoặc $N(\xi) = q_1^2$.

Nếu $N(\xi) = q_1$ thì tất yếu xảy ra (1) hoặc (2). Nếu $N(\xi) = q_1^2$, từ $\xi \mid q_1$ suy ra $\xi \sim q_1$, nên q_1 là số nguyên tố trong $\mathbb{Z}[i]$. Kết hợp với định lý 6.7, ta có $q_1 \equiv 3 \pmod{4}$, tức (3) đúng. $\square$

7. Mở rộng một số thuật toán thường dùng

Từ thảo luận trên có thể thấy $\mathbb{Z}[i]$ và $\mathbb{Z}$ có rất nhiều điểm tương tự trong cấu trúc số học. Nhờ đó, ta có thể thử mở rộng một số thuật toán thường dùng trong $\mathbb{Z}$, miễn là chúng chỉ liên quan tới cấu trúc số học, sang $\mathbb{Z}[i]$.

7.1. Thuật toán Euclid

Trong $\mathbb{Z}$, với hai số a, b không đồng thời bằng 0, ta có thể dùng thuật toán Euclid để tìm ước chung lớn nhất của a, b trong $O(\log |a|)$. Dưới đây ta phỏng theo thuật toán Euclid trong $\mathbb{Z}$ để giải bài toán ước chung lớn nhất trong $\mathbb{Z}[i]$.

¹³Bạn đọc quen với lý thuyết hàm số học có thể thấy χ ở đây thật ra là đặc trưng Dirichlet không chính modulo 4. Tuy nhiên điều này không nằm trong phạm vi thảo luận của bài viết. Ở đây chỉ nêu rằng hàm này là hoàn toàn nhân tính; chứng minh được lược bỏ.

Với $\alpha, \beta \in \mathbb{Z}[i]$ không đồng thời bằng 0, theo định lý 6.4, ta có thể viết

$$\beta = \eta\alpha + \gamma, \quad 0 \leq N(\gamma) \leq \frac{1}{2}N(\alpha), \quad (22)$$

và ký hiệu $\gamma = \beta \bmod \alpha$.

Bổ đề 7.1.1. Với $\alpha, \beta \in \mathbb{Z}[i]$, $\alpha \neq 0$, ta có

$$\gcd(\alpha, \beta) = \gcd(\beta \bmod \alpha, \alpha).$$

Chứng minh. Viết α, β theo (22). Đặt $\Delta = \gcd(\alpha, \beta)$. Từ $\Delta \mid \alpha$, $\Delta \mid \beta$, và $\gamma = \beta - \eta\alpha$, suy ra $\Delta \mid \gamma$.

Mặt khác, với mọi $\delta \mid \alpha$, $\delta \mid \gamma$, do $\beta = \eta\alpha + \gamma$, ta có $\delta \mid \beta$, nên $\delta \mid \Delta$. Theo định nghĩa,

$$\gcd(\beta \bmod \alpha, \alpha) = \Delta = \gcd(\alpha, \beta).$$

□

Vì vậy ta vẫn có thể xử lý đệ quy như thuật toán Euclid trong $\mathbb{Z}$, với biên vẫn là

$$\gcd(0, \beta) = \beta.$$

Theo (22), sau mỗi bước đệ quy trên (α, β) , chuẩn nhỏ hơn trong hai số giảm ít nhất một nửa, nên độ phức tạp thời gian của thuật toán là $O(\log N(\alpha))$.

Đến đây, nhờ định lý 6.5 và nội dung của mục nhỏ này, ta đã có thể giải bài toán ở mục 3 trong $O(\log N(x))$. Tiếp theo, ta thử mở rộng thêm một số thuật toán thường gặp khác.

7.2. Phân tích chuẩn

Như ta đã biết, mọi số nguyên hữu tỉ $n \neq 0$ đều có thể biểu diễn thành tích của một số số nguyên tố hữu tỉ, và có nhiều thuật toán giải bài toán phân tích thừa số nguyên tố, tức phân tích chuẩn, trong $\mathbb{Z}$.

Mục này bắt đầu từ phép thử chia quen thuộc nhất trong $\mathbb{Z}$, rồi tìm thuật toán phân tích chuẩn trong $\mathbb{Z}[i]$. Ký hiệu $\alpha \in \mathbb{Z}[i]$. Ta muốn biểu diễn nó dưới dạng

$$\alpha = \xi_1 \cdots \xi_r, \quad (23)$$

trong đó $\xi_1, \dots, \xi_r$ là các số nguyên tố Gauss.

7.2.1. Thử chia

Từ (23) và định lý 6.1, ta có

$$N(\alpha) = N(\xi_1) \cdots N(\xi_r).$$

Suy ra trong mọi thừa số nguyên tố của α , có nhiều nhất một thừa số có chuẩn vượt quá $\sqrt{N(\alpha)}$.

Do đó, ta có thể phỏng theo cách làm trong $\mathbb{Z}$: theo thứ tự chuẩn, lần lượt dùng mọi số nguyên Gauss có chuẩn không vượt quá $\sqrt{N(\alpha)}$ để thử chia. Phần còn lại cuối cùng hoặc là ϵ , hoặc là một số nguyên tố Gauss.

Vì số số nguyên Gauss có chuẩn không vượt quá $\sqrt{N(\alpha)}$ là $O(\sqrt{N(\alpha)})$, nên độ phức tạp thời gian của thuật toán này là $O(\sqrt{N(\alpha)})$.

7.2.2. Cải tiến thuật toán

Với $0 \neq \alpha = a + bi \in \mathbb{Z}[i]$, đặt $d = \gcd(a, b)$. Rõ ràng $d \mid \alpha$. Ký hiệu

$$\alpha_1 = \frac{\alpha}{d} = a_1 + b_1 i.$$

Từ $\gcd(a_1, b_1) = 1$, ta biết α_1 không có thừa số nguyên hữu tỉ không hiển nhiên.

Xét cách phân tích α_1 . Ta cần tìm một thừa số nguyên tố β của α_1 .

Viết phân tích chuẩn của $N(\alpha_1)$ là

$$N(\alpha_1) = \prod_{i=1}^k p_i^{r_i},$$

trong đó $p_1 < p_2 < \dots < p_k$ là các số nguyên tố hữu tỉ dương.

Khi $p_1 = 2$, vì α_1 không có thừa số nguyên hữu tỉ, lấy $\beta = 1 + i$ là thỏa yêu cầu.

Khi $p_1 \equiv 1 \pmod{4}$, ký hiệu γ_1, γ_2 là hai số nguyên Gauss không liên kết với nhau và thỏa $N(\gamma_1) = N(\gamma_2) = p_1$. Theo định lý 6.8, γ_1, γ_2 đều là số nguyên tố Gauss. Từ tính nhỏ nhất của p_1 , suy ra ít nhất một trong $\beta = \gamma_1$ hoặc $\beta = \gamma_2$ thỏa yêu cầu.

Khi $p_1 \equiv 3 \pmod{4}$, giả sử $\alpha_1 = \beta \alpha_2$. Khi đó

$$p_1 \mid N(\alpha_1) = N(\beta)N(\alpha_2).$$

Do p_1 là số nguyên tố hữu tỉ, tất yếu $p_1 \mid N(\beta)$ hoặc $p_1 \mid N(\alpha_2)$. Không mất tính tổng quát, giả sử $p_1 \mid N(\beta)$. Theo định lý 6.8, $\beta = p_1$, mâu thuẫn với việc α_1 không có thừa số nguyên hữu tỉ không hiển nhiên.

Sau khi dựng được β như trên, đặt $\alpha_1 = \beta \alpha_2$, rồi lặp lại thao tác trên với α_2 cho đến khi chỉ còn ϵ .

Cuối cùng, phân tích chuẩn d trong $\mathbb{Z}$, rồi phân tích mọi thừa số nguyên tố có modulo 4 khác 3 trong $\mathbb{Z}[i]$.

Bây giờ bài toán chuyển thành phân tích một số nguyên tố hữu tỉ $p \equiv 1 \pmod{4}$ trong $\mathbb{Z}[i]$.

Bổ đề 7.2.2.1. Cho số nguyên tố hữu tỉ $p \equiv 1 \pmod{4}$, số nguyên hữu tỉ z thỏa $z^2 \equiv -1 \pmod{p}$ và $0 \leq z < p$. Đặt $\xi = \gcd(p, z + i)$. Khi đó

$$\xi \bar{\xi} = p.$$

Chứng minh. Mệnh đề tương đương với chứng minh $N(\xi) = \xi \bar{\xi} = p$.

Theo giả thiết, $\xi \mid p$ và $\xi \mid z + i$, nên

$$N(\xi) \mid \gcd(N(p), N(z + i)) = p.$$

Theo điều kiện đề bài, tồn tại $x, y \in \mathbb{Z}$ sao cho

$$p = x^2 + y^2 = (x + yi)(x - yi).$$

Từ đó $\gcd(x, y) = \gcd(x, p) = 1$, và

$$0 \equiv x^2 + y^2 \equiv x^2 - z^2 y^2 \equiv (x + yz)(x - yz) \pmod{p}.$$

Nếu $p \mid x - yz$, giả sử $x = yz + kp$, $k \in \mathbb{Z}$. Khi đó

$$x + yi = (yz + kp) + yi = y(z + i) + kp.$$

Lại chú ý rằng $(x + yi) \mid p$ và $\gcd(x + yi, y) = 1$, nên $(x + yi) \mid (z + i)$, từ đó $(x + yi) \mid \xi$, và $N(\xi) = p$.

Nếu $p \mid x + yz$, chứng minh tương tự cho $N(\xi) = p$. $\square$

Với mỗi p cần phân tích, ta có thể dùng ngẫu nhiên để tìm¹⁴ một số c không là thặng dư bậc hai modulo p , lấy

$$z \equiv c^{(p-1)/4} \pmod{p}$$

thì sẽ thỏa $z^2 \equiv -1 \pmod{p}$.¹⁵ Sau đó thực hiện thuật toán Euclid theo bổ đề.

Giả sử số cần phân tích là α , với chuẩn $N(\alpha) = n$. Trong thuật toán này, ngoài việc phân tích số nguyên hữu tỉ trong $\mathbb{Z}$, ta chỉ thực hiện lũy thừa nhanh và thuật toán Euclid trên một phần các thừa số nguyên tố, mà số thừa số nguyên tố là $O(\log n)$, nên phần này tốn $O(\log^2 n)$.

Nút thắt độ phức tạp của thuật toán nằm ở bước phân tích số nguyên hữu tỉ trong $\mathbb{Z}$. Nếu dùng thuật toán Pollard's Rho, tổng độ phức tạp thời gian là $O(n^{1/4})$.

8. Mở rộng sơ lược bài toán tính tổng hàm số học trong số nguyên Gauss $\mathbb{Z}[i]$

Trong thi thuật toán, bài toán tính tổng hàm số học là một lớp bài toán quan trọng. Dưới đây ta phỏng theo cách làm trong $\mathbb{Z}$, mở rộng một số thuật toán tính tổng hàm số học thường gặp sang $\mathbb{Z}[i]$.

Để tiện, định nghĩa hai tập sau:

$$D^+ = \{a + bi \mid a \in \mathbb{N}^+, b \in \mathbb{N}\},$$

$$D = D^+ \cup \{0\}.$$

Rõ ràng D là một tập đại diện của $\mathbb{Z}[i]$, và D^+ là tập các phần tử khác không trong D . Mọi ký hiệu tổng sau đây đều duyệt các số nguyên Gauss thuộc D . Lại định nghĩa

$$T(n) = \{\alpha \in D \mid N(\alpha) = n\},$$

$$S(n) = \{\alpha \in D \mid 1 \leq N(\alpha) \leq n\}.$$

Phỏng theo định nghĩa gốc của hàm số học, ta đưa ra các định nghĩa sau trong $\mathbb{Z}[i]$.

Định nghĩa 8.1 (Hàm số học). Ta gọi hàm $f : D^+ \rightarrow \mathbb{C}$ là một hàm số học trong $\mathbb{Z}[i]$.

Định nghĩa 8.2 (Hàm nhân tính). Cho f là một hàm số học trong $\mathbb{Z}[i]$. Nếu với mọi $\alpha, \beta \in D^+$ và $\gcd(\alpha, \beta) = 1$, ta đều có

$$f(\alpha\beta) = f(\alpha)f(\beta),$$

thì gọi f là một hàm nhân tính trong $\mathbb{Z}[i]$.

Định nghĩa 8.3 (Hàm hoàn toàn nhân tính). Cho f là một hàm nhân tính trong $\mathbb{Z}[i]$. Nếu với mọi $\alpha, \beta \in D^+$ đều có

$$f(\alpha\beta) = f(\alpha)f(\beta),$$

thì gọi f là một hàm hoàn toàn nhân tính trong $\mathbb{Z}[i]$.

¹⁴Theo tính chất thặng dư bậc hai, kỳ vọng số lần random để gặp một số không là thặng dư bậc hai là 2.

¹⁵Kết luận này có thể suy ra từ tiêu chuẩn Euler cho thặng dư bậc hai và định lý nhỏ Fermat; ở đây không trình bày chi tiết.

Định nghĩa 8.4 (Tích chập Dirichlet). Với hai hàm số học f, g , định nghĩa tích chập Dirichlet của chúng là

$$(f * g)(\alpha) = \sum_{\delta|\alpha} f(\delta)g\left(\frac{\alpha}{\delta}\right).$$

Đặc biệt, khi $g(\alpha) = c$ là hàm hằng, tích chập này có thể viết là $f * c$.

Định lý 8.1. Cho $\alpha \in D^+$. Khi β chạy qua $S(n)$, $\alpha\beta$ chạy qua tất cả các bội của α trong $S(N(\alpha)n)$.

8.1. Sàng tuyến tính

Trong $\mathbb{Z}$, ta có thể dùng sàng Euler trong $O(n)$ để tìm mọi số nguyên tố hữu tỉ dương không vượt quá n , cũng như giá trị của các hàm nhân tính tại mọi số nguyên hữu tỉ dương không vượt quá n . Trong $\mathbb{Z}[i]$, ta có thể dùng cách tương tự để tìm mọi số nguyên tố Gauss có chuẩn không vượt quá n , cũng như giá trị của các hàm nhân tính trong $\mathbb{Z}[i]$ tại mọi số nguyên Gauss có chuẩn không vượt quá n .

Quy trình sàng Euler trong $\mathbb{Z}$ như sau: duyệt số nguyên hữu tỉ i từ 2 đến n . Nếu i chưa bị cập nhật, ghi nhận i là số nguyên tố hữu tỉ. Sau đó lần lượt duyệt các số nguyên tố hữu tỉ p không vượt quá i , cập nhật đáp án cho $i \times p$; khi $p \mid i$, cập nhật đáp án rồi dừng việc duyệt p .

Trong thuật toán này, mỗi số nguyên hữu tỉ không nguyên tố i chỉ được cập nhật một lần dưới dạng $i/p \times p$, trong đó p là thừa số nguyên tố nhỏ nhất của i . Vì vậy độ phức tạp thời gian của thuật toán là $O(n)$.

Trong $\mathbb{Z}[i]$, ta phỏng theo cách làm trên. Trước hết cần sắp xếp mọi số nguyên Gauss có chuẩn không vượt quá n theo chuẩn; sau đó duyệt α theo chuẩn từ 2 đến n . Nếu các α có cùng chuẩn thì thứ tự tùy ý. Nếu α chưa bị cập nhật, ghi nhận α là số nguyên tố Gauss. Sau đó lần lượt duyệt các số nguyên tố Gauss ξ không sau α , cập nhật đáp án cho $\alpha \times \xi$; khi $\xi \mid \alpha$, cập nhật đáp án rồi dừng việc duyệt ξ .

Tương tự, trong thuật toán này, mỗi số nguyên Gauss không nguyên tố α chỉ được cập nhật một lần dưới dạng $\alpha/\xi \times \xi$, trong đó ξ là thừa số nguyên tố đứng sớm nhất của α trong thứ tự sắp xếp. Vì số số nguyên Gauss có chuẩn không vượt quá n là $O(n)$, thuật toán có độ phức tạp $O(n)$.

8.2. Một phương pháp tính tổng nhanh dựa trên cấu tạo tích chập Dirichlet

Trong $\mathbb{Z}$, với một số hàm nhân tính, ta có thể xây dựng tích chập Dirichlet để tính tổng tiền tố trong thời gian dưới tuyến tính. Phương pháp này trong giới thi thuật toán Trung Quốc được gọi là “sàng Du”. Dưới đây ta thử mở rộng phương pháp này sang $\mathbb{Z}[i]$.

Trong $\mathbb{Z}$, đại khái thuật toán như sau. Đặt

$$F(n) = \sum_{i=1}^n f(i).$$

Ta có

$$\sum_{i=1}^n (f * g)(i) = \sum_{i=1}^n g(i)F\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Biến đổi được

$$g(1)F(n) = \sum_{i=1}^n (f * g)(i) - \sum_{i=2}^n g(i)F\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Nếu có thể xây dựng g sao cho $f * g$ và g đều dễ tính tổng, trực tiếp dùng công thức trên với ghi nhớ hóa đệ quy sẽ được thuật toán có độ phức tạp thời gian $O(n^{3/4})$. Nếu đặt một ngưỡng B , trước hết tiền xử lý

tuyến tính từ 1 đến B , phần lớn hơn B xử lý đệ quy theo công thức trên, thì độ phức tạp là

$$\begin{aligned} O(B) + \sum_{i=1}^{\lfloor n/B \rfloor} O\left(\sqrt{\frac{n}{i}}\right) &= O(B) + O\left(\int_1^{n/B} \sqrt{\frac{n}{x}} dx\right) \\ &= O\left(B + \frac{n}{\sqrt{B}}\right). \end{aligned}$$

Lấy $B = n^{2/3}$ thì đạt độ phức tạp tối ưu về lý thuyết $O(n^{2/3})$.

Trong $\mathbb{Z}[i]$, ta cũng có thể dùng thuật toán tương tự. Đặt

$$F(n) = \sum_{\alpha \in S(n)} f(\alpha).$$

Ta có

$$\sum_{\alpha \in S(n)} (f * g)(\alpha) = \sum_{i=1}^n \sum_{\alpha \in T(i)} g(\alpha) F\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Biến đổi được

$$g(1)F(n) = \sum_{\alpha \in S(n)} (f * g)(\alpha) - \sum_{i=2}^n \sum_{\alpha \in T(i)} g(\alpha) F\left(\left\lfloor \frac{n}{i} \right\rfloor\right).$$

Chỉ cần xây dựng được g thích hợp, làm theo cùng phương pháp như trong $\mathbb{Z}$ sẽ thu được thuật toán tính tổng có cùng độ phức tạp $O(n^{2/3})$.

Trong $\mathbb{Z}[i]$, việc tính $\sum_{\alpha \in S(i)} (f * g)(\alpha)$ và $\sum_{\alpha \in T(i)} g(\alpha)$ có thể cần thời gian lớn hơn $O(1)$. Nếu có thể tính trong $O(\sqrt{i})$, ta có thể dùng sàng tuyến tính tiền xử lý đáp án cho $i = 1$ đến B , còn đáp án lớn hơn B thì dùng cách $O(\sqrt{i})$ để tiền xử lý, vẫn đạt tổng độ phức tạp $O(n^{2/3})$.

8.3. Một phương pháp tính tổng nhanh dựa trên sàng Eratosthenes mở rộng

Trong $\mathbb{Z}$, còn có một thuật toán tính tổng hàm nhân tính có phạm vi áp dụng khá rộng, dựa trên sàng Eratosthenes mở rộng, trong giới thi thuật toán Trung Quốc thường gọi là “sàng Min_25”.

Trong thảo luận sau, ta coi mọi số nguyên Gauss là có thứ tự: với hai số nguyên Gauss có chuẩn khác nhau, số có chuẩn nhỏ hơn đứng trước; với hai số nguyên Gauss có cùng chuẩn, số có phần thực nhỏ hơn đứng trước.

Phỏng theo cách làm trong $\mathbb{Z}$, đặt

$$\begin{aligned} g_{n,m} &= \sum_{\substack{2 \leq N(\alpha) \leq n \\ \alpha \text{ không chứa thừa số nguyên tố thứ } m \text{ trở về trước}}} f(\alpha), \\ h_n &= \sum_{\alpha \text{ là số nguyên tố thứ } n \text{ trở về trước}} f(\alpha). \end{aligned}$$

Khi đó vẫn có

$$g_{n,m} = \sum_{\substack{\xi \in S(\sqrt{n}) \\ \xi \text{ là số nguyên tố sau } m \\ N(\xi^e) \leq n}} f(\xi^e) ([e > 1] + g_{\lfloor n/N(\xi^e) \rfloor, p} + h_n - h_m).$$

Để tính h , ta cần cần hàm hoàn toàn nhân tính f' thỏa $f'(\xi) = f(\xi)$ khi ξ là số nguyên tố. Cũng có thể tách f thành tổng của một số hàm hoàn toàn nhân tính f' , rồi cộng lại cuối cùng. Giả sử $\eta_1, \dots, \eta_r$ lần lượt là các số nguyên tố có chuẩn không vượt quá $\sqrt{n}$. Đặt

$$h'_{i,n} = \sum_{\substack{2 \leq N(\xi) \leq n \\ \xi \text{ là số nguyên tố hoặc không có thừa số } \eta_i \text{ trở về trước}}} f'(\xi).$$

Ta có

$$h'_{i,n} = h'_{i-1,n} - f'(\eta_i) \left(h'_{i-1, \lfloor n/N(\eta_i) \rfloor} - h'_{i-1, N(\xi_{i-1})} \right).$$

Biên là

$$h'_{0,n} = \sum_{\alpha \in S(n)} f'(\alpha) - f'(1).$$

Tính xong h' , ta có thể nhận được h cần tìm từ h' cuối cùng; tại các số nguyên tố Gauss có cùng chuẩn cần xử lý riêng.

Theo định lý 6.8, chuẩn của số nguyên tố Gauss luôn là một số nguyên tố hữu tỉ hoặc bình phương của một số nguyên tố hữu tỉ, và mỗi số nguyên tố hữu tỉ tương ứng với không quá $O(1)$ số nguyên tố Gauss. Vì vậy số số nguyên tố Gauss có chuẩn không vượt quá n là $O(\pi(n))$, trong đó $\pi(n)$ là số số nguyên tố hữu tỉ không vượt quá n . Do đó, chỉ cần h'_0 có thể tính nhanh, ta có thể dùng cùng phương pháp như trong $\mathbb{Z}$, với cùng độ phức tạp, để giải bài toán trong $\mathbb{Z}[i]$. Độ phức tạp thời gian vẫn là $O(n^{3/4}/\log n)$.

Với giá trị ban đầu của h'_0 , cũng có thể cần tính bằng thời gian $O(n^{2/3})$ như đã nói ở mục nhỏ trước; điều này không ảnh hưởng tới tổng độ phức tạp.

9. Một số hàm số học thường gặp và bài ví dụ

Trong mục này, với $\alpha, \beta \in \mathbb{Z}[i]$, $\gcd(\alpha, \beta)$ được hiểu là phần tử liên kết với ước chung lớn nhất của chúng trong D .

Trước hết, phỏng theo định nghĩa trong $\mathbb{Z}$, ta định nghĩa một số hàm số học thường dùng.

Hàm đơn vị $\epsilon(\alpha) = [\alpha = 1]$, tức nếu $\alpha = 1$ thì $\epsilon(\alpha) = 1$, ngược lại $\epsilon(\alpha) = 0$.

Hàm lũy thừa $\text{Id}_k(\alpha) = \alpha^k$. Đặc biệt, khi $k = 1$ cũng ký hiệu là $\text{Id}(\alpha)$.

Hàm Möbius: giả sử

$$\alpha = \prod_{i=1}^r \xi_i^{k_i},$$

trong đó $\xi_1, \dots, \xi_r$ là các số nguyên tố khác nhau. Khi đó

$$\mu(\alpha) = \begin{cases} 0, & \exists k_i > 1, \\ (-1)^r, & \text{otherwise.} \end{cases}$$

Tương tự như trong $\mathbb{Z}$, ta vẫn có tính chất sau.

Định lý 9.1. Cho $\alpha \in D^+$. Khi đó

$$\sum_{\delta|\alpha} \mu(\delta) = \epsilon(\alpha).$$

Định nghĩa 9.1 (Đồng dư). Cho $\alpha, \beta, \gamma \in \mathbb{Z}[i]$ và $\gamma \neq 0$. Ta nói α đồng dư với β khi và chỉ khi $\gamma \mid (\alpha - \beta)$, ký hiệu

$$\alpha \equiv \beta \pmod{\gamma}.$$

Định nghĩa 9.2 (Lớp thặng dư). Cho $0 \neq \gamma \in \mathbb{Z}[i]$. Ta có thể chia $\mathbb{Z}[i]$ thành các lớp tương đương đôi một rời nhau theo việc chúng có đồng dư modulo γ hay không. Các lớp tương đương này được gọi là các lớp thặng dư modulo γ , và số lượng của chúng được ký hiệu là $R(\gamma)$.

Định nghĩa 9.3 (Hệ thặng dư đầy đủ). Cho $0 \neq \gamma \in \mathbb{Z}[i]$. Lấy cố định một phần tử đại diện cho mỗi lớp thặng dư modulo γ , ta được $R(\gamma)$ phần tử gọi là một hệ thặng dư đầy đủ modulo γ .

Định lý 9.2. Cho $\gamma = a + bi \in \mathbb{Z}[i]$, $a^2 + b^2 \neq 0$. Đặt $g = \gcd(a, b)$. Khi đó

$$x_{m,n} = m + ni, \quad 0 \leq m < \frac{N(\gamma)}{g}, \quad 0 \leq n < g$$

là một hệ thặng dư đầy đủ modulo γ .

Chứng minh. Trước hết chứng minh các $x_{m,n}$ đôi một không đồng dư.

Giả sử $x_{m,n} \equiv x_{m',n'} \pmod{\gamma}$. Khi đó đặt

$$\eta\gamma = x_{m,n} - x_{m',n'} = (m - m') + (n - n')i.$$

Vì $g \mid \gamma$, ta có $g \mid (m - m') + (n - n')i$, từ đó $g \mid (n - n')$. Theo phạm vi của n , suy ra $n = n'$.

Lại đặt $\eta = c - di$. Khi đó

$$\eta\gamma = (ac + bd) + (bc - ad)i,$$

nên

$$\frac{a}{g}d = \frac{b}{g}c, \quad \gcd\left(\frac{a}{g}, \frac{b}{g}\right) = 1.$$

Theo tính chất số nguyên, có $\frac{a}{g} \mid c$, do đó

$$\frac{c}{a/g} = \frac{d}{b/g} = k \in \mathbb{Z}.$$

Suy ra

$$m - m' = ac + bd = k \cdot \frac{a^2 + b^2}{g}.$$

Tức $\frac{N(\gamma)}{g} \mid (m - m')$. Theo phạm vi của m , suy ra $m = m'$. Vì vậy $x_{m,n} = x_{m',n'}$.

Tiếp theo chứng minh với mọi $\alpha = x + yi \in \mathbb{Z}[i]$, đều tồn tại $x_{m,n}$ đồng dư với α .

Theo phép chia có dư, tồn tại $q_2 \in \mathbb{Z}$, $0 \leq n < g$, sao cho

$$y = q_2g + n,$$

và tồn tại $q_1 \in \mathbb{Z}$, $0 \leq m < \frac{N(\gamma)}{g}$, sao cho

$$x - q_2(ax_0 - by_0) = q_1 \cdot \frac{N(\gamma)}{g} + m,$$

trong đó (x_0, y_0) là một nghiệm nguyên bất kỳ của $ay_0 + bx_0 = g$. Khi đó

$$\alpha = m + ni + q_1 \cdot \frac{N(\gamma)}{g} + q_2\theta,$$

trong đó $\theta = (ax_0 - by_0) + gi$.

Do

$$\begin{aligned} \theta &= (ax_0 - by_0) + gi \\ &= (ax_0 - by_0) + (ay_0 + bx_0)i \\ &= (a + bi)(x_0 + y_0i), \end{aligned}$$

nên $\alpha \equiv m + ni \pmod{\gamma}$. $\square$

Từ định lý 9.2, ta được $R(\alpha) = N(\alpha)$.

Định nghĩa 9.4 (Hệ thặng dư thu gọn). Cho $0 \neq \gamma \in \mathbb{Z}[i]$. Trong một hệ thặng dư đầy đủ modulo γ , các phần tử α thỏa $\gcd(\alpha, \gamma) = 1$ tạo thành một hệ thặng dư thu gọn modulo γ .

Ta định nghĩa hàm Euler $\phi(\gamma)$ là kích thước của hệ thặng dư thu gọn modulo γ . Tương tự như trong $\mathbb{Z}$, ta vẫn có:

Định lý 9.3. Cho $\alpha \in D^+$. Khi đó

$$\sum_{\delta|\alpha} \phi(\delta) = N(\alpha).$$

Chứng minh. Xét mỗi số β trong một hệ thặng dư đầy đủ của α , đặt $\Delta = \gcd(\alpha, \beta)$. Khi đó

$$\gcd\left(\frac{\alpha}{\Delta}, \frac{\beta}{\Delta}\right) = 1.$$

Xét α' và β' lần lượt là các phần tử trong D liên kết với $\frac{\alpha}{\Delta}$ và $\frac{\beta}{\Delta}$. Khi đó tất cả các β' như vậy tạo thành một hệ thặng dư thu gọn của α' . Vì vậy tổng kích thước các hệ thặng dư thu gọn của mọi ước của α bằng kích thước hệ thặng dư đầy đủ của α . $\square$

Định lý 9.4. Cho $\alpha \in D^+$ và

$$\alpha = \prod_{i=1}^r \xi_i^{k_i},$$

trong đó $\xi_1, \dots, \xi_r$ là các số nguyên tố Gauss đôi một khác nhau. Khi đó

$$\phi(\alpha) = \prod_{i=1}^r N(\xi_i^{k_i}) \left(1 - \frac{1}{N(\xi_i)}\right).$$

Chứng minh. Từ định lý 9.3, ta có $\phi * 1 = N$, do đó $\phi = N * \mu$ là hàm nhân tính. Với $\beta = \eta^c$, trong đó η là số nguyên tố Gauss, ta có

$$\phi(\beta) = \phi(\eta^c) = N(\eta^c) - N(\eta^{c-1}) = N(\eta^c) \left(1 - \frac{1}{N(\eta)}\right).$$

Theo tính chất của hàm nhân tính, định lý được chứng minh. $\square$

Các ví dụ sau đều được cải biên từ những bài toán thường gặp trong số nguyên hữu tỉ.

Ví dụ 1. Cho số nguyên dương k . Mỗi truy vấn cho hai số nguyên dương n, m , hãy tính

$$\sum_{\alpha \in S(n)} \sum_{\beta \in S(m)} \gcd(\alpha, \beta)^k.$$

Số truy vấn không vượt quá 2000, $n, m, k \leq 5 \times 10^5$. Đáp án lấy modulo $10^9 + 7$.

$$\begin{aligned} & \sum_{\alpha \in S(n)} \sum_{\beta \in S(m)} \gcd(\alpha, \beta)^k \\ &= \sum_{\alpha \in S(n)} \sum_{\beta \in S(m)} \sum_{\substack{\delta|\alpha \\ \delta|\beta}} \epsilon\left(\gcd\left(\frac{\alpha}{\delta}, \frac{\beta}{\delta}\right)\right) \delta^k \\ &= \sum_{\alpha \in S(n)} \sum_{\beta \in S(m)} \sum_{\substack{\delta|\alpha \\ \delta|\beta}} \delta^k \sum_{\tau|\delta} \mu(\tau) \\ &= \sum_{\Delta \in S(n)} c\left(\left\lfloor \frac{n}{N(\Delta)} \right\rfloor\right) c\left(\left\lfloor \frac{m}{N(\Delta)} \right\rfloor\right) \sum_{\delta|\Delta} \delta^k \mu\left(\frac{\Delta}{\delta}\right) \\ &= \sum_{i=1}^n c\left(\left\lfloor \frac{n}{i} \right\rfloor\right) c\left(\left\lfloor \frac{m}{i} \right\rfloor\right) \sum_{\Delta \in T(i)} \sum_{\delta|\Delta} \delta^k \mu\left(\frac{\Delta}{\delta}\right). \end{aligned}$$

Ở đây $C(m)$ biểu thị số phần tử trong D^+ có chuẩn không vượt quá m .

Giá trị của C và của hàm ở nửa sau công thức có thể được tiền xử lý trong $O((n+m) \log k)$. Với mỗi bộ dữ liệu, chỉ cần duyệt các giá trị của $\lfloor n/i \rfloor$ và $\lfloor m/i \rfloor$ để tính. Độ phức tạp thời gian cho mỗi bộ dữ liệu là $O(\sqrt{n} + \sqrt{m})$.

Ví dụ 2. Mỗi truy vấn cho số nguyên dương n , hãy tính

$$\sum_{\alpha \in S(n)} \mu(\alpha) \quad \text{và} \quad \sum_{\alpha \in S(n)} \phi(\alpha).$$

Số truy vấn không vượt quá $10, n \leq 2^{32} - 1$.

Ta dùng phương pháp xây dựng tích chập Dirichlet để tính tổng. Theo các định lý trên, ta có $\mu * 1 = \epsilon$ và $\phi * 1 = N$. Trong đó tổng tiền tố của ϵ luôn bằng 1, nên tính nhanh được. Khi tính tổng tiền tố của hàm hằng và chuẩn N , với $x^2 + y^2 \leq m$, ta có thể duyệt x trong $O(\sqrt{m})$, rồi với mỗi x , cộng các giá trị 1 và $x^2 + y^2$ từ 1 đến $\lfloor \sqrt{m - x^2} \rfloor$. Theo phương pháp trên, tổng độ phức tạp cho mỗi truy vấn là $O(n^{2/3})$.

Ví dụ 3. Cho số nguyên dương n , hãy tính

$$\sum_{\alpha \in S(n)} \sum_{\beta \in S(n)} \frac{\alpha \beta}{\gcd(\alpha, \beta)}.$$

$n \leq 10^{10}$.

$$\begin{aligned} & \sum_{\alpha \in S(n)} \sum_{\beta \in S(n)} \frac{\alpha \beta}{\gcd(\alpha, \beta)} \\ &= \sum_{\Delta \in S(n)} \Delta^2 F\left(\frac{n}{N(\Delta)}\right) \sum_{\delta | \Delta} \frac{1}{\delta} \mu\left(\frac{\Delta}{\delta}\right) \\ &= \sum_{i=1}^n F\left(\left\lfloor \frac{n}{i} \right\rfloor\right)^2 \sum_{\Delta \in T(i)} \Delta^2 \sum_{\delta | \Delta} \frac{1}{\delta} \mu\left(\frac{\Delta}{\delta}\right). \end{aligned}$$

Ở đây $F(m)$ biểu thị tổng của mọi số nguyên Gauss trong D^+ có chuẩn không vượt quá m , có thể tiền xử lý trong $O(n^{2/3})$. Nếu đặt $(u \cdot v)(\alpha) = u(\alpha)v(\alpha)$, thì hàm phía sau là

$$f = \text{Id}_2 \cdot (\text{Id}_{-1} * \mu).$$

Có thể xây dựng tích chập Dirichlet

$$f * \text{Id}_2 = \text{Id}$$

để tính. Tổng độ phức tạp thời gian là $O(n^{2/3})$.

10. Tổng kết

Bài viết phân tích cấu trúc của $\mathbb{Z}$ và $\mathbb{Z}[i]$, rồi dựa vào các tính chất tương tự của chúng để mở rộng thuật toán Euclid, phân tích chuẩn và một số phương pháp tính tổng hàm số học thường dùng sang số nguyên Gauss. Thật ra, phần lớn thuật toán chỉ liên quan tới cấu trúc số học đều có thể được mở rộng như vậy; ngoài ra còn có rất nhiều hệ đại số khác cũng có thể được mở rộng tương tự. Bạn đọc hứng thú có thể tự thử.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn huấn luyện viên Zhang Ruizhe của đội tuyển tập huấn quốc gia đã giúp đỡ và hướng dẫn.

Xin cảm ơn cô Liu Lili của trường tôi và các anh chị khóa trên đã giúp đỡ và hướng dẫn.

Xin cảm ơn anh Wang Chengrui của trường tôi đã giúp đỡ tôi khi viết bài này.

Xin cảm ơn các anh Liu Chengao, He Jiaxuan, Du Junchen và Wang Zeyu của trường tôi đã đọc soát bài viết.

Tài liệu tham khảo

1. Pan Chengdong; Pan Chengbiao. *Đại số số học*, bản thứ hai. Nhà xuất bản Đại học Công nghiệp Cấp Nhĩ Tân.
2. Pan Chengdong; Pan Chengbiao. *Số học sơ cấp*, bản thứ ba. Nhà xuất bản Đại học Bắc Kinh.
3. Ren Zhizhou. Một số phương pháp tính tổng hàm nhân tính. Tập luận văn ứng viên đội tuyển quốc gia Olympic Tin học Trung Quốc năm 2016.
4. Zhu Zhenting. Một số bài toán tính tổng hàm số học đặc biệt. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc năm 2018.
5. <https://math.stackexchange.com/questions/5877>

Báo cáo ra đề *Bài luyện cơ bản về cây tròn-vuông*

Fan Zhiyuan, Trường Trung học Xuejun Hàng Châu

Tóm tắt

Bài viết này giới thiệu một bài cấu trúc dữ liệu truyền thống do tác giả ra trong đợt thi thử đội tuyển tập huấn quốc gia lần này.

Bài toán đòi hỏi thí sinh hiểu khá sâu các bài toán cây động, đồng thời cũng cần năng lực viết mã nhất định; vì thế đây là một bài tốt để kiểm tra trình độ cấu trúc dữ liệu của thí sinh.

Bài viết đề xuất một cấu trúc dữ liệu mới, cây ABF. So với kết quả trước đó, cấu trúc này có thể đạt độ phức tạp thời gian thấp hơn trong một số bài toán.

1. Tóm tắt đề bài

Nếu trong một đồ thị vô hướng liên thông, mỗi cạnh thuộc nhiều nhất k chu trình đơn khác nhau, ta gọi đồ thị ấy là một k -cactus.

Nếu mỗi thành phần liên thông của một đồ thị vô hướng đều là một k -cactus và đồ thị không có khuyên, ta gọi đồ thị ấy là một k -sa mạc.

Cần duy trì một k -sa mạc, hỗ trợ thêm cạnh, xóa cạnh, sửa hoặc hỏi đường đi ngắn nhất giữa hai đỉnh, hoặc sửa/hỏi một cactus con.

Khi lấy x làm gốc, cactus con y được định nghĩa là thành phần liên thông chứa y sau khi xóa mọi cạnh trên tất cả các đường đi đơn từ x đến y .

Phạm vi dữ liệu. n là số đỉnh, m là số thao tác. Với mọi dữ liệu:

$$1 \leq n \leq 50000, \quad 1 \leq m \leq 250000.$$

Tính chất đặc biệt A: mọi thao tác thêm cạnh và xóa cạnh đều xuất hiện trước các thao tác khác.

Tính chất đặc biệt B: không có truy vấn cactus con, không có cộng trên đường đi ngắn nhất và không có cộng trên cactus con.

Tính chất đặc biệt C: không có truy vấn cactus con và không có cộng trên cactus con.

Điểm	Bộ kiểm thử	Phạm vi k	Tính chất đặc biệt
6	1	$k = 0$	AB
6	2	$k = 0$	AC
6	3	$k = 0$	A
5	4	$k = 0$	B
5	5	$k = 0$	C
5	6	$k = 0$	
6	7	$k = 1$	AB
6	8	$k = 1$	AC
6	9	$k = 1$	A
5	10	$k = 1$	B
5	11	$k = 1$	C
5	12	$k = 1$	
6	13	$k = 8$	AB
6	14	$k = 8$	AC
7	15	$k = 8$	A
5	16	$k = 8$	B
5	17	$k = 8$	C
5	18	$k = 8$	

3. Thuật toán lấy điểm từng phần

Bằng cách chọn và kết hợp các thuật toán cho từng phần khác nhau, có thể nhận được nhiều mức điểm từng phần. Do giới hạn độ dài, ở đây chỉ thảo luận một số cách làm điểm cao.

3.1. Thuật toán một

Với các bộ kiểm thử $k = 0$, tức là đồ thị luôn là một rừng. Bài con này có thể làm bằng Self-adjusting top trees.

Chi tiết cụ thể có thể tham khảo [1], ở đây không nhắc lại.

Độ phức tạp thời gian là $O(m \log n)$.

Điểm kỳ vọng: 33 điểm.

3.2. Thuật toán hai

Với $k = 1$ và thỏa tính chất đặc biệt C. Bài con này trùng với “Dynamic Cactus III”¹⁶; dùng cách làm của lời giải bài ấy¹⁷ có thể giải được bài con này.

Độ phức tạp thời gian là $O(m \log n)$.

Điểm kỳ vọng: 36 điểm.

3.3. Thuật toán ba

Với $k = 1$ và thỏa tính chất đặc biệt A. Bài con này tương tự “[Tsinghua Training 2015] Static Cactus”¹⁸, có thể giải bằng cây tròn-vuông kết hợp phân rã chuỗi.

Chi tiết cụ thể về cây tròn-vuông có thể tham khảo [2], ở đây không nhắc lại.

¹⁶<http://uoj.ac/problem/65>

¹⁷<https://blog.csdn.net/VFleaKing/article/details/80747834>

¹⁸<http://uoj.ac/problem/158>

Độ phức tạp thời gian là $O(m \log n)$.

Điểm kỳ vọng: 44 điểm.

3.4. Thuật toán bốn

Với các bộ kiểm thử $k = 1$. Bài con này trùng với “Dynamic Cactus IV”¹⁹, có thể giải bằng top cactus.

Chi tiết cụ thể có thể tham khảo [3], ở đây không nhắc lại.

Độ phức tạp thời gian là $O(m \log^2 n)$.

Điểm kỳ vọng: 66 điểm.

4. Thuật toán chuẩn

Rõ ràng các cấu trúc trên đều không đủ để duy trì k -cactus. Để giải bài toán gốc, ta cần đưa vào một cấu trúc mới để duy trì k -cactus.

Vì cấu trúc này là sản phẩm của việc dùng cây AAA để duy trì cây tròn-vuông, ta gọi cấu trúc dữ liệu này là “cây ABF”.

4.1. Cây AAA

Trước khi trình bày cấu trúc của cây ABF, ta xét cây AAA.

Ghi chú: cây AAA ở phần sau hơi khác cây AAA trong [4]. Cây AAA ở đây có thể xem là một phiên bản thật sự chỉnh sửa mạnh từ SATT, và cách đặt tên đỉnh cũng tham khảo SATT.

4.1.1. Cấu trúc

Cây AAA là một cấu trúc dữ liệu dựa trên LCT. Để duy trì thông tin cây con, trên mỗi đỉnh ta gắn thêm một cây phụ duy trì tất cả các cạnh đi ra của nó.

Cây AAA có hai loại đỉnh: *compress* và *rake*. Mỗi đỉnh *compress* biểu diễn một đỉnh trong cây gốc; mỗi đỉnh *rake* biểu diễn một cạnh đi ra có hướng trong cây gốc, tức mỗi cạnh được hai đỉnh *rake* duy trì. Mọi thông tin nguyên tử, tức bản thân những thông tin không thể tách nhỏ, đều đặt trên đỉnh *compress*.

Mỗi đỉnh *compress* có ba con trở l, r, c . Hai con trở l, r trở tới các con của nó trong cây *compress*; con trở c trở tới cây *rake* duy trì tất cả các cạnh đi ra của nó.

Mỗi đỉnh *rake* cũng có ba con trở l, r, c . Hai con trở l, r trở tới các con của nó trong cây *rake*; con trở c trở tới cây *compress* duy trì chuỗi thực lấy cạnh đi ra này làm gốc. Đặc biệt, nếu cạnh đi ra ấy trở tới cha hoặc con thực của đỉnh này thì c rỗng.

Trong hình minh họa của bản gốc, chấm đen là đỉnh *compress*, chấm trắng là đỉnh *rake*; cạnh sang trái là con trở l , cạnh sang phải là con trở r , còn cạnh có mũi tên là con trở c .

Để tiện duy trì, mỗi đỉnh *compress* có hai con trở p và s , lần lượt biểu thị cạnh cha và cạnh con thực của đỉnh đó là đỉnh nào trong cây *rake*. Ví dụ, trong hình, hai con trở p, s của đỉnh B lần lượt trở tới BA, BC . Hai con trở này chỉ có tác dụng đánh dấu, không tham gia xử lý thông tin.

Ngoài ra, bản thân cấu trúc này không thể duy trì thông tin trên cạnh. Khi cạnh có thông tin, cần dựng thêm đỉnh phụ trên cạnh.

¹⁹<http://uoj.ac/problem/106>

4.1.2. Duy trì

Cây động dựa trên phân rã chuỗi có hai thao tác lỗi.

Lật (*reverse*). Thao tác lật một cây *compress* được dùng khi đổi gốc. Ta vẫn dùng đánh dấu lười để duy trì. Khi đẩy xuống đánh dấu lật, không chỉ cần đổi hai con trở l, r , mà còn cần đổi hai con trở p, s .

Chuyển ảo–thực (*splice*). Thao tác chuyển ảo–thực của con được dùng khi thao tác *expose* trích ra một chuỗi.

Khi muốn đổi con thực của đỉnh A từ đỉnh B sang đỉnh C , trước hết xoay các đỉnh AB và AC lên đỉnh, rồi đưa đỉnh C ra ngoài.

Hình minh họa trong bản gốc mô tả đúng thao tác này: sau các phép xoay trong cây phụ, cạnh AC được đưa ra khỏi cây *rake* còn cạnh AB trở thành cạnh ảo.

Rõ ràng thao tác *expose* của cây AAA hoàn toàn giống thao tác *expose* trong LCT, ngoại trừ bản thân thao tác *splice*; ở đây không nhắc lại.

Xét cách hỗ trợ một số thao tác cơ bản khác.

Đẩy thông tin lên. Với đỉnh *compress*, thông tin chia làm hai loại: tổng thông tin của các đỉnh trên chuỗi, ký hiệu $info_r$, và tổng thông tin của các đỉnh nằm trong cây con nhưng không nằm trên chuỗi, ký hiệu $info_v$.

$info_r$ là tổng $info_r$ của l, r và thông tin của bản thân đỉnh. $info_v$ là tổng $info_v$ của l, r, c .

Với đỉnh *rake*, thông tin chỉ có một loại: tổng thông tin của các đỉnh trong cây con, ký hiệu $info_v$.

$info_v$ là tổng $info_v$ của l, r và $info_r, info_v$ của c .

Đẩy đánh dấu xuống. Với đỉnh *compress*, đánh dấu chia làm hai loại: sửa các đỉnh trên chuỗi, ký hiệu tag_r , và sửa các đỉnh trong cây con nhưng không nằm trên chuỗi, ký hiệu tag_v .

tag_r được đẩy xuống tag_r của l, r , đồng thời ảnh hưởng tới thông tin nguyên tử của bản thân đỉnh.

tag_v được đẩy xuống tag_v của l, r, c .

Với đỉnh *rake*, đánh dấu chỉ có một loại: sửa các đỉnh trong cây con, ký hiệu tag_v .

tag_v được đẩy xuống tag_v của l, r , và xuống tag_r, tag_v của c .

Trích chuỗi. Khi cần trích chuỗi giữa hai đỉnh A và B , trước hết thực hiện *expose* trên đỉnh A rồi lật đỉnh A , sau đó thực hiện *expose* trên đỉnh B . Khi ấy, cây *compress* lấy A làm gốc tương ứng với đường đi ngắn nhất giữa A và B . Có thể trực tiếp hỏi trên đó, hoặc dùng đánh dấu để sửa nó.

Trích cây con. Khi cần trích cây con B với gốc là A , trước hết thực hiện *expose* trên đỉnh A rồi lật đỉnh A , sau đó thực hiện *expose* trên đỉnh B . Khi ấy, đỉnh B và cây con lấy $B.c$ làm gốc tương ứng với cây con B khi lấy A làm gốc. Có thể trực tiếp hỏi trên đó, hoặc dùng đánh dấu để sửa nó.

Thêm cạnh. Khi cần thêm một cạnh giữa hai đỉnh A và B , trước hết thực hiện *expose* trên đỉnh A rồi lật đỉnh A , sau đó thực hiện *expose* trên đỉnh B , và cho con trở l của đỉnh B trở tới đỉnh A . Trong cây *rake* của đỉnh A , chèn đỉnh AB và cho con trở s của đỉnh A trở tới nó; trong cây *rake* của đỉnh B , chèn đỉnh BA và cho con trở p của đỉnh B trở tới nó.

Xóa cạnh. Khi cần xóa cạnh giữa hai đỉnh A và B , trước hết thực hiện *expose* trên đỉnh A rồi lật đỉnh B , sau đó thực hiện *expose* trên đỉnh B và xóa con trở l của đỉnh B . Xóa đỉnh AB trong cây *rake* của đỉnh A và xóa con trở s của đỉnh A ; xóa đỉnh BA trong cây *rake* của đỉnh B và xóa con trở p của đỉnh B .

Hiện thực tất cả các thao tác trên là ta thu được một cây AAA.

4.1.3. Phân tích

Vì mọi thao tác của cây AAA đều dựa trên *expose*, còn *expose* lại dựa trên *splice*, để chứng minh một cận trên cho độ phức tạp thời gian của một thao tác cây AAA: trung bình trả góp $O(\log^2 n)$. Cụ thể, có nhiều nhất $O(\log n)$ thao tác *splice*, mỗi thao tác *splice* có độ phức tạp trung bình trả góp $O(\log n)$.

Để thu được cận chặt hơn, cần khai thác thêm tính chất của thao tác *splice*.

Theo phân tích của LCT, ta vẫn lấy tổng logarit kích thước cây con làm hàm thế.

Để tiện mô tả, ký hiệu $|x|$ là kích thước cây con của đỉnh x trước thao tác *splice*, còn $|x'|$ là kích thước cây con của đỉnh x sau thao tác *splice*.

Bổ đề 1. Trong cây AAA, chi phí trả góp của thao tác *splice* không vượt quá

$$3(\log(|A'|) - \log(|C|)) + O(1).$$

Chứng minh. Thao tác đưa C ra ngoài chỉ thay đổi thể năng của hai đỉnh AB và AC . Chi phí trả góp của nó là

$$\begin{aligned}\Phi' - \Phi + c &= (\log(|AB'|) + \log(|AC'|)) - (\log(|AB|) + \log(|AC|)) + O(1) \\ &\leq 3(\log(|A'|) - \log(|C|)) + O(1).\end{aligned}$$

Khi không có *reverse*, đỉnh AB không cần xoay lên vì nó vốn luôn ở đỉnh cây, còn chi phí trả góp khi xoay đỉnh AC lên hiển nhiên không vượt quá

$$3(\log(|AC|) - \log(|C|)) + O(1).$$

Khi có *reverse*, đỉnh AB có thể không nằm ở đỉnh cây: trước hết nó hạ xuống với vai trò cạnh cha p của A , rồi trở thành cạnh thực s do thao tác *reverse*. Vì thế ta thêm vào hàm thế một hạng

$$\log(|A.c|) - \log(|A.p|) + \text{dis}(A.c, A.p),$$

tức chi phí cần để xoay đơn đỉnh $A.p$ cũ lên khi xem nó là $A.s$. Rõ ràng đại lượng này chỉ thay đổi khi xoay đỉnh $A.s$ khiến đỉnh $A.p$ đi vào cây con của nó, và điều này chỉ xảy ra một lần.

Chi phí trả góp của lần xoay đơn này là

$$\begin{aligned}\Phi' - \Phi + c &= (\log(|A.s'|) + \log(|A.p'|) - \log(|A.p|)) \\ &\quad - (\log(|A.s|) + \log(|A.p|) - \log(|A.p|)) + O(1) \\ &\leq 3(\log(|A.s'|) - \log(|A.s|)) + O(1).\end{aligned}$$

Hiển nhiên điều này không ảnh hưởng tới phân tích thể năng của splay. Do đó chi phí trả góp để xoay đỉnh AB lên là $O(1)$.

Chi phí trả góp của thao tác *splice* là tổng ba chi phí trên, vì vậy trong cây AAA, chi phí trả góp của thao tác *splice* là

$$\begin{aligned}\Phi' - \Phi + c &\leq (3(\log(|A'|) - \log(|AC|)) + O(1)) \\ &\quad + (3(\log(|AC|) - \log(|C|)) + O(1)) + O(1) \\ &= 3(\log(|A'|) - \log(|C|)) + O(1).\end{aligned}$$

□

Từ đó có thể thu được độ phức tạp của thao tác *expose* trong cây AAA.

Bổ đề 2. Trong cây AAA, độ phức tạp thời gian của thao tác *expose* là trung bình trả góp $O(\log n)$.

Chứng minh. Gọi đỉnh đang thao tác hiện tại là đỉnh hoạt động. Sau thao tác *splice*, đỉnh hoạt động chuyển từ C thành A , vì vậy hạng thứ nhất trong chi phí trả góp của *splice* không vượt quá 3 lần thay đổi thế năng của đỉnh hoạt động. Do thế năng của đỉnh hoạt động không vượt quá $O(\log n)$, tổng độ phức tạp thời gian của hạng thứ nhất là trung bình trả góp $O(\log n)$.

Trong thao tác *expose* có nhiều nhất $O(\log n)$ thao tác *splice*, nên tổng độ phức tạp thời gian của hạng thứ hai là trung bình trả góp $O(\log n)$.

Tóm lại, trong cây AAA, độ phức tạp thời gian của thao tác *expose* là trung bình trả góp $O(\log n)$. □

Tương tự LCT, mọi thao tác của cây AAA đều dựa trên thao tác *expose*; vì vậy ta thu được độ phức tạp tổng thể của cây AAA.

Định lý 1. Với n đỉnh, cây AAA thực hiện m thao tác trong thời gian

$$O((n + m) \log n).$$

Chứng minh. Trong cây AAA, trích chuỗi, trích cây con, thêm cạnh và xóa cạnh đều có thể được thực hiện bằng $O(1)$ lần *expose* rồi sửa $O(1)$ đỉnh.

Hàm thế tại mọi thời điểm đều lớn hơn hoặc bằng 0, và không vượt quá $O(n \log n)$.

Vì vậy, với n đỉnh, cây AAA thực hiện m thao tác trong thời gian $O((n + m) \log n)$. □

4.2. Cây ABF

Với trường hợp $k = 1$, mỗi thành phần song liên thông của đồ thị đều là một chu trình. Tương tự, khi k là hằng số, cấu trúc của mỗi thành phần song liên thông trong đồ thị cũng rất đơn giản.

Bổ đề 3. Nếu mỗi cạnh của đồ thị song liên thông G thuộc nhiều nhất k chu trình đơn khác nhau, thì trong G có nhiều nhất $k + 1$ chuỗi nối hai đỉnh có bậc không nhỏ hơn ba và chỉ gồm các đỉnh bậc hai.

Chứng minh. Xét mệnh đề phản đảo: nếu trong đồ thị song liên thông G có k chuỗi nối hai đỉnh có bậc không nhỏ hơn ba và chỉ gồm các đỉnh bậc hai, thì mỗi cạnh thuộc ít nhất $k - 1$ chu trình đơn khác nhau.

Không mất tính tổng quát, xét cạnh uv trong G , thêm một đỉnh w vào giữa cạnh này và tách nó thành hai cạnh uw, vw .

Theo sự tồn tại của ear decomposition, tồn tại một cách nối chuỗi bắt đầu từ w để thu được G , sao cho tại mọi thời điểm G' đều song liên thông (chỉ cần chạy cách dựng ear decomposition với gốc w).

Mỗi khi thêm một chuỗi ab vào S để thu được S' , từ tính chất của thành phần song liên thông, trong S chắc chắn tồn tại một đường đi đơn từ a qua w tới b . Vì vậy S' chắc chắn có thêm một chu trình đơn đi qua w , tức đi qua cạnh uv trong đồ thị ban đầu.

Để thu được đồ thị G cần nối $k - 1$ chuỗi, nên mỗi cạnh trong G thuộc ít nhất $k - 1$ chu trình đơn. □

Vì thông tin có tính cộng, khi cập nhật thông tin, chuỗi có thể được co thành một đỉnh để duy trì. Do đó có thể dùng cây tròn-vuông để duy trì k -sa mạc; với mỗi thành phần song liên thông trong đồ thị, ghi lại các đỉnh bậc ba trong đó và các chuỗi nối chúng. Mỗi đỉnh vuông cần $O(k)$ thông tin khi cập nhật.

Dựa trên điều này, ta có thể thiết kế cấu trúc dữ liệu duy trì k -sa mạc.

4.2.1. Cấu trúc

Cây ABF là một cấu trúc dữ liệu dựa trên cây AAA, vẫn dùng phân rã chuỗi để duy trì hình thái của cây. Để duy trì các thành phần song liên thông, ta thêm một loại đỉnh mới vào cây AAA để duy trì thành phần song liên thông. Vì khi duy trì k -sa mạc không thể tránh việc dùng thông tin trên cạnh, trong phần sau mặc định cây AAA đã có các đỉnh phụ trên cạnh.

Ngoài *compress* và *rake*, cây ABF còn có một loại đỉnh khác: *twist*. Đỉnh *twist* xuất hiện trong cây *compress* như một biến thể của đỉnh *compress*, dùng để duy trì một thành phần song liên thông. Để bảo đảm độ phức tạp của cây ABF, tại mọi thời điểm, mọi đỉnh *twist* trong cây *compress* đều là lá.

Mỗi đỉnh *twist* có hai con trở l, r và hai bằng con trở v, e . Hai con trở l, r trở tới các con của nó trong cây *compress*; mỗi con trở trong v trở tới một điểm khóa trong thành phần song liên thông; mỗi con trở trong e trở tới một chuỗi nối các điểm khóa. Ở đây, điểm khóa là những điểm ta quan tâm trong thành phần song liên thông: điểm có ba cạnh đi ra bên trong thành phần song liên thông, điểm nối với cha của thành phần song liên thông, và điểm nối với con thực của thành phần song liên thông.

Để tiện duy trì, các con trở trong v, e không trực tiếp trở tới đỉnh *compress* mà chúng duy trì. Chúng trở tới một đỉnh *rake* rỗng, còn con trở c của đỉnh *rake* này trở tới đỉnh *compress* được nó duy trì. Tương tự trong cây AAA, nếu đỉnh cần duy trì là điểm nối với cha của thành phần song liên thông hoặc điểm nối với con thực của thành phần song liên thông, thì con trở c của đỉnh *rake* này rỗng.

Trong hình minh họa của bản gốc, chấm tròn đen là đỉnh *compress*, chấm vuông đen là đỉnh *twist*, chấm trắng là đỉnh *rake*; cạnh sang trái là con trở l , cạnh sang phải là con trở r , và cạnh có mũi tên là con trở c .

Để tiện duy trì, mỗi đỉnh *twist* có hai con trở p, s , lần lượt biểu thị điểm nối với cha và điểm nối với con thực của đỉnh này là đỉnh nào trong bằng con trở v . Ví dụ, trong hình, hai con trở p, s của O lần lượt trở tới v_1, v_2 . Hai con trở này chỉ có tác dụng đánh dấu, không tham gia xử lý thông tin.

Để ghi lại cấu trúc đồ thị, mỗi đỉnh *rake* trong bằng e cũng có hai con trở p, s , lần lượt biểu thị hai đầu mút của chuỗi này. Ví dụ, trong hình, hai con trở p, s của e_4 lần lượt trở tới v_3, v_2 . Hai con trở này cũng chỉ có tác dụng đánh dấu, không tham gia xử lý thông tin.

Khi một đỉnh *compress* A là tiền nhiệm của đỉnh *twist* B , con trở n của A có thể trở tới bất kỳ cạnh đi ra nào nối tới B trong cây *rake* của nó; khi A là hậu nhiệm của B , con trở p của nó cũng tương tự.

4.2.2. Duy trì

Tương tự cây AAA, ở đây chỉ cần xét ảnh hưởng của các thao tác *reverse* và *splice* lên đỉnh *twist*.

Cây động dựa trên phân rã chuỗi có hai thao tác lỗi.

Lật (*reverse*). Khi đẩy xuống đánh dấu lật của đỉnh *twist*, vẫn chỉ cần đổi hai con trở p, s .

Chuyển ảo-thực (*splice*). Xét cách đổi con thực của đỉnh *twist* A từ đỉnh B sang đỉnh C .

Bước một: đỉnh C có thể không phải là điểm khóa của đỉnh A , mà có thể nằm trên một chuỗi nào đó. Trong trường hợp này, trước hết xoay đỉnh C lên, rồi biến nó thành điểm khóa.

Bước hai: xoay cả tiền nhiệm và hậu nhiệm của đỉnh A trong cây *compress* lên đỉnh (dĩ nhiên, đỉnh A có thể là đỉnh đầu của một chuỗi thực, khi đó không có tiền nhiệm), rồi đưa đỉnh C ra ngoài và đưa đỉnh B vào trong.

Bước ba: đỉnh B có thể không còn là điểm khóa của đỉnh A nữa (bản thân nó là một đỉnh bậc hai); khi đó chỉ cần gộp nó với hai chuỗi kề nó.

Ba hình minh họa trong bản gốc lần lượt mô tả việc xoay C ra khỏi một chuỗi để trở thành điểm khóa, thay điểm khóa B bằng C trong đỉnh *twist*, rồi gộp B trở lại với hai chuỗi kề.

Rõ ràng thao tác *expose* của cây ABF hoàn toàn giống thao tác *expose* của cây AAA, ngoại trừ bản thân thao tác *splice*; ở đây không nhắc lại.

Xét cách hỗ trợ một số thao tác cơ bản khác. Vì cây ABF chỉ thêm đỉnh *twist* so với cây AAA, ta chỉ xét đỉnh *twist*.

Đẩy thông tin lên. Thông tin của đỉnh *twist* chia làm hai loại: tổng thông tin của các đỉnh trên đường đi ngắn nhất, ký hiệu $info_r$, và tổng thông tin của các đỉnh trong cây con nhưng không nằm trên đường đi ngắn nhất, ký hiệu $info_v$.

$info_r$ là tổng $info_r$ của l, r và $info_r$ của các đỉnh hoặc chuỗi nằm trên đường đi ngắn nhất trong v, e .

$info_v$ là tổng $info_v$ của l, r , $info_v$ của các đỉnh hoặc chuỗi nằm trên đường đi ngắn nhất trong v, e , và cả $info_r, info_v$ của các đỉnh hoặc chuỗi không nằm trên đường đi ngắn nhất trong v, e .

Đẩy đánh dấu xuống. Đánh dấu của đỉnh *twist* chia làm hai loại: sửa các đỉnh trên đường đi ngắn nhất, ký hiệu tag_r , và sửa các đỉnh trong cây con nhưng không nằm trên đường đi ngắn nhất, ký hiệu tag_v .

tag_r được đẩy xuống tag_r của l, r , và xuống tag_r của các đỉnh hoặc chuỗi nằm trên đường đi ngắn nhất trong v, e .

tag_v được đẩy xuống tag_v của l, r , tag_v của các đỉnh hoặc chuỗi nằm trên đường đi ngắn nhất trong v, e , và cả tag_r, tag_v của các đỉnh hoặc chuỗi không nằm trên đường đi ngắn nhất trong v, e .

Trích đường đi ngắn nhất. Trong cây ABF, việc trích đường đi ngắn nhất hoàn toàn giống việc trích chuỗi trong cây AAA, ngoại trừ bản thân thao tác *expose*; ở đây không nhắc lại.

Trích cactus con. Trong cây ABF, việc trích cactus con hoàn toàn giống việc trích cây con trong cây AAA, ngoại trừ bản thân thao tác *expose*; ở đây không nhắc lại.

Thêm cạnh. Khi nối hai thành phần liên thông, thao tác thêm cạnh trong cây ABF hoàn toàn giống thao tác thêm cạnh trong cây AAA; ở đây không nhắc lại.

Khi thao tác bên trong một thành phần liên thông, trích mọi thành phần song liên thông trên đường đi ngắn nhất ra, co các chuỗi nối chúng lại, rồi dựng một thành phần song liên thông lớn mới.

Xóa cạnh. Khi sau khi xóa cạnh, một thành phần liên thông bị tách thành hai, thao tác xóa cạnh trong cây ABF hoàn toàn giống thao tác xóa cạnh trong cây AAA; ở đây không nhắc lại.

Khi thao tác bên trong một thành phần song liên thông, sau khi xóa cạnh ấy, chạy Tarjan một lần. Với tất cả các thành phần song liên thông thu được, nối chúng lại theo thứ tự.

Hiện thực tất cả các thao tác trên là ta thu được một cây ABF.

4.2.3. Phân tích

Tương tự cây AAA, để chứng minh một cận trên cho độ phức tạp thời gian của một thao tác cây ABF: trung bình trả góp

$$O(\log^2 n + k \log k \log n).$$

Cụ thể, có nhiều nhất $O(\log n)$ thao tác *splice*, mỗi thao tác *splice* có độ phức tạp thời gian trung bình trả góp $O(\log n + k \log k)$ (mỗi lần *splice* cần chạy một lần đường đi ngắn nhất).

Để thu được cận chặt hơn, cần khai thác thêm tính chất của thao tác *splice*.

Ta vẫn lấy tổng logarit kích thước cây con làm hàm thế. Đỉnh *rake* phụ ngay dưới một đỉnh *twist* không được tính vào hàm thế ở đây.

Để tiện mô tả, ký hiệu $|x_0|$ là kích thước cây con của đỉnh x trước thao tác *splice*; $|x_1|$ là kích thước cây con sau bước một của *splice*; $|x_2|$ là kích thước cây con sau bước hai của *splice*; và $|x_3|$ là kích thước cây con sau thao tác *splice*.

Bổ đề 4. Trong cây ABF, chi phí trả góp của thao tác *splice* không vượt quá

$$9(\log(|T_3|) - \log(|C_0|)) + O(k \log k).$$

Chứng minh. Khi đối tượng của thao tác *splice* là một đỉnh *compress*, điều này hiển nhiên đúng. Xét thao tác *splice* trên một đỉnh *twist*.

Khi cả hai đỉnh B và C đều là đỉnh có bậc không nhỏ hơn ba, chỉ cần thực hiện bước hai trong thao tác *splice*.

Thao tác đưa đỉnh C ra ngoài chỉ thay đổi thế năng của ba đỉnh A, B, C . Chi phí trả góp của nó là

$$\begin{aligned} \Phi' - \Phi + c &= (\log(|A_2|) + \log(|B_2|) + \log(|C_2|)) \\ &\quad - (\log(|A_1|) + \log(|B_1|) + \log(|C_1|)) + O(1) \\ &\leq 3(\log(|T_2|) - \log(|C_1|)) + O(1). \end{aligned}$$

Do đỉnh A là một lá, tiền nhiệm và hậu nhiệm của nó đều là tổ tiên của nó, nên chi phí trả góp để xoay hai đỉnh này lên là

$$\begin{aligned} \Phi' - \Phi + c &\leq 3(\log(|T_2|) + \log(|C_2|) - \log(|A_1|) - \log(|A_1|)) + O(1) \\ &\leq 6(\log(|T_2|) - \log(|A_1|)) + O(1). \end{aligned}$$

Vì vậy chi phí trả góp của bước hai là

$$\Phi' - \Phi + c \leq 9 \log(|T_2|) - 6 \log(|A_1|) - 3 \log(|C_1|) + O(1).$$

Khi đỉnh B hoặc đỉnh C là đỉnh bậc hai, cần xét thêm chi phí trả góp của bước một và bước ba.

Trong bước một, chi phí xoay đỉnh C lên là

$$3(\log(|C_1|) - \log(|C_0|)) + O(1).$$

Xét cách tính thay đổi thế năng sinh ra do thay đổi điểm khóa. Chú ý có thể đổi thứ tự ba bước trong thao tác *splice*: trước hết thực hiện sửa đổi đối với đỉnh C ở bước hai, sau đó biến đỉnh C thành điểm không khóa, rồi biến đỉnh B thành điểm khóa, cuối cùng thực hiện sửa đổi đối với đỉnh B ở bước hai. Điều này không ảnh hưởng tới độ phức tạp thời gian của thao tác *splice*, nên có thể phân tích theo cách đó.

Trước khi đỉnh B trở thành điểm khóa, dựng một đỉnh ảo B' kế thừa mọi thông tin của đỉnh B . Xem B' là một đỉnh *compress* thường và đưa nó vào hàm thế, đồng thời thêm vào hàm thế một hạng $-\log(|B'|)$, tức đối của logarit kích thước cây con của B sau khi nó trở thành điểm khóa. Khi đó chi phí trả góp để biến B thành điểm khóa là

$$\Phi' - \Phi + c = (\log(|B'_3|) + \log(|B'_3|) - \log(|B'_3|)) - \log(|B_2|) + O(1) = O(1).$$

Trước khi đỉnh C trở thành điểm không khóa, thông thường đỉnh ảo C' tương ứng với nó sẽ không bị sửa. Khi đó chi phí trả góp để biến C thành điểm không khóa cũng là $O(1)$. Chỉ khi hai điểm khóa bậc hai nằm trên cùng một chuỗi và đã thực hiện thao tác *reverse* tại đỉnh *twist*, đỉnh C' mới bị sửa. Vì việc lật chỉ thể hiện trong thao tác *splice*, ta có thể tính thay đổi thế năng của nó tại đây.

Khi ấy, có thể xem như đã thực hiện một lần xoay lên đối với đỉnh C' ; chi phí của lần xoay này không vượt quá

$$\log(|A_1|) - \log(|C_1|) + O(1).$$

Do đó tổng chi phí trả góp của bước một và bước ba là

$$\begin{aligned}\Phi' - \Phi + c &\leq 3(\log(|C_1|) - \log(|C_0|)) + \log(|A_1|) - \log(|C_1|) \\ &\leq 3(\log(|A_1|) - \log(|C_0|)) + O(1).\end{aligned}$$

Mỗi thao tác *splice* còn cần chạy một lần đường đi ngắn nhất. Vì trong đồ thị có nhiều nhất $O(k)$ cạnh, độ phức tạp thời gian của lần đường đi ngắn nhất này là $O(k \log k)$.

Chi phí trả góp của thao tác *splice* là tổng của mọi chi phí trên. Vì vậy trong cây ABF, chi phí trả góp của thao tác *splice* là

$$\begin{aligned}\Phi' - \Phi + c &\leq 9 \log(|T_2|) - 6 \log(|A_1|) - 3 \log(|C_1|) + 3(\log(|A_1|) - \log(|C_0|)) \\ &\quad + O(1) + O(k \log k) \\ &\leq 9(\log(|T_3|) - \log(|C_0|)) + O(k \log k).\end{aligned}$$

□

Từ đó có thể thu được độ phức tạp thời gian của thao tác *expose* trong cây ABF.

Bổ đề 5. Trong cây ABF, độ phức tạp thời gian của thao tác *expose* là trung bình trả góp $O(k \log k \log n)$.

Chứng minh. Gọi đỉnh đang thao tác hiện tại là đỉnh hoạt động. Sau thao tác *splice*, đỉnh hoạt động chuyển từ C thành T , vì vậy hạng thứ nhất trong chi phí trả góp của *splice* không vượt quá 9 lần thay đổi thế năng của đỉnh hoạt động. Do thế năng của đỉnh hoạt động không vượt quá $O(\log n)$, tổng độ phức tạp thời gian của hạng thứ nhất là trung bình trả góp $O(\log n)$.

Trong thao tác *expose* có nhiều nhất $O(\log n)$ thao tác *splice*, nên tổng độ phức tạp thời gian của hạng thứ hai là trung bình trả góp $O(k \log k \log n)$.

Tóm lại, trong cây ABF, độ phức tạp thời gian của thao tác *expose* là trung bình trả góp $O(k \log k \log n)$.

□

Tương tự cây AAA, mọi thao tác của cây ABF cũng đều dựa trên thao tác *expose*; vì vậy ta thu được độ phức tạp tổng thể của cây ABF.

Định lý 2. Với n đỉnh, cây ABF thực hiện m thao tác trong thời gian

$$O(mk \log k \log n + n \log n).$$

Chứng minh. Trong cây ABF, trích đường đi ngắn nhất, trích cactus con, thêm cạnh và xóa cạnh đều có thể được thực hiện bằng $O(1)$ lần *expose* rồi sửa $O(1)$ đỉnh.

Hàm thế tại mọi thời điểm đều lớn hơn hoặc bằng 0, và không vượt quá $O(n \log n)$.

Vì vậy, với n đỉnh, cây ABF thực hiện m thao tác trong thời gian

$$O(mk \log k \log n + n \log n).$$

□

Bài toán gốc có thể trực tiếp duy trì bằng cây ABF.

Độ phức tạp thời gian là $O(mk \log k \log n)$.

Tương tự, “Dynamic Cactus IV” cũng có thể được duy trì bằng cây ABF.

Độ phức tạp thời gian là $O(m \log n)$.

5. Tổng kết

Bài toán này là một bài cấu trúc dữ liệu. Từ các bài toán đã có, dựa trên những cấu trúc dữ liệu cơ sở như LCT và cây tròn-vuông, bài viết đề xuất cấu trúc dữ liệu cây ABF, kiểm tra năng lực mô hình hóa và năng lực viết mã của thí sinh.

Thông qua phân tích tỉ mỉ cây ABF, có thể thu được độ phức tạp thời gian khá đẹp.

Các phần điểm bao quát nhiều bài con khác nhau của bài toán gốc, có độ khó khác nhau và có khả năng phân loại tốt.

Hy vọng bài toán này có thể gợi mở cho mọi người, giúp tăng cường khả năng vận dụng linh hoạt các cấu trúc dữ liệu cơ sở.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng trao đổi và học tập.

Xin cảm ơn thầy Xu Xianyou đã hướng dẫn và giúp đỡ tôi.

Xin cảm ơn các anh Wang Yisong, Chen Junkun và bạn Zhao Yuyang đã cung cấp ý tưởng cho bài viết này.

Xin cảm ơn các bạn Zhou Renfei và Lang Sike đã giúp đỡ trong quá trình viết bài.

Xin cảm ơn các bạn Zhang Zheyu và Gao Jiaxuan đã đọc soát bài viết.

Tài liệu tham khảo

1. Tarjan, Robert E.; Werneck, Renato F. *Self-adjusting top trees*.
2. Chen Junkun. Báo cáo ra đề *Đồ thị con kỳ diệu* và mở rộng. Tập luận văn đội tuyển tập huấn quốc gia năm 2017.
3. Wang Yisong. Thuật toán liên quan tới cactus và ứng dụng. Tập luận văn đội tuyển tập huấn quốc gia năm 2015.
4. Huang Zhiao. Bàn về các vấn đề liên quan tới cây động và một số mở rộng đơn giản. Tập luận văn đội tuyển tập huấn quốc gia năm 2014.

Báo cáo ra đề *Đếm điểm nguyên* và một số nghiên cứu về số nguyên Gauss

Xu Yixuan, Trường Trung học Cao cấp Thường Châu, tỉnh Giang Tô

Tóm tắt

Bài viết bắt đầu từ một bài toán kinh điển trong thi tin học, trình bày bối cảnh ra đề, ý tưởng và các lời giải của bài tự kiểm tra đội tuyển ứng viên *Đếm điểm nguyên*. Những kiến thức cần để giải bài, như bộ số Pythagoras, số nguyên Gauss và sàng số học, được giới thiệu có hệ thống; các định lý liên quan cũng được chứng minh hình thức. Đồng thời, khi mở rộng thêm ý tưởng giải bài này, ta còn thu được một chuỗi hội tụ tới π .

1. Lời nói đầu

Toán học là công cụ cơ bản để giải các bài toán tin học và là trọng điểm khảo sát trong thi tin học; trong đó số học số nguyên là một nhánh quan trọng. Trong thi tin học, các điểm kiểm tra liên quan tới số học số nguyên thường xuất hiện dưới dạng sàng, đảo Mobius, nghiên cứu hàm nhân tính, v.v.; chúng thường có độ khó và chiều sâu nhất định.

Trong quá trình nghiên cứu một bài toán kinh điển, tác giả được gợi mở và thu được một cách làm có khả năng mở rộng khá mạnh; từ đó tác giả ra bài tự kiểm tra đội tuyển tập huấn *Đếm điểm nguyên*. Các lời giải của *Đếm điểm nguyên* liên quan tới nhiều điểm kiến thức không xuất hiện quá thường xuyên trong thi tin học, như bộ số Pythagoras, số nguyên Gauss và sàng số học.

Tác giả hy vọng bài viết này, trong khi giới thiệu bối cảnh ra đề và lời giải của *Đếm điểm nguyên*, cũng có thể lưu lại dưới dạng luận văn những nghiên cứu liên quan của tác giả cho giới thi tin học, đóng vai trò gợi mở.

Cấu trúc bài viết như sau: mục 2 giới thiệu bài toán kinh điển mà tác giả nhắc tới, tức cơ duyên ra đề; mục 3 giới thiệu đề bài *Đếm điểm nguyên* và phạm vi dữ liệu; mục 4 khai thác một số tính chất khá hiển nhiên của bài và đưa ra một lời giải vét cạn đơn giản; mục 5 giới thiệu các tính chất liên quan tới bộ số Pythagoras và đưa ra một lời giải từng phần khả thi; mục 6 đào sâu bản chất vấn đề, kết hợp số nguyên Gauss và sàng số học để đưa ra lời giải đúng; mục 7 mở rộng thêm ý tưởng giải bài và thu được một chuỗi hội tụ tới π ; mục 8 bổ sung chứng minh cho một số định lý đã dùng ở trên; mục 9 phân tích và tổng kết điểm kiểm tra, điểm khó, thiết kế tính phân loại của đề và toàn bài viết.

2. Một bài toán kinh điển

2.1. Mô tả đề bài

Bài này bắt nguồn từ HAOI2008 *Điểm nguyên trên đường tròn*²⁰.

Với một đường tròn tâm tại gốc tọa độ và bán kính cho trước r ($1 \leq r \leq 2 \times 10^9$), hãy tính số điểm nguyên trên đường tròn. Điểm nguyên được định nghĩa là điểm có cả hoành độ lẫn tung độ đều là số nguyên.

2.2. Lời giải truyền thống

Trong đề, phạm vi của r khá lớn, không tiện trực tiếp liệt kê để đếm, nên cần một số suy luận toán học.

²⁰<https://www.lydsy.com/JudgeOnline/problem.php?id=1041>

Dễ thấy với mọi r , trên các trục tọa độ luôn có đúng 4 điểm nguyên thỏa yêu cầu, và số điểm thỏa yêu cầu trong các góc phần tư là như nhau. Vì vậy, ta có thể chỉ tính số điểm nguyên Cnt trong góc phần tư thứ nhất, rồi qua một phép tính đơn giản thu được đáp án $4Cnt + 4$.

Giả sử tọa độ một điểm nguyên trên mặt phẳng là (x, y) ($x, y \in \mathbb{N}^+$). Theo định lý Pythagoras, điều kiện cần và đủ để điểm ấy nằm trên đường tròn là $x^2 + y^2 = r^2$. Biến đổi một chút, ta có

$$(r - x)(r + x) = y^2.$$

Đặt $d = \gcd(r - x, r + x)$, khi đó

$$d^2 \times \frac{r - x}{d} \times \frac{r + x}{d} = y^2,$$

và $\frac{r-x}{d}$ nguyên tố cùng nhau với $\frac{r+x}{d}$. Do đó $\frac{r-x}{d}$ và $\frac{r+x}{d}$ đều là số chính phương. Ký hiệu

$$\frac{r - x}{d} = u^2, \quad \frac{r + x}{d} = v^2 \quad (u < v).$$

Lúc này ta có thể biểu diễn r, x, y bằng u, v, d :

$$2r = d(v^2 + u^2), \quad 2x = d(v^2 - u^2), \quad y = duv.$$

Chú ý rằng d là ước của $2r$, và $u^2 < 2r/d$. Ta có thể trực tiếp liệt kê d và u , tính

$$v = \sqrt{2r/d - u^2},$$

rồi kiểm tra $u < v$ và $\gcd(u, v) = 1$.

Trong cách làm trên, số cặp (d, u) có thể bị liệt kê xấp xỉ

$$\sum_{i=1}^{\lfloor \sqrt{r} \rfloor} \sqrt{i} + \sum_{i=1}^{\lfloor \sqrt{r} \rfloor} \sqrt{\frac{r}{i}} = \Theta \left(\int_1^{\lfloor \sqrt{r} \rfloor} \left(\sqrt{x} + \sqrt{\frac{r}{x}} \right) dx \right) = \Theta(r^{3/4}).$$

Tính thêm độ phức tạp của việc tìm $\gcd(u, v)$, tổng độ phức tạp thời gian của cách làm trên là $O(r^{3/4} \log r)$.

2.3. Tiểu kết

Mục này đưa ra lời giải truyền thống của một bài toán kinh điển; tác giả cũng dựa trên lời giải ấy để bắt đầu nghiên cứu bài toán này. Dù lời giải truyền thống không hiệu quả và khả năng mở rộng cũng tương đối kém, những phương pháp toán học như phân tích nhân tử, phân tích tính chia hết được dùng trong đó có ý nghĩa gợi mở đáng kể, và sẽ được sử dụng rộng rãi ở phần sau.

3. Đếm điểm nguyên

3.1. Mô tả đề bài

Với N, k, P cho trước, tính

$$\sum_{i=1}^N f(i)^k \bmod P.$$

Trong đó $f(x)$ biểu thị số điểm nguyên trên đường tròn tâm tại gốc tọa độ và bán kính x . Ví dụ $f(1) = 4, f(5) = 12$.

3.2. Dữ liệu vào

Một dòng gồm ba số nguyên dương N, k, P .

3.3. Dữ liệu ra

Một dòng gồm một số nguyên Ans , biểu thị kết quả của đáp án modulo P .

3.4. Ví dụ vào

5 1 998244353

3.5. Ví dụ ra

28

3.6. Giải thích ví dụ

Dễ thấy ví dụ yêu cầu tính số điểm nguyên cách gốc tọa độ không quá 5 đơn vị và có khoảng cách tới gốc là số nguyên, lấy modulo 998244353.

Các điểm thỏa yêu cầu dạng $(x, 0)$, $(-x, 0)$, $(0, x)$, $(0, -x)$ có $4 \times 5 = 20$ điểm. Ngoài ra còn có

$$(3, 4), (-3, 4), (3, -4), (-3, -4), (4, 3), (-4, 3), (4, -3), (-4, -3),$$

tổng cộng 8 điểm thỏa yêu cầu, nên tổng là 28.

3.7. Phạm vi dữ liệu và quy ước

Với mọi dữ liệu kiểm tra, bảo đảm

$$1 \leq N, k \leq 10^{11}, \quad 10^8 \leq P \leq 10^9 + 9.$$

Phạm vi dữ liệu chi tiết như bảng sau.

Test	Điểm	N	k	P
1–2	3	$\leq 10^3, \leq 8 \times 10^3$	≤ 5	P là số nguyên tố
3–5	6	$\leq 2 \times 10^5, \leq 5 \times 10^5, \leq 3 \times 10^6$	≤ 5	P là số nguyên tố
6–8	5	$\leq 2 \times 10^7$	≤ 5	P là số nguyên tố
9–10	8	$\leq 2 \times 10^8$	$= 1$	không hạn chế thêm
11–12	8	$\leq 2 \times 10^8$	$= 2$	không hạn chế thêm
13–14	1	$\leq 10^9$	$= 15, = 18$	P là lũy thừa của 2
15–16	4	$\leq 10^9$	$\leq 10^9$	P là số nguyên tố
17–18	4	$\leq 10^{10}$	$\leq 10^9$	P là số nguyên tố
19–20	6	$\leq 10^{11}$	$\leq 10^{11}$	không hạn chế thêm

4. Một số lời giải vét cạn

4.1. Thuật toán một

Xét cách tính $f(x)$ với x cho trước, tức số điểm nguyên trên đường tròn tâm tại gốc tọa độ và bán kính x .

Ta có thể liệt kê hoành độ i của một điểm trên đường tròn ($-x \leq i \leq x$), rồi theo định lý Pythagoras tính giá trị tuyệt đối của tung độ tương ứng:

$$|j| = \sqrt{x^2 - i^2}.$$

Bằng cách phán định $|j|$ có phải số nguyên dương hoặc 0 hay không, ta xác định được số nghiệm nguyên của j . Nếu $|j|$ là số nguyên dương, j có 2 nghiệm nguyên; nếu $|j| = 0$, j có 1 nghiệm nguyên; nếu không thì j không có nghiệm nguyên. Từ đó ta biết số điểm nguyên trên đường tròn với hoành độ i , rồi cộng lại để thu được $f(x)$.

Như vậy ta có một thuật toán $O(N)$ để tính $f(N)$. Gọi nhiều lần thuật toán này và tính theo công thức $\sum_{i=1}^N f(i)^k \bmod P$ trong đề, ta thu được một thuật toán $O(N^2)$. Cách này qua được test 1–2, điểm kỳ vọng 6.

4.2. Thuật toán hai

Từ cách tính $f(x)$ trong thuật toán một, ta có thể thử tính giá trị $f(x)$ của một số số:

$$\begin{aligned} f(1) &= 4, f(2) = 4, f(3) = 4, f(4) = 4, f(5) = 12, \\ f(6) &= 4, f(7) = 4, f(8) = 4, f(9) = 4, f(10) = 12, \\ f(15) &= 12, f(25) = 20, f(45) = 12, f(65) = 36, f(100) = 20. \end{aligned}$$

Dù phân bố giá trị của $f(x)$ không có quy luật rõ ràng, ta có thể chú ý rằng mọi $f(x)$ đều là bội của 4. Ta có thể hiểu trực giác hình học này như sau: nếu (x, y) là một điểm nguyên ($x > 0, y > 0$), thì các điểm thu được khi quay nó quanh gốc tọa độ ngược chiều kim đồng hồ $90^\circ, 180^\circ, 270^\circ$ cũng đều là điểm nguyên và có cùng khoảng cách tới gốc tọa độ. Vì vậy $f(x)$ nhất định là bội của 4.

Do đó $f(x)^k$ nhất định là bội của $4^k = 2^{2k}$. Khi $k \geq 15$, $f(x)^k$ nhất định là bội của 2^{30} , nên modulo một lũy thừa của 2 không vượt quá $10^9 + 9$ thì kết quả là 0.

Vì vậy, trực tiếp in 0 qua được test 13–14, điểm kỳ vọng 2. Kết hợp với thuật toán một có thể đạt 8 điểm.

5. Lời giải bằng bộ số Pythagoras

5.1. Thuật toán ba

Quan sát giá trị của $f(x)$ trong thuật toán hai còn cho ta thấy một sự thật: $f(x)$ thường không lớn, khó đạt cấp $O(x)$. Điều đó nghĩa là $\sum_{i=1}^N f(i)$ sẽ không đạt cấp $O(N^2)$. Nếu có một phương pháp hiệu quả để tìm mọi điểm có khoảng cách tới gốc tọa độ là số nguyên, ta có thể thu được một thuật toán $O(\sum_{i=1}^N f(i))$ cho bài này.

Có thể thấy với một điểm nguyên (x, y) ($x > 0, y > 0$) có khoảng cách tới gốc tọa độ là số nguyên z , ta có $x^2 + y^2 = z^2$, tức (x, y, z) là một bộ số Pythagoras. Vì vậy

$$f(x) = 4 + 4g(x),$$

trong đó $g(x)$ biểu thị số cặp bộ số Pythagoras có số lớn nhất là x .

Định nghĩa 5.1. Với bộ số Pythagoras (a, b, c) ($0 < a, b < c$, $a^2 + b^2 = c^2$), nếu $\gcd(a, b) = 1$, ta gọi (a, b, c) là một bộ số Pythagoras nguyên thủy; ngược lại gọi là bộ số Pythagoras dẫn xuất.

Có thể thấy mọi bộ số Pythagoras dẫn xuất (x, y, z) đều có thể biểu diễn duy nhất thành bội nguyên dương của một bộ số Pythagoras nguyên thủy nào đó. Tức là với một bộ dẫn xuất (x, y, z) , tồn tại duy nhất một bộ nguyên thủy (a, b, c) và số nguyên dương k ($k \geq 2$) sao cho

$$x = ka, \quad y = kb, \quad z = kc.$$

Do đó, chỉ cần ta nhanh chóng tìm được mọi bộ số Pythagoras nguyên thủy rồi nhân chúng với một dãy số nguyên dương, ta có thể nhanh chóng tìm được mọi bộ số Pythagoras và tính $g(x)$.

Về cách sinh bộ số Pythagoras nguyên thủy, có bổ đề kinh điển sau. Nếu bạn đọc đã biết thì có thể bỏ qua phần chứng minh.

Bổ đề 5.1.1. Với mọi bộ số Pythagoras nguyên thủy (a, b, c) ($0 < a, b < c$, $a^2 + b^2 = c^2$, $\gcd(a, b) = 1$), tồn tại các số nguyên dương n, m ($n < m$, $n + m \equiv 1 \pmod{2}$, $\gcd(n, m) = 1$) sao cho

$$a = 2mn, \quad b = m^2 - n^2, \quad c = m^2 + n^2$$

hoặc

$$a = m^2 - n^2, \quad b = 2mn, \quad c = m^2 + n^2.$$

Chứng minh. Nếu a, b đều là số lẻ, thì $a^2 \equiv b^2 \equiv 1 \pmod{4}$, nên $c^2 \equiv a^2 + b^2 \equiv 2 \pmod{4}$. Nhưng 2 không phải thặng dư bậc hai modulo 4, do đó trong a, b tất phải có một số lẻ và một số chẵn.

Không mất tính tổng quát, giả sử $a = 2k$. Khi đó

$$(2k)^2 = 4k^2 = c^2 - b^2 = (c + b)(c - b).$$

Vì $(c + b)(c - b)$ phải là số chẵn, c và b phải cùng là số lẻ.

Đặt

$$x = \frac{c + b}{2}, \quad y = \frac{c - b}{2};$$

rõ ràng x, y đều là số nguyên dương. Nếu tồn tại số nguyên tố p sao cho $p \mid x, p \mid y$, thì $p \mid x + y (= c)$ và $p \mid x - y (= b)$, suy ra $p \mid c, p \mid b$, rồi $p \mid a$, mâu thuẫn với $\gcd(a, b) = 1$. Vì vậy không tồn tại p như thế, tức $\gcd(x, y) = 1$.

Ghi $xy = k^2 = p_1^{k_1} p_2^{k_2} \dots p_s^{k_s}$, trong đó $p_1, p_2, \dots, p_s$ đều là số nguyên tố, còn $k_1, k_2, \dots, k_s$ đều là số chẵn dương. Do $\gcd(x, y) = 1$, suy ra x, y đều là số chính phương.

Đặt $m = \sqrt{x}, n = \sqrt{y}$. Khi đó

$$b = x - y = m^2 - n^2, \quad c = x + y = m^2 + n^2, \quad a = \sqrt{c^2 - b^2} = 2mn.$$

Hơn nữa, do $\gcd(x, y) = 1$, $\gcd(m, n) = 1$. Do $\gcd(a, b) = 1$, tức $\gcd(2mn, m^2 - n^2) = 1$, m, n khác tính chẵn lẻ, tức $n + m \equiv 1 \pmod{2}$. $\square$

Bổ đề 5.1.2. Với mọi số nguyên dương n, m ($n < m$, $n + m \equiv 1 \pmod{2}$, $\gcd(m, n) = 1$),

$$(2mn, m^2 - n^2, m^2 + n^2), \quad (m^2 - n^2, 2mn, m^2 + n^2)$$

đều là bộ số Pythagoras nguyên thủy.

Chứng minh. Trước hết, $(2mn)^2 + (m^2 - n^2)^2 = (m^2 + n^2)^2$ luôn đúng. Vì vậy hai bộ trên đều là bộ số Pythagoras.

Nếu tồn tại số nguyên tố $p \neq 2$ sao cho $p \mid 2mn$ và $p \mid m^2 - n^2$, thì $p \mid m, p \mid n$, mâu thuẫn với $\gcd(m, n) = 1$. Lại do $n + m \equiv 1 \pmod{2}$, $m^2 - n^2$ là số lẻ, nên $\gcd(2mn, m^2 - n^2) = 1$. Do đó hai bộ trên đều là bộ số Pythagoras nguyên thủy. $\square$

Hai bổ đề trên cho ta một thuật toán kinh điển để sinh hiệu quả mọi bộ số Pythagoras nguyên thủy: ta chỉ cần liệt kê mọi m, n thỏa điều kiện. Chú ý rằng bộ số Pythagoras cần xét trong bài phải thỏa $c = m^2 + n^2 \leq N$, nên phạm vi của m, n đều ở cấp $O(\sqrt{N})$.

Sinh mọi bộ số Pythagoras nguyên thủy trong phạm vi rồi tìm mọi bộ dẫn xuất tương ứng, ta thu được một thuật toán $O(\sum_{i=1}^N g(i))$ cho bài này. Khi $N = 2 \times 10^7$, số cặp bộ số Pythagoras tìm được, không tính thứ tự của a, b , là $49149097 \approx 5 \times 10^7$. Cách này qua được test 1–8, điểm kỳ vọng 38; kết hợp thuật toán hai có thể đạt 40 điểm.

6. Lời giải chuẩn

6.1. Thuật toán bốn

Cách tính $f(x)$ được nhắc tới trong thuật toán một có độ phức tạp $O(x)$ và khó mở rộng; ta cần một phương pháp hiệu quả hơn để tính $f(x)$.

Khi xét bài toán điểm nguyên trên mặt phẳng hai chiều, điểm nguyên (x, y) không chỉ biểu diễn một điểm trên mặt phẳng hai chiều, mà còn có thể biểu diễn một số nguyên Gauss²¹ $x + yi$ trên mặt phẳng phức.

Trong bài này, ta yêu cầu khoảng cách từ điểm nguyên (x, y) tới gốc tọa độ, $\sqrt{x^2 + y^2}$, là số nguyên. Một cách khác để nhìn khoảng cách này là tích của số nguyên Gauss $x + yi$ với số phức liên hợp của nó $x - yi$:

$$(x + yi)(x - yi) = x^2 + y^2.$$

Giá trị thu được đúng bằng bình phương khoảng cách từ điểm nguyên (x, y) tới gốc tọa độ.

Quan sát này giúp ta chuyển bài toán thành một bài toán phân tích thừa số nguyên tố: giá trị $f(x)$ thực ra bằng số số nguyên Gauss có tích với liên hợp của nó bằng x^2 .

Để giải bài toán mới này, ta cần làm rõ một điểm: tương tự số nguyên, vành số nguyên Gauss là miền phân tích duy nhất²². Điều đó có nghĩa số nguyên Gauss cũng có thể, giống như phân tích duy nhất trong số nguyên, viết thành tích của một dãy số nguyên tố Gauss không thể tiếp tục phân tích.

Nhắc lại phân tích duy nhất của số nguyên: với một số nguyên tùy ý, chẳng hạn 42, ta có thể biểu diễn duy nhất nó thành tích của một số số nguyên tố:

$$42 = 2 \times 3 \times 7.$$

Biểu diễn như vậy là “gần như” duy nhất, trừ khi ta cho phép số âm xuất hiện, chẳng hạn

$$42 = 2 \times 3 \times 7 = (-2) \times 3 \times (-7).$$

Tương tự số nguyên, với một số nguyên Gauss tùy ý, chẳng hạn $15 + 20i$, ta cũng có thể biểu diễn duy nhất nó thành tích của một số số nguyên tố Gauss:

$$15 + 20i = (2 - i)(2 + i)(3 + 4i).$$

Biểu diễn như vậy cũng là “gần như” duy nhất, trừ khi ta cho phép một số nhân tử trở thành số đối của chúng:

$$15 + 20i = (2 - i)(2 + i)(3 + 4i) = (-2 + i)(-2 - i)(3 + 4i),$$

hoặc cho phép một số nhân tử nhân thêm i và $-i$:

$$15 + 20i = (i(2 - i))(-i(2 + i))(3 + 4i) = (1 + 2i)(1 - 2i)(3 + 4i).$$

Trong bài này, ta cần phân tích một dãy số chính phương. Nếu biết số nguyên tố phân tích thành số nguyên Gauss như thế nào, ta có thể phân tích các số chính phương cần xét thành số nguyên Gauss.

²¹Khái niệm và thuật ngữ về số nguyên Gauss sẽ được bổ sung ở mục 8 cho bạn đọc chưa quen thuộc.

²²Điều này không hiển nhiên; để không cản trở mạch đọc, tác giả sẽ chứng minh bổ sung ở mục 8.

Định lý Fermat về tổng hai bình phương. Một số nguyên tố lẻ p có thể biểu diễn thành tổng bình phương của hai số nguyên dương a, b ,

$$p = a^2 + b^2,$$

khi và chỉ khi $p \equiv 1 \pmod{4}$; hơn nữa, nếu biểu diễn ấy tồn tại thì nó là duy nhất nếu không xét thứ tự.

Kết hợp với tính phân tích duy nhất của số nguyên Gauss, định lý Fermat về tổng hai bình phương thực chất nói rằng: với một số nguyên tố dạng $4k + 1$ là p , ta có thể tìm một cặp số nguyên Gauss liên hợp $a + bi, a - bi$ sao cho tích của chúng là p . Nếu không xét việc lấy đối hai nhân tử này hoặc lần lượt nhân chúng với i và $-i$, cặp số nguyên Gauss như thế là duy nhất; đồng thời cặp số nguyên Gauss này đều là số nguyên tố Gauss. Còn một số nguyên tố dạng $4k + 3$ bất kỳ không thể phân tích thành tích của một cặp số nguyên Gauss liên hợp, nên số nguyên tố dạng $4k + 3$ là số nguyên tố Gauss.

Số nguyên tố 2 cũng có thể phân tích thành tích của hai số nguyên tố Gauss liên hợp:

$$2 = (1 - i)(1 + i).$$

Đến đây, ta đã có thể phân tích số nguyên bất kỳ thành tích của các số nguyên tố Gauss. Vấn đề duy nhất còn lại là tính $f(x)$.

Chú ý quan sát trong thuật toán hai: nếu (x, y) là điểm nguyên ($x > 0, y > 0$), thì khi quay nó quanh gốc tọa độ ngược chiều kim đồng hồ $90^\circ, 180^\circ, 270^\circ$, ta vẫn thu được điểm nguyên có cùng khoảng cách tới gốc. Nhìn bằng số nguyên Gauss, tức là với số nguyên Gauss bất kỳ $x + yi$, sau khi nhân với $i, -1, -i$, khoảng cách của nó tới gốc trên mặt phẳng phức không đổi. Ta coi các số nguyên Gauss như vậy là về bản chất giống nhau²³. Sau đây ta thảo luận cách tính số phương án phân tích x^2 thành tích của hai số nguyên Gauss liên hợp, khác nhau về bản chất, tức $f(x)/4$.

Trước hết xét một trường hợp đơn giản: tính $f(225)/4$, trong đó

$$225 = 3^2 \times 5^2 = 3^2(2 - i)^2(2 + i)^2.$$

Theo thảo luận ở trên, các số nguyên tố khác nhau sẽ phân tích thành các số nguyên Gauss khác nhau, còn cặp số nguyên Gauss thu được cuối cùng là liên hợp. Điều này cho thấy từng số nguyên tố phải độc lập phân tích thành tích của một cặp số nguyên Gauss liên hợp. Vì vậy số phương án phân tích 225, $f(225)/4$, thực ra là tích của số phương án phân tích 9 và 25, tức $f(9)/4 \times f(25)/4$.

Phân tích lũy thừa p^k của một số nguyên tố dạng $4k + 3$ là dễ: khi và chỉ khi k chẵn mới có 1 cách phân tích hợp lệ, tức

$$p^k = p^{k/2} \times p^{k/2}.$$

Trong bài này các số cần phân tích đều là số chính phương, nên số nguyên tố dạng $4k + 3$ không ảnh hưởng tới số phương án phân tích.

Xét số phương án phân tích lũy thừa p^k của một số nguyên tố dạng $4k + 1$. Theo thảo luận ở trên,

$$p^k = (a + bi)^k(a - bi)^k.$$

Khi đó p^k có thể phân tích thành

$$(a + bi)^x(a - bi)^{k-x} \times (a + bi)^{k-x}(a - bi)^x \quad (x \in \{0, 1, 2, \dots, k\}),$$

tổng cộng $k + 1$ phương án.

Số 2 là một số nguyên tố đặc biệt, vì các số nguyên tố Gauss $(1 - i), (1 + i)$ mà nó phân tích ra là giống nhau về bản chất. Vì vậy 2 cũng không ảnh hưởng tới số phương án phân tích.

²³Thực ra quan hệ này gọi là liên kết.

Đến đây, ta có một thuật toán tính $f(x)$ trong độ phức tạp bằng độ phức tạp phân tích thừa số nguyên tố. Đặt

$$x^2 = p_1^{k_1} p_2^{k_2} \cdots p_s^{k_s},$$

trong đó $p_1, p_2, \dots, p_s$ đều là số nguyên tố, còn $k_1, k_2, \dots, k_s$ đều là số chẵn dương. Khi đó

$$f(x) = 4 \times \prod_{i=1}^s (1 + [p_i \equiv 1 \pmod{4}] \times k_i).$$

Nhờ thừa số nguyên tố nhỏ nhất của mỗi số x do sàng nguyên tố Euler tìm được, sau khi tiền xử lý $O(N)$, ta có thể phân tích một số nguyên N trong thời gian $O(\log N)$. Vì vậy ta thu được một thuật toán $O(N \log N)$, qua được test 1–8, điểm kỳ vọng 38.

Hơn nữa, ưu thế của thuật toán này so với thuật toán ba là bản thân việc phân tích thừa số nguyên tố không phụ thuộc vào bộ nhớ cấp $O(N)$, và quá trình xử lý $f(i)$ độc lập với nhau. Điều đó cho thấy ta có thể bỏ trước một ít thời gian để tính giá trị $\sum_{i=1}^x f(i)^k$ là *value*, lưu nó trong code. Khi thật sự cần tính $\sum_{i=1}^N f(i)^k$ với $N \geq x$, ta chỉ cần tính

$$value + \sum_{i=x+1}^N f(i)^k.$$

Trong test 9–12, giá trị của k chỉ là 1–2, nên giá trị thật của $\sum_{i=1}^N f(i)^k$ nằm trong phạm vi kiểu nguyên 64 bit. Giả sử đặt một ngưỡng α , tiền xử lý đáp án của bài toán khi $N = \alpha, 2\alpha, 3\alpha, \dots, n\alpha$ và lưu trong code, khi chạy thực tế ta chỉ cần tính thêm $O(\alpha)$ giá trị $f(x)$ là có thể đưa ra đáp án.

Nếu lấy $\alpha = 4 \times 10^5$, ta cần lưu

$$\frac{2 \times 10^8}{4 \times 10^5} = 500$$

đáp án đã tiền xử lý cho $k = 1, 2$, chiếm khoảng 12kb độ dài code, có thể chấp nhận được.

Độ phức tạp thời gian của phần này là $O(\alpha \cdot \text{factorize}(N))$, trong đó $\text{factorize}(N)$ biểu thị độ phức tạp thời gian để phân tích N thành thừa số nguyên tố. Cách này qua được test 9–12; tổng cộng qua được test 1–12, điểm kỳ vọng 70; kết hợp thuật toán hai có thể đạt 72 điểm.

6.2. Thuật toán năm

Trong quá trình suy ra thuật toán bốn, có một quan sát cực kỳ quan trọng: “từng số nguyên tố phải độc lập phân tích thành tích của một cặp số nguyên Gauss liên hợp”. Điều này cho thấy nếu đặt $g(x) = f(x)/4$, thì $g(x)$ là hàm nhân tính.

Do đó, nếu biến đổi một chút công thức cần tính của đề:

$$\sum_{i=1}^N f(i)^k = 4^k \times \sum_{i=1}^N g(i)^k,$$

thì việc cần làm là tính tổng tiền tố của hàm nhân tính $h(x) = g(x)^k$. Ta xét dùng sàng quen thuộc với thí sinh để giải bài toán này.

Đặt $\text{cnt}_{0,x}$ là số số nguyên tố dạng $4k + 1$ không vượt quá x , và $\text{cnt}_{1,x}$ là số số nguyên tố dạng $4k + 3$ không vượt quá x . Dù dùng sàng Zhounge hay sàng Min25, với mỗi giá trị khác nhau $x = \lfloor N/i \rfloor$, ta đều cần tính $\text{cnt}_{0,x}, \text{cnt}_{1,x}$.

Trước hết dùng sàng tuyến tính để tìm mọi số nguyên tố $\text{prime}_i \leq \sqrt{N}$, cùng số lượng $\text{pre}_{0,i}, \text{pre}_{1,i}$ của các số nguyên tố dạng $4k + 1, 4k + 3$ không vượt quá prime_i .

Định nghĩa

$$\begin{aligned} \text{getc}_{0}(N, i) &= \sum_{j=1}^N [j \text{ là số nguyên tố hoặc } \text{Min}_j > \text{prime}_i] \times [j \equiv 1 \pmod{4}], \\ \text{getc}_{1}(N, i) &= \sum_{j=1}^N [j \text{ là số nguyên tố hoặc } \text{Min}_j > \text{prime}_i] \times [j \equiv 3 \pmod{4}], \end{aligned}$$

trong đó Min_x biểu thị thừa số nguyên tố nhỏ nhất của x .

Trực giác mà nói, $\text{getc}_{0}(N, i)$, $\text{getc}_{1}(N, i)$ biểu thị số các số dạng $4k + 1$, $4k + 3$ không vượt quá N vẫn chưa bị sàng đi sau vòng thứ i của thuật toán sàng Eratosthenes. Chú ý vì ở đây ta không cần xét số chẵn, nên quy ước ở vòng thứ nhất của sàng Eratosthenes, tức $\text{prime}_i = 2$, không có số nào bị sàng đi.

Một hợp số N nhất định có một thừa số nguyên tố không vượt quá $\sqrt{N}$, nên

$$\text{getc}_{0}(x, \text{Cnt}), \text{getc}_{1}(x, \text{Cnt})$$

chính là $\text{cnt}_{0,x}$, $\text{cnt}_{1,x}$, trong đó Cnt là số số nguyên tố không vượt quá $\sqrt{N}$.

Xét cách dùng $\text{getc}_{0}(*, i-1)$, $\text{getc}_{1}(*, i-1)$ để tính $\text{getc}_{0}(*, i)$, $\text{getc}_{1}(*, i)$. Nếu $\text{prime}_i^2 > N$, vòng thứ i của sàng Eratosthenes không sàng đi số nào:

$$\text{getc}_{0}(N, i) = \text{getc}_{0}(N, i-1), \quad \text{getc}_{1}(N, i) = \text{getc}_{1}(N, i-1).$$

Nếu $\text{prime}_i^2 \leq N$, xét việc xóa khỏi $\text{getc}_{0}(N, i-1)$, $\text{getc}_{1}(N, i-1)$ những số bị vòng thứ i của sàng Eratosthenes sàng đi, tức các hợp số có thừa số nguyên tố nhỏ nhất là prime_i . Do $\text{prime}_i^2 \leq N$, ta có $\lfloor N/\text{prime}_i \rfloor \geq \text{prime}_i$. Vì vậy số các số dạng $4k + 1$, $4k + 3$ không vượt quá $\lfloor N/\text{prime}_i \rfloor$ và có thừa số nguyên tố nhỏ nhất không nhỏ hơn prime_i lần lượt là

$$\begin{aligned} &\text{getc}_{0}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{0,i-1}, \\ &\text{getc}_{1}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{1,i-1}. \end{aligned}$$

Nếu $\text{prime}_i \equiv 1 \pmod{4}$, ta có

$$\begin{aligned} \text{getc}_{0}(N, i) &= \text{getc}_{0}(N, i-1) - \left(\text{getc}_{0}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{0,i-1} \right), \\ \text{getc}_{1}(N, i) &= \text{getc}_{1}(N, i-1) - \left(\text{getc}_{1}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{1,i-1} \right). \end{aligned}$$

Ngược lại, tức $\text{prime}_i \equiv 3 \pmod{4}$, ta có

$$\begin{aligned} \text{getc}_{0}(N, i) &= \text{getc}_{0}(N, i-1) - \left(\text{getc}_{1}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{1,i-1} \right), \\ \text{getc}_{1}(N, i) &= \text{getc}_{1}(N, i-1) - \left(\text{getc}_{0}\left(\left\lfloor \frac{N}{\text{prime}_i} \right\rfloor, i-1\right) - \text{pre}_{0,i-1} \right). \end{aligned}$$

Tổng kết lại, điều kiện đầu là

$$\text{getc}_{0}(N, 0) = \left\lfloor \frac{N-1}{4} \right\rfloor, \quad \text{getc}_{1}(N, 0) = \left\lfloor \frac{N+1}{4} \right\rfloor,$$

và

$$\text{cnt}_{0,x} = \text{getc}_{0}(x, \text{Cnt}), \quad \text{cnt}_{1,x} = \text{getc}_{1}(x, \text{Cnt}).$$

Thuật toán trên có thể tính được $cnt_{0,x}, cnt_{1,x}$ với mọi giá trị khác nhau $x = \lfloor N/i \rfloor$ trong thời gian $O(N^{3/4}/\log N)$. Tiếp theo, chỉ cần lợi dụng các giá trị đã có để tính đáp án. Lấy cách làm của người ra đề làm ví dụ, phần sau suy ra một cách làm bằng sàng Min25.

Định nghĩa

$$s(N, i) = \sum_{j=1}^N [Min_j \geq prime_i] \times h(j),$$

tức tổng giá trị h của mọi số có thừa số nguyên tố nhỏ nhất không nhỏ hơn $prime_i$. Theo định nghĩa, đại lượng cần tính là

$$\sum_{i=1}^N h(i) = s(N, 2) + h(1) + [N \geq 2] \times h(2).$$

Nhờ kết quả tính ở trên, ta đã có thể nhanh chóng tính số lượng số nguyên tố dạng $4k + 1, 4k + 3$ trong một phạm vi, nên đóng góp của số nguyên tố trong $s(N, i)$ dễ tính:

$$3^k \times (cnt_{0,N} - pre_{0,i-1}) + (cnt_{1,N} - pre_{1,i-1}).$$

Tiếp theo xét đóng góp của hợp số trong đáp án. Ta liệt kê thừa số nguyên tố nhỏ nhất $prime_j$ của hợp số này và số lần xuất hiện e của nó. Do h là hàm nhân tính, đóng góp của hợp số là

$$\sum_{j=i}^{prime_j^2 \leq N} \sum_{e=1}^{prime_j^{e+1} \leq N} \left(h(prime_j^e) \times s\left(\left\lfloor \frac{N}{prime_j^e} \right\rfloor, j+1\right) + h(prime_j^{e+1}) \right).$$

Sau khi tính được $\sum_{i=1}^N h(i)$, đáp án cuối cùng là

$$4^k \times \sum_{i=1}^N h(i).$$

Độ phức tạp thời gian của phần này là $O(\text{Min25}(N))$. Với phạm vi dữ liệu $N \leq 10^{11}$, hiệu suất chạy của nó tương đương sàng Zhounge có độ phức tạp $O(N^{3/4}/\log N)$. Cách này qua được mọi test, điểm kỳ vọng 100.

7. Một số mở rộng

Có thể thấy hàm $f(x)$ trong đề luôn tính số điểm nguyên trên đường tròn có bán kính là số nguyên; trong khi đó nhiều điểm nguyên có khoảng cách tới gốc tọa độ không phải số nguyên, chẳng hạn khoảng cách từ $(1, 1)$ tới gốc tọa độ là $\sqrt{2}$, nên nó sẽ không được bất kỳ $f(x)$ nào thống kê.

Định nghĩa $f'(x)$ là số điểm nguyên trên đường tròn tâm tại gốc tọa độ và bán kính $\sqrt{x}$. Dễ thấy $f'(x^2) = f(x)$, và $f'(x)$ có thể coi là một định nghĩa tổng quát hơn của $f(x)$.

Trong thảo luận ở thuật toán bốn, ta cũng có thể thu được cách tính $f'(x)$: đặt $x = p_1^{k_1} p_2^{k_2} \cdots p_s^{k_s}$, trong đó các p_i đều là số nguyên tố và các k_i đều là số nguyên dương. Khi đó

$$f'(x) = 4 \times \prod_{i=1}^s ([p_i = 2] + [p_i \equiv 1 \pmod{4}](k_i + 1) + [p_i \equiv 3 \pmod{4}][k_i \equiv 0 \pmod{2}]).$$

Định nghĩa hàm $\chi(x)$ ($x \in \mathbb{N}^+$):

$$\chi(x) = \begin{cases} 1, & x \equiv 1 \pmod{4}, \\ -1, & x \equiv 3 \pmod{4}, \\ 0, & x \equiv 0 \pmod{2}. \end{cases}$$

Với số nguyên tố dạng $4k + 1$ là p_i ,

$$\sum_{j=0}^{k_i} \chi(p_i^j) = k_i + 1;$$

với số nguyên tố dạng $4k + 3$ là p_i ,

$$\sum_{j=0}^{k_i} \chi(p_i^j) = [k_i \equiv 0 \pmod{2}];$$

với $p_i = 2$,

$$\sum_{j=0}^{k_i} \chi(p_i^j) = 1.$$

Tóm lại, khác với cách tính $f'(x)$ ở trên vốn phải phân loại thừa số nguyên tố khác nhau, nhờ hàm $\chi(x)$, ta thậm chí có thể bỏ qua việc phân loại thừa số nguyên tố:

$$f'(x) = 4 \times \prod_{i=1}^s \sum_{j=0}^{k_i} \chi(p_i^j).$$

Đồng thời, chú ý rằng $\chi(x)$ cũng là một hàm hoàn toàn nhân tính, tức với mọi $a, b \in \mathbb{N}^+$ đều có $\chi(a) \times \chi(b) = \chi(ab)$. Kết hợp với biểu thức của $f'(x)$ ở trên, ta thấy ký hiệu $\sum$ liệt kê từng số mũ của từng thừa số nguyên tố, còn ký hiệu $\prod$ ghép các trường hợp của các thừa số nguyên tố khác nhau lại với nhau. Vì vậy biểu thức ấy cũng có thể viết thành

$$f'(x) = 4 \times \sum_{i|x} \chi(i).$$

Đến đây ta thu được một biểu thức đơn giản đến bất ngờ của $f'(x)$.

Bây giờ hãy xét một câu hỏi: trong một đường tròn bán kính R cho trước có bao nhiêu điểm nguyên?

Một mặt, số điểm nguyên bên trong nó xấp xỉ diện tích của nó, tức πR^2 . Dù đây chỉ là một ước lượng gần đúng, khi R tiến tới vô cùng dương, tỉ lệ giữa sai số và giá trị thật sẽ ngày càng nhỏ, đến mức có thể bỏ qua.

Mặt khác, ta cũng có thể lấy tổng $f'(x)$ từ 1 tới R^2 ; số điểm nguyên bên trong đường tròn xấp xỉ²⁴

$$\sum_{i=1}^{R^2} f'(i) = \sum_{i=1}^{R^2} 4 \times \sum_{j|i} \chi(j).$$

Đổi thứ tự lấy tổng, trước hết liệt kê j , ta biến đổi được thành

$$4 \times \sum_{j=1}^{R^2} \chi(j) \times \left\lfloor \frac{R^2}{j} \right\rfloor.$$

Bỏ qua một số sai số nhỏ khi R tiến tới vô cùng dương, ta thu được đẳng thức xấp xỉ

$$\pi R^2 \approx 4 \times \sum_{j=1}^{R^2} \chi(j) \times \frac{R^2}{j} = 4R^2 \times \sum_{j=1}^{R^2} \frac{\chi(j)}{j}.$$

Hơn nữa, có thể chứng minh rằng khi R tiến tới vô cùng dương, sai số giữa hai vế của đẳng thức có thể nhỏ tùy ý.

Lúc này, chia hai vế cho R^2 , ta thu được

$$\pi = 4 \times \sum_{i \in \mathbb{N}^+} \frac{\chi(i)}{i} = 4 \times \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots \right).$$

Đó chính là một chuỗi vô hạn hội tụ tới π một cách đáng kinh ngạc.

²⁴Ở đây dùng “xấp xỉ” vì bỏ qua một điểm nguyên tại gốc tọa độ; khi R tiến tới vô cùng dương, sai số này cũng có thể bỏ qua.

8. Chứng minh bổ sung

8.1. Số nguyên Gauss

Trước hết, ta cần giới thiệu các khái niệm và thuật ngữ liên quan tới số nguyên Gauss cho bạn đọc chưa quen thuộc.

Định nghĩa 8.1.1. Nếu phần thực và phần ảo của một số phức đều là số nguyên hữu tỉ, ta gọi nó là một số nguyên Gauss, tức

$$\mathbb{Z}[i] = \{a + bi \mid a, b \in \mathbb{Z}, i^2 = -1\}.$$

Phép cộng và phép nhân định nghĩa trên $\mathbb{Z}[i]$ chính là phép cộng và phép nhân số phức thông thường; các số nguyên Gauss tạo thành một vành số nguyên.

Định nghĩa 8.1.2. Định nghĩa chuẩn của số nguyên Gauss $a + bi$ là tích của số nguyên Gauss đó với số nguyên Gauss liên hợp của nó:

$$N(a + bi) = (a + bi)(a - bi) = a^2 + b^2.$$

Chú ý bản thân số nguyên Gauss không so sánh lớn nhỏ được, nhưng với định nghĩa chuẩn, ta có thể sắp thứ tự số nguyên Gauss theo một ý nghĩa nào đó.

Định nghĩa 8.1.3. Nếu số nguyên Gauss $a + bi$ thỏa $N(a + bi) = 1$, ta gọi nó là một đơn vị.

Theo định nghĩa, có tất cả 4 đơn vị, lần lượt là $1, -1, i, -i$; chúng cũng là những số nguyên Gauss khả nghịch duy nhất. Nếu hai số nguyên Gauss chỉ khác nhau bởi một thừa số đơn vị, ta gọi hai số nguyên Gauss ấy là liên kết²⁵.

Định nghĩa 8.1.4. Với các số nguyên Gauss X, Y , nếu tồn tại số nguyên Gauss Z sao cho $X = YZ$, ta nói Y chia hết X , ký hiệu $Y \mid X$.

Dễ thấy nếu Y chia hết X , thì $N(Y)$ cũng chia hết $N(X)$.

Định nghĩa 8.1.5. Với các số nguyên Gauss X, Y, g , nếu $g \mid X$ và $g \mid Y$, ta gọi g là một ước chung của X, Y . Nếu g là một ước chung của X, Y và với mọi ước chung d của X, Y đều có $d \mid g$, ta gọi g là ước chung lớn nhất của X, Y .

Ước chung lớn nhất có thể coi là ước chung có chuẩn lớn nhất. Nếu ước chung lớn nhất của hai số nguyên Gauss là một đơn vị, ta nói hai số nguyên Gauss ấy nguyên tố cùng nhau.

Định nghĩa 8.1.6. Với một số nguyên Gauss X không phải đơn vị, nếu không tồn tại các số nguyên Gauss Y, Z không phải đơn vị sao cho $X = YZ$, ta gọi X là một số nguyên tố Gauss.

Theo định nghĩa, ta có thể viết 4 số nguyên tố Gauss có chuẩn nhỏ nhất:

$$1 + i, \quad 1 - i, \quad -1 + i, \quad -1 - i.$$

Chú ý một số nguyên tố thông thường có thể không phải số nguyên tố Gauss, ví dụ $2 = (1 + i)(1 - i)$.

²⁵Ở mục 6, để bạn đọc nhanh chóng hiểu thuật toán, hai số nguyên Gauss có quan hệ liên kết được gọi là “về bản chất giống nhau”.

8.2. Phân tích duy nhất

Ở mục 6, ta đã nhắc tới tính phân tích duy nhất của số nguyên Gauss; sau đây ta chứng minh điểm này²⁶.

Bổ đề 8.2.1. (Chia có dư) Với mọi số nguyên Gauss $\alpha, \beta, \beta \neq 0$, tồn tại các số nguyên Gauss γ, λ sao cho

$$\alpha = \gamma\beta + \lambda, \quad N(\lambda) < N(\beta).$$

Chứng minh. Xét số phức $\alpha/\beta = a + bi$ ($a, b \in \mathbb{Q}$). Chọn các số nguyên c, d sao cho $|a - c| \leq 0.5, |b - d| \leq 0.5$, đặt $\gamma = c + di$. Thay vào để tính λ , ta có

$$\begin{aligned}\lambda &= \alpha - \gamma\beta \\ &= \beta \left(\frac{\alpha}{\beta} - \gamma \right) \\ &= \beta((a - c) + (b - d)i).\end{aligned}$$

Do đó

$$\begin{aligned}N(\lambda) &= N(\beta((a - c) + (b - d)i)) \\ &= N(\beta)N((a - c) + (b - d)i) \\ &= N(\beta)((a - c)^2 + (b - d)^2) \\ &\leq N(\beta)(0.5^2 + 0.5^2) \\ &< N(\beta).\end{aligned}$$

Bổ đề 8.2.1 được chứng minh. $\square$

Bổ đề 8.2.2. (Đẳng thức Bezout) Với mọi số nguyên Gauss khác 0 là α, β , gọi d là ước chung lớn nhất của chúng. Khi đó tồn tại các số nguyên Gauss γ, δ sao cho

$$\alpha\gamma + \beta\delta = d.$$

Chứng minh. Quá trình chứng minh bổ đề 8.2.1 đồng thời cho ta một cách thực hiện chia có dư trong số nguyên Gauss. Vì vậy ta có thể dùng thuật toán Euclid để tìm ước chung lớn nhất của một cặp số nguyên Gauss. Hơn nữa, đảo ngược quá trình chia Euclid và lần lượt thế vào, ta có thể dựng được một đẳng thức thỏa bổ đề 8.2.2. $\square$

Bổ đề 8.2.3. (Bổ đề Euclid) Nếu một số nguyên tố Gauss π chia hết tích $\alpha\beta$ của hai số nguyên Gauss, thì số nguyên tố Gauss ấy chia hết ít nhất một trong hai thừa số, tức ít nhất một trong $\pi \mid \alpha$ và $\pi \mid \beta$ đúng.

Chứng minh. Giả sử $\pi \nmid \alpha$. Ta chỉ cần chứng minh khi đó $\pi \mid \beta$. Do π là số nguyên tố Gauss và $\pi \nmid \alpha$, ước chung lớn nhất của π và α là một đơn vị; không mất tính tổng quát, giả sử là 1.

Khi đó tồn tại các số nguyên Gauss γ, δ sao cho

$$\pi\gamma + \delta\alpha = 1.$$

Suy ra

$$\pi\gamma\beta + \delta\alpha\beta = \beta.$$

Lại vì $\pi \mid \pi\gamma\beta$ và $\pi \mid \delta\alpha\beta$, nên $\pi \mid \beta$. Bổ đề 8.2.3 được chứng minh. $\square$

²⁶Không phải mọi vành đều có tính phân tích duy nhất. Ví dụ trong vành số chẵn, $60 = 2 \times 30 = 6 \times 10$, cách phân tích là không duy nhất.

Định lý 8.2. (Định lý phân tích duy nhất) Mọi số nguyên Gauss có chuẩn lớn hơn 1 đều có thể phân tích thành tích hữu hạn của các số nguyên tố Gauss. Nếu không xét thừa số đơn vị và thứ tự các nhân tử, cách phân tích như vậy là duy nhất.

Chứng minh. Xét quy nạp toán học. Các số nguyên Gauss có chuẩn bằng 2 đều là số nguyên tố Gauss, nên định lý phân tích duy nhất đúng với phạm vi các số nguyên Gauss có chuẩn không vượt quá 2. Giả sử định lý phân tích duy nhất đều đúng với các số nguyên Gauss có chuẩn nhỏ hơn $N(\alpha)$, và α có hai cách phân tích khác nhau thành tích các số nguyên tố Gauss:

$$\alpha = \pi_1 \pi_2 \cdots \pi_s = \pi'_1 \pi'_2 \cdots \pi'_t.$$

Ta biết $\pi_s \mid \pi'_1 \pi'_2 \cdots \pi'_t$. Theo bổ đề Euclid, π_s chia hết ít nhất một trong $\pi'_1, \pi'_2, \dots, \pi'_t$. Mà $\pi'_1, \pi'_2, \dots, \pi'_t$ đều là số nguyên tố Gauss, nên π_s liên kết với ít nhất một trong $\pi'_1, \pi'_2, \dots, \pi'_t$.

Không mất tính tổng quát, giả sử π_s liên kết với π'_t . Khi đó $\pi_1 \pi_2 \cdots \pi_{s-1}$ liên kết với $\pi'_1 \pi'_2 \cdots \pi'_{t-1}$, và

$$N(\pi_1 \pi_2 \cdots \pi_{s-1}) < N(\alpha).$$

Vì vậy định lý phân tích duy nhất đúng với số này, nên định lý cũng đúng với các số nguyên Gauss có chuẩn bằng $N(\alpha)$.

Tóm lại, định lý phân tích duy nhất đúng trong số nguyên Gauss. $\square$

8.3. Định lý Fermat về tổng hai bình phương

Trong mục nhỏ này, ta chứng minh định lý Fermat về tổng hai bình phương đã dùng ở mục 6.

Bổ đề 8.3.1. Nếu p là số nguyên tố, thì

$$(p-1)! \equiv -1 \pmod{p}.$$

Chứng minh. Dễ kiểm tra bổ đề đúng khi $p = 2$. Xét trường hợp p là số nguyên tố lẻ. Xét các số có nghịch đảo nhân không bằng chính nó. Những số này ghép cặp với nghịch đảo nhân của chúng trong $(p-1)!$, nên không có đóng góp đối với $(p-1)!$ theo nghĩa modulo p .

Các số còn lại thỏa $x^2 \equiv 1 \pmod{p}$, giải được

$$x \equiv 1 \pmod{p} \quad \text{hoặc} \quad x \equiv p-1 \pmod{p}.$$

Vì vậy

$$(p-1)! \equiv 1 \times (p-1) \equiv -1 \pmod{p}.$$

$\square$

Bổ đề 8.3.2. Nếu số nguyên tố p thỏa $p \equiv 1 \pmod{4}$, thì tồn tại số nguyên x sao cho $p \mid x^2 + 1$.

Chứng minh. Theo bổ đề 8.3.1, $(p-1)! + 1 \equiv 0 \pmod{p}$.

Chú ý $x \times y \equiv (p-x) \times (p-y) \pmod{p}$. Nếu $p-1$ là bội của 4, ta có thể chia

$$1 \times 2 \times \cdots \times (p-2) \times (p-1)$$

thành hai nửa. Nửa trước ghép từng đôi để nhân, và nửa sau tương ứng cũng ghép từng đôi để nhân; khi đó nửa sau đồng dư với nửa trước. Suy ra

$$\left(\frac{p-1}{2}!\right)^2 + 1 \equiv (p-1)! + 1 \equiv 0 \pmod{p}.$$

Bổ đề 8.3.2 được chứng minh. $\square$

Định lý 8.3. (Định lý Fermat về tổng hai bình phương) Một số nguyên tố lẻ p có thể biểu diễn thành tổng bình phương của hai số nguyên dương a, b , $p = a^2 + b^2$, khi và chỉ khi $p \equiv 1 \pmod{4}$; hơn nữa, nếu biểu diễn như vậy tồn tại thì nó là duy nhất nếu không xét thứ tự.

Chứng minh. Định lý Fermat về tổng hai bình phương gồm ba điểm: (1) số nguyên tố dạng $4k + 3$ không thể viết thành tổng bình phương của hai số nguyên dương; (2) số nguyên tố dạng $4k + 1$ có thể viết thành tổng bình phương của hai số nguyên dương; (3) cách viết một số nguyên tố dạng $4k + 1$ thành tổng bình phương của hai số nguyên dương là duy nhất.

Vì với mọi số nguyên x , x^2 modulo 4 chỉ có thể bằng 0 hoặc 1, tổng bình phương của hai số nguyên dương modulo 4 không thể có dư 3. Do đó số nguyên tố dạng $4k + 3$ không thể viết thành tổng bình phương của hai số nguyên dương; điểm (1) đúng.

Theo bổ đề 8.3.2, tồn tại số nguyên x sao cho $p \mid x^2 + 1$, tức

$$p \mid (x + i)(x - i).$$

Giả sử một số nguyên tố dạng $4k + 1$ là p cũng là số nguyên tố Gauss. Khi đó theo bổ đề 8.2.3, $p \mid x + i$ hoặc $p \mid x - i$. Nhưng $(x \pm i)/p = x/p \pm (1/p)i$ không phải số nguyên Gauss, nên giả thiết không đúng; p không phải số nguyên tố Gauss.

Đặt $p = uv$ ($u, v \in \mathbb{Z}[i]$, $N(u), N(v) > 1$). Khi đó

$$p^2 = N(p) = N(u)N(v).$$

Vì p là số nguyên tố, chỉ có thể có $N(u) = N(v) = p$. Hơn nữa tích của u và v là một số nguyên, nên u và v liên hợp. Đặt

$$u = a + bi, \quad v = a - bi \quad (a, b \in \mathbb{Z}),$$

ta có

$$p = (a + bi)(a - bi) = a^2 + b^2.$$

Điểm (2) của định lý đúng.

Giả sử p có hai cách biểu diễn khác nhau thành tổng bình phương của hai số nguyên dương:

$$p = a^2 + b^2 = c^2 + d^2.$$

Khi đó

$$(a + bi)(a - bi) = (c + di)(c - di),$$

nên $a + bi \mid (c + di)(c - di)$.

Nếu $a + bi$ là số nguyên tố Gauss, thì $a + bi \mid c + di$ hoặc $a + bi \mid c - di$; tương ứng, $a - bi \mid c - di$ hoặc $a - bi \mid c + di$. Lại vì

$$N(a + bi) = N(a - bi) = N(c + di) = N(c - di) = p,$$

các số $a + bi, a - bi$ và $c + di, c - di$ đôi một liên kết theo cặp, nên $p = a^2 + b^2 = c^2 + d^2$ thực ra là cùng một cách biểu diễn.

Nếu $a + bi$ không phải số nguyên tố Gauss, đặt

$$a + bi = uv \quad (u, v \in \mathbb{Z}[i], N(u), N(v) > 1).$$

Khi đó

$$p = (a + bi)(a - bi) = uv \times \overline{uv} = (u\overline{u}) \times (v\overline{v}).$$

Số p bị phân tích thành tích của hai số nguyên lớn hơn 1, mâu thuẫn với việc p là số nguyên tố. Vì vậy p không tồn tại hai cách biểu diễn khác nhau thành tổng bình phương của hai số nguyên dương; điểm (3) của định lý đúng.

Đến đây, định lý Fermat về tổng hai bình phương được chứng minh, và các kết luận dùng ở trên cũng đều đã được chứng minh. $\square$

9. Tổng kết

Bài *Đếm điểm nguyên* khảo sát tổng hợp các điểm kiến thức như bộ số Pythagoras, số nguyên Gauss, sàng số học, đồng thời cần dùng một số kỹ xảo lấy điểm như lưu dữ liệu đã biết trong chương trình và quan sát quy luật của hàm. Đây là một bài tốt để kiểm tra tố chất tổng hợp của thí sinh.

Trong lời giải chuẩn, tư tưởng mở rộng miền số, chuyển bài toán thành phân tích thừa số nguyên tố là cốt lõi của bài và cũng là điểm khó. Thí sinh cần có năng lực quan sát đủ nhạy bén và năng lực tư duy mạnh mới có thể phát hiện lời giải chuẩn và giải quyết vấn đề.

Trong các lời giải không chuẩn, vết cặn đơn giản khó giành được số điểm đáng kể; nhưng lời giải bằng bộ số Pythagoras tận dụng tính chất của đề và lời giải bằng số nguyên Gauss bỏ đi phần sàng số học phức tạp đều có thể đạt số điểm khá đáng kể. Hơn nữa, dữ liệu từng phần đều có độ dốc hợp lý, tạo không gian ghi điểm tốt hơn cho các lời giải mà tác giả chưa nghĩ tới hoặc các lời giải cài đặt chưa tốt.

Tình hình điểm tại buổi tự kiểm tra như sau: 4 bạn đạt 100 điểm, 3 bạn đạt 88 điểm, ngoài ra mỗi mức 38, 40, 70 điểm đều có 1 bạn; 2 bạn không có bài nộp, nên không có điểm. Có thể thấy bài này có tính phân loại tốt.

Tác giả hy vọng thông qua bài viết này, trong khi giới thiệu bối cảnh và lời giải của bài, có thể cho bạn đọc thấy sức hấp dẫn của lý thuyết số nguyên Gauss; trong tương lai không xa, các kết quả nghiên cứu liên quan cũng sẽ ngày càng xuất hiện nhiều hơn trong thi tin học.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy Cao Wen và thầy Wu Tao của Trường Trung học Cao cấp Thường Châu, tỉnh Giang Tô, đã quan tâm và hướng dẫn tôi trong nhiều năm.

Xin cảm ơn gia đình và bạn bè đã ủng hộ, khích lệ tôi.

Xin cảm ơn các thầy cô và bạn học từng giúp đỡ tôi.

Tài liệu tham khảo

1. Wikipedia. https://en.wikipedia.org/wiki/Gaussian_integer
2. Wikipedia. https://en.wikipedia.org/wiki/Pythagorean_triple
3. Wikipedia. https://en.wikipedia.org/wiki/Fermat%27s_theorem_on_sums_of_two_squares
4. Su Jianlin. *Bắt đầu từ định lý lớn Fermat (3): Số nguyên Gauss*. <https://spaces.ac.cn/archives/2811>
5. Su Jianlin. *Bắt đầu từ định lý lớn Fermat (4): Miền nguyên phân tích duy nhất*. <https://spaces.ac.cn/archives/2819>
6. Su Jianlin. *Bắt đầu từ định lý lớn Fermat (7): Định lý Fermat về tổng hai bình phương*. <https://spaces.ac.cn/archives/2886>
7. Zhu Zhenting. Một số bài toán tính tổng hàm số học đặc biệt. Tập luận văn đội tuyển quốc gia năm 2018.
8. 3Blue1Brown. *Pi hiding in prime regularities*. <https://www.patreon.com/posts/new-video-pi-in-11267915>

Bàn về thuật toán chia để trị trên cây

Zhang Zheyu, Trường Trung học Xuejun Hàng Châu

Tóm tắt

Bài viết này tóm lược các thuật toán chia để trị quen thuộc trên cây: chia để trị theo cạnh, chia để trị theo điểm và cây nhị phân cân bằng toàn cục.

Lời nói đầu

Tư tưởng cơ bản của thuật toán chia để trị là phân tách một bài toán có quy mô rất lớn thành vài bài toán con có quy mô nhỏ hơn. Các bài toán con ấy độc lập với nhau và có cùng tính chất với bài toán ban đầu. Sau khi tìm được nghiệm của các bài toán con, ta có thể gián tiếp thu được nghiệm của bài toán ban đầu.

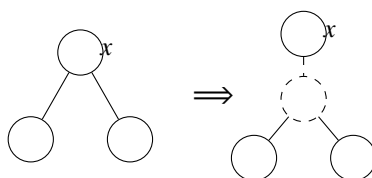
Chia để trị thông thường hay dùng trên chuỗi (dãy). Khi vận dụng trên cây, ta cần thêm một số kỹ xảo và khuôn mẫu.

1. Chia để trị theo cạnh

1.1. Giới thiệu thuật toán

Với một lớp bài toán hỏi về đường đi trên cây, ta có thể chọn một cạnh, xử lý mọi đường đi đi qua cạnh này, rồi xóa cạnh ấy và đệ quy trên hai cây nhỏ còn lại.

Để bảo đảm hiệu quả của thuật toán, sau mỗi lần xóa cạnh, cây lớn hơn trong hai cây còn lại cần càng nhỏ càng tốt. Không khó nghĩ tới việc khi mọi bậc đều là hằng số, độ phức tạp của thuật toán này là $O(n \log n)$.



Khi bậc của đỉnh x đặc biệt lớn, ta có thể thêm một đỉnh rỗng và một cạnh rỗng để bậc của x giảm đi một. Vì vậy, theo cách trong hình trên, luôn có thể thêm $O(n)$ đỉnh rỗng và cạnh rỗng để mọi đỉnh đều có bậc là hằng số.

1.2. Ví dụ

1.2.1. Mô tả đề bài

Cho hai cây đều có n đỉnh. Hãy tìm hai đỉnh sao cho tổng khoảng cách của chúng trên hai cây là lớn nhất.

$$n \leq 10^5,$$

trọng số cạnh có thể âm.

1.2.2. Lời giải

Chia để trị theo cạnh trên cây thứ nhất. Sau đó, bài toán tương đương với: cho hai tập hợp, cần lấy mỗi tập một đỉnh sao cho khoảng cách trên cây của chúng là lớn nhất. Dựng cây ảo rồi quy hoạch động là được.

1.3. Tổng kết

Vì mỗi lần chỉ cần xét chuyện giữa hai cây con, cách nghĩ thường khá đơn giản.

Nhưng do phải thêm đỉnh rỗng và cạnh rỗng, một số bài không phù hợp với cách này.

2. Chia để trị theo điểm

2.1. Giới thiệu thuật toán

Với một lớp bài toán hỏi về đường đi hoặc khối liên thông trên cây, ta có thể chọn một đỉnh, xử lý mọi đường đi hoặc khối liên thông chứa đỉnh này, rồi xóa đỉnh ấy và đệ quy trên vài cây nhỏ còn lại.

Vì kích thước của các cây nhỏ ít nhất bị chia đôi, nên tối đa chỉ đệ quy $O(\log n)$ tầng.

2.2. Ví dụ

2.2.1. Mô tả đề bài

Cho một cây n đỉnh, mỗi đỉnh có trọng số không âm. Trọng số của một khối liên thông là tổng trọng số đỉnh. Hãy tìm khối liên thông có trọng số lớn thứ k .

$$n \leq 10^5, \quad k \leq 10^5.$$

2.2.2. Lời giải

Sau khi chia để trị theo điểm, bài toán con có thêm ràng buộc rằng khối liên thông bắt buộc phải chứa gốc; khi đó bài toán trở nên rất dễ giải.

Đánh số lại theo thứ tự dfs. Tại đỉnh số i , có hai quyết định: hoặc chọn đỉnh này rồi đi tới $i + 1$, hoặc cắt bỏ cây con gốc i rồi đi tới thời điểm ra dfs của i cộng 1. Theo cách gọi thường dùng, thời điểm vào dfs là *tiền mốc*, còn thời điểm ra dfs là *hậu mốc*.

Như vậy, một khối liên thông có thể tương ứng một-một với một đường đi.

Bây giờ bài toán trở thành bài toán kinh điển: trên một DAG có $O(n \log n)$ đỉnh và $O(n \log n)$ cạnh, tìm đường đi ngắn thứ k .

Độ phức tạp thời gian:

$$O(n \log n + k \log(n \log n) + k \log k).$$

2.3. Ví dụ: xét lại 1.2

2.3.1. Muốn thử dùng chia để trị theo điểm

Rõ ràng chia để trị theo điểm và chia để trị theo cạnh cực kỳ giống nhau; ta có thể thử dùng chia để trị theo điểm để làm bài của chia để trị theo cạnh.

2.3.2. Gặp một số vấn đề

Vì trọng số cạnh có thể âm, ngoài dựng cây ảo ra không có cách hay nào để tìm đỉnh xa nhất.

Hiện tại, sau khi xóa một đỉnh, ta có thể tách ra $O(n)$ tập hợp. Số tập hợp không còn là hằng số, trực tiếp ảnh hưởng tới độ phức tạp của thuật toán.

2.3.3. Cách giải quyết

Mỗi lần xét hai tập hợp nhỏ nhất, tính đáp án chỉ gồm hai tập hợp ấy, rồi hợp nhất hai tập hợp đó. Như vậy mỗi tập hợp chỉ bị tính lặp lại $O(\log n)$ lần. Thoạt nhìn, độ phức tạp thời gian rất lớn.

2.3.4. Phân tích kỹ hơn

Vì sao chia để trị theo điểm lại vô có nhiều hơn chia để trị theo cạnh một thừa số log?

Bạn có thể cầm bút, vẽ vờ trên giấy nháp, viết ra một đồng biểu thức dạng “ $\log n - \log s$ ”, rồi đập bàn đứng dậy: “Độ phức tạp của cách làm này là $O(n \log n)$!”

Bạn cũng có thể bình tâm suy nghĩ rồi đột nhiên nhận ra: khi chia theo điểm, mỗi lần chọn hai tập hợp nhỏ nhất để hợp nhất, bản chất chính là chia để trị theo cạnh.

2.4. Ví dụ

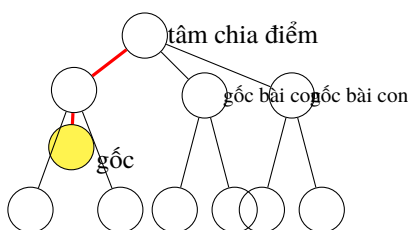
2.4.1. Mô tả đề bài

Cho một cây có gốc gồm n đỉnh, trên mỗi đỉnh có một đa thức bậc nhất. Hãy tính tổng tích các đa thức trên đường đi từ mỗi đỉnh tới gốc.

$$n \leq 10^5.$$

2.4.2. Lời giải

Chia để trị theo điểm. Trong mỗi bài toán con, giả sử ta đã tính được tích các đa thức trên đường đi từ tâm chia điểm tới gốc, thì bài toán ban đầu có thể biến thành vài bài toán con có quy mô ít nhất bị chia đôi.



Vấn đề là làm sao nhanh chóng tính được tích đa thức trên đường màu đỏ.

Xét việc sử dụng thông tin đã biết. Khi cần tính đường đi trong bài toán hiện tại, ta tìm tâm chia điểm tiếp theo trên đường đi ấy, rồi có thể trực tiếp nhận được tích trên đường đi từ tâm chia điểm tiếp theo tới gốc. Lặp lại như vậy, ta thu được $O(\log n)$ đa thức, trong đó cấp độ độ dài của đa thức thứ i không vượt quá $n/2^i$. Rõ ràng thời gian để cuộn chúng lại là $O(n \log n)$.

Vì vậy, độ phức tạp thời gian của bài toán ban đầu là $O(n \log^2 n)$.

2.5. Tổng kết

Những bài mà phần bắt buộc chứa gốc trên cây rất dễ xử lý sẽ rất phù hợp với cách này.

Bản thân hằng số không nhỏ, nhưng khi bên trong lồng thuật toán có độ phức tạp cao thì hằng số lại rất nhỏ.

3. Cây nhị phân cân bằng toàn cục

3.1. Giới thiệu thuật toán

Với mỗi đỉnh trên cây có gốc, chọn người con có kích thước cây con lớn nhất làm con nặng; cạnh nối tới nó gọi là cạnh nặng. Nếu chỉ quan tâm tới các cạnh nặng, cây được tách thành vài chuỗi.

Dùng một cây nhị phân để duy trì mỗi chuỗi. Cần chú ý rằng *mid* của cây nhị phân này không phải là *mid* trên chuỗi, mà là *mid* của cả cây con; khi đó độ sâu của toàn bộ cây là $O(\log n)$.

Đôi khi, khi số con ảo nhiều, dựng một cây nhị phân cân bằng theo kích thước cho các con ảo sẽ tiện cho việc duy trì rất nhiều thông tin.

Để thuận tiện cho phần trình bày sau, ta định nghĩa một số khái niệm.

Trong một cây con của cây nhị phân cân bằng toàn cục, tập các đỉnh mà đường đi tới gốc chỉ qua cạnh thực được gọi là *phần thực* của cây con; phần còn lại gọi là *phần ảo* của cây con.

Phần thực của cây con tương ứng với một đường đi trên cây; những đường đi như vậy được gọi là một *đoạn*. Hiển nhiên, một đường đi bất kỳ trên cây có thể chia thành $O(\log n)$ đoạn.

3.2. Ví dụ: xét lại 2.4

3.2.1. Tối ưu phân rã chuỗi nặng

Bài này có cách làm đơn giản bằng phân rã chuỗi nặng, độ phức tạp thời gian $O(n \log^3 n)$.

Với chuỗi nặng cao nhất trong bài toán hiện tại, tiến hành chia để trị, nhân đáp án của nửa dưới với tích của nửa trên rồi cộng đáp án của nửa trên.

Thay phần trên chuỗi của phân rã chuỗi nặng bằng phương pháp chia để trị của cây nhị phân cân bằng toàn cục là có thể dễ dàng đạt $O(n \log^2 n)$.

3.2.2. So sánh với cách chia để trị theo điểm

Không khó nhận ra rằng thứ tự chia để trị như vậy thực chất yêu cầu tìm chia điểm nhất định phải nằm trên chuỗi nặng nặng nhất.

Khác với chia để trị theo điểm thô sơ, nó cần chia để trị hai lần mới có thể nghiêm ngặt quy bài toán về một nửa ban đầu. Tuy nhiên, lợi ích thu được về thông tin trên chuỗi là rất lớn.

3.3. Ví dụ

3.3.1. Mô tả đề bài

Cho một cây có gốc gồm n đỉnh. Có q truy vấn, mỗi truy vấn hỏi giá trị lớn nhất của trọng số đỉnh trong các đỉnh cách p không quá d và không nằm trên đường đi $u - v$.

$$n \leq 10^5, \quad q \leq 10^5,$$

độ dài mọi cạnh của cây đều bằng 1.

3.3.2. Lời giải

Tiền xử lý thông tin của phần ảo trong mỗi cây con.

Xét p, u, v , đáy chuỗi nặng và gốc của chuỗi nặng chứa chúng. Tìm ra $O(\log n)$ đoạn giữa các đỉnh này, rồi $O(1)$ truy vấn đáp án của phần ảo trong mỗi cây con.

Đáp án của phần thực trong cây con và một vài đỉnh đặc biệt được xử lý riêng.

Một vài đỉnh đơn lẻ sẽ liên quan tới thông tin của tất cả cây con ảo của một đỉnh sau khi bỏ đi $O(1)$ cây con ảo. Vì phép *max* không thể loại trừ ngược, việc này rất khó với đỉnh có bậc lớn. Sức mạnh của việc dựng cây nhị phân cho cả các con ảo được thể hiện ở đây.

Tổng độ phức tạp thời gian là $O((n + q) \log n)$.

3.4. Tổng kết

Tuy chức năng rất mạnh, hằng số hơi lớn, cần chú ý.

Cây nhị phân cân bằng toàn cục không chỉ có thể xem là một cấu trúc chia để trị, mà cũng có thể xem là một cấu trúc dữ liệu; nhưng đây không phải trọng tâm của bài viết này.

Lời bạt

Nếu bạn chưa phải một tuyển thủ OI thành thạo cũng không sao; Baidu có mọi kiến thức nền mà bài viết này cần.

Bài viết này không có phân tích độ phức tạp, vì có rất nhiều cách phân tích. Dù bạn là tuyển thủ thích đẩy công thức $\log n - \log s$, hay là bậc thầy ước lượng thô kiểu mỗi lần số phép tính tăng thêm một thì quy mô giảm một nửa, bạn đều có thể dễ dàng thu được độ phức tạp đúng.

Nhưng điều đó cũng không có mấy tác dụng, vì độ phức tạp lý thuyết và hiệu suất chạy thực tế không có quan hệ trực tiếp. Cây nhị phân cân bằng toàn cục cùng lắm chỉ có thể dùng để ra bài tương tác nhằm siết số lần tương tác. Phân rã chuỗi nặng vẫn phải học, chia để trị theo điểm cũng vẫn phải học.

Người ra đề nói chung khá lười, thường chỉ tạo hoa cúc, chuỗi và cây nhị phân.

Tài liệu tham khảo

1. Dong Yongjian, Song Xinbo, Xu Xianyou và các tác giả khác. *Một cuốn sách thông suốt về Olympic Tin học (bản C++)*. Nhà xuất bản Văn hiến Khoa học Kỹ thuật.

Lời cảm ơn

1. Cảm ơn Wang Yisong, Ru Yizhong và Fan Zhiyuan đã giúp đỡ cho bài viết này.
2. Cảm ơn CCF đã trao cơ hội viết luận văn lần này.
3. Cảm ơn huấn luyện viên Xu Xianyou đã đọc duyệt và giúp đỡ.

Báo cáo ra đề *Tổng tổ hợp*

Wu Siyang, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này giới thiệu bài *Tổng tổ hợp* do tác giả ra trong vòng thứ ba của đợt kiểm tra lẫn nhau đội tuyển dự bị năm 2019. Bài toán khảo sát các kiến thức tổ hợp và số học như định lý nhị thức, định lý Lucas, định lý phần dư Trung Hoa; các kỹ thuật như chuỗi lũy thừa hình thức, tính tổng cấp số nhân, tính giá trị tổ hợp theo modulo; và các thuật toán như Euclid mở rộng, biến đổi Fourier nhanh. Nó đòi hỏi thí sinh có nền tảng vững và năng lực tư duy tốt.

Ngoài ra, trong những năm gần đây, các bài toán tính đáp án theo modulo ngày càng thường gặp trong lập trình thi đấu, và đã xuất hiện không ít thuật toán giải loại bài này. Tuy vậy, một số thuật toán trong đó thường chỉ phát huy tác dụng mạnh khi modulo là số nguyên tố. Bài viết này cũng mong đóng vai trò gợi mở, khơi dậy suy nghĩ về các bài toán tính đáp án theo modulo hợp số.

1. Đề bài

1.1. Mô tả đề bài

Định nghĩa

$$f(j) \equiv \sum_{i=0}^{n-1} \binom{i \cdot d}{j} \pmod{M}, \quad 0 \leq f(j) < M,$$

trong đó n, d, M là các giá trị cho trước.

Bây giờ cho m , hãy xuất giá trị

$$f(0) \text{ xor } f(1) \text{ xor } f(2) \text{ xor } \cdots \text{ xor } f(m-1).$$

Ở đây $\binom{n}{m}$ là số tổ hợp chọn m phần tử từ n phần tử, tức

$$\binom{n}{m} = \begin{cases} \frac{n!}{m!(n-m)!}, & 0 \leq m \leq n, \\ 0, & \text{ngược lại.} \end{cases}$$

Ký hiệu xor biểu thị phép xor.

1.2. Dữ liệu vào

Một dòng gồm bốn số nguyên n, m, d, M , cách nhau bởi dấu cách.

1.3. Dữ liệu ra

Một dòng chứa một số nguyên biểu thị đáp án.

1.4. Ví dụ vào

3 2 3 998244353

1.5. Ví dụ ra

10

1.6. Giải thích ví dụ

$$f(0) \equiv \binom{0}{0} + \binom{3}{0} + \binom{6}{0} \equiv 1 + 1 + 1 \equiv 3 \pmod{998244353},$$

$$f(1) \equiv \binom{0}{1} + \binom{3}{1} + \binom{6}{1} \equiv 0 + 3 + 6 \equiv 9 \pmod{998244353}.$$

Vậy $f(0) \text{ xor } f(1) = 3 \text{ xor } 9 = 10$.

1.7. Giới hạn và quy ước

Bài này dùng chấm theo nhóm.

Với mọi gói kiểm thử đều có

$$1 \leq d \leq 100, \quad 1 \leq m \cdot d \leq 3 \times 10^6, \quad 1 \leq n \cdot d \leq 10^9, \quad 10^8 \leq M \leq 10^9.$$

Với mọi gói kiểm thử, giới hạn bộ nhớ là 512 MB. Với các gói kiểm thử 1, 2, 3, 4, giới hạn thời gian là 1 giây; với các gói kiểm thử 5, 6, 7, 8, 9, 10, giới hạn thời gian là 3 giây.

Gói kiểm thử	Điểm	Quy ước khác
1	4	$n \cdot d, m \leq 2000$
2	10	$m \leq 400$
3	6	$m \leq 8000, M = 998244353$
4	6	$m \leq 8000$
5	7	$d = 1$
6	22	$\gcd(d, M) = 1$
7	7	$d = 2$
8	7	$d = 4$
9	8	d là số nguyên tố
10	23	không có

2. Quy ước và ký hiệu

Với đa thức một biến $F(x) = \sum_{i=0}^n f_i x^i$, gọi n là bậc của đa thức này. Ký hiệu $[x^i]F(x)$ là hệ số của đơn thức bậc i , nếu không tồn tại thì bằng 0, tức là

$$[x^i]F(x) = \begin{cases} f_i, & 0 \leq i \leq n, \\ 0, & \text{ngược lại.} \end{cases}$$

Đặt $(n)_k = \prod_{i=1}^k (n - i + 1)$.

Đặt $N = n \cdot d$, trong đó n, d là các biến trong đề bài.

3. Thảo luận thuật toán

3.1. Thuật toán 1

Tính trực tiếp mọi số tổ hợp cần dùng theo modulo M , cộng chúng để thu được các giá trị f , rồi xor các giá trị f để được đáp án. Có thể tính $\binom{n}{m}$ bằng công thức truy hồi

$$\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}, \quad \forall n, m, 1 \leq m \leq n-1.$$

Điều kiện biên là

$$\binom{n}{0} = \binom{n}{n} = 1, \quad \forall n, n \geq 0.$$

Độ phức tạp thời gian là $O(Nm)$, qua được gói kiểm thử thứ nhất.

3.2. Thuật toán 2

Định lý 1. Định lý nhị thức.

$$(x+y)^n = \sum_{i=0}^n \binom{n}{i} x^i y^{n-i}.$$

Khi $y = 1$, ta có

$$(x+1)^n = \sum_{i=0}^n \binom{n}{i} x^i.$$

Nếu xem đây là một đa thức một biến theo x , ta có

$$f(j) \equiv \sum_{i=0}^{n-1} \binom{i \cdot d}{j} \equiv \sum_{i=0}^{n-1} [x^j] (x+1)^{i \cdot d} \pmod{M}.$$

Đặt

$$C_n(x) = \sum_{i=0}^{n-1} (x+1)^{i \cdot d}.$$

Khi đó $f(j) \equiv [x^j] C_n(x) \pmod{M}$. Vì vậy bài toán trở thành bài toán tìm m hạng đầu của $C_n(x)$.

Xét dùng thuật toán nhân đôi để giải. Đặt $D_n(x) = (x+1)^{n \cdot d}$, khi đó

$$C_{n+1}(x) = C_n(x) \cdot (x+1)^d + (x+1)^0, \quad D_{n+1}(x) = D_n(x) \cdot (x+1)^d,$$

$$C_{2n}(x) = C_n(x) \cdot (1 + D_n(x)), \quad D_{2n}(x) = D_n(x)^2.$$

Vì chỉ cần m hạng đầu của $C_n(x)$, mọi phép nhân đa thức đều có thể thực hiện theo modulo x^m .

Nếu dùng cách $O(m^2)$ để nhân hai đa thức, độ phức tạp thời gian là $O(m^2 \log_2 N)$, qua được gói kiểm thử thứ hai.

Có thể tối ưu phép nhân đa thức bằng biến đổi Fourier nhanh, tổng độ phức tạp thời gian là $O(m \log_2 m \log_2 N)$. Cách cài đặt cụ thể có thể tham khảo bài *Xét lại biến đổi Fourier nhanh* của Mao Xiao trong tập luận văn ứng viên đội tuyển quốc gia Olympic Tin học Trung Quốc năm 2016. Vì phương pháp ấy không liên hệ nhiều với các nội dung khác của bài viết này nên không trình bày thêm. Nếu cài đặt biến đổi số học cho modulo 998244353 thì qua được gói kiểm thử thứ ba; nếu dùng chập modulo tùy ý thì qua được bốn gói kiểm thử đầu.

3.3. Thuật toán 3: cách làm khi $d = 1$

Khi $d = 1$,

$$f(j) \equiv \sum_{i=0}^{n-1} \binom{i}{j} \equiv \sum_{i=j}^{n-1} \binom{i}{j} \pmod{M}.$$

Định lý 2.

$$\binom{n+1}{k+1} = \binom{n}{k} + \binom{n}{k+1} + \dots + \binom{n}{n}.$$

Chứng minh. Theo công thức truy hồi của tổ hợp $\binom{n}{m} = \binom{n-1}{m-1} + \binom{n-1}{m}$, $\forall n, m, 1 \leq m \leq n-1$, ta có

$$\begin{aligned} \binom{n+1}{k+1} &= \binom{n}{k} + \binom{n}{k+1} \\ &= \binom{n}{k} + \binom{n-1}{k} + \binom{n-1}{k+1} \\ &= \binom{n}{k} + \binom{n-1}{k} + \binom{n-2}{k} + \binom{n-2}{k+1} \\ &= \dots \\ &= \binom{n}{k} + \binom{n-1}{k} + \dots + \binom{k+1}{k} + \binom{k+1}{k+1} \\ &= \binom{n}{k} + \binom{n-1}{k} + \dots + \binom{k+1}{k} + \binom{k}{k}. \end{aligned}$$

□

Do đó $f(j) \equiv \binom{n}{j+1} \pmod{M}$. Bài toán chuyển thành cách tính giá trị tổ hợp theo modulo M .

Quan sát các số tổ hợp cần tính, dạng của chúng là $\binom{n}{j}$, trong đó n rất lớn, không thích hợp để tiếp tục dùng thuật toán 1. Nhưng j tương đối nhỏ, nên xét tính theo

$$\binom{n}{j} = \frac{(n)_j}{j!}.$$

Do $\gcd(j!, M)$ có thể không bằng 1, $j!$ có thể không có nghịch đảo theo modulo M . Ta phân tích M thành thừa số nguyên tố

$$M = \prod_{i=1}^k p_i^{q_i},$$

và viết $\frac{(n)_j}{j!}$ dưới dạng

$$\frac{A \cdot \prod_{i=1}^k p_i^{a_i}}{B \cdot \prod_{i=1}^k p_i^{b_i}},$$

bảo đảm $\gcd(B, M) = 1$, tức là nghịch đảo của B theo modulo M tồn tại. Lại do $\forall i, a_i \geq b_i$, có thể tính

$$A \cdot B^{-1} \cdot \prod_{i=1}^k p_i^{a_i - b_i}$$

để thu được giá trị của $\binom{n}{j}$, trong đó $B \cdot B^{-1} \equiv 1 \pmod{M}$. Các giá trị A, B, a_i, b_i có thể tính theo thứ tự j tăng dần, mỗi lần sửa từ $\binom{n}{j-1}$. Độ phức tạp thời gian là $O(m \cdot (\log_2 N + \log_2 M))$, qua được gói kiểm thử thứ năm.

3.4. Thuật toán 4

Tương tự thuật toán 2, xét cách tính $C_n(x)$. Để tiện, trong phần sau nếu không nói rõ, ta dùng $C(x)$ thay cho $C_n(x)$.

Ta có

$$C(x) = (x+1)^{0 \cdot d} + (x+1)^{1 \cdot d} + \dots + (x+1)^{(n-1) \cdot d}. \quad (24)$$

$$(x+1)^d \cdot C(x) = (x+1)^{1 \cdot d} + (x+1)^{2 \cdot d} + \dots + (x+1)^{(n-1) \cdot d} + (x+1)^{n \cdot d}. \quad (25)$$

Tương tự tính tổng cấp số nhân, lấy (25) – (24) được

$$[(x+1)^d - 1] \cdot C(x) = (x+1)^{n \cdot d} - (x+1)^{0 \cdot d}.$$

Tức là

$$C(x) = \frac{(x+1)^{n \cdot d} - 1}{(x+1)^d - 1}.$$

Dễ thấy

$$[x^0]((x+1)^{n \cdot d} - 1) = [x^0]((x+1)^d - 1) = 0,$$

tức là tử và mẫu đều có thể viết thành x nhân với một đa thức, nên có thể cùng rút gọn x .

Đặt

$$A(x) = \sum_{i=1}^{n \cdot d} \binom{n \cdot d}{i} x^{i-1}, \quad B(x) = \sum_{i=1}^d \binom{d}{i} x^{i-1}.$$

Khi đó

$$C(x) = \frac{A(x)}{B(x)}, \quad B(x) \cdot C(x) = A(x).$$

Khi $d = 1$, ta nhận được cách giải giống thuật toán 3.

Từ $B(x) \cdot C(x) = A(x)$ suy ra

$$\sum_{j=0}^{d-1} ([x^j]B(x)) \cdot ([x^{i-j}]C(x)) = [x^i]A(x), \quad \forall 0 \leq i < N. \quad (26)$$

3.4.1. Thuật toán 4.1: cách làm khi $\gcd(d, M) = 1$

Từ (26) có

$$[x^i]C(x) = \frac{[x^i]A(x) - \sum_{j=1}^{d-1} ([x^j]B(x)) \cdot ([x^{i-j}]C(x))}{[x^0]B(x)}, \quad \forall 0 \leq i < m. \quad (27)$$

Do $[x^0]B(x) = \binom{d}{1} = d$, và $\gcd(d, M) = 1$, nghịch đảo của $[x^0]B(x)$ theo modulo M tồn tại. Vì vậy có thể dùng trực tiếp (27) để tính $[x^i]C(x)$ theo thứ tự i tăng dần.

Cách tính $A(x), B(x)$ giống thuật toán 3, độ phức tạp thời gian là $O(m \cdot (\log_2 N + \log_2 M))$. Thời gian tính một hạng $[x^i]C(x)$ là $O(d)$, nên tổng độ phức tạp là $O(m \cdot (\log_2 N + \log_2 M + d))$. Cách này qua được các gói kiểm thử thứ ba, thứ năm và thứ sáu.

3.4.2. Thuật toán 4.2: cách làm khi $d = 2$

Phân tích M thành dạng $2^a \cdot b$, trong đó $b \equiv 1 \pmod{2}$. Với mỗi $f(j)$, sau khi tìm giá trị theo modulo 2^a và theo modulo b , dùng định lý phần dư Trung Hoa (CRT) là tính được giá trị của $f(j)$ theo modulo M .

Giá trị theo modulo b có thể tính bằng thuật toán 4.1; vấn đề trở thành cách tính giá trị theo modulo 2^a .

Khi $d = 2$, $B(x) = x + 2$. Thay vào (26) được

$$2 \cdot [x^i]C(x) + [x^{i-1}]C(x) = [x^i]A(x), \quad \forall i \geq 0.$$

Tức là

$$[x^i]C(x) = [x^{i+1}]A(x) - 2 \cdot [x^{i+1}]C(x), \quad \forall i \geq 0. \quad (28)$$

Liên tục thay hạng $C(x)$ ở vế phải bằng (28), ta được

$$\begin{aligned} [x^i]C(x) &= [x^{i+1}]A(x) - 2 \cdot [x^{i+1}]C(x) \\ &= [x^{i+1}]A(x) - 2 \cdot [x^{i+2}]A(x) + 4 \cdot [x^{i+2}]C(x) \\ &= [x^{i+1}]A(x) - 2 \cdot [x^{i+2}]A(x) + 4 \cdot [x^{i+3}]A(x) - 8 \cdot [x^{i+3}]C(x) \\ &= \dots \\ &= \sum_{j \geq 0} (-2)^j \cdot [x^{i+j+1}]A(x). \end{aligned}$$

Khi $j \geq a$, $(-2)^j \cdot [x^{i+j+1}]A(x) \equiv 0 \pmod{2^a}$, do đó

$$[x^i]C(x) \equiv \sum_{j=0}^{a-1} (-2)^j \cdot [x^{i+j+1}]A(x) \pmod{2^a}. \quad (29)$$

Dùng (29) là có thể tính $[x^i]C(x)$ theo modulo 2^a . Thời gian cần cho một hạng là $O(a)$, tức $O(\log_2 M)$. Kết hợp phần tính theo modulo b , tổng độ phức tạp thời gian là $O(m \cdot (\log_2 N + \log_2 M + d))$. Cùng với thuật toán 4.1, cách này qua được các gói kiểm thử thứ ba, thứ năm đến thứ bảy.

3.4.3. Thuật toán 4.3: cách làm khi $d = 4$

Tương tự thuật toán 4.2, phân tích M thành $2^a \cdot b$. Giá trị theo modulo b dùng thuật toán 4.1 để tính; xét cách tính giá trị theo modulo 2^a .

Khi $d = 4$, $B(x) = 4 + 6x + 4x^2 + x^3$. Thay vào (26) được

$$4 \cdot [x^i]C(x) + 6 \cdot [x^{i-1}]C(x) + 4 \cdot [x^{i-2}]C(x) + [x^{i-3}]C(x) = [x^i]A(x), \quad \forall i \geq 0.$$

Tức là

$$[x^i]C(x) = [x^{i+3}]A(x) - 4 \cdot [x^{i+3}]C(x) - 6 \cdot [x^{i+2}]C(x) - 4 \cdot [x^{i+1}]C(x), \quad \forall i \geq 0. \quad (30)$$

Dễ thấy trong vế phải của (30), hệ số của mỗi hạng $C(x)$ đều $\equiv 0 \pmod{2}$. Tương tự cách thu được (29) trong thuật toán 4.2, ta gọi một vòng thao tác là thay mỗi hạng $C(x)$ ở vế phải của công thức bằng vế phải của (30), tức là biểu diễn nó bằng (30). Nếu trước thao tác, hệ số của mỗi hạng $C(x)$ ở vế phải đều là bội của 2^i , thì sau một vòng thao tác, hệ số của mỗi hạng $C(x)$ ở vế phải đều là bội của 2^{i+1} . Nếu thực hiện $a - 1$ vòng thao tác đối với (30), hệ số của mỗi hạng $C(x)$ sẽ $\equiv 0 \pmod{2^a}$. Khi đó vế phải chỉ còn tổng hệ số của một số hạng $A(x)$, và bài toán được giải quyết.

Thao tác lặp sẽ thực hiện a vòng, độ dài công thức sau lặp có thể đạt $O(ad)$, và khai triển mỗi hạng bằng (30) cần $O(d)$. Do đó tính công thức lặp cần độ phức tạp $O(a^2 d^2)$, tức $O((\log_2 M)^2 d^2)$. Sau đó tính mỗi hạng $[x^i]C(x)$ cần thời gian $O(\log_2 M \cdot d)$. Cộng thêm phần tính kết quả theo modulo b , tổng độ phức tạp thời gian là

$$O(m \cdot (\log_2 N + \log_2 M \cdot d) + (\log_2 M)^2 d^2).$$

Kết hợp thuật toán 4.1 và 4.2, cách này qua được các gói kiểm thử thứ ba, thứ năm đến thứ tám.

3.4.4. Thuật toán 4.4: cách làm khi d là số nguyên tố

Tương tự hai thuật toán trước (4.2 và 4.3), có thể phân tích M thành $d^a \cdot b$, trong đó $\gcd(b, d) = 1$, rồi giải riêng theo modulo d^a và modulo b . Phần modulo b dùng thuật toán 4.1; xét cách làm theo modulo d^a .

Từ (26) có

$$([x^{d-1}]B(x)) \cdot ([x^i]C(x)) = [x^{i+d-1}]A(x) - \sum_{j=0}^{d-2} ([x^j]B(x)) \cdot ([x^{i+d-1-j}]C(x)), \quad \forall i \geq 0. \quad (31)$$

Quan sát hai thuật toán trước (4.2 và 4.3), chúng đều liên tục dùng (28), (30) để lặp từng hạng $C(x)$ ở vế phải của (28), (30), cho tới khi các hệ số bằng 0 theo modulo. Vậy trong trường hợp d là số nguyên tố, chỉ cần bảo đảm sau hữu hạn vòng lặp, vế phải chỉ còn tổng của một số hạng $A(x)$ theo modulo d^a , thì có thể làm tương tự bằng cách mỗi lần thay một hạng $C(x)$ ở vế phải của (31) bằng vế phải của (31).

Định lý 3. Định lý Lucas. Với các số nguyên không âm m, n và số nguyên tố p ,

$$\binom{m}{n} \equiv \prod_{i=0}^k \binom{m_i}{n_i} \pmod{p},$$

trong đó

$$\begin{aligned} m &= m_k p^k + m_{k-1} p^{k-1} + \cdots + m_1 p + m_0, & 0 \leq m_i < p, \\ n &= n_k p^k + n_{k-1} p^{k-1} + \cdots + n_1 p + n_0, & 0 \leq n_i < p. \end{aligned}$$

Theo định lý Lucas, $\forall 0 < i < d$,

$$\binom{d}{i} \equiv \binom{1}{0} \binom{0}{i} \equiv 0 \pmod{d}.$$

Tức là, trừ $[x^{d-1}]B(x)$, các hệ số còn lại của $B(x)$ đều là bội của d . Vì vậy nếu trước khi lặp, các hệ số của $C(x)$ đều là bội của d^i , thì sau lặp các hệ số sẽ là bội của d^{i+1} ; do đó sau $a - 1$ vòng lặp, các hệ số của $C(x)$ đều bằng 0.

Cách tính tổng độ phức tạp thời gian tương tự thuật toán 4.3, là

$$O(m \cdot (\log_2 N + \log_2 M \cdot d) + (\log_2 M)^2 d^2).$$

Kết hợp thuật toán 4.1 và 4.3, cách này qua được các gói kiểm thử thứ ba, thứ năm đến thứ chín.

3.4.5. Thuật toán 4.5: cách làm đạt điểm tối đa

Phân tích M thành thừa số nguyên tố

$$M = \prod_{i=1}^k p_i^{q_i}.$$

Với mỗi $p_i^{q_i}$, tính giá trị theo modulo đó. Bài toán trở thành cách giải theo modulo một lũy thừa p^q của số nguyên tố.

Khi $\gcd(d, p) = 1$, vẫn dùng thuật toán 4.1.

Khi $\gcd(d, p) > 1$, tức $d \equiv 0 \pmod{p}$, vẫn muốn dùng lặp (31) để làm cho hệ số của $C(x)$ ở vế phải bằng 0 theo modulo. Khi $i \equiv 0 \pmod{p}$, theo định lý Lucas,

$$\binom{d}{i} \equiv \binom{d/p}{i/p} \binom{0}{0} \equiv \binom{d/p}{i/p} \pmod{p},$$

kết quả không nhất định bằng 0, nên không thể áp dụng trực tiếp cách của thuật toán 4.4.

Nguyên nhân chính khiến việc lặp (31) thất bại là công thức lặp tồn tại một hạng $C(x)$ có bậc cao hơn i , mà hệ số của nó không nhất định là bội của p . Điều này làm cho sau khi lặp, hệ số mới sinh ra chưa chắc khiến mọi hệ số đều là bội của lũy thừa cao hơn của p . Nếu ta chọn hạng $C(x)$ có bậc cao nhất trong các hạng có hệ số không phải bội của p để đưa sang vế trái, tức là tìm t nhỏ nhất sao cho $[x^t]B(x) \not\equiv 0 \pmod{p}$, thì (26) có thể viết thành

$$\begin{aligned} ([x^t]B(x)) \cdot ([x^i]C(x)) &= [x^{i+t}]A(x) \\ &- \sum_{j=0}^{t-1} ([x^j]B(x)) \cdot ([x^{i+t-j}]C(x)) \\ &- \sum_{j=t+1}^{d-1} ([x^j]B(x)) \cdot ([x^{i+t-j}]C(x)), \quad \forall i \geq 0. \end{aligned} \quad (32)$$

Trong đó, khi $j < t$ thì $[x^j]B(x)$ là bội của p , còn khi $j > t$ thì không nhất định.

Chỉ cần biến hệ số của các hạng $C(x)$ có bậc cao hơn i trong (32) thành 0 theo modulo p^q , ta có thể tương tự thuật toán 4.1, lần lượt tính từng $[x^i]C(x)$ cần dùng theo thứ tự tăng dần. Vì vậy chỉ cần lặp các hạng $C(x)$ có bậc cao hơn i . Nếu lặp trực tiếp bằng (32), khi lặp một hạng bậc j , có thể sinh ra hệ số không phải bội của p ở một hạng bậc k , $i < k < j$; như vậy không nhất định bảo đảm sau hữu hạn vòng lặp sẽ kết thúc. Vấn đề này chỉ sinh một chiều, tức là sinh sang các hạng bậc thấp hơn. Vì thế, một ý tưởng tự nhiên là: giả sử hệ số của mọi hạng $C(x)$ bậc cao hơn i hiện đều là bội của p^a . Mỗi lần ta tìm hạng $C(x)$ bậc cao nhất có hệ số không phải bội của p^{a+1} , rồi lặp hạng đó, cho tới khi hệ số của mọi hạng $C(x)$ đều là bội của p^{a+1} . Ta gọi quá trình này là một vòng. Dễ thấy trong một vòng thao tác, bậc của các hạng cần lặp giảm đơn điệu, nên độ phức tạp thời gian của một vòng là $O(l)$, trong đó l là số hạng $C(x)$ ở vế phải trước khi thao tác. Sau $q - 1$ vòng, hệ số của các hạng có bậc cao hơn i đều trở thành 0. Trong quá trình lặp, vì $[x^i]C(x)$ mỗi lần chỉ được cộng thêm một bội của p , nó vẫn nguyên tố cùng nhau với p^q , nên nghịch đảo vẫn tồn tại. Chỉ cần nhân hai vế với nghịch đảo này là thu được chuyển tiếp của $[x^i]C(x)$ theo $A(x)$ và $[x^j]C(x)$, $j < i$.

Tính $A(x)$ có độ phức tạp thời gian $O(m \cdot (\log_2 N + \log_2 M))$. Khi tính giá trị theo modulo tích của mọi p^q thỏa $\gcd(d, p) = 1$, có thể tính trước tích T , rồi dùng thuật toán 4.1 để tính theo modulo T , độ phức tạp thời gian là $O(md)$. Khi tính giá trị theo modulo tích của mọi p^q thỏa $\gcd(d, p) \neq 1$, cần tính riêng với từng p^q ; với mỗi p^q , độ phức tạp thời gian là $O(mdq + d^2q^2)$. Dùng thuật toán CRT để tính đáp án cuối cùng có độ phức tạp thời gian $O(m \cdot \log_2 M^2)$. Do đó tổng độ phức tạp thời gian là

$$O(m \cdot (\log_2 N + d \cdot \log_2 M) + d^2(\log_2 M)^2),$$

qua được mọi điểm kiểm thử.

4. Cách sinh dữ liệu

Dễ thấy việc đầu vào n và m có ngẫu nhiên hay không không liên quan nhiều tới bài này, nên n và m đều được sinh ngẫu nhiên.

d và M chỉ liên quan tới phân tích thừa số nguyên tố. Vì vậy tác giả sinh chúng theo những cách cấu thành thừa số nguyên tố khác nhau, chẳng hạn số nguyên tố, lũy thừa của một số nguyên tố, 1, tích của nhiều số nguyên tố, v.v., đồng thời thỏa các giới hạn đặc biệt của từng gói kiểm thử.

5. Ý tưởng ra đề

Ý tưởng ra bài này bắt nguồn từ bài *F Test Data Generation* do tourist ra trong VK Cup 2017 Round 3. Khi làm bài đó, đầu tiên tác giả chuyển bài toán thành dạng tính

$$\sum_{i=0}^n \sum_{j=0}^m \binom{2i}{2j} \pmod{M}.$$

Sau đó tác giả nghĩ ra thuật toán 4.2 và dùng nó để qua bài ấy, còn lời giải chính thức đưa ra rất giống thuật toán 2. Cách của tác giả tốt hơn lời giải chính thức khi tính một lần giá trị của công thức trên, nhưng trong bài đó cần tính giá trị ứng với $n = N, N/2, N/4, \dots$. Điều này khiến thuật toán của tác giả phải làm $\log_2 N$ lần, còn thuật toán nhân đôi có thể xử lý đồng thời $\log_2 N$ truy vấn, nên độ phức tạp thời gian là như nhau. Tuy vậy, cách của tác giả vẫn tốt hơn về hằng số; do đó tính tới tháng 3 năm 2019, bài nộp của tác giả cho bài ấy vẫn là bài chạy nhanh nhất.

Nhiều truy vấn không thể hiện ưu thế của thuật toán này, nên trước hết tác giả sửa đề thành chỉ hỏi một giá trị n .

Tuy nhiên, nếu chỉ lấy tổng các $f(j)$, có thể giải bằng các kỹ thuật như nội suy. Vì vậy tác giả lại sửa đề thành yêu cầu giá trị của từng $f(j)$; để tránh lượng xuất quá lớn, dùng cách xuất tổng xor.

Sau khi nghĩ ra cách làm cho $d = 2$, tác giả lại thử giải bài toán với d bất kỳ. Ban đầu tác giả vẫn muốn lặp liên tục như các thuật toán 4.2, 4.3 và 4.4, nhưng do tồn tại các hệ số không phải bội của p , tác giả không biết khi nào quá trình lặp kết thúc. Từ đó nảy sinh ý tưởng tìm i nhỏ nhất sao cho $\binom{d}{i}$ không là bội của p , rồi lặp phần có bậc cao hơn nó trong $C(x)$. Sau đó tác giả lại quan sát thấy phần này chỉ đóng góp hệ số không phải bội của p sang các hạng $C(x)$ bậc thấp hơn, nên nghĩ tới cách lặp lần lượt từ phải sang trái, tức thuật toán 4.5, và thu được bài này.

6. Thiết kế điểm kiểm tra, độ khó và độ phân hóa

Bài này khảo sát các kiến thức tổ hợp và số học như định lý nhị thức, định lý Lucas, định lý phần dư Trung Hoa; các kỹ thuật như chuỗi lũy thừa hình thức, tính tổng cấp số nhân, tính giá trị tổ hợp theo modulo; và các thuật toán như Euclid mở rộng, biến đổi Fourier nhanh.

Nếu biết các kiến thức cơ bản như số tổ hợp, modulo, tổng xor thì có thể làm được gói kiểm thử thứ nhất. Phần này nhằm phân hóa việc thí sinh có biết các kiến thức cơ bản ấy hay không, cũng như mức độ hiểu đề.

Thông qua suy luận về số tổ hợp có thể làm được gói kiểm thử $d = 1$. Phần này nhằm phân hóa năng lực suy luận toán học của thí sinh.

Muốn làm được các gói kiểm thử có N lớn, cần nghĩ tới định lý nhị thức và dùng chuỗi lũy thừa hình thức để chuyển bài toán thành tính tổng đa thức. Đây là điểm khó thứ nhất của bài. Trên cơ sở đó không khó thu được thuật toán 2; phần này nhằm phân hóa việc thí sinh có biết định lý nhị thức, thuật toán nhân đôi hay không, có dùng thành thạo chuỗi lũy thừa hình thức để hỗ trợ giải bài hay không, đồng thời cũng khảo sát tối ưu phép nhân đa thức.

Trên cơ sở đã nghĩ tới tính tổng đa thức, quan sát mỗi hạng của đa thức tổng đều là một lũy thừa của $(x + 1)^d$, liên tưởng tới kỹ thuật tính tổng cấp số nhân trong toán học, rồi chuyển bài toán thành phép chia đa thức. Đây là điểm khó thứ hai của bài. Trên cơ sở đó, nếu hiểu phép chia đa thức thì không khó thu được lời giải cho trường hợp $\gcd(d, M) = 1$. Phần này nhằm phân hóa mức độ hiểu của thí sinh về kỹ thuật tính tổng và phép chia đa thức.

Các gói kiểm thử $d = 2$, $d = 4$ và d là số nguyên tố nhằm dẫn dắt thí sinh giải các trường hợp tương đối đặc biệt. Dùng định lý phần dư Trung Hoa (CRT) để tách bài toán thành hai phần: modulo nguyên

tổ cùng nhau với d , và modulo là lũy thừa của thừa số nguyên tố của d . Trong đó, trường hợp modulo không nguyên tố cùng nhau với d là điểm khó thứ ba của bài. Phần này nhằm phân hóa mức độ nắm vững CRT và thuật toán lặp của thí sinh.

Sau khi biết các trường hợp d tương đối đặc biệt, phân tích vì sao thuật toán lặp trước đó không thể giải tiếp trường hợp tổng quát, rồi khai thác tính chất của công thức chuyển tiếp để thu được lời giải đúng cuối cùng. Đây là điểm khó cuối cùng của bài. Phần này nhằm phân hóa việc thí sinh có hiểu sâu công thức chuyển tiếp và các cách làm trước đó hay không.

Trong đợt kiểm tra lẫn nhau thực tế, có 6 thí sinh nộp bài này. Ngoại trừ gói kiểm thử cuối, các gói kiểm thử khác đều có thí sinh qua. Tình hình cụ thể như sau:

Gói kiểm thử	1	2	3	4	5	6	7	8	9	10
Số người qua	4	1	3	1	3	2	1	1	1	0

7. Tổng kết

Nhìn chung, các kiến thức và thuật toán cần dùng trong bài này không quá khó; độ khó của phần lớn thuật toán gần với cấp độ NOI. Quá trình đi tới lời giải đúng cũng không nhiều, lượng mã không lớn, nhưng độ khó tư duy không nhỏ, ý tưởng mới mẻ và có ý nghĩa gợi mở lớn. Với tư cách một bài OI, thiết kế điểm từng phần khuyến khích nhiều phương pháp giải bài toán này, đồng thời dẫn dắt thí sinh đi từng bước từ các trường hợp đặc biệt tới lời giải đúng.

Ngoài ra, trong những năm gần đây, các bài toán tính đáp án theo modulo ngày càng thường gặp trong lập trình thi đấu, và đã xuất hiện không ít thuật toán giải loại bài này. Tuy vậy, một số thuật toán trong đó thường chỉ phát huy tác dụng mạnh khi modulo là số nguyên tố, chẳng hạn thuật toán Berlekamp–Massey. Khi modulo là hợp số, do nghịch đảo nhân không nhất định tồn tại, các thuật toán này không nhất định có thể thực hiện phép chia, từ đó sinh ra hàng loạt vấn đề. Các cách xử lý thường gặp gồm: dùng kiểu số thực như double để lưu biến; dùng số chính xác cao để chuyển số hữu tỉ sang dạng phân số rồi lưu trữ. Cách trước có thể có sai số độ chính xác, và khi số phép toán nhiều có thể dẫn tới kết quả sai; cách sau tuy bảo đảm tính đúng đắn của kết quả, nhưng độ phức tạp thời gian quá cao, đôi khi khó chấp nhận. Tư tưởng được dùng trong bài này là trước hết phân tích modulo thành thừa số nguyên tố, biểu diễn nó thành tích của một số lũy thừa của các số nguyên tố khác nhau. Nếu có thể giải riêng cho từng lũy thừa nguyên tố, ta có thể dùng thuật toán định lý phần dư Trung Hoa để thu được đáp án cuối cùng. Khi giải đáp án theo modulo p^q với p nguyên tố, thường có thể thử giải trước trường hợp lũy thừa thấp hơn, chẳng hạn p^{q-1} , rồi sửa trên cơ sở đó để thu được đáp án cuối cùng. Tư tưởng này cũng xuất hiện trong các thuật toán khác, chẳng hạn thuật toán Reeds–Sloane. Bài viết này cũng mong đóng vai trò gợi mở, khơi dậy suy nghĩ về các bài toán tính đáp án theo modulo hợp số.

8. Lời cảm ơn

Cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn cha mẹ và thầy cô đã quan tâm, chỉ dẫn trong nhiều năm.

Cảm ơn bạn Kong Chaozhe, Trường Trung học Ôn Châu, đã kiểm thử đề và đọc duyệt bài viết này.

Cảm ơn các bạn học khác đã trao đổi, thảo luận với tôi.

Tài liệu tham khảo

1. Mao Xiao. *Xét lại biến đổi Fourier nhanh*. Tập luận văn đội tuyển quốc gia năm 2016.

2. Wang Leping. *Hàm sinh, thuật toán đa thức và đếm đồ thị*. Slide lớp một WC2019.
3. Wikipedia. Binomial coefficient. https://en.wikipedia.org/wiki/Binomial_coefficient.
4. Wikipedia. Chinese remainder theorem. [English Wikipedia](#).
5. Wikipedia. Lucas's theorem. https://en.wikipedia.org/wiki/Lucas%27s_theorem.
6. Wikipedia. Reeds–Sloane algorithm. [English Wikipedia](#).

Bàn về một lớp cấu trúc dữ liệu ngắn gọn

Wang Siqi, Trường Trung học số 7 Thành Đô

Tóm tắt

Cấu trúc dữ liệu ngắn gọn là một loại cấu trúc dữ liệu có hằng số không gian nhỏ. Bài viết này giới thiệu một lớp cấu trúc dữ liệu ngắn gọn và đưa ra một số ứng dụng của chúng.

1. Lời nói đầu

Biểu diễn dữ liệu và truy vấn dữ liệu một cách tiết kiệm không gian là một vấn đề quan trọng trong khoa học máy tính. Với việc biểu diễn dữ liệu, phương pháp thường gặp nhất là nén; nhưng sau khi nén, truy vấn dữ liệu gốc thường trở nên khó hơn. Nếu lưu trực tiếp dữ liệu gốc rồi dùng cấu trúc dữ liệu thông thường để duy trì, không gian lại chưa đủ hiệu quả.

Vì các vấn đề ấy, những cấu trúc dữ liệu tiết kiệm không gian ra đời. Các loại thường gặp gồm những loại sau, giả sử dữ liệu gốc cần n bit để biểu diễn:

- Cấu trúc dữ liệu nén (compressed data structure): không gian lưu trữ phụ thuộc vào dữ liệu cụ thể, thường liên quan tới entropy thông tin của dữ liệu được biểu diễn.
- Cấu trúc dữ liệu ngắn gọn (succinct data structure): dùng $n + o(n)$ bit.
- Cấu trúc dữ liệu ẩn (implicit data structure): dùng $n + O(1)$ bit.
- Cấu trúc dữ liệu compact: dùng $O(n)$ bit.

Bài viết này giới thiệu một cấu trúc dữ liệu ngắn gọn để xử lý hai bài toán rank và select trên chuỗi 01; sau đó, dựa trên cấu trúc dữ liệu này, dẫn nhập cấu trúc dữ liệu xử lý cây và wavelet tree.

2. Chuỗi 01

Xét một chuỗi 01 S độ dài n , cần hỗ trợ hai thao tác:

- $\text{rank}_v(k)$: hỏi từ vị trí 1 đến vị trí k có bao nhiêu ký tự v .
- $\text{select}_v(k)$: hỏi vị trí của ký tự v thứ k .

Rõ ràng, nếu giá trị ở vị trí i là v , thì $\text{select}_v(\text{rank}_v(i)) = i$.

Bài toán này có cách dùng tổng tiền tố với không gian $O(n \log n)$, thời gian $O(1)$, và cách vét cạn với không gian n , thời gian $O(n)$. Nhưng cách tốt hơn có thể đạt không gian

$$n + O\left(\frac{n \log \log n}{\log n}\right)$$

và thời gian $O(1)$.

Dưới đây trước hết giới thiệu một cách làm không gian $O(n)$, thời gian $O(1)$, rồi giới thiệu cách tốt hơn ở trên.

2.1. Cách làm không gian $O(n)$, thời gian $O(1)$

Với thao tác rank, trước hết chia S thành các khối kích thước $\log n/2$, rồi lưu tổng tiền tố tại cuối mỗi khối. Phần này cần

$$\frac{n}{\log n/2} \cdot \log n = 2n$$

không gian.

Trong nội bộ khối, có thể lưu trực tiếp tổng của mọi chuỗi 01 có độ dài không vượt quá $\log n/2$, cần $O(\sqrt{n} \log \log n)$ không gian. Khi truy vấn, ta tra tổng của phần gồm các khối liên tiếp đầy đủ, còn phần chưa đủ một khối có thể tính bằng hai lần tra bảng.

Với thao tác select₁, trước hết chia giá trị được hỏi thành các khối lớn theo $\log n$, tức là tính trước mọi giá trị select₁ ($k \log n$). Phần này cần

$$\frac{n}{\log n} \cdot \log n = n$$

không gian.

Nếu một khối lớn ứng với một đoạn trong chuỗi gốc có độ dài lớn hơn $\log^2 n$, thì số khối như vậy không vượt quá $n/\log^2 n$. Có thể lưu trực tiếp đáp án của mọi khối như vậy, tổng cộng cần

$$O\left(\frac{n}{\log^2 n} \cdot \log n \cdot \log n\right) = O(n)$$

không gian.

Với các khối lớn còn lại, tiếp tục chia trong khối theo $\log \log n$ thành khối nhỏ, rồi lưu đáp án của mỗi vị trí trong khối nhỏ ấy. Vì đoạn tương ứng trong chuỗi gốc có độ dài không vượt quá $\log^2 n$, biểu diễn mỗi đáp án cũng chỉ cần $O(\log \log n)$ không gian. Do đó phần này tổng cộng cần

$$O\left(\frac{n}{\log n} \cdot \frac{\log n}{\log \log n} \cdot \log \log n\right) = O(n)$$

không gian.

Với các đoạn có độ dài không vượt quá $\log n/2$, có thể lưu trực tiếp đáp án của mọi thao tác select trên mọi đoạn như vậy, tổng cộng cần $O(\sqrt{n} \log n \log \log n)$ không gian.

Sau hai lần chia khối, nếu một khối nhỏ ứng với đoạn trong chuỗi gốc có độ dài không vượt quá $\log n/2$, ta tra trực tiếp bảng ở trên; nếu không, số khối như vậy hiển nhiên là $O(n/\log n)$, nên có thể lưu trực tiếp đáp án của mọi vị trí trong khối ấy tương đối với khối lớn $\log n$ chứa nó, cần

$$O\left(\frac{n(\log \log n)^2}{\log n}\right)$$

không gian.

Vì vậy tổng cộng cần

$$O\left(n + \sqrt{n} \log \log n + \sqrt{n} \log n \log \log n + \frac{n(\log \log n)^2}{\log n}\right) = O(n)$$

không gian, và số lần phân loại trong mỗi thao tác là hằng số, nên thời gian đều là $O(1)$.

2.2. Cách làm không gian $n + O\left(\frac{n \log \log n}{\log n}\right)$, thời gian $O(1)$

Trước hết chia S thành các khối kích thước $\log^2 n$, rồi lưu tổng tiền tố tại cuối mỗi khối. Phần này cần

$$\frac{n}{\log^2 n} \cdot \log n = \frac{n}{\log n}$$

không gian.

Tiếp theo, chia mỗi khối thành các khối nhỏ kích thước $\log n$, rồi lưu tổng trên đoạn từ cuối mỗi khối nhỏ đến đầu khối lớn. Như vậy tổng cộng lưu $n/\log n$ giá trị; mỗi giá trị có cỡ $\log^2 n$, nên cần

$$\frac{n \log \log n}{\log n}$$

không gian.

Cuối cùng, dùng một mảng lưu tổng của mọi chuỗi 01 có độ dài không vượt quá $\log n/2$, cần $O(\sqrt{n} \log \log n)$ không gian.

Với thao tác rank, trước hết định vị khối lớn chứa k , rồi định vị khối nhỏ chứa k . Sau khi lấy hai tổng tiền tố phía trước, phần cuối chưa đủ $\log n$ có thể tính bằng hai lần tra bảng.

Với thao tác select, có thể dùng tương tự phương pháp chia khối nhiều lần; chi tiết có thể tham khảo [3]. Do giới hạn độ dài, bài viết không trình bày thêm.

2.3. Cài đặt

Hai cách làm phía trên đạt độ phức tạp tốt, nhưng khi cài đặt thực tế còn phụ thuộc vào độ lớn của các hằng số.

Chuỗi 01 có thể được lưu trực tiếp bằng cách dùng một số nguyên 64 bit cho mỗi 64 bit liên tiếp.

Với thao tác rank, có thể chia khối với kích thước là bội của 64 và lưu tổng tiền tố giữa các khối. Phần chưa đủ một khối có thể tính trực tiếp bằng lệnh CPU. Có thể điều chỉnh kích thước khối để cân bằng thời gian và không gian.

Với thao tác select, một phương pháp là dùng lại bảng của thao tác rank, tìm kiếm nhị phân trong đó rồi xử lý phần chưa đủ một khối. Cách này dùng ít không gian hơn, nhưng độ phức tạp thời gian cao hơn.

Một phương pháp khác tương tự mục 2.1, nhưng để giảm hằng số truy vấn thì chỉ chia khối một lần. Với phần ánh xạ sang đoạn ngắn trong chuỗi gốc, có thể tính vết cận trực tiếp; với phần dài hơn thì vẫn lưu đáp án.

3. Cây

Mục này giới thiệu ba cấu trúc dữ liệu ngắn gọn biểu diễn cây và ứng dụng của chúng. Ba cấu trúc dữ liệu đó lần lượt là BP (Balanced Parentheses), LOUDS (Level-ordered Unary Degree Sequence) và DFUDS (Depth-first Unary Degree Sequence).

Cả ba cấu trúc dữ liệu đều mã hóa cây thành chuỗi 01 hoặc dãy ngoặc độ dài $2n + O(1)$, rồi dùng cấu trúc dữ liệu cho chuỗi 01 đã nêu ở trên, hoặc cấu trúc dữ liệu cho dãy ngoặc trong [4], để duy trì.

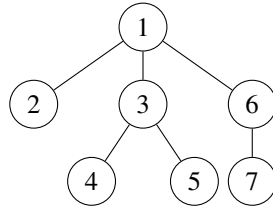
Ba cấu trúc dữ liệu này đều có thời gian $O(1)$ trên một số thao tác, còn trên các thao tác khác thì khác nhau. Ngoài ra còn có một số cấu trúc dữ liệu khác có thể thực hiện các thao tác này và nhiều thao tác hơn nữa trong cùng không gian $2n + o(n)$; chi tiết có thể tham khảo [6,7,8], ở đây không nhắc lại.

3.1. BP

BP là cách dùng dãy ngoặc để biểu diễn cây.

Dãy ngoặc tương ứng với Hình 10 là:

Dãy ngoặc	(((((((()))))))
Số hiệu đỉnh tương ứng	1 2 2 3 4 4 5 3 3 6 7 7 6 1



Hình 10: Một cây.

Sau khi bỏ hai ngoặc của nút gốc, hiển nhiên dãy ngoặc cân bằng độ dài $2n - 2$ (hai loại ngoặc có số lượng bằng nhau) và cây n đỉnh tương ứng một-một với nhau. Số dãy ngoặc độ dài $2n - 2$ là

$$C_{n-1} = \frac{1}{n} \binom{2n-2}{n-1}.$$

Theo công thức Stirling, $\log C_n$ tiến tới $2n$. Vì vậy, dùng không gian $2n + o(n)$ bit là ngắn gọn.

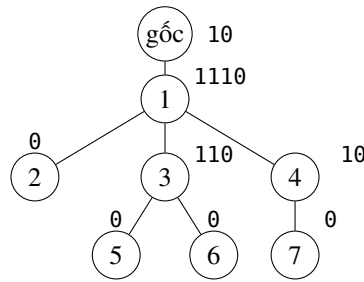
Theo phương pháp trong [4], với một dãy ngoặc độ dài $2n$, ta có thể trong thời gian $O(1)$ và không gian $2n + o(n)$ tìm được ngoặc phải ứng với một ngoặc trái, ngoặc trái ứng với một ngoặc phải, và ngoặc bao ngoài một ngoặc.

Dùng các thao tác này, có thể cài đặt trong $O(1)$ các thao tác tìm con đầu tiên, con cuối cùng, nút anh em, cha, cũng như tính kích thước cây con và độ sâu của đỉnh.

3.2. LOUDS

LOUDS là cách mã hóa bậc theo dạng nhất phân theo thứ tự BFS.

Trước hết thêm cho cây một gốc mới, trong đó con duy nhất của gốc mới là gốc của cây ban đầu. Sau đó đánh số cây này theo thứ tự BFS và nối lần lượt mã của từng nút để thu được mã của cả cây. Mã của mỗi nút phụ thuộc vào số con của nó: nếu có x con thì mã là x ký tự 1, rồi thêm một ký tự 0.



Hình 11: Một cây và mã của từng đỉnh.

Mã của cây trong Hình 11 là

10.1110.0.110.10.0.0.0,

trong đó dấu chấm chỉ để tiện đọc. Mỗi nút tương ứng đúng một ký tự 1 trong mã. Quan hệ giữa số hiệu nút và vị trí trong mã như sau:

- $v = \text{rank}_1(i)$: tìm số hiệu nút v ứng với vị trí i .
- $i = \text{select}_1(v)$: tìm vị trí i của nút có số hiệu v .

Các thao tác liên quan tới nút trên cây cũng có thể được cài đặt dễ dàng:

- $\text{firstchild}(i) = \text{select}_0(\text{rank}_1(i)) + 1$: tìm vị trí con đầu tiên của nút ứng với vị trí i .

- $\text{lastchild}(i) = \text{select}_0(\text{rank}_1(i) + 1) - 1$: tìm vị trí con cuối cùng của nút ứng với vị trí i .
- $\text{parent}(i) = \text{select}_1(\text{rank}_0(i))$: tìm vị trí cha của nút ứng với vị trí i .
- $\text{child}(i, k) = \text{firstchild}(i) + k - 1$: tìm vị trí con thứ k của nút ứng với vị trí i .
- $\text{degree}(i) = \text{lastchild}(i) - \text{firstchild}(i) + 1$: tìm số con của nút ứng với vị trí i .

3.3. DFUDS

DFUDS là cách mã hóa bậc theo dạng nhất phân theo thứ tự DFS.

DFUDS của một cây được cho như sau:

- Với cây chỉ có một nút, mã của nó là $()$.
- Với cây có nhiều hơn một nút, trước hết xóa ngoặc trái ngoài cùng trong mã của mọi cây con. Sau đó mã của cây là $((\dots))$, trong đó số ngoặc trái bằng số con cộng 1, rồi nối tiếp lần lượt mã của từng con.

Nếu xóa ngoặc trái ngoài cùng, thì phần mã mới được thêm vào cho mỗi nút giống LOUDS, trong đó 1 ứng với ngoặc trái và 0 ứng với ngoặc phải.

Mã của cây trong Hình 10 là

$(.(((.)).((.)).).().)$

trong đó dấu chấm chỉ để dễ nhìn; mã của từng nút có thể xem ở Hình 11.

Không khó thấy DFUDS nhất định là một dãy ngoặc hợp lệ.

Khi xây dựng thực tế, có thể DFS một lần, rồi với mỗi nút lần lượt thêm số ngoặc trái bằng số con của nó và một ngoặc phải. Cuối cùng thêm ngoặc trái ở đầu. Thuật toán cụ thể như sau.

Thuật toán 1 Tìm mã DFUDS của một cây

Require: cây T

Ensure: mã $Result$

```

1:  $Result \leftarrow ""$ 
2: procedure Dfs( $x$ )
3:    $Result \leftarrow Result + "(" \times x.children\_count + ")"$ 
4:   for  $y \in x.children$  do
5:     Dfs( $y$ )
6:   end for
7: end procedure
8: Dfs( $T.root$ )
9:  $Result \leftarrow "(" + Result$ 

```

Để thực hiện nhanh các thao tác trên cây, ta cần dùng các thao tác đã nêu ở mục 2.2 và các thao tác trên dãy ngoặc trong [4] để duy trì.

- $\text{rank}_v(k)$: hỏi từ đầu đến vị trí k có bao nhiêu ký tự v , trong đó v là *open* hoặc *close*, biểu diễn ngoặc trái và ngoặc phải.
- $\text{select}_v(k)$: hỏi vị trí ký tự v thứ k .
- $\text{findclose}(k)$: hỏi ngoặc phải ứng với vị trí k , bảo đảm vị trí k là ngoặc trái.
- $\text{findopen}(k)$: hỏi ngoặc trái ứng với vị trí k , bảo đảm vị trí k là ngoặc phải.
- $\text{enclose}(k)$: hỏi ngoặc trái bao ngoài vị trí k .

Trong DFUDS, mỗi nút và mỗi ngoặc phải tương ứng một-một, với quan hệ như sau:

- $v = \text{rank}_{\text{close}}(i)$: tìm số hiệu nút v ứng với vị trí i .
- $i = \text{select}_{\text{close}}(v)$: tìm vị trí i của nút có số hiệu v .

Dễ thấy số hiệu của mỗi nút chính là vị trí của nó trong thứ tự DFS.

Các thao tác được cài đặt như sau:

- $\text{pre}(k) = \text{select}_{\text{close}}(\text{rank}_{\text{close}}(k) - 1)$: tìm vị trí ngoặc phải đứng trước.
 - $\text{degree}(k) = k - \text{select}_{\text{close}}(\text{rank}_{\text{close}}(k) - 1)$: tìm số con của nút ứng với vị trí k .
 - $\text{child}(i, k) = \text{select}_{\text{close}}(\text{rank}_{\text{close}}(\text{findclose}(k - i)) + 1)$: tìm con thứ i của nút ứng với vị trí k .
 - $\text{parent}(k) = \text{select}_{\text{close}}(\text{rank}_{\text{close}}(\text{findopen}(\text{pre}(k))) + 1)$: tìm cha của nút ứng với vị trí k .
 - $\text{subtreesize}(k) = (\text{findclose}(\text{enclose}(\text{pre}(k))) - \text{pre}(k)) / 2 + 1$: tìm kích thước cây con của vị trí k .
- Thao tác này cần xử lý riêng khi k là gốc.

3.4. Ứng dụng

Trong nhiều tình huống cần biểu diễn cây, đều có thể dùng ba cấu trúc dữ liệu này để tối ưu.

Chẳng hạn với trie, khi xây dựng trie cần dùng cấu trúc con trỏ; nhưng sau khi xây xong, ta có thể không dùng con trỏ nữa mà chỉ giữ lại cấu trúc cây. Dưới đây là hai cách tối ưu.

Giả sử bảng chữ cái là Σ , số nút của trie là n .

Cách thứ nhất: với mỗi nút của trie, thêm một số nút ảo để bù đầy số con lên $|\Sigma|$. Sau đó mã hóa trực tiếp cây này; có thể dựa vào số con của một nút để phán đoán nó có phải nút ảo hay không.

Cách này có độ phức tạp không gian $O(n|\Sigma|)$, và độ phức tạp thời gian để tìm nút con ứng với một ký tự là $O(1)$.

Cách thứ hai: mã hóa trực tiếp trie ban đầu, rồi ghi lại ký tự ứng với mỗi nút. Khi tìm nút con ứng với một ký tự, có thể tìm kiếm nhị phân hoặc duyệt; độ phức tạp thời gian lần lượt là $O(\log |\Sigma|)$ và $O(|\Sigma|/w)$, trong đó w là độ rộng từ máy. Cách sau có thể xem gần như $O(1)$ khi $|\Sigma|$ nhỏ.

Cách này có độ phức tạp không gian $O(n)$.

4. Wavelet Tree

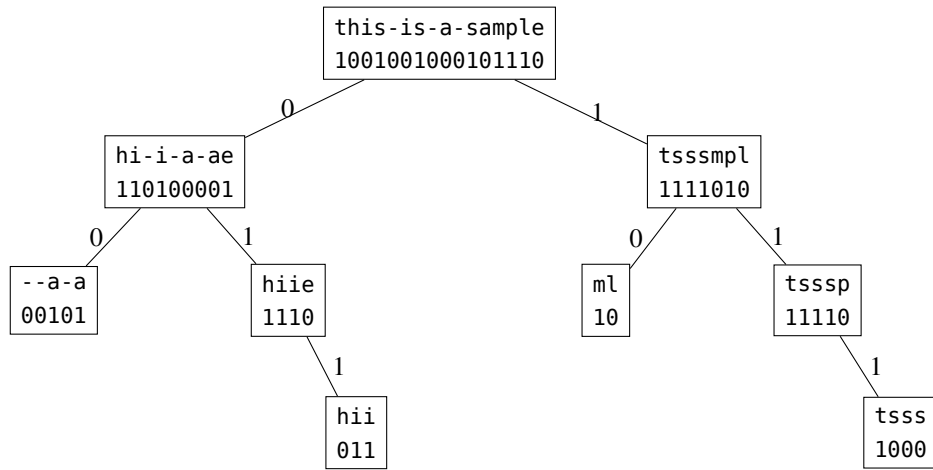
Xét một chuỗi S độ dài n , bảng chữ cái là Σ , cần hỗ trợ hai thao tác:

- $\text{rank}_v(k)$: hỏi từ vị trí 1 đến vị trí k có bao nhiêu ký tự v .
- $\text{select}_v(k)$: hỏi vị trí của ký tự v thứ k .

Wavelet tree có thể thực hiện hai thao tác này với không gian $(1 + o(1))n \log |\Sigma|$ và thời gian mỗi truy vấn $O(\log |\Sigma|)$.

Với một chuỗi, ta chia bảng chữ cái của nó thành hai phần, thường là chia đều trực tiếp theo thứ tự ký tự, rồi thay các ký tự thuộc phần thứ nhất bằng 0 và các ký tự thuộc phần thứ hai bằng 1. Sau đó lấy lần lượt các ký tự thuộc hai phần trong chuỗi gốc để thu được hai chuỗi mới, rồi đệ quy xây cây trên hai chuỗi mới ấy, cho tới khi kích thước bảng chữ cái bằng 1.

Hình 12 là wavelet tree của chuỗi gốc `this-is-a-sample`, với bảng chữ cái $\{-, a, e, h, i, l, m, p, s, t\}$.



Hình 12: Ví dụ về wavelet tree.

Cây xây theo cách này có $|\Sigma| - 1$ nút, độ sâu lớn nhất là $\lceil \log |\Sigma| \rceil$, và tổng độ dài các chuỗi 01 được duy trì trên mọi nút không vượt quá $n \lceil \log |\Sigma| \rceil$.

Sau khi duy trì nhanh các thao tác $\text{rank}_0, \text{rank}_1, \text{select}_0, \text{select}_1$ trên chuỗi 01 của mỗi nút, các thao tác cần hỏi có thể được thực hiện như sau:

- Với thao tác $\text{rank}_v(k)$, ta tìm ở nút hiện tại xem v bị chia sang 0 hay 1. Giả sử v bị chia sang x , ta tính $\text{rank}_x(k)$ trên chuỗi 01 của nút hiện tại, rồi đệ quy sang nút con phía ấy để tiếp tục truy vấn. Chẳng hạn, tại gốc trong Hình 12 nếu hỏi $\text{rank}_a(10)$, ta thấy a được chia sang 0, và trên chuỗi 01 tại nút ấy có $\text{rank}_0(10) = 7$, nên có thể đệ quy sang con trái và tiếp tục hỏi $\text{rank}_a(7)$.
- Với thao tác $\text{select}_v(k)$, có thể xử lý ngược với thao tác rank . Trước hết tìm nút thấp nhất chứa v , xác định ở nút ấy v được chia thành giá trị x , rồi tính $\text{select}_x(k)$ trên chuỗi 01 của nút đó, cuối cùng đệ quy lên nút cha để xử lý.

Chẳng hạn, trong Hình 12 nếu hỏi $\text{select}_a(1)$, ta thấy nút thấp nhất chứa a là $--a-a$. Ở nút ấy truy vấn $\text{select}_1(1) = 3$, rồi có thể chuyển thành truy vấn $\text{select}_0(3) = 6$ ở cha của nó, tiếp tục đệ quy lên cha thành $\text{select}_0(6) = 9$, cuối cùng thu được $\text{select}_a(1) = 9$.

Mã giả của hai thao tác này như sau:

Thuật toán 2 Các thao tác rank và select trên wavelet tree

Require: nút gốc T , bảng chữ cái $\{1, \dots, |\Sigma|\}$

```

1:  function Rank( $T, v, k, l, r$ )
2:    if  $l = r$  then
3:      return  $k$ 
4:    end if
5:     $mid \leftarrow (l + r) / 2$ 
6:    if  $v \leq mid$  then
7:      return Rank( $T.left, v, T.rank_0(k), l, mid$ )
8:    else
9:      return Rank( $T.right, v, T.rank_1(k), mid + 1, r$ )
10:   end if
11: end function
12:
13: function Select( $T, v, k, l, r$ )
14:   if  $l = r$  then
15:     return  $k$ 
16:   end if
17:    $mid \leftarrow (l + r) / 2$ 
18:   if  $v \leq mid$  then
19:     return  $T.select_0(Select(T.left, v, k, l, mid))$ 
20:   else
21:     return  $T.select_1(Select(T.right, v, k, mid + 1, r))$ 
22:   end if
23: end function
24:
25: function rank( $v, k$ )
26:   return Rank( $T, v, k, 1, |\Sigma|$ )
27: end function
28:
29: function select( $v, k$ )
30:   return Select( $T, v, k, 1, |\Sigma|$ )
31: end function

```

Dùng wavelet tree còn có thể hoàn thành các thao tác sau:

- $quantile(l, r, k)$: tìm giá trị lớn thứ k trong $[l, r]$. Có thể tính số lượng 0 trong $[l, r]$, từ đó quyết định giá trị lớn thứ k bị chia sang 0 hay 1, rồi đệ quy sang nút con phía ấy.
- $rangefreq(l, r, x, y)$: tìm số phần tử trong $[l, r]$ có miền giá trị thuộc $[x, y]$. Có thể truy vấn đệ quy tương tự cây đoạn.
- $nextvalue(l, r, x)$: tìm giá trị nhỏ nhất lớn hơn x trong $[l, r]$. Có thể trước hết xác định x bị chia sang 0 hay 1, rồi đệ quy sang nút con phía ấy để xử lý.

Các thao tác này đều có thể hoàn thành trong thời gian $\log |\Sigma|$. Nhiều bài toán khác đồng thời đặt ràng buộc trên đoạn và trên miền giá trị cũng có thể được thực hiện dễ dàng.

5. Tổng kết

Bài viết này từ chuỗi 01 dẫn nhập cấu trúc dữ liệu xử lý cây và wavelet tree. Với chuỗi 01 và cây, ngoài các cấu trúc dữ liệu được nhắc tới trong bài viết này, còn có những cấu trúc dữ liệu đạt độ phức tạp thấp hơn hoặc thực hiện được nhiều chức năng hơn.

Ba loại cấu trúc dữ liệu này đều có ứng dụng rộng, đặc biệt là trong các bối cảnh như thuật toán nén, cơ sở dữ liệu và chỉ mục sâu.

Trong thi tin học, do giới hạn bộ nhớ thường khá lớn nên thông thường không cần dùng các cấu trúc dữ liệu này; còn nếu muốn khảo sát trực tiếp các cấu trúc dữ liệu ấy, kỹ thuật chấm hiện có lại khó giới hạn bộ nhớ thật chính xác, vì vậy hiện nay chúng ít được khảo sát. Tuy nhiên, các cấu trúc dữ liệu này vẫn có không ít công dụng trong tối ưu hằng số; nếu muốn khảo sát trực tiếp, có thể dùng một ngôn ngữ tùy biến để giới hạn bộ nhớ chính xác, chẳng hạn ngôn ngữ Potato từng xuất hiện trong đợt tập huấn IOI2019.

Lời cảm ơn

Cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn thầy Zhang Junliang và thầy Lin Yang của Trường Trung học số 7 Thành Đô đã quan tâm và chỉ dẫn trong nhiều năm.

Cảm ơn các thầy cô và bạn học đã đọc duyệt, góp ý cho bài viết này.

Cảm ơn các thầy cô và bạn học khác từng giúp đỡ tôi.

Tài liệu tham khảo

1. G. Jacobson, Succinct Static Data Structures, Tech. report CMU-CS-89-112, Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA, 1989.
2. G. Jacobson, Space-efficient static trees and graphs, in Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science, pp. 549–554, 1989.
3. A. Golynski, Optimal lower bounds for rank and select indexes, *Theoretical Computer Science*, 387, pp. 348–359, 2007.
4. J. I. Munro and V. Raman, Succinct representation of balanced parentheses, static trees and planar graphs, in Proceedings of the 38th Annual IEEE Symposium on Foundations of Computer Science, pp. 118–126, 1997.
5. D. Benoit, E. D. Demaine, J. I. Munro, and V. Raman, Representing trees of higher degree, in Proceedings of the 6th Workshop on Algorithms and Data Structures (WADS), Lecture Notes in Comput. Sci. 1663, Springer-Verlag, Berlin, pp. 169–180, 1999.
6. R. F. Geary, R. Raman and V. Raman. Succinct ordinal trees with level-ancestor queries. In Proc. 15th ACM-SIAM SODA, pp. 1–10, 2004.
7. M. He, J. I. Munro, S. S. Rao, Succinct ordinal trees based on tree covering, in: Proceedings of the International Conference on Automata, Language and Programming, LNCS, vol. 4596, pp. 509–520, 2007.
8. A. Farzan, R. Raman, and S. S. Rao. Universal succinct representations of trees? In Proceedings of the 36th International Colloquium on Automata, Languages and Programming (ICALP). 451–462, 2009.
9. H. Zhang, D. G. Andersen, M. Kaminsky, A. Pavlo, H. Lim, V. Leis, and K. Keeton. SuRF: Practical Range Query Filtering with Fast Succinct Tries. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD). ACM, 323–336, 2018.

10. R. Grossi, A. Gupta, and J. S. Vitter, High-order entropy-compressed text indexes, in Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms, SIAM, Philadelphia, pp. 841–850, 2003.
11. M. Burrows and D. J. Wheeler, A block-sorting lossless data compression algorithm. Digital SRC Reports 124, 1994.

Thuật toán liên quan tới truy vấn chu kỳ xâu con và ứng dụng

Chen Sunli, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này giới thiệu cách giải bài toán truy vấn chu kỳ xâu con, cùng một số kết luận và công cụ xâu dùng trong lời giải, đó là từ điển xâu con cơ bản (Dictionary of Basic Factors). Sau đó bài viết trình bày một số ứng dụng của những kết luận và công cụ nói trên.

Mục 2 đưa vào một số định nghĩa và ký hiệu cơ sở.

Mục 3 chứng minh ba bổ đề quan trọng và giới thiệu từ điển xâu con cơ bản.

Mục 4 mô tả phương pháp giải bài toán truy vấn chu kỳ xâu con.

Mục 5 giới thiệu một số ứng dụng của từ điển xâu con cơ bản.

1. Dẫn nhập

Ở giai đoạn hiện nay, nghiên cứu về xâu trong khoa học máy tính ngày càng chú trọng khảo sát tính chất của chu kỳ, tức các “mẫu lặp”. Hướng này cũng có ý nghĩa thực tế lớn: chẳng hạn nhận dạng tiếng nói và khớp DNA đều cần mô tả tinh tế những mẫu xâu lặp lại với số lượng lớn.

Bài toán truy vấn chu kỳ xâu con là một bài toán kinh điển có mô tả đơn giản nhưng lời giải phức tạp. Từ đó còn có thể mở rộng thành bài toán khớp mẫu nội bộ (Internal Pattern Matching) có phạm vi ứng dụng rộng hơn. Bài viết này tập trung giới thiệu cách giải bài toán ấy và một số mở rộng.

2. Ký hiệu, định nghĩa cơ sở và mô tả bài toán

2.1. Ký hiệu và định nghĩa

Gọi S là một xâu, $|S|$ là độ dài của nó, $S[i]$ là ký tự thứ i của S , và $S[l, r]$ là xâu mới tạo bởi các ký tự từ vị trí l đến vị trí r của S . Nếu $l > r$ thì $S[l, r] = \emptyset$.

Ký hiệu $\text{pref}(S, x) = S[1, \dots, x]$ là tiền tố độ dài x của S . Tương tự, $\text{suf}(S, x) = S[|S| - x + 1, \dots, |S|]$ là hậu tố độ dài x của S .

Nếu $T[x, \dots, x + |S| - 1] = S$, ta nói S xuất hiện trong T tại vị trí x , và x là vị trí xuất hiện hay vị trí khớp của S trong T .

Định nghĩa 1. Nếu xâu S và số nguyên x thỏa mãn $1 \leq x \leq |S|$, đồng thời với mọi $1 \leq i \leq |S| - x$ đều có $S[i] = S[i + x]$, thì gọi x là một chu kỳ, hay độ dài chu kỳ, của S . Đặc biệt, nếu x còn là ước của $|S|$, thì gọi x là một chu kỳ nguyên của S , và gọi S là xâu chu kỳ nguyên.

Hiển nhiên, nếu x là chu kỳ của S , thì kx ($k \in \mathbb{Z}^+$, $kx \leq |S|$) cũng là chu kỳ của S ; ngoài ra x cũng là chu kỳ của $\text{pref}(S, y)$ với $y \geq x$.

Định nghĩa 2. $\text{per}(S)$ biểu thị tập hợp tất cả các chu kỳ của S , còn $\text{minper}(S)$ biểu thị giá trị nhỏ nhất trong $\text{per}(S)$. Dễ thấy với xâu không rỗng S , $|S|$ luôn là một chu kỳ của S , nên $|\text{per}(S)| > 0$.

Định nghĩa 3. Nếu xâu S và số nguyên x thỏa mãn $0 \leq x < |S|$, đồng thời $\text{pref}(S, x) = \text{suf}(S, x)$, thì gọi $\text{pref}(S, x)$ là một border của S .

Hệ quả 1. $|S| - x$ là chu kỳ của S khi và chỉ khi $\text{pref}(S, x)$ là một border của S .

Chứng minh. Theo định nghĩa của border và của hai xâu bằng nhau, $\text{pref}(S, x)$ là border của S khi và chỉ khi với mọi $1 \leq i \leq |\text{pref}(S, x)|$ đều có

$$\text{pref}(S, x)[i] = \text{suf}(S, x)[i],$$

tức $S[i] = S[|S| - x + i]$, đúng với định nghĩa chu kỳ. $\square$

Hệ quả 2. Nếu p là chu kỳ của S , và một xâu con $S[l, \dots, r]$ thỏa $r - l + 1 \geq p$ có chu kỳ q , trong đó $q \mid p$, thì q cũng là chu kỳ của S .

Hệ quả này khá hiển nhiên.

2.2. Mô tả bài toán

Cho một xâu S và nhiều truy vấn. Mỗi truy vấn cho hai số l, r , yêu cầu đưa ra $\text{per}(S[l, \dots, r])$.

Nhờ các kết luận sẽ nêu bên dưới, ta có thể dùng một cách ngắn gọn để biểu diễn $\text{per}(S[l, \dots, r])$, chỉ tốn $O(\log n)$ không gian thay vì $O(|S|)$ trong trường hợp xấu nhất. Đồng thời, ta sẽ đưa ra một thuật toán tiền xử lý trong $O(n \log n)$ thời gian và không gian, rồi trả lời mỗi truy vấn trong $O(\log n)$ thời gian và không gian.

3. Chuẩn bị cho lời giải

3.1. Một số tính chất về “lặp”

Về chu kỳ của xâu có một bổ đề quan trọng gọi là Periodicity Lemma. Chứng minh của nó tương đối rườm rà; dưới đây ta chứng minh một phiên bản yếu hơn.

Bổ đề 1 (Weak Periodicity Lemma). Nếu $p, q \in \text{per}(S)$ và $p + q \leq |S|$, thì $\text{gcd}(p, q) \in \text{per}(S)$.

Chứng minh. Không mất tính tổng quát, giả sử $p < q$. Với mọi $1 \leq i \leq |S|$, nếu $p < i \leq |S| - q + p$, thì

$$S[i] = S[i - p] = S[i + q - p].$$

Nếu $i \leq p$, thì

$$S[i] = S[i + q] = S[i + q - p].$$

Suy ra $q - p$ cũng là chu kỳ của S . Ở đây ta dùng định nghĩa chu kỳ và điều kiện $p + q \leq |S|$. Lại có $p + (q - p) \leq |S|$, nên có thể áp dụng thuật toán Euclid liên tiếp để thu được $\text{gcd}(p, q)$ là chu kỳ của S . $\square$

Hệ quả 3. Nếu $p = \text{minper}(S) \leq |S|/2$, thì với mọi $x \in \text{per}(S)$ thỏa $x + p \leq |S|$, ta có $p \mid x$.

Chứng minh. Nếu $p \nmid x$, theo Bổ đề 1 có $\text{gcd}(p, x) \in \text{per}(S)$. Vì $p < x$, nên $\text{gcd}(p, x) < p$, mâu thuẫn với $p = \text{minper}(S)$. $\square$

Từ đây trở đi, ta viết tắt Weak Periodicity Lemma là WPL.

Bổ đề 2. Nếu hai xâu S, T thỏa $2|S| \geq |T|$, thì các vị trí xuất hiện của S trong T tạo thành một cấp số cộng.

Chứng minh. Nếu S xuất hiện trong T một hoặc hai lần thì bổ đề đã đúng, nên chỉ cần xét trường hợp S xuất hiện trong T ít nhất ba lần.

Với hai vị trí xuất hiện $p < q$, do S là border của $T[p, \dots, q + |S| - 1]$, $q - p$ là chu kỳ của $T[p, \dots, q + |S| - 1]$. Đồng thời, từ $1 \leq p, q \leq |T| - |S| + 1$, ta có

$$p + |S| - 1 \geq |S| \geq |T| - |S| \geq q,$$

nên $q - p < |S|$. Do đó $q - p$ cũng là chu kỳ của S .

Xét hai vị trí xuất hiện đầu tiên $p_1 < p_2$ của S trong T , và một vị trí xuất hiện $q > p_2$. Ta có

$$p_2 - p_1, q - p_2 \in \text{per}(S).$$

Lại có $q - p_1 \leq |T| - |S| \leq |S|$. Theo WPL,

$$r = \gcd(p_2 - p_1, q - p_2) \in \text{per}(S).$$

Đồng thời

$$r \leq \min(p_2 - p_1, q - p_2) \leq \frac{q - p_1}{2} \leq \frac{|S|}{2}.$$

Đặt $r' = \text{minper}(S)$. Theo Hệ quả 3, $r' \mid r$. Theo Hệ quả 2, r' là chu kỳ của $T[p_1, \dots, p_2 + |S| - 1]$. Giả sử $r' < r$, theo kết luận phía trên, S cũng xuất hiện tại vị trí $p_1 + r'$, mâu thuẫn với việc p_2 là vị trí xuất hiện thứ hai. Vì vậy $r = r' = \text{minper}(S)$.

Đến đây, áp dụng Hệ quả 3 thêm một lần nữa, ta có $r \mid q - p_2$. Mặt khác, nếu gọi q là vị trí khớp cuối cùng của S trong T , thì mọi $x = p_1 + kr \leq q$, $k \in \mathbb{Z}$, theo kết luận trên đều là vị trí xuất hiện của S . Kết hợp hai kết luận này, Bổ đề 2 được chứng minh. $\square$

Không chỉ vậy, ta còn thu được: nếu S xuất hiện trong T ít nhất ba lần, thì công sai của cấp số cộng tương ứng bằng $\text{minper}(S)$.

Bổ đề 3. Tất cả độ dài border của xâu S không nhỏ hơn $|S|/2$ tạo thành một cấp số cộng.

Chứng minh. Chỉ cần chứng minh mọi chu kỳ của S không lớn hơn $|S|/2$ tạo thành một cấp số cộng; kết luận này suy ra trực tiếp từ Hệ quả 3. $\square$

Đến đây, ba bổ đề cần thiết để giải bài toán truy vấn chu kỳ xâu con đã được chứng minh.

3.2. Cấu trúc dữ liệu: từ điển xâu con cơ bản

Để giải bài toán truy vấn chu kỳ xâu con, ta còn cần một cấu trúc dữ liệu gọi là từ điển xâu con cơ bản.

Định nghĩa 4. Từ điển xâu con cơ bản (Dictionary of Basic Factors) của một xâu S , ký hiệu $\text{DBF}(S)$, gồm $\lceil \log_2 |S| \rceil + 1$ mảng. Cụ thể, mảng thứ t được ký hiệu Name_t , có độ dài $|S| - 2^t + 1$, và thỏa mãn

$$\text{Name}_t(i) \leq \text{Name}_t(j)$$

khi và chỉ khi

$$S[i, \dots, i + 2^t - 1] \leq S[j, \dots, j + 2^t - 1].$$

Thông thường, ta có thể đặt thêm ràng buộc rằng các giá trị trong mỗi Name_t là số nguyên dương và miền giá trị đúng bằng $\{1, 2, \dots, k\}$, với $k \in \mathbb{Z}$.

Dễ thấy mảng Name_t có thể được suy ra từ Name_{t-1} . Vì vậy, để tìm từ điển xâu con cơ bản của xâu S , chỉ cần dùng thuật toán nhân đôi tìm mảng hậu tố tương tự Manber–Myers, với độ phức tạp $O(n \log n)$.

Chỉ có các mảng Name có thể vẫn chưa đủ. Một mở rộng đơn giản là tách mỗi mảng $Name_i$ theo giá trị: với mỗi x , duy trì một danh sách có thứ tự ghi lại mọi i thỏa $Name_i[i] = x$. Không khó thấy bước này cũng có thể hoàn thành trong quá trình xây dựng từ điển xâu con cơ bản, và độ phức tạp vẫn là $O(n \log n)$.

Phần sau sẽ cho thấy để đạt độ phức tạp tối ưu hiện tại $O(\log n)$ cho một truy vấn, ta còn phải mở rộng từ điển xâu con cơ bản thêm một lần nữa.

4. Giải bài toán truy vấn chu kỳ xâu con

4.1. Thuật toán cơ bản

Định nghĩa 5. Với hai xâu S, T thỏa $|S| = |T|$, định nghĩa

$$PS(S, T) = \{k \mid \text{pref}(S, k) = \text{suf}(T, k)\},$$

$$\text{LargePS}(S, T) = \{k \mid k \in PS(S, T), k \geq |S|/2\}.$$

Hệ quả 4. Các phần tử của $\text{LargePS}(S, T)$ tạo thành một cấp số cộng.

Chứng minh. Xét phần tử lớn nhất p trong tập này. Với mọi $q \in \text{LargePS}(S, T)$, $\text{pref}(S, q)$ là tiền tố của $\text{pref}(S, p)$, còn $\text{suf}(T, q)$ là hậu tố của $\text{suf}(T, p)$. Do đó $\text{pref}(S, q)$ là border của $\text{pref}(S, p)$. Theo Bổ đề 3, các phần tử này tạo thành một cấp số cộng. $\square$

Hệ quả 5. Với một xâu S và số nguyên $k \leq |S|/2$, tất cả độ dài border x của S thỏa $k \leq x < 2k$ tạo thành một cấp số cộng.

Chứng minh. Tập các x nói trên đúng bằng

$$\text{LargePS}(\text{pref}(S, 2k), \text{suf}(S, 2k)).$$

$\square$

Nhờ kết luận này, ta có thể chia các border của S thành $O(\log |S|)$ lớp. Lớp thứ i gồm mọi border có độ dài thuộc $[2^{i-1}, 2^i)$, nhưng nếu $|S|$ không phải lũy thừa của 2 thì khoảng cuối cùng cần đổi thành $[2^{\lfloor \log_2 |S| \rfloor}, |S|)$. Trong mỗi khoảng, các độ dài border đều có thể được biểu diễn bằng một cấp số cộng. Vì vậy ta thu được một cách biểu diễn các độ dài border của S , tức $\text{per}(S)$, chỉ tốn $O(\log |S|)$ không gian.

Với khoảng kiểu $[2^{i-1}, 2^i)$, thứ ta cần chính là

$$\text{LargePS}(\text{pref}(S, 2^i), \text{suf}(S, 2^i)).$$

Ta dùng bổ đề sau; chứng minh của nó khá hiển nhiên.

Bổ đề 4. Với $2^{k-1} \leq x < |S| = |T| = 2^k$, ta có

$$x \in \text{LargePS}(S, T)$$

khi và chỉ khi

$$\text{suf}(\text{pref}(S, x), 2^{k-1}) = \text{suf}(T, 2^{k-1}),$$

và tương ứng

$$\text{pref}(\text{suf}(T, x), 2^{k-1}) = \text{pref}(S, 2^{k-1}).$$

Có bổ đề này, trước hết ta tìm mọi vị trí xuất hiện của $\text{pref}(S, 2^{i-1})$ trong $\text{suf}(S, 2^i)$, và mọi vị trí xuất hiện của $\text{suf}(S, 2^{i-1})$ trong $\text{pref}(S, 2^i)$, đều biểu diễn bằng cấp số cộng. Để thu được $\text{LargePS}(\text{pref}(S, 2^i), \text{suf}(S, 2^i))$, ta chỉ cần tịnh tiến hai cấp số cộng nói trên rồi lấy giao.

Việc tìm mọi vị trí xuất hiện của $\text{pref}(S, 2^{i-1})$ trong $\text{suf}(S, 2^i)$ chính là tìm vị trí xuất hiện đầu tiên, thứ hai và cuối cùng. Vì hai xâu này đều là xâu con của xâu mẹ, chỉ cần cài đặt hai thao tác $\text{succ}(S, i)$ và $\text{pred}(S, i)$, tức tìm vị trí xuất hiện đầu tiên của xâu S không trước hoặc không sau vị trí i . Cụ thể, giả sử xâu mẹ là T , xâu thứ nhất là $S_1 = T[l, \dots, r]$, xâu thứ hai là $S_2 = T[l', \dots, r']$, trong đó

$$2(r - l + 1) = r' - l' + 1.$$

Đặt

$$s = \text{succ}(S_1, l'), \quad d = \text{succ}(S_1, s + 1) - s, \quad t = \text{pred}(S_1, r' - |S_1| + 1).$$

Cấp số cộng cần tìm có số hạng đầu s , số hạng cuối t , công sai d .

Vì vậy, ta xây dựng từ điển xâu con cơ bản của xâu mẹ, rồi trong Name_{i-1} tìm danh sách có thứ tự chứa $\text{pref}(S, 2^{i-1})$. Với mỗi truy vấn succ hoặc pred , chỉ cần tìm kiếm nhị phân trên danh sách có thứ tự này.

Với khoảng kiểu $[2^{\lceil \log_2 |S| \rceil}, |S|]$, cách làm tương tự như trên, nhưng sau khi lấy giao hai cấp số cộng còn phải cắt kết quả để chỉ giữ đoạn nhỏ hơn $|S|$.

Đến đây, khung thuật toán cơ bản cho bài toán truy vấn chu kỳ xâu con đã hoàn thành.

4.2. Lấy giao cấp số cộng và tối ưu

Ta dùng ba tham số (s, d, k) để mô tả cấp số cộng

$$s, s + d, \dots, s + (k - 1)d.$$

Xét việc tìm phần chung của hai cấp số cộng (s_1, d_1, k_1) và (s_2, d_2, k_2) , tức tìm tất cả nghiệm nguyên của

$$s_1 + xd_1 = s_2 + yd_2,$$

rồi cắt lấy đoạn thích hợp. Với bài toán tổng quát, chỉ có thể dùng thuật toán Euclid mở rộng để giải phương trình Diophantine này, độ phức tạp $O(\log \max(|d_1|, |d_2|))$.

Tuy nhiên trong bài toán này, ta có thể chứng minh bổ đề sau, nhờ đó lấy giao hai cấp số cộng trong $O(1)$.

Bổ đề 5. Nếu bốn xâu thỏa $|x_1| = |y_1| \geq |x_2| = |y_2|$, x_1 khớp trong y_2y_1 ít nhất 3 lần, và y_1 khớp trong x_1x_2 ít nhất 3 lần, thì

$$\text{minper}(x_1) = \text{minper}(y_1).$$

Chứng minh. Dùng phản chứng, giả sử $\text{minper}(x_1) \neq \text{minper}(y_1)$. Nếu $\text{minper}(x_1) > \text{minper}(y_1)$, xét vị trí khớp cuối cùng của x_1 trong y_2y_1 , và gọi phần giao của lần khớp đó với y_1 là z . Theo Bổ đề 2, khoảng cách giữa hai lần khớp kề nhau của x_1 trong y_2y_1 đúng bằng $\text{minper}(x_1)$. Lại có $|y_2| \leq |x_1|$, nên

$$|y_2| + |z| \geq |x_1| + 2 \text{minper}(x_1) \geq |y_2| + 2 \text{minper}(x_1),$$

tức $|z| \geq 2 \text{minper}(x_1)$.

Lúc này có thể thấy z có hai chu kỳ $\text{minper}(x_1)$ và $\text{minper}(y_1)$. Vì

$$|z| \geq 2 \text{minper}(x_1) \geq \text{minper}(x_1) + \text{minper}(y_1),$$

theo WPL, z có chu kỳ

$$d = \gcd(\text{minper}(x_1), \text{minper}(y_1)) \mid \text{minper}(x_1).$$

Do đó d cũng là chu kỳ của x_1 , mâu thuẫn với $\text{minper}(x_1)$.

Khi $\text{minper}(x_1) < \text{minper}(y_1)$, quá trình dẫn đến mâu thuẫn gần như tương tự, chỉ khác ở ý nghĩa “lật ngược”, nên không lặp lại. Mâu thuẫn trên chứng tỏ $\text{minper}(x_1) = \text{minper}(y_1)$. $\square$

Nếu $k_1 < 3$ hoặc $k_2 < 3$, có thể trực tiếp liệt kê mọi phần tử trong cấp số cộng tương ứng rồi kiểm tra nó có thuộc cấp số cộng còn lại hay không. Ngược lại, theo Bổ đề 5, $d_1 = d_2$, và có thể lấy giao hai cấp số cộng trực tiếp trong $O(1)$.

Kết hợp hai phần, ta thu được cách lấy giao cấp số cộng trong $O(1)$ cho bài toán này.

4.3. Tối ưu truy vấn succ và pred

Trước hết phân tích độ phức tạp hiện tại. Thời gian tiền xử lý đã là $O(n \log n)$. Một truy vấn chu kỳ sâu con được chia thành $O(\log |S|)$ truy vấn LargePS, mỗi truy vấn LargePS cần $O(1)$ truy vấn succ, pred, và một lần lấy giao cấp số cộng. Phần sau đã được tối ưu thành $O(1)$ ở mục trước, nhưng phần đầu vẫn cần tìm kiếm nhị phân, mỗi lần $O(\log |S|)$, khiến tổng độ phức tạp mỗi truy vấn là $O(\log^2 n)$. Độ phức tạp không gian lúc này là $O(n \log n)$.

Nếu xét việc thực sự cài đặt chức năng succ và pred, có vẻ chỉ có một phương án khả thi là tìm kiếm nhị phân trên danh sách chỉ số. Nhưng trong bài toán truy vấn chu kỳ sâu con, ta dùng chúng để tìm cấp số cộng của các vị trí xuất hiện, dường như hơi quá mức cần thiết. Vì vậy ta xét từ chính bài toán để tìm vật thay thế cho hai thao tác này.

Trong bài toán này, succ và pred không cần được cài đặt đầy đủ, vì các truy vấn của ta có dạng “mọi vị trí xuất hiện của $\text{pref}(S, 2^{i-1})$ trong $\text{suf}(S, 2^i)$ ”. Nói cách khác, ta chỉ quan tâm tới lớp bài toán “mọi vị trí xuất hiện của một xâu độ dài 2^{i-1} trong một khoảng có độ dài không quá 2^i ”.

Xét mở rộng nội dung của từ điển xâu con cơ bản thêm một lần nữa. Trong lần mở rộng thứ nhất, trong Name_t , ta lưu mọi vị trí xuất hiện của các xâu bằng nhau độ dài 2^t vào cùng một danh sách có thứ tự, rồi tìm kiếm nhị phân trên danh sách ấy để thực hiện truy vấn succ. Trên cơ sở đó, ta chia xâu mẹ S thành các đoạn dài 2^t ký tự, tức các đoạn

$$[1, 2^t], [2^t + 1, 2^{t+1}], [2^{t+1} + 1, 3 \cdot 2^t], \dots, [k \cdot 2^t + 1, (k + 1) \cdot 2^t].$$

Với mỗi đoạn, lưu lại mọi chỉ số trong danh sách có thứ tự hiện tại nằm trong đoạn ấy.

Cụ thể, với mỗi xâu T độ dài 2^t khác nhau về bản chất, ta lưu $\text{seg}(T, x)$, $1 \leq x \leq \lfloor |S|/2^t \rfloor$, biểu thị mọi vị trí xuất hiện của T trong xâu mẹ S nằm trong khoảng

$$[2^t(x - 1) + 1, 2^t x].$$

Theo Bổ đề 2, mỗi $\text{seg}(T, x)$ có thể được biểu diễn bằng một cấp số cộng. Nếu cấp số cộng được biểu diễn là rỗng, thì $\text{seg}(T, x) = \emptyset$.

Nếu lưu trực tiếp như vậy, không gian sẽ lên tới mức $O(|S|^2 \log |S|)$. Nhưng để ý rằng không cần cấp phát không gian cho mọi $\text{seg}(T, x)$. Với một t cố định, số cặp thỏa $\text{seg}(T, x) \neq \emptyset$ không vượt quá tổng số lần xuất hiện của mọi xâu T , mà tổng này bằng $|S| - 2^t + 1$. Vì vậy trong toàn bộ từ điển xâu con cơ bản, số lượng $\text{seg}(T, x)$ khác rỗng là $O(|S| \log |S|)$.

Để độ phức tạp không gian cùng bậc với số lượng seg khác rỗng, với mỗi T ta cần duy trì tập mọi giá trị x sao cho $\text{seg}(T, x)$ khác rỗng, đồng thời hỗ trợ truy vấn nhanh xem một số có thuộc tập này hay không. Không khó thấy có thể dùng bảng băm để hoàn thành thao tác truy vấn.

Cuối cùng, nếu cần truy vấn mọi vị trí xuất hiện của

$$S[i, \dots, i + 2^t - 1]$$

trong khoảng $[j, j + 2^t - 1]$, chỉ cần tìm mọi khoảng trong $\text{seg}(S[i, \dots, i + 2^t - 1])$ có giao với $[j, j + 2^t - 1]$. Số khoảng như vậy hiển nhiên nhiều nhất là hai. Ta lấy ra các giá trị $\text{seg}(T, x)$ tương ứng rồi cắt đoạn thích hợp. Do bảo đảm đáp án cuối cùng cũng là một cấp số cộng, chỉ cần hợp nhất trực tiếp các $\text{seg}(T, x)$ sau khi cắt.

Như vậy, ta đã thay thế thành công các truy vấn succ và pred dùng trong thuật toán bằng những truy vấn kỳ vọng $O(1)$ trên bảng băm của seg.

Tổng hợp phân tích phía trên, bài toán truy vấn chu kỳ xâu con đạt tiền xử lý kỳ vọng $O(n \log n)$ thời gian và không gian, và mỗi truy vấn có độ phức tạp kỳ vọng $O(\log n)$.

5. Mở rộng và ứng dụng của từ điển xâu con cơ bản

Mục này chủ yếu giới thiệu một số mở rộng và ứng dụng của từ điển xâu con cơ bản. Vì từ điển xâu con cơ bản có nhiều liên hệ với mảng hậu tố, cũng có thể xem nó là hướng ứng dụng mảng hậu tố vào các xâu con cơ bản, tức các xâu con có độ dài là lũy thừa của 2.

5.1. So sánh xâu con

Một ứng dụng cơ sở của từ điển xâu con cơ bản là so sánh thứ tự hai xâu con trong $O(1)$. Dù bài toán này cũng có thể được giải với cùng độ phức tạp bằng mảng hậu tố, mảng độ cao và truy vấn giá trị nhỏ nhất trên đoạn (RMQ), thậm chí đạt tiền xử lý tốt hơn $O(|S|)$, cách dùng từ điển xâu con cơ bản đơn giản, theo tôi có tính gọi mở nhất định, và khả năng mở rộng mạnh hơn cách làm RMQ.

5.1.1. Mô tả bài toán

Cho xâu S . Mỗi truy vấn cho i, j, i', j' , yêu cầu so sánh thứ tự của $S[i, \dots, j]$ và $S[i', \dots, j']$.

5.1.2. Cách làm

Nếu $j - i \neq j' - i'$, hai xâu con không thể bằng nhau. Có thể xóa bớt một số ký tự cuối của xâu con dài hơn để nó có độ dài bằng xâu con ngắn hơn, rồi thực hiện một truy vấn mới. Nếu câu trả lời của truy vấn mới là không bằng nhau, câu trả lời của truy vấn ban đầu giống câu trả lời mới; nếu không, xâu con dài hơn lớn hơn.

Bây giờ giả sử $j - i = j' - i'$. Tìm số nguyên dương m thỏa

$$2^m < j - i + 1 \leq 2^{m+1}.$$

Khi đó việc so sánh hai xâu con có thể hoàn thành bằng cách so sánh

$$S[i, \dots, i + 2^m - 1] \quad \text{với} \quad S[i', \dots, i' + 2^m - 1],$$

và so sánh

$$S[j - 2^m + 1, \dots, j] \quad \text{với} \quad S[j' - 2^m + 1, \dots, j'].$$

Hai phép so sánh đầu có thể được thực hiện bằng cách so sánh $\text{Name}_m[i]$ với $\text{Name}_m[i']$, và $\text{Name}_m[j - 2^m + 1]$ với $\text{Name}_m[j' - 2^m + 1]$.

Tóm lại, ta thu được một cách làm tiền xử lý $O(n \log n)$, truy vấn $O(1)$.

5.1.3. Mở rộng đơn giản

Đổi bài toán ban đầu thành: cho một cây trie kích thước n . Định nghĩa $up(i, l)$ là xâu nhận được bằng cách từ đỉnh i đi lên l cạnh và nối lần lượt các ký tự trên các cạnh đó. Mỗi truy vấn cho i, l, i', l' , yêu cầu so sánh $up(i, l)$ và $up(i', l')$.

Lúc này thuật toán nhân đôi tìm mảng hậu tố vẫn dùng được, nhưng thuật toán tuyến tính tìm mảng độ cao lại mất hiệu lực. Khi gặp tình huống này, thường chỉ có thể dùng cách nhị phân cộng băm xâu với hằng số lớn, hoặc dùng cây hậu tố tổng quát hay tự động hậu tố khó hiểu hơn.

Tuy nhiên, cách dựa trên từ điển xâu con cơ bản vẫn còn hiệu lực, thậm chí có thể kết hợp với cách RMQ để thu được một phương pháp mới tìm mảng độ cao. Vì phần băm trong nhị phân cộng băm chỉ dùng để phán định hai xâu có bằng nhau hay không, có thể thay trực tiếp nó bằng phép so sánh xâu dựa trên từ điển xâu con cơ bản, qua đó có hằng số tốt hơn.

Dĩ nhiên, có thể thấy mọi thuật toán trên đều không tránh được nhu cầu nhanh chóng tìm tổ tiên cấp k , $anc(x, k)$, của một đỉnh x . Ghi lại

$$f(i, k) = anc(i, 2^k),$$

và dùng chuyển tiếp

$$f(i, k) = f(f(i, k-1), k-1)$$

để suy ra mọi giá trị f , khi đó truy vấn anc có thể được giải trong $O(\log n)$ thời gian. Nhưng như vậy độ phức tạp mỗi truy vấn tăng lên $O(\log n)$. Để đạt truy vấn $O(1)$, còn cần kết hợp phân rã chuỗi dài và kỹ thuật ladder; chi tiết có thể tham khảo các tài liệu liên quan.

5.2. Liên quan tới chu kỳ nguyên

Trong một số bài toán về chu kỳ nguyên, xâu lũy thừa (String Powers) là đối tượng nghiên cứu quan trọng.

Định nghĩa 6. Nếu một xâu T có chu kỳ $|T|/k$, với $k \in \mathbb{Z}$, $k > 1$, thì gọi T là một xâu lũy thừa bậc k . Khi đó T có thể biểu diễn dưới dạng

$$\text{pref}(T, |T|/k)^k.$$

Nếu còn thỏa $k \cdot \text{minper}(T) = |T|$, thì gọi T là một xâu lũy thừa bậc k nguyên thủy. Đặc biệt, khi $k = 2$, T là xâu bình phương (xâu bình phương nguyên thủy); khi $k = 3$, T là xâu lập phương (xâu lập phương nguyên thủy). Nếu không có k nào thỏa điều kiện, thì gọi T là xâu nguyên thủy.

5.2.1. Tính chất đơn giản của chu kỳ nguyên

Tính chất 1. Nếu T vừa là xâu lũy thừa bậc k_1 , vừa là xâu lũy thừa bậc k_2 , thì T là xâu lũy thừa bậc

$$\frac{k_1 k_2}{\gcd(k_1, k_2)}.$$

Tính chất 2. T là xâu nguyên thủy khi và chỉ khi T^k là xâu lũy thừa bậc k nguyên thủy. Đặc biệt, khi $k = 2$, T là xâu nguyên thủy khi và chỉ khi trong T^2 , T chỉ xuất hiện với vai trò tiền tố và hậu tố.

Hai tính chất này dễ chứng minh bằng WPL.

5.2.2. Tìm xâu con lũy thừa bậc k

Cho một xâu S , cần tìm một xâu con $S[l, \dots, r]$ của S sao cho nó là xâu lũy thừa bậc k . Ta trực tiếp đưa ra cách làm và giải thích tính đúng đắn của nó. Chú ý thuật toán có thể đưa ra nhiều xâu con lũy thừa bậc k giống nhau.

Algorithm 1 Detect k -th Powers

Require: S, k

Require: Function $\text{IsPower}_k(pos, root)$

Ensure: Report if S contains a k -th power

```

1  Name  $\leftarrow$  DBF( $S$ )
2  for  $t = 0$  to  $\lfloor \log_2 |S| \rfloor$  do
3    Prev  $\leftarrow (0, 0, \dots, 0)$ 
4    for  $i = 1$  to  $|S| - 2^t + 1$  do
5      name  $\leftarrow$  Name $_t(i)$ 
6      pos  $\leftarrow$  Prev[name]
7      root  $\leftarrow i - pos$ 
8      if  $\text{IsPower}_k(pos, root)$  then
9        Report that  $S$  contains a  $k$ -th power  $S[pos, \dots, pos + k \cdot root - 1]$ 
10     end if
11     Prev[name]  $\leftarrow i$ 
12   end for
13 end for
14 if No  $k$ -th power detected then
15   Report that  $S$  does not contain a  $k$ -th power
16 end if
```

Hàm $\text{IsPower}_k(pos, root)$ trong thuật toán trả về việc $S[pos, \dots, pos + k \cdot root - 1]$ có phải là một xâu lũy thừa bậc k hay không. Có thể giải quyết bằng cách phán định

$$S[pos, \dots, pos + (k - 1)root - 1]$$

và

$$S[pos + root, \dots, pos + k \cdot root - 1]$$

có bằng nhau hay không, từ đó áp dụng phương pháp phán định $O(1)$ phía trước.

Hiển nhiên nếu thuật toán trả lời rằng tồn tại xâu con lũy thừa bậc k , thì câu trả lời ấy chắc chắn đúng. Ta chỉ cần giải thích rằng nếu thuật toán trả lời không tồn tại, thì trong S chắc chắn không có xâu con lũy thừa bậc k .

Định lý 1. Khi $k = 2$, nếu S tồn tại xâu con bình phương, thì Algorithm 1 nhất định sẽ tìm được mọi xâu con bình phương có độ dài ngắn nhất.

Bổ đề 6. Nếu

$$w = S[i, \dots, j] = S[i', \dots, j']$$

và hai khoảng $[i, j], [i', j']$ có giao không rỗng, thì trong S tồn tại một xâu bình phương có độ dài không vượt quá $2|w| - 1$.

Chứng minh. Đặt $l = \min(i, i')$, $r = \max(j, j')$. Xét xâu $S[l, \dots, r]$. w là border của nó, nên $r - l + 1 - |w|$ là một chu kỳ của nó. Vì $r - l + 1 < 2|w|$, suy ra

$$S[l, \dots, l - 1 + 2(r - l + 1 - |w|)]$$

là một xâu bình phương. $\square$

Chứng minh Định lý 1. Gọi $S[j, \dots, j + 2r - 1]$ là một xâu con bình phương có độ dài ngắn nhất, và đặt $v = S[j, \dots, j + r - 1]$. Tìm s thỏa $2^s \leq |v| < 2^{s+1}$, rồi xét thời điểm Algorithm 1 chạy tới $t = s$, $i = j + r$. Vì

$$S[j, \dots, j + 2^s - 1] = S[i, \dots, i + 2^s - 1],$$

ta có $i > pos \geq j$, nên chỉ cần chứng minh $pos = j$.

Đặt $w = S[i, \dots, i + 2^s - 1]$. Giả sử $pos > j$. Khi đó

$$S[i, \dots, i + 2^s - 1] = S[pos, \dots, pos + 2^s - 1] = S[j, \dots, j + 2^s - 1] = w.$$

Do $j - i = |v| < 2^{s+1} = 2|w|$ theo độ dài tuyệt đối của khoảng cách, khoảng $[pos, pos + 2^s - 1]$ nhất định giao với một trong hai khoảng $[i, i + 2^s - 1]$ và $[j, j + 2^s - 1]$. Theo Bổ đề 6, ta thu được một xâu bình phương ngắn hơn $2|v|$, mâu thuẫn. $\square$

Định lý 2. Khi $k \geq 3$, Algorithm 1 sẽ tìm được mọi xâu con lũy thừa bậc k nguyên thủy trong S .

Chứng minh. Gọi $S[j, \dots, j + kr - 1]$ là một xâu con lũy thừa bậc k nguyên thủy. Đặt $v = S[j, \dots, j + r - 1]$. Tìm s thỏa $2^{s-1} < |v| \leq 2^s$, rồi xét thời điểm Algorithm 1 chạy tới $t = s$, $i = j + r$. Vì

$$S[j, \dots, j + 2^s - 1] = S[i, \dots, i + 2^s - 1],$$

ta có $i > pos \geq j$, nên chỉ cần chứng minh $pos = j$.

Xét $S[j, \dots, j + 2r - 1] = v^2$. Nếu $i > pos > j$, thì v còn xuất hiện một lần nữa tại vị trí pos . Theo Tính chất 1 và Tính chất 2, v không phải xâu nguyên thủy, mâu thuẫn với việc v^k là xâu lũy thừa bậc k nguyên thủy. $\square$

Kết hợp Định lý 1 và Định lý 2, ta đã chứng minh tính đúng đắn của Algorithm 1.

5.2.3. Runs và Cubic Runs

Một khái niệm quan trọng khác liên quan tới chu kỳ nguyên là khái niệm Run.

Định nghĩa 7. Một Run của xâu S là một bộ ba (i, j, l) thỏa

$$l \in \text{per}(S[i, \dots, j]), \quad l \leq \frac{j - i + 1}{2},$$

và $S[i - 1, \dots, j]$ cũng như $S[i, \dots, j + 1]$ đều không xác định hoặc không thỏa điều kiện trên. Nếu $l \leq (j - i + 1)/3$, thì bộ ba (i, j, l) được gọi là một Cubic Run.

Theo WPL, cũng không khó thu được kết luận sau: các khoảng $[i, j]$ tương ứng với mọi Run của xâu S không có quan hệ chứa nhau.

Có thể hiểu Run là một xâu con có chu kỳ, lặp ít nhất hai lần và không thể mở rộng sang trái hay sang phải. Định lý quan trọng dưới đây cho biết cận trên của số lượng Run. Chứng minh của định lý có thể tham khảo tài liệu liên quan.

Định lý 3 (The Runs Theorem). Một xâu S có nhiều nhất $|S|$ Run.

Độ phức tạp tối ưu để tìm mọi Run của xâu S là $O(|S|)$, nhưng cần dùng xây dựng tuyến tính mảng hậu tố, cùng RMQ tiền xử lý tuyến tính và truy vấn $O(1)$ dựa trên Method of Four Russians, nên khá rườm rà và không phù hợp để cài đặt trong thi thuật toán. Bài toán này có một cách làm $O(|S| \log |S|)$ dễ hiểu và dễ cài đặt hơn; độc giả quan tâm có thể đọc tài liệu tham khảo.

Trong một số bài toán, điều ta quan tâm không phải mọi Run, mà là Cubic Run có tính chất mạnh hơn. Vì Cubic Run là tập con của Run, số lượng Cubic Run cũng nhiều nhất là $|S|$. Không khó thấy bất kỳ xâu lập phương nguyên thủy nào cũng được chứa đúng trong một Cubic Run có chu kỳ bằng chu kỳ của nó, và một Cubic Run nhất định chứa ít nhất một xâu lập phương nguyên thủy. Điều này gợi ý rằng ta có thể giải những bài toán kiểu này bằng cách tìm mọi xâu lập phương nguyên thủy. Ta nêu không chứng minh định lý sau, nhờ đó có thể trực tiếp dùng Algorithm 1 để tìm mọi xâu lập phương nguyên thủy.

Định lý 4. Mọi xâu lũy thừa bậc k do Algorithm 1 tìm được đều là nguyên thủy.

Sau khi tìm được mọi xâu lập phương nguyên thủy, việc thu được mọi Cubic Run trở nên dễ dàng: mỗi lần lấy xâu lập phương nguyên thủy có vị trí xuất hiện sớm nhất, rồi mở rộng tối đa để thu được một Cubic Run.

6. Tổng kết

Trong bài viết này, chúng ta đã giải trọn vẹn bài toán kinh điển truy vấn chu kỳ xâu con. Tuy bài toán đã được giải, cách làm của nó, ba bổ đề được dùng và cấu trúc dữ liệu từ điển xâu con cơ bản vẫn đáng được nghiên cứu thêm.

Hy vọng bài viết này có thể đóng vai trò gợi mở, dẫn dắt độc giả bước đầu cảm nhận vẻ đẹp của chu kỳ trong xâu, đồng thời hiểu sâu hơn những tri thức và kết quả nghiên cứu liên quan không chỉ giới hạn trong bài viết này.

Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã chỉ dẫn.

Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã chỉ dẫn và hỗ trợ.

Xin cảm ơn các bạn Yang Junzhao và Wang Zeyuan đã đọc duyệt bản thảo.

Xin cảm ơn mọi thầy cô và bạn học từng giúp đỡ tôi.

Xin cảm ơn cha mẹ tôi đã luôn hết lòng ủng hộ và khích lệ.

Tài liệu tham khảo

1. Jin Ce. *Chuyên đề thuật toán xâu*, trại đông CCF NOI 2017.
2. Tomasz Kociumaka, Jakub Radoszewski, Wojciech Rytter, Tomasz Walen. Efficient Data Structure for the Factor Periodicity Problem.
3. Maxime Crochemore, Costas Iliopoulos, Marcin Kubica, Jakub Radoszewski, Wojciech Rytter, Krzysztof Stencel, Tomasz Walen. New Simple Efficient Algorithms Computing Powers and Runs in Strings.

4. Udi Manber, Gene Myers. Suffix Arrays: a new method for on-line string searches.
5. Hideo Bannai, Tomohiro I, Shunsuke Inenaga, Yuto Nakashima, Masayuki Takeda, Kazuya Tsuruta. The “Runs” Theorem.
6. Xu Mingkuan. *Bước đầu về thuật toán chia khối kích thước phi thông thường*. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2018.

Báo cáo ra đề Công viên

Wu Zuotong, Trường Trung học số 1 Phúc Châu, tỉnh Phúc Kiến

Tóm tắt

Công viên là một bài toán tôi ra trong kỳ kiểm tra chéo đội tuyển tập huấn năm 2019. Bài viết này đưa vào khái niệm “đồ thị nối tiếp-song song tổng quát”, đồng thời chứng minh rằng mọi đồ thị nối tiếp-song song tổng quát đều có thể được biến đổi thành đồ thị chỉ còn một đỉnh bằng ba phép toán: “xóa đỉnh bậc 1”, “co đỉnh bậc 2” và “chồng cạnh song song”. Bài viết giới thiệu ý tưởng giải và lựa chọn thuật toán của bài toán dưới các điều kiện đặc biệt khác nhau. Trong lời giải chuẩn, để hỗ trợ sửa tham số nhanh, ta xây cây biểu thức biểu diễn quá trình quy hoạch động trên đồ thị nối tiếp-song song tổng quát của bài toán, dùng dạng ma trận để biểu diễn chuyển tiếp, rồi dùng cây đoạn theo phân rã chuỗi để tăng tốc cập nhật. Tôi hy vọng bài toán *Công viên* có thể gợi thêm cho mọi người nhiều suy nghĩ về đồ thị nối tiếp-song song tổng quát và phép co đỉnh bậc 2.

1. Đề bài

1.1. Tóm tắt đề

Ta gọi một đồ thị vô hướng liên thông là đồ thị nối tiếp-song song tổng quát nếu với bất kỳ 4 đỉnh nào cũng không tồn tại 6 đường đi đôi một không chung cạnh nối từng cặp trong 4 đỉnh ấy.

PDF nguồn đưa ra một hình ví dụ là đồ thị nối tiếp-song song tổng quát và một hình ví dụ không phải đồ thị nối tiếp-song song tổng quát. Ở hình thứ hai, với các đỉnh 1, 4, 5, 6, tồn tại sáu đường đi đôi một không chung cạnh

$$1 \rightarrow 4, \quad 4 \rightarrow 5, \quad 5 \rightarrow 6, \quad 6 \rightarrow 1, \quad 4 \rightarrow 3 \rightarrow 6, \quad 1 \rightarrow 2 \rightarrow 5$$

nối từng cặp trong bốn đỉnh này.

Cho một đồ thị nối tiếp-song song tổng quát đơn giản (không khuyên, không cạnh song song) bậc n , ký hiệu $G = (V, E)$. Mỗi đỉnh có hai trọng số đỉnh b_i, w_i , mỗi cạnh có hai trọng số cạnh s_i, d_i . Cần tô mỗi đỉnh thành một trong hai màu đen, trắng, và tìm giá trị lớn nhất của một phương án tô. Gọi màu của đỉnh i sau khi tô là c_i : nếu $c_i = 0$ thì đỉnh i màu đen, nếu $c_i = 1$ thì đỉnh i màu trắng. Giá trị của một phương án tô được định nghĩa là

$$\sum_{v \in V} (b_v[c_v = 0] + w_v[c_v = 1]) + \sum_{e=(u,v) \in E} (s_e[c_u = c_v] + d_e[c_u \neq c_v]).$$

Ở đây $[x]$ có nghĩa là $[x] = 1$ nếu mệnh đề x đúng, và $[x] = 0$ nếu x sai.

Cần hỗ trợ sửa động hai trọng số của một đỉnh hoặc hai trọng số của một cạnh. Trước mọi lần sửa và sau mỗi lần sửa, hãy in đáp án. Có tất cả Q lần sửa.

1.2. Định dạng vào

Dòng đầu gồm hai số nguyên dương n, m , trong đó n là số đỉnh và m là số cạnh của G .

Tiếp theo là n dòng, mỗi dòng gồm hai số nguyên không âm; dòng thứ i ($1 \leq i \leq n$) biểu diễn b_i, w_i .

Tiếp theo là m dòng, mỗi dòng gồm bốn số nguyên dương x_i, y_i, s_i, d_i ; dòng thứ i ($1 \leq i \leq m$) biểu diễn hai đầu mút của cạnh thứ i là x_i, y_i , và hai trọng số cạnh của nó là s_i, d_i .

Dòng thứ $n + m + 2$ gồm một số nguyên không âm Q .

Tiếp theo là Q dòng, mỗi dòng gồm ba số nguyên dương x, a, b biểu diễn một lần sửa. Nếu $x \leq n$, sửa b_x thành a và w_x thành b ; ngược lại, sửa s_{x-n} thành a và d_{x-n} thành b .

1.3. Định dạng ra

In $Q + 1$ dòng. Dòng thứ nhất là đáp án trước mọi lần sửa. Với $2 \leq i \leq Q + 1$, dòng thứ i là đáp án sau lần sửa thứ $i - 1$.

1.4. Ví dụ vào

```
2 1
2 3
4 7
1 2 5 7
1
1 2 6
```

1.5. Ví dụ ra

```
16
18
```

1.6. Giải thích ví dụ

Trước khi sửa, $b_1 = 2, w_1 = 3, b_2 = 4, w_2 = 7, s_1 = 5, d_1 = 7$. Phương án tối ưu là $c_1 = 0, c_2 = 1$, có giá trị $b_1 + w_2 + d_1 = 16$.

Sau khi sửa, $b_1 = 2, w_1 = 6, b_2 = 4, w_2 = 7, s_1 = 5, d_1 = 7$. Phương án tối ưu là $c_1 = 1, c_2 = 1$, có giá trị $w_1 + w_2 + s_1 = 18$.

1.7. Quy mô dữ liệu và ràng buộc

Bảo đảm $2 \leq n \leq 10^5$, $Q \leq 10^5$. Bảo đảm $x_i, y_i \leq n$, $x \leq n + m$, $b_i, w_i, s_i, d_i, a, b \leq 10^6$.

Tập	Điểm	Dạng đồ thị	n	Q	Ràng buộc khác
1	2	cây		$= 0$	
2	3	xương rồng	≤ 17	≤ 17	
3	7	xương rồng		$= 0$	
4	5		≤ 1000	$= 0$	
5	13		≤ 1000	≤ 1000	
6	19	xương rồng			$b_i = w_i = 0, x > n$
7	17				$b_i = w_i = 0, x > n$
8	23	cây			
9	11				

Cây là đồ thị liên thông không chu trình. Xương rồng là đồ thị liên thông mà mỗi cạnh thuộc không quá một chu trình đơn. Ô trống nghĩa là không có ràng buộc bổ sung.

1.8. Giới hạn thời gian và bộ nhớ

Giới hạn thời gian: 6 giây. Giới hạn bộ nhớ: 1 GB.

2. Phân tích sơ bộ

2.1. Thuật toán một

Bài toán yêu cầu tìm phương án tô có giá trị lớn nhất. Rõ ràng với một phương án tô cố định, ta có thể tính giá trị của nó trong $O(m)$ thời gian bằng cách duyệt cạnh và cộng đóng góp.

Với tập con $2, n, Q$ đều không quá 17. Với một truy vấn, số phương án tô rất ít, không quá $2^{17} = 131072$, nên có thể xét mọi phương án tô, tính giá trị từng phương án rồi lấy lớn nhất.

Độ phức tạp thời gian là $O(2^n Qm)$. Đề không trực tiếp cho cận của m , nhưng có thể chứng minh $m = O(n)$; chứng minh xem ở mục 5.3.1.

Điểm kỳ vọng: 3 điểm.

Thuật toán này kém hiệu quả vì không dùng tính chất G là đồ thị nối tiếp-song song tổng quát, và sau mỗi lần sửa đều tính lại đáp án từ đầu, không tận dụng lượng lớn dữ liệu giống nhau trước và sau sửa.

Tiếp theo ta bắt đầu từ vài trường hợp đặc biệt để phân tích bài toán.

3. Bài toán trên cây

3.1. Thuật toán hai: cây, không có sửa

Với các bài toán tối ưu trên cây kiểu này, cách thường dùng là quy hoạch động trên cây.

Biến cây vô căn thành cây có gốc tại 1. Ký hiệu $f(i, j)$ ($j \in \{0, 1\}$) là đáp án trong cây con của đỉnh i khi $c_i = j$. Gọi $C(u)$ là tập con của u . Ta có

$$\begin{aligned} f(u, 0) &= b_u + \sum_{v \in C(u)} \max\{f(v, 0) + s_{(u,v)}, f(v, 1) + d_{(u,v)}\}, \\ f(u, 1) &= w_u + \sum_{v \in C(u)} \max\{f(v, 0) + d_{(u,v)}, f(v, 1) + s_{(u,v)}\}. \end{aligned}$$

Đáp án là $\max\{f(1, 0), f(1, 1)\}$. Độ phức tạp thời gian $O(n)$. Có thể qua tập con 1, kỳ vọng 2 điểm.

3.2. Thuật toán ba: cây, có sửa

Xét việc tận dụng thuật toán quy hoạch động của thuật toán hai. Từ công thức chuyển, chỉ các giá trị f ²⁷ trên đường đi từ đỉnh/cạnh bị sửa tới gốc mới có thể thay đổi. Với loại bài toán này, có thể dùng phân rã theo chuỗi.

Trên cây, ta gọi đỉnh không phải gốc có bậc 1 là lá; các đỉnh còn lại là đỉnh không lá. Có thể tính trực tiếp giá trị f của đỉnh bị sửa hoặc của cha của cạnh bị sửa. Phần còn lại có thể được tách thành $O(\log n)$ chuỗi nặng bằng phân rã cây theo chuỗi nặng-nhẹ²⁸. Khi đó chỉ cần xét cách duy trì nhanh giá trị f trên chuỗi nặng.

²⁷Ta gọi chung $f(u, 0)$ và $f(u, 1)$ là giá trị f của đỉnh u .

²⁸Trong bài viết này, mọi chỗ nói “phân rã cây theo chuỗi” đều là phân rã nặng-nhẹ. Cụ thể, con nặng của đỉnh u là đỉnh $v \in C(u)$ có $\text{size}(v)$ lớn nhất, nếu có nhiều thì lấy chỉ số nhỏ nhất; $\text{size}(v)$ là số đỉnh trong cây con gốc v . Cạnh nối một đỉnh với con nặng của nó là cạnh nặng; các cạnh còn lại là cạnh nhẹ. Một chuỗi nặng là một thành phần liên thông của đồ thị sinh bởi các cạnh nặng, hoặc một lá u sao cho cạnh nối u với cha là cạnh nhẹ. Con nhẹ của u là con không phải con nặng.

Gọi son_u là con nặng của u . Với đỉnh không lá u , viết lại công thức chuyển:

$$\begin{aligned} f(u, 0) &= b_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + s_{(u,v)}, f(v, 1) + d_{(u,v)}\} \\ &\quad + \max\{f(\text{son}_u, 0) + s_{(u, \text{son}_u)}, f(\text{son}_u, 1) + d_{(u, \text{son}_u)}\}, \\ f(u, 1) &= w_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + d_{(u,v)}, f(v, 1) + s_{(u,v)}\} \\ &\quad + \max\{f(\text{son}_u, 0) + d_{(u, \text{son}_u)}, f(\text{son}_u, 1) + s_{(u, \text{son}_u)}\}. \end{aligned}$$

Khi dùng giá trị f mới của son_u để cập nhật giá trị f của u , chỉ hạng cuối chứa $f(\text{son}_u, 0)$ và $f(\text{son}_u, 1)$ có thể thay đổi; phần còn lại có thể xem là hằng số. Vì vậy có thể viết

$$\begin{aligned} f(u, 0) &= \max\{f(\text{son}_u, 0) + A_u, f(\text{son}_u, 1) + B_u\}, \\ f(u, 1) &= \max\{f(\text{son}_u, 0) + C_u, f(\text{son}_u, 1) + D_u\}, \end{aligned}$$

trong đó A_u, B_u, C_u, D_u được xác định bởi đóng góp của các con nhẹ và cạnh nặng tương ứng:

$$\begin{aligned} A_u &= b_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + s_{(u,v)}, f(v, 1) + d_{(u,v)}\} + s_{(u, \text{son}_u)}, \\ B_u &= b_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + s_{(u,v)}, f(v, 1) + d_{(u,v)}\} + d_{(u, \text{son}_u)}, \\ C_u &= w_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + d_{(u,v)}, f(v, 1) + s_{(u,v)}\} + d_{(u, \text{son}_u)}, \\ D_u &= w_u + \sum_{\substack{v \in C(u) \\ v \neq \text{son}_u}} \max\{f(v, 0) + d_{(u,v)}, f(v, 1) + s_{(u,v)}\} + s_{(u, \text{son}_u)}. \end{aligned}$$

Chuyển trên một chuỗi nặng vì thế có thể biểu diễn bằng dạng giống ma trận.

3.2.1. Biểu diễn ma trận

Đặt

$$f_u = \begin{bmatrix} f(u, 0) \\ f(u, 1) \end{bmatrix} = \begin{bmatrix} \max\{f(\text{son}_u, 0) + A_u, f(\text{son}_u, 1) + B_u\} \\ \max\{f(\text{son}_u, 0) + C_u, f(\text{son}_u, 1) + D_u\} \end{bmatrix}.$$

Dạng này rất giống kết quả của phép nhân ma trận, chỉ khác là phép cộng của phép nhân ma trận thường được thay bằng max, còn phép nhân được thay bằng phép cộng. Vì phép cộng và max đều giao hoán, kết hợp, đồng thời phép cộng phân phối với max, phép nhân ma trận mới định nghĩa như vậy vẫn kết hợp. Mọi phép nhân ma trận trong bài này đều hiểu theo định nghĩa đó.

Đặt

$$g_u = \begin{bmatrix} A_u & B_u \\ C_u & D_u \end{bmatrix}.$$

Khi đó

$$g_u f_{\text{son}_u} = \begin{bmatrix} \max\{f(\text{son}_u, 0) + A_u, f(\text{son}_u, 1) + B_u\} \\ \max\{f(\text{son}_u, 0) + C_u, f(\text{son}_u, 1) + D_u\} \end{bmatrix} = f_u.$$

3.2.2. Phân rã

Ở trạng thái ban đầu, mọi A_u, B_u, C_u, D_u có thể được tiền xử lý bằng thuật toán hai. Sau đó, các giá trị này của u chỉ cần cập nhật khi sửa trọng số của cạnh nối u với một con, hoặc sửa trọng số đỉnh trong cây con của u hay một con nhẹ của u . Vì trên một đường đơn từ một đỉnh tới gốc chỉ có $O(\log n)$ cạnh nhẹ, mỗi lần sửa chỉ làm thay đổi A_u, B_u, C_u, D_u của $O(\log n)$ đỉnh.

Để cập nhật A_u, B_u, C_u, D_u , hoặc để tính đáp án, ta cần biết giá trị f của đỉnh đầu²⁹ của một chuỗi nặng. Dùng cây đoạn để duy trì giá trị này. Nhờ phép nhân ma trận ở trên có tính kết hợp, ta duy trì tích các ma trận g trên một đoạn của chuỗi nặng bằng cây đoạn; khi sửa g_u , cập nhật điểm tương ứng là đủ. Chú ý g_u không định nghĩa với lá, nên cây đoạn chỉ cần lưu thông tin của đỉnh không lá.

Nếu chuỗi nặng gồm $u_1, u_2, \dots, u_l$, với u_1 là đỉnh đầu, u_i là cha của u_{i+1} , và u_l là lá, thì

$$f_{u_1} = g_{u_1} g_{u_2} \cdots g_{u_{l-1}} f_{u_l}.$$

Do đó chỉ cần truy vấn tích ma trận g của cả chuỗi rồi nhân phải với f_{u_l} .

Tóm lại, trong một lần sửa:

1. Sửa trọng số của đỉnh hoặc cạnh; gọi u là cha của cạnh bị sửa hoặc là đỉnh bị sửa.
2. Nếu u không phải lá, cập nhật g_u và cây đoạn của chuỗi tương ứng.
3. Đặt u thành đỉnh đầu của chuỗi nặng chứa u .
4. Tính lại f_u , gọi giá trị này là v .
5. Nếu u là gốc, trả v làm đáp án; ngược lại đặt u thành cha của u .
6. Dùng v đã tính để cập nhật g_u .
7. Quay lại bước 3.

Mỗi lần sửa cần $O(\log n)$ lần cập nhật điểm trên cây đoạn, nên độ phức tạp thời gian là $O(n + Q \log^2 n)$. Thuật toán qua được các tập con 1 và 8, kỳ vọng 25 điểm.

4. Bài toán trên xương rồng

4.1. Thuật toán bốn: xương rồng, không có sửa

Với bài toán trên xương rồng, một phương pháp kinh điển là quy hoạch động trên xương rồng. Độ phức tạp $O(n)$, có thể qua các tập con 1 và 3, được 9 điểm. Phần này không liên quan đến chủ đề chính của bài viết nên không nói kỹ; có thể tham khảo bài của Chen Junkun về *Đồ thị con kỳ diệu* và mở rộng.

Tập con 6 có ràng buộc “mọi trọng số đỉnh đều bằng 0”. Ràng buộc này giúp gì cho lời giải? Trước hết xét trường hợp trên cây.

4.2. Thuật toán năm: cây, mọi trọng số đỉnh bằng 0

Khi mọi trọng số đỉnh bằng 0, giá trị phương án tô chỉ liên quan đến việc hai đầu của mỗi cạnh có cùng màu hay khác màu. Ta gọi cạnh có hai đầu cùng màu là cạnh đồng màu, cạnh có hai đầu khác màu là cạnh dị màu.

Một ý tưởng là xét mỗi cạnh là đồng màu hay dị màu, rồi kiểm tra có tồn tại phương án tô thỏa điều kiện hay không. Trên cây, giả sử đã quyết định trạng thái đồng/dị màu của từng cạnh, ta có thể đặt $c_1 = 0$

²⁹Đỉnh đầu của chuỗi nặng là đỉnh gần gốc nhất trên chuỗi; đỉnh cuối là đỉnh xa gốc nhất trên chuỗi.

rồi DFS một lần. Với mỗi đỉnh không phải gốc, màu của nó được xác định bởi màu của cha và trạng thái đồng/dị màu của cạnh nối với cha. Điều này chứng minh rằng mọi cách chọn trạng thái cạnh đều có một phương án tô hợp lệ. Vì vậy trên cây với trọng số đỉnh bằng 0, có thể tham lam lấy đóng góp lớn hơn của mỗi cạnh, đáp án là

$$\sum_{e \in E} \max\{s_e, d_e\}.$$

4.3. Thuật toán sáu: xương rồng, mọi trọng số đỉnh bằng 0

Vẫn có thể dùng DFS để kiểm tra một phương án trạng thái cạnh có tương ứng với một cách tô hợp lệ hay không. Lấy một cây DFS bất kỳ, xác định màu từng đỉnh như ở 4.2, rồi kiểm tra mọi cạnh không thuộc cây.

Có thể thấy phương án tô hợp lệ tồn tại khi và chỉ khi không có chu trình đơn nào chứa số cạnh dị màu lẻ. Vì trên xương rồng mỗi cạnh thuộc nhiều nhất một chu trình đơn, ta có thể tính đáp án độc lập cho từng chu trình đơn và từng cầu rồi cộng lại.

Với cầu, cộng trực tiếp $\max\{s_i, d_i\}$ vào đáp án. Với chu trình, trước hết cộng $\max\{s_i, d_i\}$ của mọi cạnh. Nếu số cạnh dị màu là chẵn, ta đạt được giá trị lớn nhất lý thuyết và hợp lệ. Nếu số đó là lẻ, tìm cạnh có $|s_i - d_i|$ nhỏ nhất và đảo trạng thái đồng/dị màu của hai đầu cạnh ấy, thu được nghiệm tối ưu. Khi sửa, lấy đáp án của truy vấn trước cộng với độ biến thiên của chu trình hoặc cầu chứa cạnh bị sửa.

Độ phức tạp thời gian là $O((n + Q) \log n)$. Có thể qua tập con 6, kỳ vọng 19 điểm. Kết hợp các phần trên cây và xương rồng (thuật toán ba, bốn, sáu) được 51 điểm.

5. Dạng đồ thị tổng quát hơn

Tính chất của đồ thị nối tiếp-song song tổng quát là với bất kỳ 4 đỉnh nào cũng không tồn tại 6 đường đi đôi một không chung cạnh nối từng cặp trong 4 đỉnh ấy. Các thuật toán trước đòi hỏi dạng đồ thị đặc biệt hơn, nên không phù hợp cho tình huống tổng quát hơn này. Tuy nhiên ta vẫn có thể xuất phát từ cây và xương rồng để tìm cách làm phổ quát hơn. Để tiện phân tích, tạm thời không xét sửa đổi.

5.1. Một cách giải khác cho bài toán trên cây

Ví dụ 1. Bài toán tập độc lập trọng số lớn nhất trên cây. Cho một cây T có n đỉnh, đỉnh i có trọng số a_i . Hãy tìm một tập con S của tập đỉnh sao cho không có hai đỉnh trong S kề nhau, và tổng trọng số các đỉnh trong S là lớn nhất.

Ràng buộc: $1 \leq n \leq 10^5$, $1 \leq a_i \leq 10^9$. Giới hạn thời gian 1 giây, bộ nhớ 128 MB. Đây là bài toán kinh điển.

Bài này tất nhiên có quy hoạch động giống thuật toán hai, nhưng giờ xét một thuật toán khác. Khi mọi $a_i = 1$, có cách tham lam kinh điển: mỗi lần tìm một đỉnh u có bậc không quá 1, đưa nó vào S , rồi xóa khỏi T đỉnh kề v với nó (nếu tồn tại; nếu $u \in S$ thì theo đề $v \notin S$) và xóa chính u . Lặp cho tới khi T không còn đỉnh; khi đó S là một nghiệm tối ưu.

Chứng minh. Giả sử tồn tại một nghiệm tối ưu $S^* \neq S$. Khi mọi $a_i = 1$, tối đa hóa tổng trọng số tương đương tối đa hóa $|S|$.

Nếu u có bậc 0 và không thuộc S^* , rõ ràng có thể thêm u vào S^* để tăng kích thước, mâu thuẫn với tối ưu.

Nếu u có bậc 1 và không thuộc S^* , gọi v là đỉnh kề của nó. Nếu $v \notin S^*$, thêm u vào S^* cũng cho nghiệm tốt hơn, mâu thuẫn. Nếu $v \in S^*$, thay v bằng u trong S^* vẫn cho một nghiệm hợp lệ và tối ưu.

Vì vậy ta có thể giải tối ưu bài toán sau khi xóa u và v (nếu có), gọi nghiệm là S' , rồi lấy $S = S' \cup \{u\}$. Biên độ quy là đồ thị rỗng, khi đó $S = \emptyset$ hiển nhiên tối ưu.

Mỗi phép toán xóa ít nhất một đỉnh. Mọi đồ thị con không rỗng của cây đều có một đỉnh bậc không quá 1, vì mỗi thành phần liên thông của nó vẫn là cây, và theo nguyên lý chuồng bồ câu cây luôn có đỉnh bậc không quá 1. Do đó thuật toán kết thúc sau hữu hạn bước. $\square$

Nếu bỏ ràng buộc mọi $a_i = 1$, thuật toán tham lam này sai, vì trong chứng minh, thay v bằng u có thể làm nghiệm kém đi. Nhưng phần còn lại của chứng minh vẫn dùng được.

Do đó ta cải tiến như sau: sau khi đưa u vào S , cho phép về sau hủy thao tác ấy và đưa v vào S . Điều này tương đương với việc sau này có thể xét chọn v , nhưng phải trả thêm chi phí a_u để loại u đã chọn khỏi S . Ta làm bằng cách đổi a_v thành $a_v - a_u$. Nếu $a_v < a_u$, nghiệm tối ưu chắc chắn chứa u và không chứa v , vì nếu chứa v thì thay v bằng u sẽ tốt hơn. Thuật toán cải tiến xử lý được trường hợp tổng quát.

Áp dụng ý tưởng cải tiến này cho bài toán trên cây của ta cũng tương tự. Khi gặp một đỉnh bậc 1 là u , gọi đỉnh kề của nó là v . Nếu tô v màu đen, phần u và cạnh (u, v) đóng góp nhiều nhất

$$\max\{s_{(u,v)} + b_u, d_{(u,v)} + w_u\}.$$

Nếu tô v màu trắng, đóng góp nhiều nhất là

$$\max\{d_{(u,v)} + b_u, s_{(u,v)} + w_u\}.$$

Vậy có thể đổi

$$b_v \leftarrow b_v + \max\{s_{(u,v)} + b_u, d_{(u,v)} + w_u\},$$

$$w_v \leftarrow w_v + \max\{d_{(u,v)} + b_u, s_{(u,v)} + w_u\},$$

rồi xóa đỉnh u và cạnh (u, v) . Sau thao tác này đáp án không đổi, nhưng quy mô bài toán giảm một đỉnh. Ta gọi thao tác xóa một đỉnh bậc 1 và sửa trọng số đỉnh kề là phép xóa đỉnh bậc 1.

Một cây có hơn một đỉnh luôn có đỉnh bậc 1, và sau khi xóa một đỉnh bậc 1 vẫn là một cây. Vì vậy liên tục xóa đỉnh bậc 1 sẽ biến cây thành đồ thị chỉ còn một đỉnh, trong khi đáp án không đổi. Với đồ thị chỉ có một đỉnh u , đáp án là $\max\{b_u, w_u\}$. Do đó bài toán tính trên cây được giải trong $O(n)$ thời gian.

Phép xóa đỉnh bậc 1 không yêu cầu hình dạng tổng thể của đồ thị; hễ đồ thị có đỉnh bậc 1 thì có thể áp dụng.

5.2. Ứng dụng trên xương rồng

Trước hết nêu vài kiến thức về thành phần song liên thông đỉnh mà không chứng minh.

Định lý 5.1. Với mọi đồ thị vô hướng không rỗng G , tồn tại một thành phần song liên thông đỉnh B của G sao cho trong B có không quá một đỉnh là khớp của G . Nếu trong B không có khớp của G , thì $B = G$.

Định lý 5.2. Nếu một thành phần song liên thông đỉnh không phải K_2 ³⁰, thì trong thành phần ấy có ít nhất một chu trình đơn.

Bổ đề 5.1. Mỗi thành phần song liên thông đỉnh trên xương rồng hoặc là K_2 , hoặc là một chu trình đơn.

³⁰ K_n là đồ thị đầy đủ gồm n đỉnh.

Bổ đề 5.2. Với một đỉnh bất kỳ u trong một thành phần song liên thông đỉnh không phải K_2 , tồn tại một chu trình đơn C chứa u .

Trên cây ta có thể xóa đỉnh bậc 1 (lá) để đơn giản hóa bài toán. Xương rồng có thể làm tương tự hay không?

Trước hết, nếu tồn tại đỉnh bậc 1 thì vẫn dùng phép xóa đỉnh bậc 1. Ngược lại, theo định lý 5.1, tìm một thành phần song liên thông đỉnh B sao cho trong B có không quá một đỉnh là khớp của G ; kết hợp bổ đề 5.1, B nhất định là một chu trình đơn.

Khi đó vai trò của B trong đồ thị giống vai trò của đỉnh bậc không quá 1 trên cây. Liệu có thể biến B thành một đỉnh để giảm số thành phần song liên thông đỉnh của G hay không? Câu trả lời là có.

Vì B là một chu trình, ngoài khớp (nếu có), các đỉnh của B đều có bậc 2. Khi gặp đỉnh bậc 1, ta tính đóng góp của nó theo màu của đỉnh kề; khi gặp đỉnh bậc 2 cũng có thể làm tương tự.

Gọi x là đỉnh bậc 2, hai đỉnh kề là u, v . Khi c_u, c_v lần lượt là 0, 0, đóng góp lớn nhất của x và hai cạnh kề với nó là

$$w_{00} = \max\{s_{(u,x)} + b_x + s_{(x,v)}, d_{(u,x)} + w_x + d_{(x,v)}\}.$$

Tương tự,

$$w_{01} = \max\{s_{(u,x)} + b_x + d_{(x,v)}, d_{(u,x)} + w_x + s_{(x,v)}\},$$

$$w_{10} = \max\{d_{(u,x)} + b_x + s_{(x,v)}, s_{(u,x)} + w_x + d_{(x,v)}\},$$

$$w_{11} = \max\{d_{(u,x)} + b_x + d_{(x,v)}, s_{(u,x)} + w_x + s_{(x,v)}\}.$$

Vì đóng góp phụ thuộc đồng thời vào c_u, c_v , ta không cập nhật trọng số đỉnh u, v như khi xóa đỉnh bậc 1, mà tạo một cạnh mới nối u, v , trên cạnh ghi đóng góp cho từng giá trị của c_u, c_v .

Dùng $(u, v, (w_{00}, w_{01}, w_{10}, w_{11}))$ để biểu diễn cạnh có đóng góp $w_{c_u c_v}$. Trọng số của nó là

$$[w_{00} \ w_{01} \ w_{10} \ w_{11}]^T.$$

Cạnh đầu vào dạng $(u, v, (s, d))$ có thể đổi thành $(u, v, (s, d, d, s))$ để thống nhất định dạng.

Ta gọi phép biến một đỉnh bậc 2 và hai cạnh kề thành một cạnh là phép co đỉnh bậc 2. Chú ý phép co đỉnh bậc 2 không làm thay đổi bậc của u, v .

Vì thế với một chu trình n đỉnh ($n \geq 3$), một lần co đỉnh bậc 2 biến nó thành chu trình $n - 1$ đỉnh. Riêng với tam giác, sau một lần co sẽ xuất hiện hai cạnh song song.

Do đóng góp có thể cộng dồn, nếu gặp hai cạnh song song

$$(u, v, (w_{00}, w_{01}, w_{10}, w_{11})) \quad \text{và} \quad (u, v, (w'_{00}, w'_{01}, w'_{10}, w'_{11})),$$

có thể hợp nhất thành

$$(u, v, (w_{00} + w'_{00}, w_{01} + w'_{01}, w_{10} + w'_{10}, w_{11} + w'_{11})).$$

Ta gọi phép hợp nhất hai cạnh song song là phép chồng cạnh song song. Nếu hai cạnh song song có hướng biểu diễn ngược nhau, ví dụ

$$(u, v, (w_{00}, w_{01}, w_{10}, w_{11})) \quad \text{và} \quad (v, u, (w'_{00}, w'_{01}, w'_{10}, w'_{11})),$$

chỉ cần đổi hướng một cạnh rồi chồng; cạnh thu được là

$$(u, v, (w_{00} + w'_{00}, w_{01} + w'_{01}, w_{10} + w'_{10}, w_{11} + w'_{11})).$$

Giả sử B có k đỉnh. Vì trong B chỉ có một khớp của G , các đỉnh còn lại đều có bậc 2, và phép co đỉnh bậc 2 cùng phép chồng cạnh song song không làm đỉnh bậc 2 tăng bậc quá 2, sau $k - 2$ lần co và một lần chồng, B biến thành K_2 . Tiếp tục xóa một đỉnh bậc 1, số thành phần song liên thông đỉnh của G giảm một. Do đó sau $O(n)$ phép toán, có thể biến G thành một đỉnh và giải bài toán tính trên xương rồng trong $O(n)$.

5.3. Đồ thị nối tiếp-song song tổng quát

Sau đây chúng minh không chỉ xương rỗng, mà mọi đồ thị nối tiếp-song song tổng quát đều có thể biến thành đồ thị chỉ còn một đỉnh sau một số phép co đỉnh bậc 2, chồng cạnh song song và xóa đỉnh bậc 1.

5.3.1. Chứng minh

Ta chứng minh mệnh đề phản đảo: mọi đồ thị vô hướng liên thông không thể dùng các phép trên để biến thành đồ thị chỉ còn một đỉnh đều không phải đồ thị nối tiếp-song song tổng quát. Trước hết chứng minh hai kết luận.

Định lý 5.3. Nếu một đồ thị vô hướng liên thông G có ba đỉnh bậc 1 khác nhau x, y, z , thì tồn tại một đỉnh $u \notin \{x, y, z\}$ sao cho có 3 đường đi đơn đôi một không chung cạnh, cùng có một đầu là u , đầu còn lại lần lượt là x, y, z .

Chứng minh. Lấy một cây khung bất kỳ T của G . Vì x, y, z là đỉnh bậc 1 trong G , chúng cũng là đỉnh bậc 1 trong T . Do số đỉnh bậc lẻ trong mọi đồ thị là chẵn, G có một đỉnh khác x, y, z . Chọn một đỉnh như vậy làm gốc, biến T thành cây có gốc. Đặt $u = \text{LCA}(x, y)$. Vì x, y, z đều là lá và không phải gốc, nên $u \neq x, u \neq y, u \neq z$.

Nếu z không nằm trong cây con của u , hoặc cả $\text{LCA}(x, z)$ và $\text{LCA}(y, z)$ đều bằng u , thì ba đường trong T từ u tới x, y, z đôi một không chung cạnh.

Ngược lại, đúng một trong $\text{LCA}(x, z)$ và $\text{LCA}(y, z)$ khác u . Không mất tính tổng quát, giả sử $w = \text{LCA}(x, z) \neq u$. Khi đó y không nằm trong cây con của w , và vì $\text{LCA}(x, z) = w$, ba đường trong T từ w tới x, y, z đôi một không chung cạnh. $\square$

Định lý 5.4. Với một đồ thị vô hướng G , nếu sau một số lần xóa đỉnh bậc 1, co đỉnh bậc 2 và chồng cạnh song song ta thu được đồ thị không phải nối tiếp-song song tổng quát, thì G cũng không phải nối tiếp-song song tổng quát.

Chứng minh. Xét việc hoàn nguyên các phép toán. Giả sử trong đồ thị sau thao tác có bốn đỉnh a, b, c, d làm vi phạm tính chất nối tiếp-song song tổng quát, với sáu đường đi đôi một không chung cạnh

$$p(a, b), p(a, c), p(a, d), p(b, c), p(b, d), p(c, d).$$

Nghịch đảo của xóa đỉnh bậc 1 là chọn một đỉnh $u \in V$, thêm đỉnh mới v và cạnh (u, v) . Nghịch đảo của chồng cạnh song song là chọn một cạnh $(u, v) \in E$ rồi thêm một cạnh song song mới (u, v) . Hai nghịch đảo này không xóa đỉnh hay cạnh, nên nếu đồ thị sau phép xóa đỉnh bậc 1 hoặc chồng cạnh song song không phải nối tiếp-song song tổng quát, đồ thị trước thao tác cũng không phải.

Nghịch đảo của co đỉnh bậc 2 là chọn cạnh $(u, v) \in E$, xóa cạnh ấy, thêm đỉnh mới w và hai cạnh $(u, w), (w, v)$. Nếu khi hoàn nguyên không xóa cạnh nào trên sáu đường đi nêu trên, chính sáu đường cũ vẫn chứng minh đồ thị trước thao tác không phải nối tiếp-song song tổng quát. Nếu có, không mất tính tổng quát giả sử cạnh bị xóa nằm trên $p(a, b)$. Thay cạnh đó trong $p(a, b)$ bằng hai cạnh và đỉnh mới của nghịch đảo, ta vẫn có sáu đường đôi một không chung cạnh. Vậy sau bao nhiêu thao tác nếu đồ thị thu được không phải nối tiếp-song song tổng quát thì đồ thị ban đầu cũng không phải. $\square$

Một đồ thị vô hướng đơn giản không còn thao tác nào để làm được khi và chỉ khi mỗi thành phần liên thông của nó hoặc là một đỉnh bậc 0, hoặc là một đồ thị có mọi đỉnh bậc ít nhất 3. Ta có định lý sau.

Định lý 5.5. Mọi đồ thị vô hướng đơn giản liên thông mà mọi đỉnh đều có bậc ít nhất 3 đều không phải đồ thị nối tiếp-song song tổng quát.

Chứng minh. Giả sử G là một đồ thị vô hướng đơn giản liên thông mà mọi đỉnh đều có bậc ít nhất 3. Theo định lý 5.1, trong G có một thành phần song liên thông đỉnh B sao cho trong B có không quá một đỉnh là khớp của G . Thành phần B chắc chắn không phải K_2 , nếu không G sẽ có một đỉnh bậc 1.

Nếu B có khớp của G , theo bổ đề 5.2, trong B tồn tại một chu trình đơn chứa khớp duy nhất ấy; gọi nó là C . Nếu không, theo định lý 5.2, trong B tồn tại một chu trình đơn; lấy tùy ý một chu trình như vậy làm C .

Gọi các đỉnh trên C theo thứ tự là $v_0, v_1, \dots, v_{l-1}$, với v_0 kề v_{l-1} , và v_i kề v_{i+1} với $0 \leq i < l-1$. Đặt $V' = V \setminus V_C$, $E' = E \setminus E_C$. Trên $V' \cup E'$, định nghĩa quan hệ nhị nguyên $\sim$: với $u, e \in V' \cup E'$, $u \sim e$ khi và chỉ khi $u \in V'$, $e \in E'$, và u là một đầu mút của e . Lại định nghĩa $\leftrightarrow$ là quan hệ tương đương nhỏ nhất chứa $\sim$. Trực giác, nếu a, b là đỉnh hoặc cạnh của G không nằm trên C , thì $a \leftrightarrow b$ khi và chỉ khi từ a có thể đi qua một chuỗi đỉnh và cạnh không nằm trên C để tới b .

Ta gọi mỗi lớp tương đương của $\leftrightarrow$ là một mảnh. Với mảnh U , tập đỉnh của nó là $U \cap V$, tập cạnh là $U \cap E$, và tập điểm liên hệ với C là

$$A(U) = \{v \in V_C \mid \exists u \in V, e = (u, v) \in U \cap E\}.$$

Nói trực giác, sau khi xóa mọi đỉnh và cạnh trên C khỏi G (nhưng các cạnh không nằm trên C vẫn giữ lại dù một hoặc hai đầu mút bị xóa), các phần còn “liên thông” là mảnh; $A(U)$ là tập đầu mút bị thiếu của những cạnh trong mảnh có không đủ hai đầu mút.

Vì mọi đỉnh của G có bậc ít nhất 3, với mỗi đỉnh v_i trên chu trình, tồn tại ít nhất một mảnh U sao cho $v_i \in A(U)$. Nếu B có khớp của G , gọi khớp ấy là v_k ; nếu không có khớp thì đặt $k = -1$. Nếu tồn tại mảnh U với $|A(U)| = 1$, xóa phần tử duy nhất trong $A(U)$ sẽ làm G không liên thông, nên phần tử đó là khớp của G . Vì vậy mọi mảnh thỏa $|A(U)| = 1$ đều có $A(U) = \{v_k\}$. Khi $k = -1$, điều này có nghĩa là không tồn tại mảnh với $|A(U)| = 1$.

Ta chia ba trường hợp để chứng minh G không phải đồ thị nối tiếp-song song tổng quát.

Trường hợp 1. Tồn tại mảnh U sao cho $|A(U)| \geq 3$.

Giả sử U có một cạnh e mà cả hai đầu mút đều không thuộc U . Khi đó e không thể có quan hệ $\sim$ với bất kỳ phần tử $u \in V'$ nào, và do đó không có phần tử $a \in V' \cup E'$ nào thỏa $e \leftrightarrow a$. Suy ra $U = \{e\}$, và $A(U)$ chỉ gồm hai đầu mút của e , mâu thuẫn với $|A(U)| \geq 3$. Vậy trong U không có cạnh nào mà cả hai đầu mút đều không thuộc U .

Thêm mọi phần tử của $A(U)$ vào U , thu được đồ thị

$$U' = ((U \cap V) \cup A(U), U \cap E').$$

Trong U' , tập $U \cap V$ liên thông vì U là một lớp tương đương của $\leftrightarrow$, và các cạnh của U có một đầu ngoài U không làm mất tính liên thông ấy. Xóa một số cạnh kề các đỉnh trong $A(U)$ sao cho mỗi đỉnh của $A(U)$ có bậc 1; khi đó các đỉnh trong $A(U)$ vẫn nối với tập đỉnh của U bằng một cạnh trong U' , nên U' liên thông.

Theo định lý 5.3, với ba đỉnh $x, y, z \in A(U)$, trong U' tồn tại một đỉnh u và ba đường đôi một không chung cạnh nối u lần lượt với x, y, z . Mặt khác, x, y, z nằm trên cùng chu trình C , nên trên C có ba đường đôi một không chung cạnh nối x với y , y với z , và z với x . Vì U' và C không chung cạnh, ta tìm được bốn đỉnh x, y, z, u và sáu đường đôi một không chung cạnh nối từng cặp trong bốn đỉnh ấy. Vậy G không phải đồ thị nối tiếp-song song tổng quát.

Trường hợp 2. Với mọi mảnh U đều có $|A(U)| \leq 2$, và tồn tại U_1, U_2 sao cho

$$A(U_1) = \{v_x, v_y\}, \quad A(U_2) = \{v_z, v_w\},$$

trong đó $x < z < y < w$.

Khi đó trên chu trình C tìm được bốn đường đôi một không chung cạnh nối v_x với v_z , v_z với v_y , v_y với v_w , và v_w với v_x . Trong G có một đường nối v_x với v_y có tập cạnh nằm trong tập cạnh của U_1 , và một đường nối v_z với v_w có tập cạnh nằm trong tập cạnh của U_2 . Vì tập cạnh của U_1 , tập cạnh của U_2 và tập cạnh của C đôi một rời nhau, ta tìm được sáu đường đôi một không chung cạnh nối từng cặp trong bốn đỉnh, nên G không phải nối tiếp-song song tổng quát.

Trường hợp 3. Với mọi mảnh U đều có $|A(U)| \leq 2$, và tồn tại U_0, i sao cho $A(U_0) = \{v_i, v_{i+1}\}$.

Trước hết giải thích rằng ba trường hợp trên bao quát mọi khả năng. Nếu không thỏa trường hợp 1, ta biết: với mọi v_i trên chu trình có mảnh U sao cho $v_i \in A(U)$; mọi U thỏa $|A(U)| = 1$ đều có $A(U) = \{v_k\}$; chu trình đơn C có ít nhất 3 đỉnh; và không có U nào với $|A(U)| \geq 3$. Từ bốn sự kiện này, chắc chắn tìm được một mảnh U với $|A(U)| = 2$.

Giả sử $A(U) = \{v_x, v_y\}$ ($x < y$). Vì có thể định hướng chu trình theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ, không mất tính tổng quát giả sử $k \notin (x, y)$. Ta chứng minh nếu không thỏa trường hợp 1 và 2, thì nhất định tồn tại U_0, i với $A(U_0) = \{v_i, v_{i+1}\}$. Nếu $y - x = 1$, lấy $U_0 = U, i = x$. Ngược lại, tìm U_1, z sao cho $A(U_1) = \{v_{x+1}, v_z\}$. Theo các sự kiện trên và $k \notin (x, y)$, U_1, z như vậy luôn tồn tại. Nếu $z = x$, lấy $U_0 = U_1, i = x$. Nếu $z \notin [x, y]$, thì U và U_1 thỏa trường hợp 2, mâu thuẫn. Do đó nếu $z \neq x$ thì $z \in (x + 1, y]$. Đặt $x' = x + 1, y' = z$, khi đó

$$y' - x' = y' - x - 1 \leq y - x - 1$$

và vẫn có $y' > x', k \notin (x', y')$. Mỗi lần lặp từ x, y sang x', y' làm $y - x$ giảm, nên sau hữu hạn lần sẽ tìm được U_0, i cần thiết.

Gọi $G' = (V_{G'}, E_{G'})$ là đồ thị thu được từ U_0 sau khi thêm hai đỉnh v_i, v_{i+1} và cạnh (v_i, v_{i+1}) . Khi đó $\deg(v_i) \geq 2, \deg(v_{i+1}) \geq 2$, và với mọi $u \in U_0, \deg(u) \geq 3$. Vì vậy G' không có đỉnh bậc không quá 1, số đỉnh bậc 2 không quá 2, và nếu có 2 đỉnh bậc 2 thì hai đỉnh ấy kề nhau.

Có thể chứng minh $V_{G'} \subseteq V_B$. Nếu $k \neq -1$, giả sử tồn tại $u \in V_{G'}$ mà $u \notin V_B$. Mọi đường nối u với một điểm bất kỳ trong B đều phải qua v_k , mà $v_k \in V_C$, suy ra $A(U_0) = \{v_k\}$, mâu thuẫn với $|A(U_0)| = 2$. Nếu $k = -1$, theo định lý 5.1 có $B = G$, nên $V_{G'} \subseteq V = V_B$.

Tiếp theo chứng minh G' là đồ thị song liên thông đỉnh, tức là G' không có khớp. Trước hết v_i và v_{i+1} không phải khớp của G' , nếu không U_0 không liên thông. Các đỉnh còn lại của G' cũng không phải khớp của G' , nếu không chúng đồng thời là khớp của G . Khi $k = -1$, B không có khớp của G ; khi $k \neq -1$, khớp duy nhất của G trong B là v_k , nhưng do $V_{G'} \cap V_C = \{v_i, v_{i+1}\}$, chắc chắn $v_k \notin V_{G'} \setminus \{v_i, v_{i+1}\}$. Vậy G' không có khớp.

Bằng cách liệt kê các đồ thị đơn có không quá 3 đỉnh, có thể thấy không đồ thị nào đồng thời song liên thông đỉnh và không có đỉnh bậc không quá 1 trong khi số đỉnh bậc 2 không quá 2. Do đó G' có ít nhất 4 đỉnh.

Xét việc thực hiện co đỉnh bậc 2 và chồng cạnh song song trên G' . Phép co đỉnh bậc 2 không ảnh hưởng đến bậc của hai đỉnh u, v kề với đỉnh bị co; phép chồng cạnh song song sau đó, nếu cần, chỉ làm bậc của u, v giảm 1.

Nếu G' chỉ có một đỉnh bậc 2, sau thao tác, G' vẫn không có đỉnh bậc không quá 1, số đỉnh bậc 2 vẫn không quá 2, và nếu có 2 đỉnh bậc 2 thì chúng kề nhau. Nếu G' có hai đỉnh bậc 2 kề nhau x, y , gọi đỉnh kề còn lại của x là z , của y là w . Khi đó $z \neq w$, nếu không z có bậc 2 hoặc là khớp của G' . Không

mất tính tổng quát, co x . Vì giữa y và z không có cạnh, không cần chồng cạnh song song; bậc của y, z không đổi, nên sau thao tác G' chỉ còn một đỉnh bậc 2 và không có đỉnh bậc 1.

Ta còn cần chứng minh sau một lần co đỉnh bậc 2 và một lần chồng cạnh song song nếu xuất hiện cạnh song song, G' vẫn song liên thông đỉnh. Gọi đồ thị sau thao tác là G'' . Cạnh song song không thay đổi tính liên thông theo đỉnh, nên phép chồng cạnh song song không ảnh hưởng đến song liên thông đỉnh. Xét phép co đỉnh bậc 2: giả sử đỉnh bậc 2 bị co trong G' là x , hai đỉnh kề là y, z . Nếu trong cạnh (y, z) của G'' “chèn” lại đỉnh x , tức là thay (y, z) bằng $(y, x), (x, z)$, ta thu được G' . Vì thế chứng minh xóa bất kỳ đỉnh nào của G'' thì G'' vẫn liên thông tương đương chứng minh xóa bất kỳ đỉnh nào khác x trong G' thì G' vẫn liên thông. Điều này đúng vì G' song liên thông đỉnh.

Do đó tại mọi thời điểm trong quá trình thao tác, G' vẫn song liên thông đỉnh, không có đỉnh bậc không quá 1, và có không quá 2 đỉnh bậc 2. Mỗi lần co đỉnh bậc 2 làm số đỉnh của G' giảm 1. Nếu một lúc nào đó G' có 3 đỉnh, vì G' song liên thông đỉnh nên G' phải là K_3 , nhưng K_3 có 3 đỉnh bậc 2, trái với bất biến. Vì mọi lúc G' không có đỉnh bậc 1, sau một số hữu hạn đủ lớn (có thể bằng 0) các phép co đỉnh bậc 2 và chồng cạnh song song, mọi đỉnh của G' đều có bậc ít nhất 3.

Ban đầu G' chỉ có hai đỉnh nằm trên C , trong khi C có ít nhất 3 đỉnh, nên số đỉnh của G' nhỏ hơn số đỉnh của G . Lấy đồ thị G' sau thao tác, vốn có mọi đỉnh bậc ít nhất 3, làm G mới để lặp lại. Đồ thị mới vẫn rơi vào một trong ba trường hợp trên. Nếu rơi vào trường hợp 1 hoặc 2, nó không phải nối tiếp-song song tổng quát; nếu rơi vào trường hợp 3, số đỉnh tiếp tục giảm nhưng luôn ít nhất 4. Vì vậy sau hữu hạn lần lặp, chắc chắn tìm được sáu đường đôi một không chung cạnh nối từng cặp trong bốn đỉnh. Theo định lý 5.4, G ban đầu không phải nối tiếp-song song tổng quát.

Vậy mọi đồ thị vô hướng đơn giản liên thông mà mọi đỉnh có bậc ít nhất 3 đều không phải đồ thị nối tiếp-song song tổng quát. $\square$

Hiển nhiên, với đồ thị liên thông, các phép co đỉnh bậc 2, chồng cạnh song song và xóa đỉnh bậc 1 đều không thể làm đồ thị trở nên không liên thông. Theo định lý 5.4 và 5.5, mọi đồ thị vô hướng liên thông không thể dùng các phép ấy để biến thành một đồ thị đều không phải đồ thị nối tiếp-song song tổng quát.

Vì phép xóa đỉnh bậc 1 và co đỉnh bậc 2 đều làm số đỉnh và số cạnh giảm 1, còn phép chồng cạnh song song chỉ có thể xảy ra sau co đỉnh bậc 2 và làm số cạnh giảm 1, suy ra mọi đồ thị nối tiếp-song song tổng quát có thể biến thành đồ thị một đỉnh sau $O(n)$ lần co đỉnh bậc 2, chồng cạnh song song và xóa đỉnh bậc 1. Ngoài ra, cạnh của đồ thị nối tiếp-song song tổng quát đơn giản là $O(n)$; cụ thể $m \leq 2n$.

5.4. Thuật toán bảy: đồ thị nối tiếp-song song tổng quát, không có sửa

Với kết luận trên, có thể dùng trực tiếp thuật toán ở 5.2 để qua các tập con 1, 3, 4. Nếu sau mỗi lần sửa tính lại thô bạo, có thể qua các tập con 2, 5. Độ phức tạp thời gian $O(nQ)$, kỳ vọng 30 điểm.

6. Xét sửa đổi

Nay đã có thuật toán giải một truy vấn trong thời gian tuyến tính, ta tối ưu trên cơ sở đó để hỗ trợ sửa hiệu quả hơn. Trước hết xét tình huống đơn giản hơn: trong tập con 7, mọi trọng số đỉnh đều bằng 0, nên không cần xét ảnh hưởng của trọng số đỉnh.

6.1. Thuật toán tám: đồ thị nối tiếp-song song tổng quát, mọi trọng số đỉnh bằng 0

Xét các phép toán cần dùng. Phép chồng cạnh song song cộng tương ứng trọng số của hai cạnh song song và thay bằng một cạnh mới. Phép co đỉnh bậc 2 lấy hai cạnh kề của một đỉnh bậc 2, tính rồi thay

bằng một cạnh mới. Vì trọng số đỉnh đều bằng 0, trọng số của đỉnh bậc 2 không ảnh hưởng tới trọng số cạnh mới.

Khi trọng số đỉnh bằng 0, ta có $w_{00} = w_{11}$ và $w_{01} = w_{10}$, nên như mô tả đề bài, chỉ cần dùng s_e, d_e để biểu diễn trọng số cạnh e . Với phép xóa đỉnh bậc 1, vì có thể chọn màu của đỉnh bậc 1 để tối đa hóa đóng góp của cạnh kề, ta cộng trực tiếp $\max\{s_i, d_i\}$ của cạnh ấy vào đáp án.

Vì phép chồng cạnh song song và co đỉnh bậc 2 đều biến trọng số của hai cạnh thành trọng số của một cạnh mới, ta có thể biểu diễn chúng bằng phép toán.

6.1.1. Định nghĩa phép toán

Gọi trọng số của cạnh e là

$$v_e = \begin{bmatrix} s_e \\ d_e \end{bmatrix}.$$

Với trọng số của hai cạnh, định nghĩa phép $+$ là kết quả sau phép chồng cạnh song song:

$$\begin{bmatrix} s_1 \\ d_1 \end{bmatrix} + \begin{bmatrix} s_2 \\ d_2 \end{bmatrix} = \begin{bmatrix} s_1 + s_2 \\ d_1 + d_2 \end{bmatrix}.$$

Định nghĩa phép $*$ là kết quả sau phép co đỉnh bậc 2:

$$\begin{bmatrix} s_1 \\ d_1 \end{bmatrix} * \begin{bmatrix} s_2 \\ d_2 \end{bmatrix} = \begin{bmatrix} \max\{s_1 + s_2, d_1 + d_2\} \\ \max\{s_1 + d_2, d_1 + s_2\} \end{bmatrix}.$$

Dễ thấy $+$ và $*$ đều giao hoán. Với phép xóa đỉnh bậc 1, có thể định nghĩa đáp án ban đầu là $\text{ans} = [0 \ 0]^T$; khi gặp đỉnh bậc 1 thì đặt $\text{ans} \leftarrow \text{ans} * v_e$, trong đó e là cạnh kề đỉnh bậc 1.

Vì trọng số của mỗi cạnh đều có thể biểu diễn từ trọng số ban đầu bằng các phép $+$ và $*$, đáp án toàn bài cũng có thể biểu diễn bằng một biểu thức. Ta xây cây biểu thức của biểu thức ấy, biến bài toán lại thành quy hoạch động trên cây, rồi dùng phân rã chuỗi để tối ưu sửa đổi.

6.1.2. Xây cây biểu thức

Quá trình xây cây biểu thức như sau. Gọi op_v là toán tử ở đỉnh không lá v của cây biểu thức. Trước hết với mỗi cạnh e , tạo đỉnh tương ứng $v_e = [s_e \ d_e]^T$ trong cây biểu thức; với đáp án ban đầu, tạo đỉnh $\text{ans} = [0 \ 0]^T$.

Sau đó dùng thuật toán bảy thao tác trên G tới khi chỉ còn một đỉnh:

1. Nếu cần co đỉnh bậc 2, sau thao tác tạo đỉnh $v_{e'}$ của cây biểu thức ứng với cạnh mới e' . Đặt hai đỉnh ứng với hai cạnh cũ làm con của $v_{e'}$, và đặt $\text{op}_{e'} = *$.
2. Nếu cần chồng cạnh song song, sau thao tác tạo đỉnh $v_{e'}$ ứng với cạnh mới e' . Đặt hai đỉnh ứng với hai cạnh cũ làm con của $v_{e'}$, và đặt $\text{op}_{e'} = +$.
3. Nếu cần xóa đỉnh bậc 1, sau thao tác tạo đỉnh v trong cây biểu thức. Đặt ans và đỉnh ứng với cạnh bị xóa làm con của v , đặt $\text{op}_v = *$, rồi đặt $\text{ans} = v$.

6.1.3. Phân rã

Tiếp tục dùng phân rã của thuật toán ba. Trong cây biểu thức, mỗi đỉnh không lá có đúng hai con. Ký hiệu $c_{v,0}, c_{v,1}$ là hai con của v , và sn_v là chỉ số tham số của con nặng, tức là $c_{v,\text{sn}_v} = \text{son}_v$.

Để duy trì nhanh thông tin trên chuỗi nặng, vẫn dùng định nghĩa g_v trong thuật toán ba:

$$f_v = g_v f_{\text{son}_v}.$$

Với đỉnh không lá v , giả sử

$$\text{son}_v = \begin{bmatrix} x \\ y \end{bmatrix}, \quad c_{v,1-\text{sn}_v} = \begin{bmatrix} z \\ w \end{bmatrix}.$$

Nếu $\text{op}_v = +$, thì

$$v = \begin{bmatrix} x + z \\ y + w \end{bmatrix}, \quad g_v = \begin{bmatrix} z & -\infty \\ -\infty & w \end{bmatrix}.$$

Nếu $\text{op}_v = *$, thì

$$v = \begin{bmatrix} \max\{x + z, y + w\} \\ \max\{x + w, y + z\} \end{bmatrix}, \quad g_v = \begin{bmatrix} z & w \\ w & z \end{bmatrix}.$$

Sau đó có thể áp dụng thuật toán ba. Điểm khác biệt là trong thuật toán ba, g không thể tính thô bạo mà phải duy trì bằng chênh lệch; còn ở đây g có thể tính trực tiếp. Độ phức tạp $O(n + Q \log^2 n)$, qua được các tập con 6, 7, kỳ vọng 36 điểm.

Cuối cùng xét ảnh hưởng của trọng số đỉnh.

6.2. Thuật toán chín: đồ thị nối tiếp-song song tổng quát, có sửa

Cải tiến trên cơ sở thuật toán tám. Lúc này phép co đỉnh bậc 2 và xóa đỉnh bậc 1 đều phải xét trọng số đỉnh, và trọng số cạnh cần ghi bốn giá trị $w_{00}, w_{01}, w_{10}, w_{11}$, tức là

$$v_e = \begin{bmatrix} w_{00} \\ w_{01} \\ w_{10} \\ w_{11} \end{bmatrix}.$$

Ngoài ra cần ghi trọng số đỉnh; với đỉnh u ,

$$v_u = \begin{bmatrix} b_u \\ w_u \end{bmatrix}.$$

Các phép $+$ và $*$ của thuật toán tám không còn phù hợp, nên cần định nghĩa lại.

6.2.1. Mở rộng phép toán

Tương tự, phép $+$ là kết quả của phép chồng cạnh song song:

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} + \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} = \begin{bmatrix} a + x \\ b + y \\ c + z \\ d + w \end{bmatrix}.$$

Phép $*$ là trọng số cạnh mới sinh ra khi co đỉnh bậc 2; lúc này nó phụ thuộc vào trọng số của một đỉnh bị xóa và hai cạnh. Giả sử

$$e_0 = (u_0, u_1, (a, b, c, d)), \quad e_1 = (u_1, u_2, (x, y, z, w)), \quad u_1 = (\alpha, \beta).$$

Khi đó

$$* \left(\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}, \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \right) = \begin{bmatrix} \max\{a + \alpha + x, b + \beta + z\} \\ \max\{a + \alpha + y, b + \beta + w\} \\ \max\{c + \alpha + x, d + \beta + z\} \\ \max\{c + \alpha + y, d + \beta + w\} \end{bmatrix}.$$

Với phép xóa đỉnh bậc 1, định nghĩa phép $\oplus$ là trọng số của đỉnh bị sửa, tức là đỉnh kề với đỉnh bị xóa. Giả sử

$$e_0 = (u_0, u_1, (a, b, c, d)), \quad u_0 = (\alpha, \beta), \quad u_1 = (\gamma, \delta),$$

và u_1 là đỉnh bậc 1 được chọn để xóa, thì

$$\oplus \left(\begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}, \begin{bmatrix} \gamma \\ \delta \end{bmatrix} \right) = \begin{bmatrix} \max\{\alpha + a + \gamma, \alpha + b + \delta\} \\ \max\{\beta + c + \gamma, \beta + d + \delta\} \end{bmatrix}.$$

Ngoài ra còn cần xét đổi hướng một cạnh: biến

$$(u, v, (w_{00}, w_{01}, w_{10}, w_{11})) \quad \text{thành} \quad (v, u, (w_{00}, w_{10}, w_{01}, w_{11})).$$

Định nghĩa phép R :

$$R \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} a \\ c \\ b \\ d \end{bmatrix}.$$

6.2.2. Xây cây biểu thức

Cách xây cây biểu thức tương tự thuật toán tám, nhưng $*$ và $\oplus$ không giao hoán, nên cần xét thứ tự con. Lúc này $c_{v,0}, c_{v,1}, c_{v,2}$ lần lượt là ba tham số của phép $*$ hoặc $\oplus$, tức là

$$v = *(c_{v,0}, c_{v,1}, c_{v,2}) \quad \text{hoặc} \quad v = \oplus(c_{v,0}, c_{v,1}, c_{v,2}).$$

Dùng thuật toán bảy thao tác trên G đến khi chỉ còn một đỉnh:

1. Nếu có đỉnh bậc 2, sau thao tác tạo đỉnh $v_{e'}$ ứng với cạnh mới e' trong cây biểu thức. Đặt các đỉnh ứng với một đỉnh bị xóa và hai cạnh bị xóa làm con của $v_{e'}$, xác định $c_{v_{e'},0}, c_{v_{e'},1}, c_{v_{e'},2}$, và đặt $\text{op}_{e'} = *$.
2. Nếu chồng cạnh song song, sau thao tác tạo đỉnh $v_{e'}$ ứng với cạnh mới e' . Đặt hai đỉnh ứng với hai cạnh cũ làm con của $v_{e'}$, và đặt $\text{op}_{e'} = +$.
3. Nếu xóa đỉnh bậc 1, sau thao tác tạo đỉnh $v_{u'}$ ứng với đỉnh được sửa u' . Đặt hai đỉnh và một cạnh liên quan đến thao tác làm con của $v_{u'}$, xác định $c_{v_{u'},0}, c_{v_{u'},1}, c_{v_{u'},2}$, và đặt $\text{op}_{u'} = \oplus$.
4. Khi thao tác phải xét hướng cạnh, nếu cần đổi hướng cạnh thì sau thao tác tạo đỉnh $v_{e'}$ của cây biểu thức, đặt đỉnh ứng với cạnh cần đổi hướng làm con của $v_{e'}$, và đặt $\text{op}_{e'} = R$.

6.2.3. Phân rã

Tương tự, chỉ cần tính được ma trận g là có thể dùng cách phân rã của thuật toán tám. Với đỉnh không lá v :

1. $\text{op}_v = +$. Giả sử

$$\text{son}_v = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}, \quad c_{v,1-\text{sn}_v} = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}.$$

Theo phép +,

$$v = \begin{bmatrix} x + a \\ y + b \\ z + c \\ w + d \end{bmatrix},$$

nên

$$g_v = \begin{bmatrix} a & -\infty & -\infty & -\infty \\ -\infty & b & -\infty & -\infty \\ -\infty & -\infty & c & -\infty \\ -\infty & -\infty & -\infty & d \end{bmatrix}.$$

2. $\text{op}_v = R$. Theo định nghĩa R ,

$$g_v = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ -\infty & -\infty & 0 & -\infty \\ -\infty & 0 & -\infty & -\infty \\ -\infty & -\infty & -\infty & 0 \end{bmatrix}.$$

3. $\text{op}_v = *$. Giả sử

$$c_{v,0} = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}, \quad c_{v,1} = \begin{bmatrix} a \\ \beta \end{bmatrix}, \quad c_{v,2} = \begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}.$$

Khi đó

$$v = \begin{bmatrix} \max\{a + \alpha + x, b + \beta + z\} \\ \max\{a + \alpha + y, b + \beta + w\} \\ \max\{c + \alpha + x, d + \beta + z\} \\ \max\{c + \alpha + y, d + \beta + w\} \end{bmatrix}.$$

Nếu $\text{sn}_v = 0$,

$$g_v = \begin{bmatrix} \alpha + x & \beta + z & -\infty & -\infty \\ \alpha + y & \beta + w & -\infty & -\infty \\ -\infty & -\infty & \alpha + x & \beta + z \\ -\infty & -\infty & \alpha + y & \beta + w \end{bmatrix}.$$

Nếu $\text{sn}_v = 1$,

$$g_v = \begin{bmatrix} \alpha + x & b + z \\ \alpha + y & b + w \\ c + x & d + z \\ c + y & d + w \end{bmatrix}.$$

Nếu $\text{sn}_v = 2$,

$$g_v = \begin{bmatrix} \alpha + \alpha & -\infty & b + \beta & -\infty \\ -\infty & \alpha + \alpha & -\infty & b + \beta \\ c + \alpha & -\infty & d + \beta & -\infty \\ -\infty & c + \alpha & -\infty & d + \beta \end{bmatrix}.$$

4. $\text{op}_v = \oplus$. Giả sử

$$c_{v,0} = \begin{bmatrix} a \\ \beta \end{bmatrix}, \quad c_{v,1} = \begin{bmatrix} b \\ c \\ d \end{bmatrix}, \quad c_{v,2} = \begin{bmatrix} \gamma \\ \delta \end{bmatrix}.$$

Theo định nghĩa,

$$v = \begin{bmatrix} \max\{a + a + \gamma, a + b + \delta\} \\ \max\{\beta + c + \gamma, \beta + d + \delta\} \end{bmatrix}.$$

Nếu $\text{sn}_v = 0$,

$$g_v = \begin{bmatrix} \max\{a + \gamma, b + \delta\} & -\infty \\ -\infty & \max\{c + \gamma, d + \delta\} \end{bmatrix}.$$

Nếu $\text{sn}_v = 1$,

$$g_v = \begin{bmatrix} a + \gamma & a + \delta & -\infty & -\infty \\ -\infty & -\infty & \beta + \gamma & \beta + \delta \end{bmatrix}.$$

Nếu $\text{sn}_v = 2$,

$$g_v = \begin{bmatrix} a + a & a + b \\ \beta + c & \beta + d \end{bmatrix}.$$

Cách dựng ma trận g ở đây tương tự dựng ma trận chuyển của một quan hệ tuyến tính thông thường. Khi cài đặt, không cần phân trường hợp theo sn_v để tính g_v ; chi tiết có thể xem hàm ‘tran’ trong mã nguồn của tôi tại <https://loj.ac/submission/433979>. Các bước còn lại giống thuật toán tám nên không nhắc lại.

Độ phức tạp thời gian của thuật toán là $O(n + Q \log n)$, qua được toàn bộ điểm, kỳ vọng 100 điểm.

6.3. Ưu nhược điểm

6.3.1. Ưu điểm

Giải bài toán bằng cách thu nhỏ quy mô giúp quá trình quy hoạch động trực quan hơn, qua đó giảm lượng suy nghĩ cần thiết.

So với thuật toán phân rã chuỗi thông thường, cách dùng phân rã chuỗi trên cây biểu thức không cần xử lý tình huống một đỉnh có nhiều con nhẹ, nên giảm độ khó cài đặt.

6.3.2. Hạn chế

Cách dùng phân rã chuỗi trên cây biểu thức thường chỉ hỗ trợ truy vấn toàn cục hoặc truy vấn cây con, khó hỗ trợ truy vấn trên đường đi. Ngoài ra, phương pháp này thường phải duy trì tích ma trận trên cây đoạn, kéo theo hằng số không nhỏ.

7. Mở rộng

Trong một số bài toán, ta cần đếm số phương án tối ưu. Khi đó có thể đồng thời ghi số phương án tối ưu trong từng phần tử ma trận. Cho mỗi phần tử ma trận là một cặp (a, cnt) , trong đó a là giá trị tối ưu và cnt là số phương án tối ưu. Định nghĩa

$$(a_1, \text{cnt}_1) + (a_2, \text{cnt}_2) = (\max\{a_1, a_2\}, \text{cnt}_1[a_1 \geq a_2] + \text{cnt}_2[a_2 \geq a_1]),$$

$$(a_1, cnt_1) \times (a_2, cnt_2) = (a_1 + a_2, cnt_1 \times cnt_2).$$

Không khó thấy phép cộng và phép nhân định nghĩa ở đây đều giao hoán, kết hợp, và phép nhân phân phối với phép cộng, nên có thể dùng dạng nhân ma trận để chuyển. Vì vậy nếu *Công viên* hoặc một bài tương tự yêu cầu in số phương án tối ưu (lấy modulo một số nguyên tố lớn), vẫn có thể dùng cây đoạn theo phân rã cây thành chuỗi để duy trì tích ma trận.

8. Ý tưởng ra đề

Bài toán này ban đầu xuất phát từ một số suy nghĩ của tôi về bài toán tập độc lập. Khi tìm kiếm tập độc lập lớn nhất³¹, có một cách cắt nhánh: nếu tồn tại một đỉnh bậc 1, thì luôn tồn tại một nghiệm tối ưu chứa đỉnh bậc 1 ấy, nên có thể xóa đỉnh bậc 1 và đỉnh kề của nó để giảm quy mô. Nếu đặt vào bài toán tập độc lập trọng số lớn nhất³², liệu có cách tương tự hay không?

Có thể thấy nếu trọng số của đỉnh bậc 1 không nhỏ hơn trọng số của đỉnh kề, kết luận trên³³ vẫn đúng. Ngược lại, ta có thể trước hết giả định cạnh kề với đỉnh bậc 1 ấy không tồn tại, và chọn đỉnh bậc 1. Nếu về sau chọn đỉnh kề với đỉnh bậc 1 ban đầu, điều đó đồng nghĩa không thể chọn đỉnh bậc 1 ấy nữa; và vì vậy có thể trừ trọng số của đỉnh bậc 1 khỏi trọng số của đỉnh kề để xử lý tình huống này.

Trong luận văn *Bước đầu về bài toán tập độc lập trong thi tin học* của đội tuyển ứng viên IOI Trung Quốc 2017, Zhong Zhixian nhắc đến một thuật toán ưu tiên tìm kiếm các đỉnh bậc ít nhất 3; độ phức tạp của thuật toán ấy phụ thuộc nhiều vào số đỉnh bậc ít nhất 3 cần liệt kê. Tôi nhận ra rằng xóa đỉnh bậc 1 có thể giảm phần nào số đỉnh bậc ít nhất 3 cần liệt kê (ví dụ bài toán tập độc lập trên cây không cần liệt kê đỉnh bậc ít nhất 3 nào). Từ đó tôi tự hỏi có thể làm thao tác tương tự với đỉnh bậc 2 hay không, để tiếp tục giảm số đỉnh bậc ít nhất 3 cần liệt kê, tăng hiệu quả thuật toán hoặc thậm chí cải thiện độ phức tạp. Qua nghiên cứu tương tự cách xóa đỉnh bậc 1, tôi phát hiện điều này làm được, và thiết kế một thuật toán tốt hơn cho bài toán tập độc lập trọng số lớn nhất.

Tiếp đó tôi nghĩ: thuật toán này không cần liệt kê bất kỳ đỉnh bậc ít nhất 3 nào khi đồ thị thỏa tính chất gì? Đầu tiên tôi nghĩ tới xương rồng; rõ ràng xương rồng thỏa điều kiện. Nhưng tôi cảm thấy không chỉ xương rồng mới thỏa, có thể xét trường hợp tổng quát hơn. Nghiên cứu cho thấy thuật toán không cần liệt kê đỉnh bậc ít nhất 3 khi và chỉ khi mỗi thành phần liên thông của đồ thị là đồ thị nối tiếp-song song tổng quát. Điều này tạo nên ý tưởng ban đầu của bài *Công viên*.

Sau đó tôi cho rằng tập độc lập trọng số lớn nhất là bài toán kinh điển; nếu đưa thẳng vào kiểm tra chéo đội tập huấn thì cả độ mới lẫn độ khó đều hơi thiếu. Vì vậy tôi suy nghĩ cách tăng cường và cải tạo bài toán. Trước hết tôi nghĩ tới việc đưa đóng góp đáp án lên cạnh, làm đề trông bớt kinh điển hơn nhưng độ khó vẫn hơi thấp. Sau đó, xét tới việc gần đây có nhiều bài sửa động trên cây, thậm chí trên xương rồng, rồi hỏi nghiệm tối ưu toàn cục, tôi cũng thử đột phá theo hướng sửa động trên đồ thị nối tiếp-song song tổng quát.

Ban đầu tôi xét việc dùng hình dạng đồ thị đối ngẫu (tương tự một cây) rồi dùng phân rã chuỗi và cây đoạn để duy trì thông tin cạnh sau khi co đỉnh bậc 2 (khi ấy bài toán của tôi bảo đảm đồ thị song liên thông đỉnh và không có trọng số đỉnh). Nhưng tôi phát hiện nếu cho sẵn đồ thị đối ngẫu, bài toán trực tiếp trở thành bài toán trên cây, vì không cần xây đồ thị gốc vẫn giải được; còn nếu không cho đồ thị đối ngẫu, việc thu được hình dạng đồ thị đối ngẫu lại khá khó. Vì vậy tôi chuyển hướng sang trực tiếp duy trì quá trình thay đổi thông tin trên đồ thị khi co đỉnh bậc 2 (trong bài này chính là trọng số cạnh). Tôi nhận ra làm vậy chính là xây một cây biểu thức. Với các suy nghĩ trước đó, tôi giải được bài toán khá thuận lợi.

³¹Bài toán tập độc lập lớn nhất: cho đồ thị $G = (V, E)$, tìm tập con lớn nhất $S \subseteq V$ sao cho với mọi cạnh trong E , ít nhất một trong hai đầu mút không thuộc S .

³²Bài toán tập độc lập trọng số lớn nhất: cho đồ thị $G = (V, E)$ và hàm trọng số $w : V \rightarrow \mathbb{R}^+$, tìm $S \subseteq V$ sao cho với mọi cạnh trong E , ít nhất một đầu mút không thuộc S , và $\sum_{u \in S} w(u)$ lớn nhất.

³³Tức là “luôn tồn tại một nghiệm tối ưu chứa đỉnh bậc 1 ấy”.

Nhưng tôi vẫn chưa hoàn toàn hài lòng, vì cây biểu thức này thực chất biểu diễn trực tiếp cấu trúc nối tiếp-song song của đồ thị gốc; với lớp đồ thị này, tính một số đáp án chỉ liên quan đến cạnh (chẳng hạn tính diện tích tương đương giữa hai điểm trong mạch nối tiếp-song song) vốn tương đối dễ, bởi bản thân “nối tiếp-song song” mô tả quan hệ giữa các cạnh. Để bài toán tổng quát hơn, tôi bắt đầu xét vấn đề liên quan đến đỉnh. Tôi phát hiện cách xây cây biểu thức rồi phân rã chuỗi vẫn dùng được sau khi thêm trọng số đỉnh, chỉ cần bổ sung một lượng chi tiết xử lý nhất định. Vì vậy *Công viên* trở thành hình thức cuối cùng như trên.

Về thiết kế tập con, các tập 1, 2, 3 giúp thí sinh phân tích thuật toán đơn giản, đồng thời cung cấp ý tưởng cho tập 4, 5. Tập 4, 5 và tập 8 lần lượt đại diện cho hai hướng then chốt để giải bài toán. Tập 6, 7 đơn giản hóa chi tiết từ góc nhìn đặc biệt không cần xét trọng số đỉnh, giúp suy ra lời giải chuẩn cho tập 9 dễ hơn.

9. Tổng kết

Công viên là một bài toán có độ khó nhất định. Bài này thiết kế điểm thành ba mảng bộ phận để dẫn dắt thí sinh tới lời giải chuẩn, đồng thời cung cấp nhiều hướng đột phá; nhưng điều đó cũng có nghĩa là muốn lấy nhiều điểm bộ phận, thí sinh phải viết nhiều đoạn mã. Để qua đúng bài này cần tư duy lập trình rõ ràng và năng lực xử lý chi tiết thích hợp.

10. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn cô Chen Ying của Trường Trung học số 1 Phúc Châu đã quan tâm và chỉ dẫn.

Xin cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã chỉ dẫn và giúp đỡ.

Xin cảm ơn các đàn anh Dong Kefan, Chen Junkun, Zhong Zhixian của Đại học Thanh Hoa, và đàn anh Yang Haoxiang của Đại học Bắc Kinh đã quan tâm, chỉ dạy và khích lệ.

Xin cảm ơn các bạn Chen Yanxu và Chen Yuxin của Trường Trung học số 1 Phúc Châu đã hỗ trợ một số phần trong chứng minh.

Xin cảm ơn các bạn Chen Hongji, Chen Yanxu, Chen Yuxin và những bạn khác trong nhóm tin học Trường Trung học số 1 Phúc Châu đã đọc duyệt bản thảo.

Tài liệu tham khảo

1. Wikipedia. Biconnected component.
2. Wikipedia. Series-parallel graph.
3. Nhà xuất bản Công nghiệp Cơ khí. *Đại số tuyến tính*.
4. Qi Zichao. *Ứng dụng của thuật toán chia để trị trong các bài toán đường đi trên cây*.
5. Zhong Zhixian. *Bước đầu về bài toán tập độc lập trong thi tin học*. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2017.
6. Chen Junkun. *Báo cáo ra đề “Đồ thị con kỳ diệu” và mở rộng*. Tập luận văn ứng viên đội tuyển quốc gia IOI Trung Quốc 2017.

Bàn về cấu trúc dữ liệu có thể truy vết lịch sử

Kong Chaozhe, Trường Trung học Ôn Châu

Tóm tắt

Bài viết này chủ yếu giới thiệu cấu trúc dữ liệu có thể truy vết lịch sử. Trước hết bài viết giới thiệu cách truy vết lịch sử cho DSU, sau đó giới thiệu cách truy vết lịch sử cho hàng đợi, ngăn xếp và hàng đợi hai đầu; tiếp theo giới thiệu phần truy vết lịch sử một phần của hàng đợi ưu tiên, và cuối cùng dùng truy vết lịch sử hoàn toàn cho hàng đợi ưu tiên làm ví dụ để giới thiệu một phương pháp truy vết lịch sử hoàn toàn mang tính tổng quát.

1. Dẫn nhập

Giả sử hiện có một cấu trúc dữ liệu, hỗ trợ một số thao tác sửa đổi và thao tác truy vấn.

Sau khi thực hiện vài thao tác sửa đổi, ta phát hiện một thao tác sửa đổi nào đó bị nhập sai, muốn sửa thao tác ấy rồi tiếp tục.

Vì vậy xuất hiện cấu trúc dữ liệu có thể truy vết lịch sử.

2. Định nghĩa

Với một cấu trúc dữ liệu, ta duy trì một dãy thao tác, hỗ trợ chèn một thao tác vào một vị trí nào đó, hoặc xóa một thao tác; đồng thời hỗ trợ một số truy vấn trên trạng thái thu được sau khi thực hiện các thao tác ấy theo đúng thứ tự.

Nếu thực hiện được các chức năng này, ta gọi đó là truy vết lịch sử một phần (*Partial Retroactivity*) của cấu trúc dữ liệu ban đầu.

Nếu còn hỗ trợ truy vấn trên một tiền tố bất kỳ của dãy thao tác, ta gọi đó là truy vết lịch sử hoàn toàn (*Full Retroactivity*) của cấu trúc dữ liệu ban đầu.

3. DSU có thể truy vết lịch sử

Trước hết xét ví dụ truy vết lịch sử cho DSU.

Duy trì một dãy thao tác của DSU, hỗ trợ thao tác sau:

$\text{Insert}(t, \text{union}(x, y))$

chèn thao tác hợp nhất hai tập chứa x và y sau thao tác thứ t .

Để đơn giản hóa bài toán, ở đây không xét thao tác xóa.

(Khi có thao tác xóa, truy vết lịch sử một phần thực chất chính là bài toán liên thông động trên đồ thị.)

3.1. DSU truy vết lịch sử một phần

DSU truy vết lịch sử một phần cần trả lời truy vấn sau:

$\text{Query_sameset}(x, y)$

hỏi ở thời điểm hiện tại x và y có thuộc cùng một tập hay không.

Ở đây xem mọi thao tác xảy ra tại các thời điểm đôi một khác nhau, tăng dần theo thứ tự trong dãy thao tác; thời điểm hiện tại là $+\infty$.

3.1.1. Phân tích

Thao tác union có tính giao hoán, nên dùng trực tiếp DSU là được.

3.2. DSU truy vết lịch sử hoàn toàn

DSU truy vết lịch sử hoàn toàn cần trả lời truy vấn sau:

`Query_sameset( $t, x, y$ )`

hỏi tại thời điểm t , x và y có thuộc cùng một tập hay không.

3.2.1. Phân tích

Chỉ cần dùng LCT để duy trì cây khung nhỏ nhất lấy thời điểm làm khóa.

4. Hàng đợi có thể truy vết lịch sử

Duy trì một dãy thao tác của hàng đợi, hỗ trợ các thao tác sau:

1. `Insert( $t$ , enqueue( $x$ ))`: chèn thao tác đưa phần tử x vào hàng đợi sau thao tác thứ t .
2. `Insert( $t$ , dequeue())`: chèn thao tác lấy phần tử ra khỏi hàng đợi sau thao tác thứ t .
3. `Delete( $t$ )`: xóa thao tác thứ t .

4.1. Hàng đợi truy vết lịch sử một phần

Hàng đợi truy vết lịch sử một phần phải trả lời được các truy vấn sau:

1. `Query(front())`: hỏi phần tử tiếp theo sẽ bị lấy ra ở thời điểm hiện tại.
2. `Query(back())`: hỏi phần tử được thêm vào sau cùng trong hàng đợi ở thời điểm hiện tại.

4.1.1. Phân tích

Không khó thấy các thao tác của hàng đợi đều có thể gộp lại; không cần xét thứ tự giữa một thao tác `enqueue( $x$ )` và một thao tác `dequeue()`. Ta có thể thực hiện trước toàn bộ các thao tác `enqueue( $x$ )`, rồi mới thực hiện toàn bộ các thao tác `dequeue()`; hàng đợi thu được vẫn như cũ.

Như vậy chỉ cần duy trì thứ tự trước sau của các thao tác `enqueue( $x$ )`, tức là mỗi lần chèn hiệu quả một số x sau số thứ t .

Cây cân bằng?

Quá khó viết, độ phức tạp quá cao.

Thực ra danh sách liên kết là đủ.

Lưu các thao tác enqueue(x) theo thứ tự thời gian trong danh sách liên kết. Đồng thời duy trì hai con trỏ F và B , lần lượt biểu diễn phần tử đầu hàng đợi và phần tử cuối hàng đợi; khi trả lời Query($front()$) và Query($back()$), chỉ cần trả về giá trị của F và B .

Với ví dụ

```
enqueue(6) enqueue(5) enqueue(4) enqueue(3)
enqueue(2) enqueue(1) dequeue() dequeue()
```

các sửa đổi trên danh sách liên kết như sau.

Một thao tác Insert(t , enqueue(x)) sửa danh sách liên kết:

1. Chèn phần tử x .
2. Nếu nó trở thành đuôi hàng mới thì cập nhật B .
3. Nếu chèn phần tử sau F , thì cho F trở tới phần tử kế tiếp của F ; nếu không thì giữ nguyên.

Một thao tác Delete(t) sửa danh sách liên kết, trong đó t là thao tác enqueue(x):

1. Xóa phần tử x .
2. Nếu nó vốn là đuôi hàng thì cập nhật B .
3. Nếu xóa phần tử sau F , thì cho F trở tới phần tử trước của F ; nếu không thì giữ nguyên.

Một thao tác Insert(t , dequeue()) sửa danh sách liên kết bằng cách cho F trở tới phần tử trước của F .

Một thao tác Delete(t) sửa danh sách liên kết, trong đó t là thao tác dequeue(), bằng cách cho F trở tới phần tử kế tiếp của F .

Độ phức tạp của một thao tác là $O(1)$.

4.2. Hàng đợi truy vết lịch sử hoàn toàn

Hàng đợi truy vết lịch sử hoàn toàn phải trả lời được các truy vấn sau:

1. Query(t , front()): hỏi phần tử tiếp theo sẽ bị lấy ra sau thời điểm t .
2. Query(t , back()): hỏi phần tử được thêm vào sau cùng trong hàng đợi sau thời điểm t .

4.2.1. Phân tích

Do thao tác enqueue(x) và thao tác dequeue() độc lập với nhau, ta duy trì hai cây cân bằng T_e và T_d để xử lý riêng hai loại thao tác.

T_e lưu toàn bộ thao tác enqueue(x) theo thứ tự thời gian.

T_d lưu toàn bộ thao tác dequeue() theo thứ tự thời gian.

Thao tác sửa đổi chỉ cần thực hiện thao tác tương ứng trên cây cân bằng tương ứng, độ phức tạp $O(\log m)$.

Trả lời truy vấn Query(t , back()): tìm trong T_e thao tác cuối cùng trước t .

Trả lời truy vấn Query(t , front()): trước hết tìm trong T_d số lượng thao tác dequeue() trước t , giả sử số đó là k . Sau đó tìm trong T_e giá trị được chèn ở lần thứ $k + 1$, có thể nhị phân trên cây cân bằng.

Độ phức tạp của một truy vấn là $O(\log m)$.

5. Ngăn xếp có thể truy vết lịch sử

Duy trì một dãy thao tác của ngăn xếp, hỗ trợ các thao tác sau:

1. $\text{Insert}(t, \text{push}(x))$: chèn thao tác đưa phần tử x vào ngăn xếp sau thao tác thứ t .
2. $\text{Insert}(t, \text{pop}())$: chèn thao tác lấy phần tử ra khỏi ngăn xếp sau thao tác thứ t .
3. $\text{Delete}(t)$: xóa thao tác thứ t .

Do độ phức tạp mỗi thao tác của truy vết lịch sử một phần và truy vết lịch sử hoàn toàn đều là $O(\log m)$, sau đây chỉ xét ngăn xếp truy vết lịch sử hoàn toàn.

5.1. Ngăn xếp truy vết lịch sử hoàn toàn

Ngăn xếp truy vết lịch sử hoàn toàn phải trả lời được truy vấn sau:

$\text{Query}(t, \text{top}())$

hỏi phần tử trên đỉnh ngăn xếp sau thời điểm t .

5.1.1. Phân tích

Xét phần tử nào mới có thể là $\text{top}()$ cuối cùng.

Có thể thấy một thao tác $\text{Insert}(t', \text{push}(x))$ là phần tử $\text{top}()$ cuối cùng khi và chỉ khi trong đoạn $(t', t]$, số thao tác push bằng số thao tác pop .

Vì vậy bài toán trở thành làm thế nào để nhanh chóng tìm thao tác như vậy.

Dùng cây cân bằng để giải. Duy trì mọi thao tác theo thứ tự thời gian trong một cây cân bằng; thao tác $\text{push}(x)$ có trọng số $+1$, thao tác $\text{pop}()$ có trọng số -1 .

Khi đó bài toán trở thành tìm điểm cuối cùng có tổng hậu tố bằng 1.

Do giá trị chỉ có ± 1 , nên các tổng hậu tố trong đoạn $[l_t, r_t]$ chắc chắn tạo thành một đoạn giá trị liên tục $[l_x, r_x]$. Vì vậy với mỗi nút của cây cân bằng, ghi min và max, lần lượt biểu diễn tổng hậu tố nhỏ nhất và lớn nhất trong cây con, cùng với tổng trọng số sum của cây con; khi đó có thể nhị phân trên cây cân bằng để tìm điểm có tổng hậu tố bằng 1.

Độ phức tạp của mỗi thao tác sửa đổi và truy vấn là $O(\log m)$.

6. Hàng đợi hai đầu có thể truy vết lịch sử

Duy trì một dãy thao tác của hàng đợi hai đầu, hỗ trợ các thao tác sau:

1. $\text{Insert}(t, \text{pushL}(x))$: chèn thao tác đưa phần tử x vào hàng đợi hai đầu từ đầu trái sau thao tác thứ t .
2. $\text{Insert}(t, \text{pushR}(x))$: chèn thao tác đưa phần tử x vào hàng đợi hai đầu từ đầu phải sau thao tác thứ t .
3. $\text{Insert}(t, \text{popL}())$: chèn thao tác lấy số đầu tiên ở đầu trái sau thao tác thứ t .
4. $\text{Insert}(t, \text{popR}())$: chèn thao tác lấy số đầu tiên ở đầu phải sau thao tác thứ t .
5. $\text{Delete}(t)$: xóa thao tác thứ t .

Do độ phức tạp mỗi thao tác của truy vết lịch sử một phần và truy vết lịch sử hoàn toàn đều là $O(\log m)$, sau đây chỉ xét hàng đợi hai đầu truy vết lịch sử hoàn toàn.

6.1. Hàng đợi hai đầu truy vết lịch sử hoàn toàn

Hàng đợi hai đầu truy vết lịch sử hoàn toàn phải trả lời được các truy vấn sau:

1. `Query(t, left())`: hỏi giá trị của phần tử ngoài cùng bên trái trong hàng đợi hai đầu sau thời điểm t .
2. `Query(t, right())`: hỏi giá trị của phần tử ngoài cùng bên phải trong hàng đợi hai đầu sau thời điểm t .

6.1.1. Phân tích

Đã có ngăn xếp truy vết lịch sử hoàn toàn, liệu có thể tách hàng đợi hai đầu thành hai nửa, dùng các thao tác `pushL(x)` và `popL()` để duy trì một ngăn xếp đầu trái, và dùng các thao tác `pushR(x)` và `popR()` để duy trì một ngăn xếp đầu phải hay không?

Vì thao tác ở một phía của hàng đợi hai đầu, nếu trở nên không hợp lệ, sẽ ảnh hưởng tới phía còn lại, nên điểm cuối cùng có tổng hậu tố bằng 1 chưa chắc là đáp án. Chẳng hạn ví dụ sau:

`pushR(1) popL() pushL(2) Query(t, right())`.

Dĩ nhiên, thuật toán sai này cũng gợi ra một số ý tưởng. Một giá trị chắc chắn được đưa vào hàng đợi hai đầu bởi một thao tác `pushL(x)` hoặc một thao tác `pushR(x)`. Nếu lấy được ứng viên có khả năng là đáp án nhất trong các thao tác `pushL(x)` và ứng viên có khả năng là đáp án nhất trong các thao tác `pushR(x)`, rồi kiểm tra thêm, là đủ.

Xét một cách tự viết hàng đợi hai đầu. Tạo một mảng a , ban đầu $L = 1, R = 0$.

1. Thao tác `pushL(x)` tương ứng với $a[-L] = x$.
2. Thao tác `pushR(x)` tương ứng với $a[++R] = x$.
3. Thao tác `popL()` tương ứng với $--L$.
4. Thao tác `popR()` tương ứng với $--R$.

Như vậy mỗi truy vấn chỉ cần biết i và $a[i]$.

Duy trì hai cây cân bằng T_l và T_r . Cây T_l lưu các thao tác `pushL(x)` và `popL()` theo thời gian, với trọng số lần lượt là $+1$ và -1 ; T_r cũng tương tự. Muốn tìm giá trị i tại thời điểm t , chỉ cần tính tổng tiền tố trong T_l hoặc T_r .

Giá trị của $a[i]$ chắc chắn đến từ thao tác `pushL(x)` cuối cùng mà sau đó $L = i$, hoặc thao tác `pushR(x)` cuối cùng mà sau đó $R = i$. Cả hai thao tác này đều có thể được tìm bằng cách nhị phân trên cây cân bằng để lấy điểm có tổng hậu tố bằng k ; chọn thao tác muộn hơn trong hai thao tác ấy là được.

Độ phức tạp mỗi thao tác sửa đổi và truy vấn là $O(\log m)$.

Có thể thấy khi trả lời truy vấn, ta không hề dùng tới ràng buộc là điểm ngoài cùng bên trái hay ngoài cùng bên phải, nên việc trả lời giá trị của một phần tử bất kỳ trong hàng đợi hai đầu cũng làm được.

7. Hàng đợi ưu tiên có thể truy vết lịch sử

Duy trì một dãy thao tác của hàng đợi ưu tiên (heap), hỗ trợ các thao tác sau:

1. $\text{Insert}(t, \text{insert}(k))$: chèn thao tác đưa phần tử k vào heap sau thao tác thứ t .
2. $\text{Insert}(t, \text{delete_min}())$: chèn thao tác lấy phần tử nhỏ nhất ra sau thao tác thứ t .
3. $\text{Delete}(t)$: xóa thao tác thứ t .

7.1. Hàng đợi ưu tiên truy vết lịch sử một phần

Hàng đợi ưu tiên truy vết lịch sử một phần phải trả lời được truy vấn sau:

$\text{Query}()$

hỏi phần tử nhỏ nhất của hàng đợi ở thời điểm hiện tại.

7.1.1. Phân tích

Ký hiệu Q_t là tập phần tử trong hàng đợi tại thời điểm t ; ta chỉ cần có thể truy vấn Q_{now} .

Xét ảnh hưởng của $\text{Insert}(t, \text{insert}(k))$ lên Q_{now} . Có thể thấy nó tương đương với việc chèn vào Q_{now} giá trị

$$\max\{k, k' \mid k' \text{ bị xóa tại thời điểm } \geq t\}.$$

Vấn đề là khó duy trì những phần tử nào đã bị xóa.

Ta gọi một thời điểm t là một cầu (*bridge*) khi và chỉ khi $Q_t \subseteq Q_{\text{now}}$.

Nếu t' là cầu đầu tiên trước thời điểm t , thì

$$\max\{k' \mid k' \text{ bị xóa tại thời điểm } \geq t\} = \max\{k' < Q_{\text{now}} \mid k' \text{ được chèn tại thời điểm } \geq t'\}. \quad (37)$$

Tương tự, xét ảnh hưởng của $\text{Insert}(t, \text{delete_min}())$. Nếu t' là cầu đầu tiên sau t , thì ảnh hưởng của thao tác này là xóa

$$\min\{k' \in Q_{\text{now}} \mid k' \text{ được chèn tại thời điểm } \leq t'\}. \quad (38)$$

Còn với việc hủy một thao tác $\text{insert}(k)$, nếu k thuộc Q_{now} , ảnh hưởng là xóa k ; nếu không, ảnh hưởng là xóa

$$\min\{k' \in Q_{\text{now}} \mid k' \text{ được chèn tại thời điểm } \leq t'\}.$$

Tương tự, ảnh hưởng của việc hủy một thao tác $\text{delete_min}()$ là

$$\max\{k' < Q_{\text{now}} \mid k' \text{ được chèn tại thời điểm } \geq t'\}.$$

Nhờ đó ta có thể duy trì Q_{now} .

Xét việc dùng cây cân bằng để duy trì Q_{now} . Để xử lý sửa đổi, ta cần thêm các cây cân bằng khác.

Dùng một cây cân bằng khác để duy trì các thao tác chèn. Với mỗi nút u , ta lưu

$$\max\{k' < Q_{\text{now}} \mid k' \text{ được chèn trong cây con của } u\}$$

và

$$\min\{k' \in Q_{\text{now}} \mid k' \text{ được chèn trong cây con của } u\}.$$

Ta còn cần một cây cân bằng để tìm cầu. Ta lưu các thao tác Insert và gán trọng số như sau:

1. Với thao tác $\text{insert}(k)$ và $k \in Q_{\text{now}}$, gán trọng số 0.

2. Với thao tác $\text{insert}(k)$ và $k < Q_{\text{now}}$, gán trọng số $+1$.
3. Với thao tác $\text{delete_min}()$, gán trọng số -1 .

Tại mỗi nút, ta lưu tổng cây con, giá trị nhỏ nhất của tổng tiền tố và giá trị lớn nhất của tổng tiền tố.

Vì cầu tương đương với tổng tiền tố bằng 0, nên với mỗi lần sửa đổi, có thể tìm một cầu trước t trong thời gian $O(\log^2 m)$ bằng cách nhị phân trên cây.

Sau khi tìm được cầu, ta dùng công thức (37) và (38) để tính ảnh hưởng của thao tác này lên Q_{now} , từ đó duy trì Q_{now} theo thời gian thực.

Sau khi tìm được ảnh hưởng lên Q_{now} , ta cũng có thể đồng thời duy trì thông tin của hai cây cân bằng còn lại.

Độ phức tạp thời gian: mỗi thao tác sửa đổi và truy vấn đều là $O(\log^2 m)$.

7.2. Hàng đợi ưu tiên truy vết lịch sử hoàn toàn

Truy vết lịch sử hoàn toàn cho heap tương đối khó, nên các phương pháp được dùng đều khá tổng quát và có phạm vi áp dụng rộng.

Có một phương pháp tổng quát có thể đưa độ phức tạp của *full* về độ phức tạp của *partial* nhân với $O(\sqrt{m})$.

Chia dãy thao tác thành các khối, mỗi khối gồm $O(\sqrt{m})$ thao tác.

Với mỗi khối, dùng phương pháp *partial* để duy trì dãy thao tác gồm tất cả thao tác trước khối ấy.

Chèn, xóa và truy vấn đều có thể làm rất thô bạo.

Bằng một phương pháp gọi là *hierarchical checkpointing*, có thể đạt $O(\log^2 m)$ cho mỗi thao tác chèn hoặc xóa, và $O(\log^2 m \log \log m)$ cho mỗi truy vấn.

Dùng một cây scapegoat để duy trì toàn bộ dãy thao tác.

Với mỗi nút, xét dãy thao tác tạo bởi cây con mà nút ấy đại diện, duy trì một heap truy vết lịch sử một phần. Kết quả duy trì được ghi thành hai cây cân bằng Q_{now} và Q_{del} , lần lượt chứa các phần tử chưa bị xóa và các phần tử đã bị xóa (nếu một thao tác xóa không có phần tử để xóa, xem phần tử bị xóa là vô cùng lớn).

Khi chèn nút, thực hiện một lần chèn trên mỗi tổ tiên.

Nếu việc đó làm một cây con mất cân bằng, cần tái dựng cây con ấy (cách cài đặt tái dựng sẽ nói sau).

Xóa cũng tương tự.

Để cài đặt thao tác xóa, có thể đánh dấu xóa, cũng có thể chỉ lưu thông tin ở các nút lá, tức là cấu trúc tương tự cây đoạn.

Khi truy vấn, cần hợp nhất $O(\log m)$ heap truy vết lịch sử một phần, rồi truy vấn trên kết quả hợp nhất.

Bổ đề. Ký hiệu kết quả hợp nhất hai heap truy vết lịch sử một phần Q_1, Q_2 là Q_3 . Khi đó

$$Q_{3,\text{now}} = Q_{2,\text{now}} \cup \max\text{-A}\{Q_{1,\text{now}} \cup Q_{2,\text{del}}\},$$

$$Q_{3,\text{del}} = Q_{1,\text{del}} \cup \min\text{-D}\{Q_{1,\text{now}} \cup Q_{2,\text{del}}\}.$$

Ở đây $A = |Q_{1,\text{now}}|$, $D = |Q_{2,\text{del}}|$, và $\max\text{-C}\{S\}$ biểu diễn C phần tử lớn nhất trong tập S .

Để hợp nhất nhanh, ta dùng hợp của nhiều cây cân bằng để biểu diễn Q_{now} và Q_{del} .

Định nghĩa heap truy vết lịch sử một phần hợp nhất được Q^k : cả Q_{now}^k và Q_{del}^k đều được biểu diễn bằng không quá k cây cân bằng.

Khi cần hợp nhất Q_1^k và Q_2^k , đặt

$$T = Q_{1,\text{now}} \cup Q_{2,\text{del}},$$

và ký hiệu T_i ($1 \leq i \leq 2k$) là cây cân bằng thứ i trong T . Đặt

$$x = \max\text{-A}\{T\}.$$

Tách mỗi T_i theo x thành hai cây cân bằng $T_{i,<}$ và $T_{i,>}$. Khi đó

$$Q_{3,\text{now}} = Q_{2,\text{now}} \cup T_{1,>} \cup \dots \cup T_{2k,>},$$

$$Q_{3,\text{del}} = Q_{1,\text{del}} \cup T_{1,<} \cup \dots \cup T_{2k,<}.$$

Như vậy từ Q_1^k và Q_2^k , ta thu được Q_3^{3k} .

Độ phức tạp thời gian là $O(k \log m)$.

Việc tính $\max\text{-A}\{T\}$ hơi phiền, ở đây không trình bày thêm; bạn đọc quan tâm có thể tự đọc bài báo.

Giả sử hiện cần hợp nhất k heap truy vết lịch sử một phần $Q_{1..k}$, và ký hiệu kết quả hợp nhất là Q^* .

Nếu chia để trị trực tiếp và dùng phương pháp trên để hợp nhất, thì Q_{now}^* và Q_{del}^* sẽ được biểu diễn bằng $O(3^{\log_2 k}) = O(k^{\log_2 3})$ cây cân bằng.

Bổ đề. Tồn tại a_i và a'_i sao cho

$$Q_{\text{now}}^* = \bigcup_i \max\text{-}a_i Q_{i,\text{now}} \cup \bigcup_i \max\text{-}a'_i Q_{i,\text{del}}.$$

Vì vậy Q_{now}^* và Q_{del}^* đều có thể được biểu diễn bằng $2k$ cây cân bằng.

Do đó thời gian hợp nhất k heap truy vết lịch sử một phần là $O(k \log k \log m)$.

Độ phức tạp khi truy vấn là $O(\log^2 m \log \log m)$.

Về tái dựng cây con, chỉ cần dùng thao tác trộn để hợp nhất $Q_{1,\text{now}}$ và $Q_{2,\text{del}}$. Với cây con kích thước m , độ phức tạp tái dựng là $O(m \log m)$.

Vì vậy độ phức tạp chèn và xóa là $O(\log^2 m)$ tính khấu hao.

8. Lời cảm ơn

Xin cảm ơn Hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy Shu Chunping đã bồi dưỡng và chỉ dẫn tôi.

Xin cảm ơn các bạn Wu Siyang và Dai Yan đã giúp đỡ bài viết này.

Xin cảm ơn cha mẹ đã thấu hiểu và quan tâm.

Tài liệu tham khảo

1. Erik D. Demaine, John Iacono and Stefan Langerman. *Retroactive Data Structures*.
2. Erik D. Demaine, Tim Kaler, Quanquan Liu, Aaron Sidford, and Adam Yedidia. *Polylogarithmic Fully Retroactive Priority Queues via Hierarchical Checkpointing*.